

[1-32] - Data Science

Tasks

Codecademy



A/B Testing for ShoeFly.com

Our favorite online shoe store, ShoeFly.com is performing an A/B Test. They have two different versions of an ad, which they have placed in emails, as well as in banner ads on Facebook, Twitter, and Google. They want to know how the two ads are performing on each of the different platforms on each day of the week. Help them analyze the data using aggregate measures.

If you get stuck during this project or would like to see an experienced developer work through it, click “**Get Unstuck**” to see a **project walkthrough video**.

Tasks

11/11 complete

[Mark the tasks as complete by checking them off](#)

Analyzing Ad Sources

1.
Examine the first few rows of `ad_clicks`.

Hint

Try pasting the following code:

```
print(ad_clicks.head())
```

Copy to Clipboard

2.
Your manager wants to know which ad platform is getting you the most views.
How many views (i.e., rows of the table) came from each `utm_source`?

Stuck? Get a hint

3.
If the column `ad_click_timestamp` is not null, then someone actually clicked on the ad that was displayed.
Create a new column called `is_click`, which is `True` if `ad_click_timestamp` is not null and `False` otherwise.

Stuck? Get a hint

4.
We want to know the percent of people who clicked on ads from each `utm_source`.
Start by grouping by `utm_source` and `is_click` and counting the number of `user_id`'s in each of those groups.
Save your answer to the variable `clicks_by_source`.

Stuck? Get a hint

5.
Now let's pivot the data so that the columns are `is_click` (either `True` or `False`), the index is `utm_source`, and the values are `user_id`.
Save your results to the variable `clicks_pivot`.

Stuck? Get a hint

6.
Create a new column in `clicks_pivot` called `percent_clicked` which is equal to the percent of users who clicked on the ad from each `utm_source`.
Was there a difference in click rates for each source?

Stuck? Get a hint

Analyzing an A/B Test

7.

The column `experimental_group` tells us whether the user was shown Ad A or Ad B. Were approximately the same number of people shown both ads?

Hint

We can group by `experimental_group` and count the number of users.

8.

Using the column `is_click` that we defined earlier, check to see if a greater percentage of users clicked on Ad A or Ad B.

Hint

Group by both `experimental_group` and `is_click` and count the number of `user_id`'s. You might want to use a pivot table like we did for the `utm_source` exercises.

9.

The Product Manager for the A/B test thinks that the clicks might have changed by day of the week. Start by creating two DataFrames: `a_clicks` and `b_clicks`, which contain only the results for A group and B group, respectively.

Stuck? Get a hint

10.

For each group (`a_clicks` and `b_clicks`), calculate the percent of users who clicked on the ad by `day`.

Stuck? Get a hint

11.

Compare the results for A and B. What happened over the course of the week? Do you recommend that your company use Ad A or Ad B?

SQL

Usage Funnels with Warby Parker

WARBY PARKER

<https://www.codecademy.com/paths/data-science-inf/tracks/dsinf-advanced-sql-for-data-science/modules/dsinf-usage-funnels/projects/warby-parker>

[Warby Parker](#) is a transformative lifestyle brand with a lofty objective: to offer designer eyewear at a revolutionary price while leading the way for socially conscious businesses. Founded in 2010 and named after two characters in an early Jack Kerouac journal, Warby Parker believes in creative thinking, smart design, and doing good in the world. For every pair of eyeglasses and sunglasses sold, a pair is distributed to someone in need.

In this project, you will analyze different Warby Parker marketing funnels in order to calculate conversion rates. Here are the funnels and the tables that you are given:

Quiz Funnel:

survey

Home Try-On Funnel:

quiz

home_try_on

purchase

This project was a collaboration with Warby Parker's Data Science team (thank you!) and uses fictional data.

Let's get started!

Tasks

2/6 complete

Mark the tasks as complete by checking them off

Quiz Funnel:

1.

To help users find their perfect frame, Warby Parker has a [Style Quiz](#) that has the following questions:

“What are you looking for?”

“What’s your fit?”

“Which shapes do you like?”

“Which colors do you like?”

“When was your last eye exam?”

The users’ responses are stored in a table called `survey`.

Select all columns from the first 10 rows. What columns does the table have?

Stuck? Get a hint

2.

Users will “give up” at different points in the survey. Let’s analyze how many users move from Question 1 to Question 2, etc.

Create a quiz funnel using the `GROUP BY` command.

What is the number of responses for each question?

Stuck? Get a hint

3.

Using a spreadsheet program like Excel or Google Sheets, calculate the percentage of users who answer each question:

Which question(s) of the quiz have a lower completion rates?

What do you think is the reason?

Stuck? Get a hint

Home Try-On Funnel:

4.

Warby Parker’s purchase funnel is:

Take the Style Quiz → Home Try-On → Purchase the Perfect Pair of Glasses

During the Home Try-On stage, we will be conducting an A/B Test:

50% of the users will get **3** pairs to try on

50% of the users will get **5** pairs to try on

Let's find out whether or not users who get more pairs to try on at home will be more likely to make a purchase.

The data will be distributed across three tables:

```
quiz
home_try_on
purchase
```

Examine the first five rows of each table

What are the column names?

Stuck? Get a hint

5.

We'd like to create a new table with the following layout:

<code>user_id</code>	<code>is_home_try_on</code>	<code>number_of_pairs</code>	<code>is_purchase</code>
4e8118dc	True	3	False
291f1cca	True	5	False
75122300	False	NULL	False

Each row will represent a single user from the browse table:

If the user has any entries in `home_try_on`, then `is_home_try_on` will be `True`.

`number_of_pairs` comes from `home_try_on` table

If the user has any entries in `purchase`, then `is_purchase` will be `True`.

Use a `LEFT JOIN` to combine the three tables, starting with the top of the funnel (quiz) and ending with the bottom of the funnel (purchase).

Select only the first 10 rows from this table (otherwise, the query will run really slowly).

Stuck? Get a hint

6.

Once we have the data in this format, we can analyze it in several ways:

We can calculate overall conversion rates by aggregating across all rows.

We can compare conversion from `quiz→home_try_on` and `home_try_on→purchase`.

We can calculate the difference in purchase rates between customers who had 3 `number_of_pairs` with ones who had 5.

And more!

We can also use the original tables to calculate things like:

The most common results of the style `quiz`.

The most common types of `purchase` made.

And more!

What are some actionable insights for Warby Parker?

Stuck? Get a hint

Query Results	
Database Schema	
home_try_on	
name	type
user_id	TEXT
number_of_pairs	TEXT
address	TEXT

Rows: 750

purchase	
name	type
user_id	TEXT

product_id	INTEGER
style	TEXT
model_name	TEXT
color	TEXT
price	INTEGER

Rows: 495

survey

name	type
question	TEXT
user_id	TEXT
response	TEXT

Rows: 1986

quiz

name	type
user_id	TEXT
style	TEXT
fit	TEXT

shape	TEXT
color	TEXT

Rows: 1000

Data Scientist: Inference Specialist

Attribution Queries

<https://www.codecademy.com/paths/data-science-inf/tracks/dsinf-advanced-sql-for-data-science/modules/dsinf-marketing-attribution/projects/analyze-data-sql-attribution-queries>

CoolTShirts sells shirts of all kinds, as long as they are T-shaped and cool. Recently, CTS started a few marketing campaigns to increase website visits and purchases. Using touch attribution, they'd like to map their customers' journey: from initial visit to purchase. They can use that information to optimize their marketing campaigns. This project will guide you through some of that process.

At this point you should have taken the [lesson on touch attribution](#). This project will guide you through a number of queries on CoolTShirts' user data (see the [schema here](#)).

Check off the steps as you go. In the rare chance that your project gets stuck in a bad state — maybe you accidentally edited the data —, refresh the page.

If you get stuck during this project or would like to see an experienced developer work through it, click “**Get Unstuck**” to see a **project walkthrough video**.

Tasks

7/7 complete

[Mark the tasks as complete by checking them off](#)

Get familiar with CoolTShirts

1.

How many campaigns and sources does CoolTShirts use? Which source is used for each campaign?

Use three queries:

- one for the number of distinct campaigns,
- one for the number of distinct sources,
- one to find how they are related.

2.

What pages are on the CoolTShirts website?

Find the distinct values of the `page_name` column.

What is the user journey?

3.

How many first touches is each campaign responsible for?

You'll need to use a first-touch query. Group by campaign and count the number of first touches for each.

4.

How many last touches is each campaign responsible for?

Starting with the last-touch query from the lesson, group by campaign and count the number of last touches for each.

5.

How many visitors make a purchase?

Count the distinct users who visited the page named `4 - purchase`.

6.

How many last touches *on the purchase page* is each campaign responsible for?

This query will look similar to your last-touch query, but with an additional `WHERE` clause.

7.

CoolTShirts can re-invest in 5 campaigns. Given your findings in the project, which should they pick and why?

Calculating Churn Rates

<https://www.codecademy.com/paths/analyze-data-with-sql/tracks/analyze-data-sql-analyze-real-data/modules/analyze-data-sql-user-churn/projects/calc-churn-proj>

Four months into launching Codeflix, management asks you to look into subscription churn rates. It's early on in the business and people are excited to know how the company is doing.

The marketing department is particularly interested in how the churn compares between two segments of users. They provide you with a dataset containing subscription data for users who were acquired through two distinct channels.

The dataset provided to you contains one SQL table, `subscriptions`. Within the table, there are 4 columns:

`id` - the subscription id

`subscription_start` - the start date of the subscription

`subscription_end` - the end date of the subscription

`segment` - this identifies which segment the subscription owner belongs to

Codeflix requires a minimum subscription length of 31 days, so a user can never start and end their subscription in the same month.

If you get stuck during this project or would like to see an experienced developer work through it, click **"Get Unstuck"** to see a **project walkthrough video**.

Tasks

9/9 complete

[Mark the tasks as complete by checking them off](#)

Get familiar with the data

1.

Take a look at the first 100 rows of data in the `subscriptions` table. How many different segments do you see?

Hint

Use a `SELECT` statement and be sure to `LIMIT 100`.

2.

Determine the range of months of data provided. Which months will you be able to calculate churn for?

Hint

Use `MIN` and `MAX` to examine the range of `subscription_start`.

Calculate churn rate for each segment

3.

You'll be calculating the churn rate for both segments (87 and 30) over the first 3 months of 2017 (you can't calculate it for December, since there are no `subscription_end` values yet). To get started, create a temporary table of `months`.

Hint

```
WITH months AS
(
  SELECT
    '2017-01-01' as first_day,
    '2017-01-31' as last_day
  UNION
  ...
)
```

Copy to Clipboard

4.

Create a temporary table, `cross_join`, from `subscriptions` and your `months`. Be sure to `SELECT` every column.

Hint

The syntax to join `table1` and `table2` is:

```
desired_temp_table AS
(
  SELECT table1.*, table2.*
  FROM table1
  CROSS JOIN table2
)
```

Copy to Clipboard

5.

Create a temporary table, `status`, from the `cross_join` table you created. This table should contain:

`id` selected from `cross_join`

`month` as an alias of `first_day`

`is_active_87` created using a `CASE WHEN` to find any users from segment 87 who existed prior to the beginning of the month. This is `1` if true and `0` otherwise.

`is_active_30` created using a `CASE WHEN` to find any users from segment 30 who existed prior to the beginning of the month. This is `1` if true and `0` otherwise.

Stuck? Get a hint

6.

Add an `is_canceled_87` and an `is_canceled_30` column to the `status` temporary table.

This should be `1` if the subscription is canceled during the month and `0` otherwise.

Stuck? Get a hint

7.

Create a `status_aggregate` temporary table that is a `SUM` of the active and canceled subscriptions for each segment, for each month.

The resulting columns should be:

`sum_active_87`

`sum_active_30`

`sum_canceled_87`

`sum_canceled_30`

Stuck? Get a hint

8.

Calculate the churn rates for the two segments over the three month period. Which segment has a lower churn rate?

Stuck? Get a hint

Bonus

9.

How would you modify this code to support a large number of segments?

Stuck? Get a hint

Code Review

<https://www.codecademy.com/paths/analyze-data-with-sql/tracks/analyze-data-sql-analyze-real-data/modules/analyze-data-sql-marketing-attribution/projects/analyze-data-sql-attribution-queries>

Analyze Data with SQL

Climate Change

<https://www.codecademy.com/paths/analyze-data-with-sql/tracks/advanced-sql/modules/sql-window-functions/projects/sql-window-functions-project>

To practice what you've learned about window functions, you are going to use climate data from each state in the United States. A labeled map of the United States can be found [here](#).

These data will show the average annual temperature for each state – this is the *average* temperature of every day in all parts of the state for that year.

For this project, you will be working with one table:

```
state_climate
```

Tasks

9/9 complete

Mark the tasks as complete by checking them off

Understanding the Data

1. Let's see what our table contains by running the following command:

```
SELECT *  
FROM state_climate;
```

Copy to Clipboard

Check the hint if you are unsure what each column represents.

Stuck? Get a hint

Aggregate and Value Functions

2.

Let's start by looking at how the average temperature changes over time in each state.

Write a query that returns the `state`, `year`, `tempf` or `tempc`, and `running_avg_temp` (in either Celsius or Fahrenheit) for each state.

(The `running_avg_temp` should use a window function.)

Hint

For this you need to use the aggregate function `AVG` for temperature. You will also need `PARTITION BY` for `state` and `ORDER BY` for `year` within your window function.

3.

Now let's explore the *lowest* temperatures for each state.

Write a query that returns `state`, `year`, `tempf` or `tempc`, and the lowest temperature (`lowest_temp`) for each state.

Are the lowest recorded temps for each state more recent or more historic?

Hint

To find the lowest temperature for each state, you need to use the `FIRST_VALUE` function with the temperature (`tempf` or `tempc` as the parameter). Then you need to `PARTITION BY` the `state` and `ORDER BY` the temperature.

4.

Like before, write a query that returns `state`, `year`, `tempf` or `tempc`, except now we will also return the highest temperature (`highest_temp`) for each state.

Are the highest recorded temps for each state more recent or more historic?

Hint

We need to use `LAST_VALUE` in order to find the highest recorded temperature for each state but unlike `FIRST_VALUE`, we also need to specify a *frame* for our query: `RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`

5.

Let's see how temperature has changed each year in each state.

Write a query to select the same columns but now you should write a window function that returns the `change_in_temp` from the *previous year* (no `null` values should be returned).

Which states and years saw the largest changes in temperature?

Is there a particular part of the United States that saw the largest yearly changes in temperature?

Hint

This one is a little tricky! In order to answer the questions you need to:

find the difference in temperature from the previous year using subtraction and

`LAG`

you need all three parameters that `LAG` accepts in order to look to the previous

row and never return a `null`

`PARTITION BY state and ORDER BY year`

lastly your *entire results set* needs to use `ORDER BY change_in_temp` (use `ASC` or

`DESC`!)

Ranking Functions

6.

Write a query to return a rank of the coldest temperatures on record (`coldest_rank`)

along with `year`, `state`, and `tempf` or `tempc`. Are the coldest ranked years recent or

historic? The coldest years should be from *any state or year*.

Hint

This will require you to use the `RANK` window function with `ORDER BY` on `tempf` or `tempc`. Because we want to see the overall coldest for any state or year, we do not need to partition the data.

7.

Modify your `coldest_rank` query to now instead return the `warmest_rank` *for each state*,

meaning your query should return the warmest temp/year for each state. Again, are the

warmest temperatures more recent or historic for each state?

Hint

Since we now want to return the *warmest* year we should add `DESC` to our `ORDER BY` as this will return the highest temperatures first. We will also need to add a `PARTITION BY` for each `state` so we can see the highest temperatures for each state.

8.

Let's now write a query that will return the average yearly temperatures in *quartiles*

instead of in rankings for each state.

Your query should return `quartile`, `year`, `state` and `tempf` or `tempc`. The top `quartile`

should be the coldest years.

Are the coldest years more recent or historic?

Stuck? Get a hint

9.

Lastly, we will write a query that will return the average yearly temperatures in *quintiles* (5).

Your query should return `quintile`, `year`, `state` and `tempf` or `tempc`. The top `quintile` should be the coldest years **overall**, not by state.

What is different about the coldest quintile now?

Hint

For this, we need to use the `NTILE()` function for quintiles (5). Because we are looking at *all* the data, we do not need a `PARTITION BY` for this query.

Analyze Data with SQL

Analyze Twitch Gaming Data

<https://www.codecademy.com/paths/data-science-inf/tracks/dsinf-advanced-sql-for-data-science/modules/dsinf-gaming-trends/projects/analyze-data-with-sql-twitch>

<https://www.codecademy.com/paths/analyze-data-with-sql/tracks/sql-interview-prep/modules/sql-interview-prep/projects/analyze-data-with-sql-twitch>



[Twitch](#) is the world's leading live streaming platform for gamers, with 15 million daily active users. Using data to understand its users and products is one of the main responsibilities of the Twitch Data Science Team.

In this project, you will be working with two tables that contain Twitch users' stream viewing data and chat room usage data.

Stream viewing data:

`stream` table

Chat usage data:

`chat` table

The Twitch Science Team provided this practice dataset. You can download the `.csv` files (800,000 rows) from [GitHub](#).

Let's get started!

If you get stuck during this project or would like to see an experienced developer work through it, click **Get Unstuck** to see a walkthrough video.

Tasks

12/12 complete

[Mark the tasks as complete by checking them off](#)

Getting Started:

1.

Start by getting a feel for the `stream` table and the `chat` table.

Select the first 20 rows from each of the two tables.

What are the column names?

Stuck? Get a hint

2.

What are the *unique* `games` in the `stream` table?

Stuck? Get a hint

3.

What are the *unique* channels in the `stream` table?

Hint

The `DISTINCT` channels should be:

frank

george

estelle

morty

kramer

jerry

helen

newman

elaine

susan

Yada yada yada...

Aggregate Functions:

4.

What are the most popular games in the `stream` table?

Create a list of games and their number of viewers using `GROUP BY`.

Hint

You can `GROUP BY` the games:

```
SELECT game, COUNT(*)
FROM stream
GROUP BY game
ORDER BY COUNT(*) DESC;
```

Copy to Clipboard

You can rewrite this using column reference numbers instead of column names:

```
SELECT game, COUNT(*)
FROM stream
GROUP BY 1
ORDER BY 2 DESC;
```

Copy to Clipboard

5.

These are some big numbers from the game League of Legends (also known as LoL).

Where are these LoL stream viewers located?

Create a list of countries and their number of LoL viewers using `WHERE` and `GROUP BY`.

Hint

```
SELECT country, COUNT(*)
FROM stream
WHERE game = 'League of Legends'
GROUP BY 1
ORDER BY 2 DESC;
```

Copy to Clipboard

Oh wow, that is very international!

You can also add a `LIMIT 10` at the bottom to cap the results.

6.

The `player` column contains the source the user is using to view the stream (`site`, `iphone`, `android`, etc).

Create a list of `players` and their number of streamers.

Hint

```
SELECT player, COUNT(*)
FROM stream
GROUP BY 1
ORDER BY 2 DESC;
```

Copy to Clipboard

7.

Create a new column named `genre` for each of the games.

Group the games into their genres: Multiplayer Online Battle Arena (MOBA), First Person Shooter (FPS), Survival, and Other.

Using `CASE`, your logic should be:

If `League of Legends` → MOBA

If `Dota 2` → MOBA

If `Heroes of the Storm` → MOBA

If `Counter-Strike: Global Offensive` → FPS

If `DayZ` → Survival

If `ARK: Survival Evolved` → Survival

Else → Other

Use `GROUP BY` and `ORDER BY` to showcase only the unique game titles.

Hint

A `CASE` statement allows us to create different outputs. It is SQL's way of handling [if-then](#) logic.

In this question, we are trying to figure out the different genres of games, and each of its viewer counts.

Before we do that, however, we will select the `game` column and create a new column called `genre` that has these genres: MOBA, FPS, Survival, and Other.

After that, we will make a third column that is `COUNT(*)` and we are going to `GROUP BY` the `game` column and sort the result by `COUNT(*)` in a descending order:

```
SELECT game,
CASE
  WHEN game = 'Dota 2'
    THEN 'MOBA'
  WHEN game = 'League of Legends'
    THEN 'MOBA'
  WHEN game = 'Heroes of the Storm'
    THEN 'MOBA'
  WHEN game = 'Counter-Strike: Global Offensive'
    THEN 'FPS'
  WHEN game = 'DayZ'
    THEN 'Survival'
  WHEN game = 'ARK: Survival Evolved'
    THEN 'Survival'
  ELSE 'Other'
END AS 'genre',
COUNT(*)
FROM stream
GROUP BY 1
ORDER BY 3 DESC;
```

Copy to Clipboard

How does view count change in the course of a day?

8.

Before we get started, let's run this query and take a look at the `time` column from the `stream` table:

```
SELECT time
FROM stream
LIMIT 10;
```

Copy to Clipboard

The data type of the `time` column is `DATETIME`. It is for storing a date/time value in the database.

Notice that the values are formatted like:

```
2015-01-01 18:33:52
```

So the format is:

```
YYYY-MM-DD HH:MM:SS
```

Hint

The `HH` is the hour of the day, from 0 to 23.

The `MM` is the minute of the hour, from 0 to 59.

The `SS` is the seconds within a minute, from 0 to 59.

9.

SQLite comes with a `strftime()` function - a very powerful function that allows you to return a formatted date.

It takes two arguments:

```
strftime(format, column)
```

Let's test this function out:

```
SELECT time,
       strftime('%S', time)
FROM stream
GROUP BY 1
LIMIT 20;
```

Copy to Clipboard

What do you think this does? Open the hint if you'd like to learn more.

Hint

This returns the seconds, `SS`, of the timestamp column!

For `strftime(__, timestamp)`:

`%Y` returns the year (YYYY)

`%m` returns the month (01-12)

`%d` returns the day of the month (1-31)

`%H` returns 24-hour clock (00-23)

`%M` returns the minute (00-59)

`%S` returns the seconds (00-59)

if `time` format is `YYYY-MM-DD HH:MM:SS`.

Read more on the [SQLite documentation](#).

10.

Okay, now we understand how `strftime()` works. Let's write a query that returns two columns:

The hours of the `time` column

The view count for each hour

Lastly, filter the result with only the users in *your country* using a `WHERE` clause.

Hint

```
SELECT strftime('%H', time),  
       COUNT(*)  
FROM stream  
GROUP BY 1;
```

Copy to Clipboard

Because this data is international and there are multiple time differences, let's filter the result to a specific country:

```
SELECT strftime('%H', time),  
       COUNT(*)  
FROM stream  
WHERE country = 'US'  
GROUP BY 1;
```

Copy to Clipboard

Joining the two tables:

11.

The `stream` table and the `chat` table share a column: `device_id`.

Let's join the two tables on that column.

Hint

```
SELECT *  
FROM stream
```



```
JOIN chat
  ON stream.device_id = chat.device_id;
```

Copy to Clipboard

12.

Woohoo! You have completed the the guided practice, see what else you can dig up. For example:

What are your favorite games? Can you find some insights about its viewers and chat room users?

Is there anything you can do after joining the two tables?

The dataset **.csv** files are on [GitHub](#).

Hint

P.S. Shout out to June Dershewitz ([@jdersh](#)), previous Head of Data Governance and Analytics, for working with us on this project!

Healthcare in Different States - Python

In this project, we will use boxplots to investigate the way hospitals in different states across the United States charge their patients for medical procedures.

This dataset might look a bit familiar to you — we used it in our lesson about describing histograms. The data originally came from the [United State Health and Human Services Department](#).

Let's use boxplots to find more meaning in this data!

Tasks

12/12 complete

[Mark the tasks as complete by checking them off](#)

Investigate the Data

1.

We've imported the dataset into a variable named `healthcare`. Let's take a look at what data we have to work with. Print `healthcare.head()`. This will print the first five rows of the dataset.

Scroll through the table to see what information we have. We know that we want to eventually look at the way healthcare works in different states. What column will be useful to do that?

Note that we're going to ask you to print out multiple values over the course of the project. Feel free to comment out old print statements using `#` to avoid clutter:

```
# This is a comment
```

[Copy to Clipboard](#)

Stuck? Get a hint

2.

We're going to focus on only the ways in which hospitals charge patients for chest pain. But you could do this for any of the diagnoses in our dataset. Print `healthcare["DRG Definition"].unique()` to see all of the different diagnoses in our dataset.

Can you find the official name of the diagnosis related to chest pain?

Stuck? Get a hint

3.

Let's grab only the rows in the dataset that are about chest pain. Use this line of code to do that.

```
chest_pain = healthcare[healthcare['DRG Definition'] == '313 - CHEST PAIN']
```

[Copy to Clipboard](#)

If you're interested in investigating a different diagnosis, replace `'313 - CHEST PAIN'` with the name of the other diagnosis. You might want to change the new variable name to be something other than `chest_pain` if you do this!

Separating By State

4.

We now want to separate the dataset by state. Eventually, we'll use a `for` loop to do this for every state, but for now, let's prove to ourselves that we can make a boxplot for one state.

When we printed the head, we saw the first few rows had a `'Provider State'` of `"AL"`. Those were hospitals in Alabama.

To get every chest pain diagnosis in Alabama, use this line of code:

```
alabama_chest_pain = chest_pain[chest_pain['Provider State'] == "AL"]
```

Copy to Clipboard

Make sure to use the variable that you created in the previous step. We called it `chest_pain`, but you might have named it something else if you used a different diagnosis.

Again, feel free to use a state of your choosing instead of Alabama. Different state abbreviations should work, like `"CO"` or `"NY"`. Name your variable appropriately!

5.

We're almost there! We now have all of the hospitals in Alabama that have a diagnosed chest pain.

We now want to find the average cost of those diagnoses. These value is stored in the column `'Average Covered Charges '` (Note the spaces at the start and the end of the string!)

To get only these values, call this line of code:

```
costs = alabama_chest_pain[' Average Covered Charges '].values
```

Copy to Clipboard

Again, make sure that you're using the correct variable names — yours might be different.

6.

Let's now make a boxplot of those values! Call `plt.boxplot()` using `costs` as the first parameter.

Then call `plt.show()` to see your boxplot!

Stuck? Get a hint

Making a Boxplot for All States

7.

Nice work! We've made a boxplot for one state. But how does that state compare to the others?

Let's make a boxplot for every state! Comment out (or delete) the two lines of code you just wrote to make the boxplot.

To begin, we first need to create a list of all the states in our dataset. We can once again use the `unique()` function. We used this function when we looked at all of the diagnoses.

Find all of the unique states from the dataset `chest_pain` and store it in a variable named `states`.

Stuck? Get a hint

8.

We'll now use a `for` loop to separate the dataset into a dataset for each state:

```
datasets = []
for state in states:
    datasets.append(chest_pain[chest_pain['Provider State'] == state]['Average Covered Charges'].values)
```

Copy to Clipboard

`datasets` now contains 50 datasets — one for each state.

9.

We're about to draw 50 boxplots. Before we draw them, let's make sure there's enough room. Call

`plt.figure(figsize=(20,6))`. This will make your figure long to allow room for so many boxplots!

10.

Draw the boxplot using `datasets` as the first parameter. Add the second parameter `labels = states` to label your boxplots.

Finally, make sure to call `plt.show()`.

Stuck? Get a hint

Interpret and Explore

11.

Nice work! You should now see 50 boxplots. If the graph is too small to see, you can expand it by clicking on the arrows in the top right corner of the right panel. You can also take a look at [this image](#) to see what your final results should look like.

What information have you learned by looking at these boxplots side by side? What state has the largest spread? What state has the largest median? Which states have the most outliers?

Check the hint to see our thoughts.

Stuck? Get a hint

12.

Take some time to explore more from here. Here are some ways in which you can investigate the data more:

Look at diagnoses other than `'313 - CHEST PAIN'`.

Group states by regions. Maybe hospitals in the Northeast charge patients differently than hospitals in the South.

Plot something other than `'Average Covered Charges'`. You have data about how much

Medicare pays in the `'Average Medicare Payments'` column.

Summarizing Automobile Evaluation Data

In the following project you'll use what you've learned about summarizing categorical data to analyze a sample from a popular open source dataset. This dataset contains information on the cost and physical attributes of several thousand cars. Originally, this dataset was used for to train a classification model that assigned an acceptability score/category to cars based on these attributes.

The car evaluation dataset has been sourced from the UCI Machine Learning Repository and has been slightly modified for this project. Specifically, one additional field `manufacturer_country` has been simulated for illustrative purposes. You can read more about the details, features, and original uses of this dataset in research on the [UCI data description page](#).

Setup Instructions

You have two options of completing this assignment. Either here, within Codecademy's output terminal, or on your own, in case you're more comfortable using a Jupyter notebook. If you choose to do this project on your computer instead of Codecademy, you can download what you'll need by clicking the "Download" button below. If you need help setting up your computer, be sure to check out our setup guides:

[Command Line Interface Setup](#)

[Introducing Jupyter Notebook](#)

[Setting up Jupyter Notebook](#)

[Getting Started with Jupyter](#)

[Getting More out of Jupyter Notebook](#)

Open **Summarizing Automobile Evaluation Data.ipynb** and follow the steps in the Jupyter Notebook. If you get stuck, you can look at **Summarizing Automobile Evaluation Data_Solution.ipynb** for the answer.

Download

Tasks

11/11 complete

[Mark the tasks as complete by checking them off](#)

Summarizing Manufacturing Country

1.

`manufacturer_country` is a *nominal categorical variable* that indicates the country of the manufacturer of each car reviewed. Create a table of frequencies of all the cars reviewed by `manufacturer_country`. What is the modal category? Which country appears 4th most frequently? Print out your results.

Hint

You can call `.value_counts()` on `manufacturer_country` to get a summary of counts by country. You can then use `.index[N-1]` to find the N-th most frequent observation in the data.

2.

Calculate a table of proportions for countries that appear in `manufacturer_country` in the dataset.
What percentage of cars were manufactured in Japan?

Hint

To calculate a table of proportions you can still use pandas `.value_counts()` function. However, you should include `normalize = True` to normalize the frequencies/counts to a percentage.

Summarizing Buying Costs

3.

`buying_cost` is a categorical variable which describes the cost of buying any car in the dataset. Print out a list of the possible values for this variable.

Hint

You can find the unique values in each field using `car_data["column_name"].unique()`.

4.

`buying_cost` is an *ordinal categorical variable*, which means we can create an order associated with the values in the data and perform additional numeric operations on the variable. Create a new list, `buying_cost_categories`, that contains the unique values in `buying_cost`, ordered from lowest to highest.

Hint

The categories (from lowest to highest price) should be as follows. Create a list with the categories in this order.

```
"low"  
"med"  
"high"  
"v-high"
```

5.

Convert `buying_cost` to type 'category' using the list you created in the previous exercise.

Hint

You can convert a field to type category using the function `pandas.Categorical()`. Adapt the code below to change the type of `buying_cost` to category.

```
car_data["column_name"] = pd.Categorical(  
    car_data["column_name"],  
    buying_cost_categories,  
    ordered=True  
)
```

Copy to Clipboard

6.

Calculate the median category of the `buying_cost` variable and print the result.

Hint

In Python, you can use `np.median()` to calculate the median value of a numerical series. In this case, you also must access the numerical values of the categories. This can be done with the `.cat.codes` attribute. Complete the code below to calculate the median category for `buying_cost`.

```
median_category_num = np.median(car_data['column_name'].cat.codes)
```

```
print(median_category_num)

median_category = ordered_list_of_categories[int(median_category_num)]
print(median_category)
```

Copy to Clipboard

Summarizing Luggage Capacity

7.

`luggage` is a categorical variable in the car evaluations dataset that records the luggage capacity for each reviewed car. Calculate a table of proportions for this variable and print the result.

Stuck? Get a hint

8.

Are there any missing values in this column? Replicate the table of proportions from the previous exercise, but do not drop any missing values from the count. Print your result.

Hint

Using `.value_counts()`, missing values are removed by default. To keep missing values in the summary you can pass `dropna = False` to `.value_counts()`. If the result using `dropna = False` is the same, then you can conclude there are no missing values.

```
car_data["column_name"].value_counts(____ = ____, normalize=True)
```

Copy to Clipboard

9.

Without passing `normalize = True` to `.value_counts()`, can you replicate the result you got in the previous exercises?

Hint

One way to replicate the behavior of `.value_counts(normalize=True)` is to divide the table of frequencies by the total count of observations in the dataset. Adapt the code below to check your answer.

This method returns the same result as `.value_counts(normalize=True)` only when there are no NaN values in the target column. Fortunately, we confirmed there is no missing data in the previous exercise.

```
car_data["column_name"].value_counts() / len(car_data["column_name"])
```

Copy to Clipboard

Summarizing Passenger Capacity

10.

`doors` is a categorical variable in the car evaluations dataset that records the count of doors for each reviewed car. Find the count of cars that have 5 or more doors. You can identify cars with 5+ doors by looking for cars that have a value of `'5more'` in the `doors` column. Print your result.

Hint

You can identify cars with 5+ doors using the label '5more'. You can then create a variable that identifies cars with that value by modifying the code below:

```
frequency = (car_data["column_name"] == '____').sum()
```

Copy to Clipboard

11.

Find the proportion of cars that have 5+ doors and print the result.

Hint

This is very similar to the previous exercise, instead of the `.sum()` function, you can use the `.mean()` function to calculate the proportion.

```
proportion = (car_data["column_name"] == '____').mean()
```

Healthcare in Different States

In this project, we will use boxplots to investigate the way hospitals in different states across the United States charge their patients for medical procedures.

This dataset might look a bit familiar to you — we used it in our lesson about describing histograms. The data originally came from the [United State Health and Human Services Department](#).

Let's use boxplots to find more meaning in this data!

Tasks

2/12 complete

[Mark the tasks as complete by checking them off](#)

Investigate the Data

1.

We've imported the dataset into a variable named `healthcare`. Let's take a look at what data we have to work with. Print `healthcare.head()`. This will print the first five rows of the dataset.

Scroll through the table to see what information we have. We know that we want to eventually look at the way healthcare works in different states. What column will be useful to do that?

Note that we're going to ask you to print out multiple values over the course of the project. Feel free to comment out old print statements using `#` to avoid clutter:

```
# This is a comment
```


Copy to Clipboard

Stuck? Get a hint

2.

We're going to focus on only the ways in which hospitals charge patients for chest pain.

But you could do this for any of the diagnoses in our dataset. Print `healthcare["DRG Definition"].unique()` to see all of the different diagnoses in our dataset.

Can you find the official name of the diagnosis related to chest pain?

Stuck? Get a hint

3.

Let's grab only the rows in the dataset that are about chest pain. Use this line of code to do that.

```
chest_pain = healthcare[healthcare['DRG Definition'] == '313 - CHEST PAIN']
```

Copy to Clipboard

If you're interested in investigating a different diagnosis, replace `'313 - CHEST PAIN'` with the name of the other diagnosis. You might want to change the new variable name to be something other than `chest_pain` if you do this!

Separating By State

4.

We now want to separate the dataset by state. Eventually, we'll use a `for` loop to do this for every state, but for now, let's prove to ourselves that we can make a boxplot for one state.

When we printed the head, we saw the first few rows had a `'Provider State'` of `"AL"`. Those were hospitals in Alabama.

To get every chest pain diagnosis in Alabama, use this line of code:

```
alabama_chest_pain = chest_pain[chest_pain['Provider State'] == "AL"]
```

Copy to Clipboard

Make sure to use the variable that you created in the previous step. We called it `chest_pain`, but you might have named it something else if you used a different diagnosis.

Again, feel free to use a state of your choosing instead of Alabama. Different state abbreviations should work, like `"CO"` or `"NY"`. Name your variable appropriately!

5.

We're almost there! We now have all of the hospitals in Alabama that have a diagnosed chest pain. We now want to find the average cost of those diagnoses. This value is stored in the column `' Average Covered Charges '` (Note the spaces at the start and the end of the string!)

To get only these values, call this line of code:

```
costs = alabama_chest_pain[' Average Covered Charges '].values
```

Copy to Clipboard

Again, make sure that you're using the correct variable names — yours might be different.

6.

Let's now make a boxplot of those values! Call `plt.boxplot()` using `costs` as the first parameter.

Then call `plt.show()` to see your boxplot!

Stuck? Get a hint

Making a Boxplot for All States

7.

Nice work! We've made a boxplot for one state. But how does that state compare to the others? Let's make a boxplot for every state! Comment out (or delete) the two lines of code you just wrote to make the boxplot.

To begin, we first need to create a list of all the states in our dataset. We can once again use the `unique()` function. We used this function when we looked at all of the diagnoses. Find all of the unique states from the dataset `chest_pain` and store it in a variable named `states`.

Stuck? Get a hint

8.

We'll now use a `for` loop to separate the dataset into a dataset for each state:

```
datasets = []
for state in states:
    datasets.append(chest_pain[chest_pain['Provider State'] ==
state][' Average Covered Charges '].values)
```

Copy to Clipboard

`datasets` now contains 50 datasets — one for each state.

9.

We're about to draw 50 boxplots. Before we draw them, let's make sure there's enough room. Call `plt.figure(figsize=(20,6))`. This will make your figure long to allow room for so many boxplots!

10.

Draw the boxplot using `datasets` as the first parameter. Add the second parameter `labels = states` to label your boxplots.

Finally, make sure to call `plt.show()`.

Stuck? Get a hint

Interpret and Explore

11.

Nice work! You should now see 50 boxplots. If the graph is too small to see, you can expand it by clicking on the arrows in the top right corner of the right panel. You can also take a look at [this image](#) to see what your final results should look like.

What information have you learned by looking at these boxplots side by side? What state has the largest spread? What state has the largest median? Which states have the most outliers?

Check the hint to see our thoughts.

Stuck? Get a hint

12.

Take some time to explore more from here. Here are some ways in which you can investigate the data more:

Look at diagnoses other than `'313 - CHEST PAIN'`.

Group states by regions. Maybe hospitals in the Northeast charge patients differently than hospitals in the South.

Plot something other than `'Average Covered Charges'`. You have data about how much Medicare pays in the `'Average Medicare Payments'` column.

Summarizing Automobile Evaluation Data

In the following project you'll use what you've learned about summarizing categorical data to analyze a sample from a popular open source dataset. This dataset contains information on the cost and physical attributes of several thousand cars. Originally, this dataset was used for to train a classification model that assigned an acceptability score/category to cars based on these attributes.

The car evaluation dataset has been sourced from the UCI Machine Learning Repository and has been slightly modified for this project. Specifically, one additional field `manufacturer_country` has been simulated for illustrative purposes. You can read more about the details, features, and original uses of this dataset in research on the [UCI data description page](#).

Setup Instructions

You have two options of completing this assignment. Either here, within Codecademy's output terminal, or on your own, in case you're more comfortable using a Jupyter notebook. If you choose to do this project on your computer instead of Codecademy, you can download what you'll need by clicking the "Download" button below. If you need help setting up your computer, be sure to check out our setup guides:

[Command Line Interface Setup](#)

[Introducing Jupyter Notebook](#)

[Setting up Jupyter Notebook](#)

[Getting Started with Jupyter](#)

[Getting More out of Jupyter Notebook](#)

Open **Summarizing Automobile Evaluation Data.ipynb** and follow the steps in the Jupyter Notebook. If you get stuck, you can look at **Summarizing Automobile Evaluation Data_Solution.ipynb** for the answer.

Download

Tasks

11/11 complete

[Mark the tasks as complete by checking them off](#)

Summarizing Manufacturing Country

1.

`manufacturer_country` is a *nominal categorical variable* that indicates the country of the manufacturer of each car reviewed. Create a table of frequencies of all the cars reviewed by `manufacturer_country`. What is the modal category? Which country appears 4th most frequently? Print out your results.

Stuck? Get a hint

2.

Calculate a table of proportions for countries that appear in `manufacturer_country` in the dataset. What percentage of cars were manufactured in Japan?

Stuck? Get a hint

Summarizing Buying Costs

3.

`buying_cost` is a categorical variable which describes the cost of buying any car in the dataset. Print out a list of the possible values for this variable.

Stuck? Get a hint

4.

`buying_cost` is an *ordinal categorical variable*, which means we can create an order associated with the values in the data and perform additional numeric operations on the variable. Create a new list, `buying_cost_categories`, that contains the unique values in `buying_cost`, ordered from lowest to highest.

Stuck? Get a hint

5.

Convert `buying_cost` to type `'category'` using the list you created in the previous exercise.

Stuck? Get a hint

6.

Calculate the median category of the `buying_cost` variable and print the result.

Stuck? Get a hint

Summarizing Luggage Capacity

7.

`luggage` is a categorical variable in the car evaluations dataset that records the luggage capacity for each reviewed car. Calculate a table of proportions for this variable and print the result.

Stuck? Get a hint

8.

Are there any missing values in this column? Replicate the table of proportions from the previous exercise, but do not drop any missing values from the count. Print your result.

Stuck? Get a hint

9.

Without passing `normalize = True` to `.value_counts()`, can you replicate the result you got in the previous exercises?

Stuck? Get a hint

Summarizing Passenger Capacity

10.

`doors` is a categorical variable in the car evaluations dataset that records the count of doors for each reviewed car. Find the count of cars that have 5 or more doors. You can identify cars with 5+ doors by looking for cars that have a value of `'5more'` in the `doors` column. Print your result.

Hint

You can identify cars with 5+ doors using the label `'5more'`. You can then create a variable that identifies cars with that value by modifying the code below:

```
frequency = (car_data["column_name"] == '____').sum()
```

Copy to Clipboard

11.

Find the proportion of cars that have 5+ doors and print the result.

Hint

This is very similar to the previous exercise, instead of the `.sum()` function, you can use the `.mean()` function to calculate the proportion.

```
proportion = (car_data["column_name"] == '____').mean()
```

Copy to Clipboard

In this project, you'll work as a data analyst for a tech startup that is looking to improve its operations after a global pandemic has taken the world by storm.

You will apply data transformation techniques to make better sense of the company's data and help answer important questions such as:

Is the company in good financial health?

Does the company need to let go of any employees?

Should the company allow employees to work from home permanently?

Let's get started!

Tasks

1/21 complete

Mark the tasks as complete by checking them off

Analyzing Revenue and Expenses

1.

The management team of the company you work for is concerned about the status of the company after a global pandemic.

The CFO (Chief Financial Officer) asks you to perform some data analysis on the past six months of the company's financial data, which has been loaded in the variable `financial_data`.

First, examine the first few rows of the data using `print()` and `.head()`.

Hint

Use the following code to examine the first few rows of `financial_data`:

```
print(financial_data.head())
```

Copy to Clipboard

2.

Notice that `financial_data` has three columns – `Month`, `Revenue`, and `Expenses`.

Store each column in three separate variables called `month`, `revenue`, and `expenses`.

Hint

Use the following code to store the `Month` column, then use similar syntax for the `Revenue`, and `Expenses` columns:

```
month = financial_data['Month']
```

Copy to Clipboard

3.

Next, use the following code to create a plot of revenue over the past six months:

```
plt.plot(month, revenue)
plt.show()
```

Copy to Clipboard

4.

On the right, you should now see a plot of revenue over time. You can label and format the figure using the following functions:

```
plt.xlabel('Month')
plt.ylabel('Amount ($)')
plt.title('Revenue')
```

Copy to Clipboard

These should be added before `plt.show()`. Add the labels to your plot.

5.

Repeat steps 3 and 4 for monthly expenses. In other words, create a second plot of monthly expenses over the past 6 months. Note that you'll need to use the function `plt.clf()` prior to creating this new plot. Otherwise, it will be plotted on-top of the revenue plot. The code should look something like this:

```
plt.clf()
#insert code to create plot here
#add labels to the plot here
plt.show()
```

Copy to Clipboard

How are monthly expenses changing over time?

Hint

Use the following code:

```
plt.clf()
plt.plot(month, expenses)
plt.xlabel('Month')
plt.ylabel('Amount ($)')
plt.title('Expenses')
plt.show()
```

Copy to Clipboard

6.

As shown, revenue seems to be quickly decreasing while expenses are increasing. If the current trend continues, expenses will soon surpass revenues, putting the company at risk.

After you show this chart to the management team, they are alarmed. They conclude that expenses must be cut immediately and give you a new file to analyze called `expenses.csv`.

Use `pandas` to read in `expenses.csv` and store it in a variable called `expense_overview`.

Print the first seven rows of the data.

Hint

Use the following code to read in `expenses.csv` and to print the first few rows of the data:

```
expense_overview = pd.read_csv('expenses.csv')
print(expense_overview.head(7))
```

Copy to Clipboard

Pie Chart and Collapsing Categories

7.

Notice that there are two columns:

`Expense`: indicates the expense category

`Proportion`: indicates how much of the overall expenses a specific category takes up

Store the `Expense` column in a variable called `expense_categories` and the `Proportion` column in a variable called `proportions`.

Hint

Use the following code to store the expense categories and proportions:

```
expense_categories = expense_overview['Expense']
proportions = expense_overview['Proportion']
```

Copy to Clipboard

8.

Next, we want to create a pie chart of the different expense categories. Use `plt.clf()` again to clear the previous plot, then create a pie chart using the `plt.pie()` method, passing in two arguments:

```
proportions
labels = expense_categories
```

Give your pie chart a title using `plt.title()`, then use `plt.show()` at the end to show the plot.

Hint

Use the following code to create a pie chart for the expense categories:

```
plt.clf()
plt.pie(proportions, labels = expense_categories)
plt.title('Expense Categories')
plt.show()
```

Copy to Clipboard

9.

Notice that the pie chart currently looks deformed.

Above `plt.show()`, add in the following two lines of code to set the axis and adjust the spacing:

```
plt.axis('Equal')
plt.tight_layout()
```

Copy to Clipboard

Take a moment to look at the pie chart. Which expense categories make up most of the data, and which ones aren't so significant?

Hint

Right above `plt.show()`, add the following two lines of code to format the pie chart:


```
plt.axis('Equal')
plt.tight_layout()
```

Copy to Clipboard

10.

It seems that `Salaries`, `Advertising`, and `Office Rent` make up most of the expenses, while the rest of the categories make up a small percentage.

Before you hand this pie chart back to management, you would like to update the pie chart so that all categories making up less than 5% of the overall expenses (`Equipment`, `Utilities`, `Supplies`, and `Food`) are collapsed into an “Other” category.

Update the pie chart accordingly.

Hint

Replace the code for the current pie chart with the following:

```
expense_categories = ['Salaries', 'Advertising', 'Office Rent',
                     'Other']
proportions = [0.62, 0.15, 0.15, 0.08]
plt.clf()
plt.pie(proportions, labels = expense_categories)
plt.title('Expense Categories')
plt.axis('Equal')
plt.tight_layout()
plt.show()
```

Copy to Clipboard

11.

You should now see four categories in your updated pie chart:

```
Salaries
Advertising
Office Rent
Other
```

This simplified pie chart helps the management team see a big picture view of the company's expenses without getting distracted by noisy data.

If the company wants to cut costs in a big way, which category do you think they should focus on?

Put your answer in a string variable called `expense_cut`.

Hint

Salaries make up a majority of the expenses, so this is the category that the company should focus on in order to cut costs in a meaningful way.

```
expense_cut = 'Salaries'
```

Copy to Clipboard

Employee Productivity

12.

Salaries make up 62% of expenses. The management team determines that to cut costs in a meaningful way, they must let go of some employees.

Each employee at the company is assigned a productivity score based on their work. The management would like to keep the most highly productive employees and let go of the least productive employees.

First, use `pandas` to load in `employees.csv` and store it in a variable called `employees`.

Print the first few rows of the data.

Hint

Use the following code to read in `employees.csv` and to print the first few rows of the data:

```
employees = pd.read_csv('employees.csv')
print(employees.head())
```

Copy to Clipboard

13.

Notice that there is a `Productivity` column, which indicates the productivity score assigned to that employee.

Sort the `employees` data frame (in ascending order) by the `Productivity` column and store the result in a variable called `sorted_productivity`.

To sort a data frame, you can do the following:

```
sorted_data = dataframe_name.sort_values(by=['Column Name'])
```

Copy to Clipboard

Print `sorted_productivity`.

Hint

Use the following code to sort `employees` by `Productivity` and print the result:

```
sorted_productivity = employees.sort_values(by=['Productivity'])
print(sorted_productivity)
```

Copy to Clipboard

14.

You should now see the employees with the lowest productivity scores at the top of the data frame.

The company decides to let go of the 100 least productive employees.

Store the first 100 rows of `sorted_productivity` in a new variable called `employees_cut` and print out the result.

Unfortunately, this batch of employees won't be so lucky.

Hint

Use the following code to store the first 100 rows of `sorted_productivity` and print out the result:

```
employees_cut = sorted_productivity.head(100)
print(employees_cut)
```

Copy to Clipboard

15.

Your colleague Sarah, a data scientist at the company, would like to explore the relationship between `Income` and `Productivity` more in depth, but she points out that these two features are on vastly different scales.

For example, productivity is a feature that ranges from 0-100, but income is measured in the thousands of dollars.

Moreover, there are outliers in the data that add an additional layer of complexity.

She asks you for advice on how she should transform the data. Should she perform normalization, standardization, log transformation, or something else?

Put your answer in a string in a variable called `transformation`.

Hint

Your code should look like:

```
transformation = 'standardization'
```

Copy to Clipboard

Since there are outliers in the data, Sarah should use standardization in this situation. Unlike normalization, standardization does not have a bounding range.

Once the standardization is complete, all the features will have a mean of zero, a standard deviation of one, and therefore, the same scale.

Commute Times and Log Transformation

16.

The COO (Chief Operating Officer) is debating whether to allow employees to continue to work from home post-pandemic.

He first wants to take a look at roughly how long the average commute time is for employees at the company. He asks for your help to analyze this data.

The `employees` data frame has a column called `Commute Time` that stores the commute time (in minutes) for each employee.

Create a variable called `commute_times` that stores the `Commute Time` column.

Hint

Use the following code to store the `Commute Time` column in a variable called `commute_times`:

```
commute_times = employees['Commute Time']
```

Copy to Clipboard

17.

Let's do some quick analysis on the commute times of employees.

Use `print()` and `.describe()` to print out descriptive statistics for `commute_times`.

What are the average and median commute times? Might it be worth it for the company to explore allowing remote work indefinitely so employees can save time during the day?

Hint

Use the following code to print out descriptive statistics for `commute_times`:

```
print(commute_times.describe())
```

Copy to Clipboard

The average and median commute times are 33 and 31 minutes, respectively.

18.

Let's explore the shape of the commute time data using a histogram.

First, use `plt.clf()` to clear the previous plots. Then use `plt.hist()` to plot the histogram of `commute_times`. Finally, use `plt.show()` to show the plot. Feel free to add labels above

`plt.show()` if you would like to practice!

What do you notice about the shape of the data? Is it symmetric, left skewed, or right skewed?

Hint

Use the following lines of code:

```
plt.clf()
plt.hist(commute_times)
plt.title("Employee Commute Times")
plt.xlabel("Commute Time")
plt.ylabel("Frequency")
plt.show()
```

Copy to Clipboard

The commute time data appears to be right-skewed.

19.

The data seems to be skewed to the right. To make it more symmetrical, we might try applying a log transformation.

Right under the `commute_times` variable, create a variable called `commute_times_log` that stores a log-transformed version of `commute_times`.

To apply log-transform, you can use numpy's `log()` function.

Hint

Use the following code to apply log transform to `commute_times`:

```
commute_times_log = np.log(commute_times)
```

Copy to Clipboard

20.

Replace the histogram for `commute_times` with one for `commute_times_log`.

Notice how the shape of the data changes from being right skewed to a more symmetrical (and even slightly left-skewed) in shape. After applying log transformation, the transformed data is more "normal" than before.

Hint

Replace the following line of code:

```
plt.hist(commute_times)
```

Copy to Clipboard
with:

```
plt.hist(commute_times_log)
```

Copy to Clipboard

Extra

21.

Congratulations! In this project, you performed data analysis to help a management team answer important questions about the status of the company during a difficult time.

You did this by analyzing data sets and applying common data transformation techniques. These are important skills to have as a data analyst.

If you'd like extra practice with data transformations, here are some suggestions to extend this project:

Apply standardization to the employees data using `StandardScaler()` from `sklearn`.

Refer to [this](#) article if you need help.

Explore the income and productivity features in more detail. Can you find a relationship between productivity and income?

Constellations

<https://www.codecademy.com/paths/data-analyst/tracks/dsaly-visualization-for-data-science-applications/modules/dsaly-matplotlib-challenge-projects/projects/constellations>

Overview

This project is slightly different than others you have encountered thus far on Codecademy. Instead of a step-by-step tutorial, this project contains a series of open-ended requirements which describe the project you'll be building. There are many possible ways to fulfill all of these requirements correctly, and you should expect to use the internet, Codecademy, and other resources when you encounter a problem that you cannot easily solve.

Project Goals

In this project, you are Dr. Jillian Bellovary, a real-life astronomer for the Hayden Planetarium at the American Museum of Natural History. As an astronomer, part of your job is to study the stars. You've recently become interested in the constellation Orion, a collection of stars that appear in our night sky and form the shape of [Orion](#), a warrior God from ancient Greek mythology.

As a researcher on the Hayden Planetarium team, you are in charge of visualizing the Orion constellation using the Matplotlib function `.scatter()`. To learn more about the `.scatter()` you can see the Matplotlib documentation [here](#).

You will create both a 2D and a 3D visualization of the position of Orion's stars and get a better sense of their actual positions. To achieve this, you will be mapping real data from outer space that maps the position of the stars in the sky.

The goal of the project is to understand spatial perspective. Once you visualize Orion in both 2D and 3D, you will be able to see the difference in the constellation shape humans see from earth versus the actual position of the stars that make up this constellation.

This will be your first reviewable project, which will give you the opportunity to receive some feedback on what you've learned so far.

Setup Instructions

You have two options for completing this assignment: you can complete the project here, within Codecademy's Learning Environment, or you can complete it using a Jupyter notebook if that is more comfortable for you.

If you choose to complete this project using Codecademy's Learning Environment, then proceed to the tasks section below when you are ready!

If you choose to do this project on a Jupyter notebook, you can download what you'll need by clicking the "Download" button below. If you need help setting up your computer, be sure to check out our setup guides:

[Command Line Interface Setup](#)

[Introducing Jupyter Notebook](#)

[Setting up Jupyter Notebook](#)

[Getting Started with Jupyter](#)

[Getting More out of Jupyter Notebook](#)

Download the [Orion_Constellation.zip](#) project folder.

Open **orion_project.ipynb** and follow the steps in the Jupyter Notebook. If you get stuck, you can look at **orion_solution.ipynb** for the answer.

Download

Tasks

0/7 complete

[Mark the tasks as complete by checking them off](#)

Project Requirements

1.

In order to complete this project, we will need to import two Python packages. First, import pyplot as you have been importing it in previous lessons:

```
import matplotlib.pyplot as plt.
```

We also need some way to visualize the stars in 3D. To do this, import Axes3D in the following way:

```
from mpl_toolkits.mplot3d import Axes3D
```

2.

Astronomers describe a star's position in the sky by using a pair of angles: declination and right ascension. Declination is similar to longitude, but it is projected on the celestial sphere. Right ascension is known as the "hour angle" because it accounts for the time of day and earth's rotation. Both angles are relative to the celestial equator. You can learn more about star position [here](#).



The x, y, z coordinates for each star in the collection of stars that make up the Orion constellation are documented in a paper by Nottingham Trent University on "The Orion constellation as an installation" found [here](#). These coordinates have been organized and stored in Python lists named `x`, `y`, and `z` in the **script.py** file.

Spend some time looking at `x`, `y`, and `z`. Does each fall within some range?

3.

Before we visualize the stars in 3D, let's get a sense of what they look like in 2D.

Create a 2D plot that will use the (x,y) coordinates to form the Orion constellation.

Does the 2D visualization look like the Orion constellation we see in the night sky?

Hint

Create a figure for the 2D plot and save it to a variable called `fig`.

Using `.add_subplot()`, add your subplot as the single subplot.

Use the scatter [function](#) to visualize your `x` and `y` coordinates.

Render the visualization.

4.

Now, create a 3D visualization of the Orion Constellation with the `(x,y,z)` coordinates.

Observe the 3D representation of the Orion constellation. Remember, this will never look exactly like the Orion we see from Earth. The visualization does not curve as the night sky does.

Stuck? Get a hint

5.

Now that we have the stars plotted, let's make the plot look more like a night sky. Modify the parameters of the 2D and the 3D scatter plot to change the color of the background and of the stars to resemble a night sky.

Hint

Use the `.set_facecolor` method to change the color of the background.

Inside the `.scatter` method call, use the `color` parameter to change the color of the stars.

Solution

6.

Great work! Visit the [Codecademy forum](#) to post your project solution. You can also host your own Github repository with your solution for other learners to see. The solution you provide may be different from our suggested solution, and that is okay! There are many ways to complete on-platform projects, and you may learn more by viewing others' solutions.

Extra Challenge

7.

Congratulations!!

You've done fantastic work! Now that you have had a chance to model the Orion constellation, let's try to make the space in our visualization a more accurate depiction of the celestial body. (Note: This task is considered a challenge; you do not need to complete this in order to move on to the next section.)

If you wish to view the Orion constellation amongst a sky of stars, feel free to plot more stars! First, look up other celestial `x, y, z` coordinates [here](#). Append those coordinates to the `x`, `y`, and `z` lists and update your plots.

Board Slides for FoodWheel

Overview

<https://www.codecademy.com/paths/data-analyst/tracks/dsaly-visualization-for-data-science-applications/modules/dsaly-matplotlib-challenge-projects/projects/board-slides-for-food-wheel>

This project is slightly different than others you have encountered thus far on Codecademy. The project instructions contain a series of open-ended requirements which describe the project you'll be building, and the hints will give you more step-by-step instructions if you get stuck. There are many possible ways to fulfill all of these requirements correctly so feel free to use the internet, Codecademy, and other resources when you encounter a problem that you cannot easily solve.

Project Goals

Great job! You've passed your Data Analyst interview for FoodWheel!

FoodWheel is a startup delivery service that takes away the struggle of deciding where to eat.

FoodWheel picks you an amazing local restaurant and lets you order through the app. Senior leadership is getting ready for a big board meeting, and as the resident Data Analyst, you have been enlisted to help decipher data and create a presentation to answer several key questions:

What cuisines does FoodWheel offer? Which areas should the company search for more restaurants to partner with?

How has the average order amount changed over time? What does this say about the trajectory of the company?

How much has each customer on FoodWheel spent over the past six months? What can this tell us about the average FoodWheel customer?

Over this project, you will analyze several DataFrames and create several visualizations to help answer these questions.

Use your pandas and matplotlib skills to analyze and visualize the FoodWheel dataset.

Setup Instructions

You have two options for completing this assignment. Either here, within Codecademy's output terminal, or on your own, in case you're more comfortable using a Jupyter notebook.

If you choose to do this project on your computer instead of Codecademy, you can download what you'll need by clicking the "Download" button below. If you need help setting up your computer, be sure to check out our setup guides:

[Command Line Interface Setup](#)

[Introducing Jupyter Notebook](#)

[Setting up Jupyter Notebook](#)

[Getting Started with Jupyter](#)

[Getting More out of Jupyter Notebook](#)

Click "Download" below to download the project folder. Open **foodwheel_project.ipynb** and follow the steps in the Jupyter Notebook. If you get stuck, you can look at **foodwheel_solution.ipynb** for the answer.

Download

Tasks

0/11 complete

[Mark the tasks as complete by checking them off](#)

Project Requirements

1.

Before we begin analyzing our data, we need to import two Python packages. To be able to load in and inspect our datasets, we must first import the pandas package.

We will also need to create visualizations to help us analyze the data. This requires us to import pyplot, a subset of the Matplotlib package.

Hint

```
import pandas as pd
```

Copy to Clipboard

```
import matplotlib.pyplot as plt
```

Copy to Clipboard

Cuisine Offerings

2.

Now that we have imported the pandas package, we can begin looking at our data. Load and inspect the restaurant data from `restaurants.csv` into a DataFrame called `restaurants`.

Hint

Inspect the elements of your dataset using `.head()`.

3.

Now that we have our data loaded in and inspected, we can start to ask some questions about the data.

Suppose we wanted to know how many different types of cuisine FoodWheel offers and how many different restaurants serve that type of cuisine. Print out your answers to both questions.

Hint

Use `.nunique()` to find the number of unique types of food.

Use `.groupby()` and `.count()` to aggregate our data. Save your results to a variable called `cuisine_counts` and print your result.

4.

The board wants to make sure that FoodWheel offers a wide variety of restaurants. Having many different options makes customers more likely to come back. Create a pie chart that shows the different types of cuisines available on FoodWheel.

Hint

Make sure that your pie chart includes:

Labels for each cuisine (i.e, "American", "Chinese", etc.)

Percent labels using `autopct`.

A title.

Use `plt.axis('equal')` to make the pie chart a perfect circle.

`plt.show()` to display your chart.

Order Price Over Time

5.

FoodWheel is a relatively new start-up. They launched one year ago and have grown more popular since then.

Management suspects that the average amount spent on each order has increased over time.

First, load in the `orders.csv` dataset as `orders` and inspect it.

Hint

Start by loading the data from `orders.csv` into the DataFrame called `orders`.

Examine the first few rows of `orders` using `head()`.

6.

Perhaps tracking the orders made per month can help us determine a trend. Group the `orders` data by month.

Stuck? Get a hint

7.

While we are interested in the average order amount spent each month (average price), it wouldn't hurt to include the standard deviation as well. Calculate the average and standard deviation of the amount spent per month (`price`) and store each as its own separate variable.

Hint

Group `orders` by `month` and get the average order amount in each `month` using `.mean()`. Save your answer to `avg_order`.

Calculate the standard deviation for each month using `.std()`. Save this to `std_order`.

8.

At last, we can now visualize our data. Create a bar plot that demonstrates the trend in average order size over time. Make sure to include the following information:

The height of each bar should come from the `price` field from your DataFrame in step 7 (`avg_order`).

Use the standard deviations from step 7 (`std_order`) as the `yerr`.

The error capsize should be 5.

Make sure that you label each bar with the name of the month (i.e., 4 = April).

Also be sure to label the y-axis.

Give your plot a descriptive title.

Hint

The height of the bar plot should be set to `avg_order.price`, and the `yerr` should be set to `std_order.price`.

Customer Orders

9.

There is a range of amounts that customers spend at FoodWheel. Create a `customer_amount` DataFrame that stores each customer and how much they order.

Hint

Start by grouping `orders` by `customer_id` and calculating the total amount spent by each customer. Save your results to `customer_amount`.

10.

Create a histogram of the amount spent by each customer over the past six months with the following format:

The range should be from 0 to 200

The number of bins should be 40

Label the x-axis `Total Spent`

Label the y-axis `Number of Customers`

Add a title

Extra Challenge

11.

Congratulations!!

The team loves your analysis, and plan to use the information you have gathered to make beneficial decisions for the company and community. There is an additional task of adding the links for each of the paintings to the main list. This task can be considered a challenge that does not need to be completed to move on to the next section.

Extra Challenge

After a great round of applause, a member of the senior leadership board would quickly like to see the neighborhoods that have the most restaurants.

Create a visualization to display the restaurant count for each neighborhood.

Hint

View the unique neighborhoods.
Calculate the value counts of the neighborhood variable.
Create a list of the neighborhood count values.

Roller Coaster

Overview

This project is slightly different than others you have encountered thus far on Codecademy. Instead of a step-by-step tutorial, this project contains a series of open-ended requirements which describe the project you'll be building. There are many possible ways to correctly fulfill all of these requirements, and you should expect to use the internet, Codecademy, and other resources when you encounter a problem that you cannot easily solve.

Project Goals

You will work to create several data visualizations that will give you insight into the world of roller coasters.

Setup Instructions

You have two options of completing this assignment. Either here, within Codecademy's output terminal, or on your own, in case you're more comfortable using a Jupyter notebook. If you choose to do this project on your computer instead of Codecademy, you can download what you'll need by clicking the "Download" button below. If you need help setting up your computer, be sure to check out our setup guides:

[Command Line Interface Setup](#)

[Introducing Jupyter Notebook](#)

[Setting up Jupyter Notebook](#)

[Getting Started with Jupyter](#)

[Getting More out of Jupyter Notebook](#)

Open **Roller Coaster.ipynb** and follow the steps in the Jupyter Notebook. If you get stuck, you can look at **Roller Coaster_Solution.ipynb** for the answer.

Download

Tasks

0/12 complete

Mark the tasks as complete by checking them off

Prerequisites

1.

In order to complete this project, you should have completed the first two lessons in the [Data Analysis with Pandas course](#) and the first two lessons in the [Data Visualization in Python course](#). This content is also covered in the [Data Scientist Career Path](#).

Project Requirements

2.

Roller coasters are thrilling amusement park rides designed to make you squeal and scream! They take you up high, drop you to the ground quickly, and sometimes even spin you upside down before returning to a stop. Today you will be taking control back from the roller coasters and visualizing data covering international roller coaster rankings and roller coaster statistics.

Roller coasters are often split into two main categories based on their construction material: **wood** or **steel**.

Rankings for the best wood and steel roller coasters from the 2013 to 2018 [Golden Ticket Awards](#) are provided in `'Golden_Ticket_Award_Winners_Wood.csv'` and `'Golden_Ticket_Award_Winners_Steel.csv'`, respectively. Load each csv into a DataFrame and inspect it to gain familiarity with the data.

Open the hint for more information about each column of the datasets.

Hint

The Golden Ticket Awards datasets include the following columns:

`Rank`: rank of the roller coaster in a given year

`Name`: name of the roller coaster

`Park`: amusement park where the roller coaster is located

`Location`: location of the amusement park where the roller coaster is located

`Supplier`: manufacturer of the roller coaster

`Year_Built`: year the roller coaster was built

`Points`: points scored in a given year's ranking

`Year of Rank`: year the roller coaster achieved a given rank

How many roller coasters are included in each ranking dataset? How many different roller coaster suppliers are included in the rankings? Do some years include more rankings than other years?

If you have a favorite amusement park or there is one near where you live, subset one of the DataFrames on your chosen park. Are there any ranked roller coasters there?

3.

Write a function that will plot the ranking of a given roller coaster over time as a line. Your function should take a roller coaster's name and a ranking DataFrame as arguments. Make sure to include informative labels that describe your visualization.

Call your function with "El Toro" as the roller coaster name and the wood ranking DataFrame. What issue do you notice? Update your function with an additional argument to alleviate the problem, and retest your function.

Hint

Since "El Toro" is not a unique roller coaster name, the plot generated by your function will show multiple rankings for a given year. To fix this issue, we added an additional argument `park_name` to our function that allows us to differentiate between roller coasters of the same name at different amusement parks. Our solution then used logical statements to select a subset of the ranking DataFrame, only selecting the rows that are relevant to our particular roller coaster of interest.

```
coaster_rankings = rankings_df[(statement_checking_name) &
                                (statement_checking_park)]
```

Copy to Clipboard

We then used `plt.plot()` to plot the "rank" column against the "year of rank" column to produce the visualization.

We also inverted the y-axis of our plot in order to better represent increases and decreases in rank. You can do this by calling `.invert_yaxis()` on the axes of your plot. For more information on inverting the y-axis, check out the [matplotlib documentation here](#).

4.

Write a function that will plot the ranking of two given roller coasters over time as lines. Your function should take both roller coasters' names and a ranking DataFrame as arguments. Make sure to include informative labels that describe your visualization.

Call your function with "El Toro" as one roller coaster name, "Boulder Dash" as the other roller coaster name, and the wood ranking DataFrame. What issue do you notice? Update your function with two additional arguments to alleviate the problem, and retest your function.

Hint

Like in the last task, "El Toro" is not a unique roller coaster name, and the plot generated by your function will show multiple rankings for a given year. To fix this issue, we added two additional arguments `park_name_1` and `park_name_2` to our function that allow us to differentiate between roller coasters of the same name at different amusement parks.

The rest of our function is very similar to the function we wrote for the previous task, except we plotted two lines on the same figure. We also used the `'color'` and `'label'` keywords on each call to `plt.plot()` to differentiate the two lines. A legend will also help to distinguish between the two lines.

5.

Write a function that will plot the ranking of the top `n` ranked roller coasters over time as lines. Your function should take a number `n` and a ranking DataFrame as arguments. Make sure to include informative labels that describe your visualization.

For example, if `n == 5`, your function should plot a line for each roller coaster that has a rank of 5 or lower.

Call your function with a value for `n` and either the wood ranking or steel ranking DataFrame.

Hint

Our function selected all rows from the ranking DataFrame that have a rank less than or equal to `n`.

```
top_n_rankings = rankings_df[rankings_df['Rank'] <= n]
```

Copy to Clipboard

We then created a figure on which to plot. After creating the figure, we iterated through each unique coaster in our `top_n_rankings` DataFrame as used each coaster to filter `top_n_rankings` by that coaster and plot the ranking over time.

```
for coaster in set(top_n_rankings['Name']):
    coaster_rankings = top_n_rankings[top_n_rankings['Name'] == coaster]
    ax.plot(coaster_rankings['Year of
Rank'],coaster_rankings['Rank'],label=coaster)
```

Copy to Clipboard

6.

Now that you've visualized rankings over time, let's dive into the actual statistics of roller coasters themselves.

[Captain Coaster](#) is a popular site for recording roller coaster information. Data on all roller coasters documented on Captain Coaster has been accessed through its API and stored in `roller_coasters.csv`. Load the data from the csv into a DataFrame and inspect it to gain familiarity with the data.

Open the hint for more information about each column of the dataset.

Hint

The Captain Coaster dataset includes the following columns:

- `name`: name of the roller coaster
- `material_type`: the material used to build the roller coaster
- `seating_type`: the type of seating used on the roller coaster
- `speed`: maximum speed of the roller coaster, in kilometers/hour
- `height`: maximum height of the roller coaster, in meters
- `length`: length of the roller coaster, in meters
- `num_inversions`: total number of inversions on the roller coaster
- `manufacturer`: manufacturer of the roller coaster
- `park`: amusement park where the roller coaster is located
- `status`: operating status of the roller coaster

7.

Write a function that plots a histogram of any numeric column of the roller coaster DataFrame. Your function should take a DataFrame and a column name for which a histogram should be constructed as arguments.

Make sure to include informative labels that describe your visualization.

Call your function with the roller coaster DataFrame and one of the column names.

Hint

Our function uses `plt.hist()` to plot a histogram. We also used the pandas function `.dropna()` to remove all missing values before plotting the histogram.

Plotting a histogram of "height" is tricky since there are some outliers that skew the data. We also made a separate function that creates a histogram of height and used the following code to remove the outliers.

```
heights = coaster_df[coaster_df['height'] <= 140]
```

Copy to Clipboard

With the outliers removed, we could plot the histogram and better visualize the distribution of heights.

8.

Write a function that creates a bar chart showing the number of inversions for each roller coaster at an amusement park. Your function should take the roller coaster DataFrame and an amusement park name as arguments. Make sure to include informative labels that describe your visualization.

Call your function with the roller coaster DataFrame and an amusement park name.

Stuck? Get a hint

9.

Write a function that creates a pie chart that compares the number of operating roller coasters ('status.operating') to the number of closed roller coasters ('status.closed.definitely'). Your function should take the roller coaster DataFrame as an argument. Make sure to include informative labels that describe your visualization.

Call your function with the roller coaster DataFrame.

Hint

Our function selected the rows of our roller coaster DataFrame that indicated the roller coaster was either operating or not operating and saved them into new, separated DataFrames.

```
operating_coasters = coaster_df[coaster_df['status'] ==  
'status.operating']  
closed_coasters = coaster_df[coaster_df['status'] ==  
'status.closed.definitely']
```

Copy to Clipboard

We then found the length of these new DataFrames, saved them into a list called `status_counts`, and used them to create the pie chart.

```
plt.pie(status_counts, autopct='%0.1f%%', labels=['Operating', 'Closed'])
```

Copy to Clipboard

We added the keyword `autopct='%0.1f%%'` to have the percentages formatted on our pie chart.

10.

`.scatter()` is another useful function in matplotlib that you might not have seen before. `.scatter()` produces a scatter plot, which is similar to `.plot()` in that it plots points on a figure. `.scatter()`, however, does not connect the points with a line. This allows you to analyze the relationship between two variables. Find `.scatter()`'s [documentation here](#).

Write a function that creates a scatter plot of two numeric columns of the roller coaster DataFrame. Your function should take the roller coaster DataFrame and two-column names as arguments. Make sure to include informative labels that describe your visualization.

Call your function with the roller coaster DataFrame and two-column names.

Hint

Our function uses `plt.hist()` to create the scatter plot.

Plotting a scatter plot of "speed" vs "height" is tricky since there are some outliers that skew the data. We also made a separate function that specifically creates a scatter plot of "speed" vs "height" and used the following code to remove the "height" outliers.

```
coaster_df = coaster_df[coaster_df['height'] < 140]
```

Copy to Clipboard

With the outliers removed, we could create the scatter plot and better visualize the relationship between "height" and "speed".

11.

Part of the fun of data analysis and visualization is digging into the data you have and answering questions that come to your mind.

Some questions you might want to answer with the datasets provided include:

What roller coaster seating type is most popular? And do different seating types result in higher/faster/longer roller coasters?

Do roller coaster manufacturers have any specialties (do they focus on speed, height, seating type, or inversions)?

Do amusement parks have any specialties?

What visualizations can you create that answer these questions, and any others that come to you? Share the questions you ask and the accompanying visualizations you create on the Codecademy forums.

Solution

12.

Great work! Visit [our forums](#) to compare your project to our sample solution code. You can also learn how to host your own solution on GitHub so you can share it with other learners! Your solution might look different from ours, and that's okay! There are multiple ways to solve these projects, and you'll learn more by seeing others' code.

Code Review

NYC Tree Census and Income: Data Setup - TABLEAU

<https://www.codecademy.com/paths/data-analyst/tracks/dsaly-tableau/modules/dsaly-tableau-data-setup/projects/tableau-nyc-tree-census-and-income-data-setup>

<https://www.youtube.com/watch?v=itLufyC00bk>

This is Part I of the NYC Tree Census and Income project. By the end of this project, we will have...

- loaded multiple datasets into Tableau
- renamed and formatted columns for the best readability
- joined datasets on a common field
- saved our work privately to Tableau Public
- become familiar with the data and started looking towards making visualizations

You've probably already downloaded all the data you'll need for this project as you worked on the Data Setup lesson. If not, or if you'd like more info on our data, check out the [Github Repository](#) for this course.

Heads up: the data filename shown in the video is different from the one you'll use. We deleted some unused columns from the dataset to make it run faster and smoother in Tableau. In the video, Eric is working from a file called

2015StreetTreesCensus_TREES (which has *all* the columns), and the corresponding file in our Github repo is `tree-census-NYC_2015.csv`.

In this project, we'll use the NYC Tree Census, NYC Native Trees dataset, and NYC Income data.

Tasks

0/5 complete

Mark the tasks as complete by checking them off

Check off tasks as you complete them in Tableau.

1.
Load all datasets into Tableau.
2.
Join datasets on a common field.
3.
Rename and format columns for the best readability.
4.
Save work privately to Tableau Public.
5.
Ideate on 1-3 data visualizations.

FINANCIAL DATA

NPV Analysis - Project Greenspace

During a weekly project meeting, one of your team members presents an exciting new project, Project Greenspace. After she presents, you have a chance to review the documents with the team and discuss some possible scenarios to help the team make a recommendation about whether the company should pursue the project.

You decide to perform a Net Present Value (NPV) analysis to determine the profitability of Project Greenspace. The Net Present Value determines the present value of the future cash inflows and outflows over the life the project.

Tasks

0/5 complete

Mark the tasks as complete by checking them off

NPV Analysis

1.

The initial investment requires \$2,000,000, or a cash flow for Year 1 of `-2000000`. Construction will take place in Years 2 and 3 (no cash flows will be generated by the project).

The project will generate revenue in Year 4 and will continue to generate cash flows until the project is sold in Year 10 for \$1,000,000, or a cash flow for Year 10 of `1000000`.

Run the code by clicking "Save" and view the NPV profile. Based on these projections, what is the discount rate where the NPV of Project Greenspace equals 0? Open the hint to see the answer.

Hint

The discount rate where the NPV of Project Greenspace equals 0 is approximately 11%.

2.

After some discussions, it looks like construction may take longer and the expected revenue in Year 4 is only \$50,000, instead of \$250,000. Change the cash flow for Year 4 (the 4th value in `project_a`) to `50000` and run the code.

What happens to the NPV profile curve? At what discount rate does NPV = 0? Open the hint to see the answer.

Hint

After updating the year 4 cash flow to `50000`, `project_a` should be defined as shown below:

```
project_a = [-2000000, 0, 0, 50000, 250000, 500000, 500000, 750000, 750000, 1000000]
```

Copy to Clipboard

The NPV profile curve shifts to the left when the cash flow for Year 4 decreases from \$250,000 to \$50,000. The discount rate where the NPV of Project Greenspace equals 0 is now just under 10%.

3.

Based on another update from the project team, the project is estimated to provide cash flows for 15 years rather than 10.

The project will generate \$500,000 years 10-14, and the final cash flow of \$1,000,000 from selling the project will be realized in Year 15 when the project is sold.

Begin by updating the year 10 cash flow (the last number in `project_a`) to `500000`.

Hint

After updating the year 10 cash flow to `500000`, `project_a` should be defined as shown below:

```
project_a = [-2000000, 0, 0, 50000, 250000, 500000, 500000, 750000, 750000, 500000]
```

Copy to Clipboard

4.

Since years 11, 12, 13 and 14 will realize cash flows of \$500,000, add four more entries to the end of `project_a` with the value `500000`. Each entry should be separated by a comma.

Hint

After adding the year 11, 12, 13, and 14 cash flows of `500000`, `project_a` should be defined as shown below:

```
project_a = [-2000000, 0, 0, 50000, 250000, 500000, 500000, 750000, 750000, 500000, 500000, 500000, 500000, 500000]
```

Copy to Clipboard

5.

In year 15 the project is sold for \$1,000,000. Add as final entry to `project_a` the value `1000000` and then run the code by clicking "Save".

What happens to the NPV profile curve? In this scenario, at what discount rate does Project Greenspace become unattractive?

Hint

Analyzing Stock Data

It's time to put your Python skills and financial knowledge to the test!

You are given the monthly stock prices of two E-commerce companies, Amazon (AMZN) and eBay (EBAY). Help us analyze the risk and return for each investment! You will calculate the rates of return from this data, as well as other key statistics such as variance and correlation for assessing risk.

Let's get started!

Setup Instructions

You have two options of completing this assignment. Either here, within Codecademy's output terminal, or on your own, in case you're more comfortable using a Jupyter notebook. If you choose to do this project on your computer instead of Codecademy, you can download what you'll need by clicking the "Download" button below. If you need help setting up your computer, be sure to check out our setup guides:

[Command Line Interface Setup](#)

[Introducing Jupyter Notebook](#)

[Setting up Jupyter Notebook](#)

[Getting Started with Jupyter](#)

[Getting More out of Jupyter Notebook](#)

Open **Analyzing Stock Data.ipynb** and follow the steps in the Jupyter Notebook. If you get stuck, you can look at **Analyzing Stock Data_Solution.ipynb** for the answer.

Download

Tasks

10/10 complete

[Mark the tasks as complete by checking them off](#)

Inspect Data and Code

1.

In **script.py**, you are provided with the monthly stock prices of Amazon (AMZN) and eBay (EBAY) over the course of a year (June 2018 to June 2019). You are also given a helper function that displays decimal values (such as 0.075) in percentage form (7.5%).

In **utils.py**, you will find functions for calculating the log rate of return, variance, standard deviation, and correlation coefficient. They are imported at the top of **script.py**.

Take a moment to familiarize yourself with these functions and proceed to the next step when you are ready!

Hint

The `display_as_percentage()` function takes a numerical value as input and returns a formatted string that displays the value in percentage form.

```
my_value = 0.075
print('The value formatted as a percentage is',
      display_as_percentage(my_value))

# Prints:
# The value formatted as a percentage is 7.5%
```

Copy to Clipboard

Calculate Rate of Return

2.

Let's start by calculating the logarithmic rates of return from the stock prices. Define a function called `get_returns()` that takes a parameter called `prices`, which will be a list of stock prices.

The function will eventually return a list of log returns calculated from each adjacent pair of prices. For now, create a variable called `returns` inside the function body, and set it equal to an empty list.

Hint

A function can be defined with the following syntax:

```
def my_function(my_parameter):
    # code goes here
```

Copy to Clipboard

3.

Next, use a `for` loop to iterate over the `prices` list, from the 1st element to the 2nd to last.

You will be accessing the elements by their indices, so use the `range()` function to help generate the sequence of numbers to iterate over. Recall that Python uses zero-based indexing, meaning the index numbering of the list starts with 0 and ends with `n - 1`, where `n` is the number of elements in the list.

Hint

Use the built-in `len()` function to help find the number of elements in a list.

The syntax for iterating over a sequence of numbers generated by `range()` is:

```
for i in range(...):
    # code goes here
```

Copy to Clipboard

See [this exercise on the range function](#) for a review!

4.

As you iterate over each index `i`, the element in the `prices` list that is at position `i` will be the start price and the element with index `i + 1` will be the end price. Use the `calculate_log_return()` function imported from `utils.py` to calculate the rate of return from the start and end prices. Then, append the rate of return to the `returns` list. After your for loop, return `returns` from the `get_returns()` function.

Hint

Elements in a list can be accessed by index using square brackets:

```
for i in range(...):
    start_price = prices[i]
    end_price = prices[i + 1]

    # code goes here
```

Copy to Clipboard

5.

Use the `get_returns()` function to find the monthly log rates of return from the Amazon and eBay stock prices. Store those list of returns in the variables `amazon_returns` and `ebay_returns`, respectively.

Hint

The value returned from a function call can be stored in a variable using the syntax:

```
my_variable = my_function(...)
```

Copy to Clipboard

6.

Time to print out the lists of monthly returns! Since rates of return is often expressed as a percentage, use the `display_as_percentage()` function and list comprehension to display each value in `amazon_returns` and `ebay_returns` as a percentage.

How do the monthly returns of the two stocks compare? Are they on average profitable?

Hint

The syntax for calling a function on each element of a list using list comprehension is:

```
[my_function(item) for item in my_list]
```

Copy to Clipboard

For both stocks, there are more months where the rate of return is positive (indicating a *profit*) rather than negative (indicating a *loss*). Amazon has more profitable months than eBay.

7.

Now, let's calculate the annual rate of return for each stock!

Recall that log returns can easily be aggregated over time. Since `amazon_returns` and `ebay_returns` contain the monthly log returns for all 12 months in the past year, the annual return is simply the sum of all monthly returns. Use the `display_as_percentage()` function to help format the annual return as a percentage when you print out the results. How do the annual returns of the two stocks compare?

Hint

Use Python's built-in `sum()` function to find the sum of all values in a list.

We can see that Amazon's annual return is slightly higher than eBay's annual return.

Assess Investment Risk

8.

Let's move on to assessing the risk of each investment! Start by calculating the variance of each stock's monthly returns. Use the `calculate_variance()` function imported at the top of `script.py`, and print out the results.

How do the variance for each stock compare? What does this tell you about their relative risk?

Hint

The variance for Amazon's monthly returns is slightly higher than eBay's. A greater variance generally signifies a riskier investment.

9.

Now, calculate the standard deviation of each stock's monthly returns using the `calculate_stddev()` function, and print out the results.

Recall that the standard deviation has the same unit of measurement as the original dataset, or the monthly returns in this case. Since rates of return are often expressed as a percentage, use `display_as_percentage()` to help format the standard deviation for easier interpretation.

Hint

Amazon's monthly returns have a greater standard deviation than eBay's. As we've already seen previously, investing in Amazon stock is likely riskier.

It is worthwhile to note again that Amazon also has the higher rate of return. This demonstrates the risk-return tradeoff that often exists in investments.

10.

Finally, calculate the correlation between the stock returns using the `calculate_correlation()` function, and print out the results.

Are Amazon and eBay stock returns strongly or weakly correlated? Is the correlation positive or negative?

Hint

Betty's Bakery

Betty has always used her grandmother's recipe book to make cookies, cakes, pancakes, and bread for her friends and family. She's getting ready to open a business and will need to start buying all of her milk, eggs, sugar, flour, and butter in bulk.

Help Betty figure out how much she needs to buy using NumPy arrays describing her recipes.

If you get stuck during this project or would like to see an experienced developer work through it, click **"Get Unstuck"** to see a **project walkthrough video**.

Tasks

10/10 complete

[Mark the tasks as complete by checking them off](#)

Reading Recipes

1.

Start by importing NumPy as `np`.

Hint

```
import numpy as np
```

Copy to Clipboard

2.

All of Betty's recipes call for milk, eggs, sugar, flour, and butter. For example, her cupcake recipe calls for:

Flour	Sugar	Eggs	Milk	Butter
2 cups	0.75 cups	2 eggs	1 cups	0.5 cups

Create a NumPy array that represents this data. Each element should be a number (i.e., 2 for "2 cups"). Save this array as `cupcakes`.

Hint

```
cupcakes = np.array([2, 0.75, 2, 1, 0.5])
```

Copy to Clipboard

3.

Betty's assistant has compiled all of her recipes into a csv (comma-separated variable) file called `recipes.csv`. Load this file into a variable called `recipes`.

Hint

```
recipes = np.genfromtxt('recipes.csv',  
                        delimiter=',')
```

Copy to Clipboard

4.

Display `recipes` using `print`.

Each row represents a different recipe. Each column represents a different ingredient.

Recipe	Cups of Flour	Cups of Sugar	Eggs	Cups of Milk	Cups of Butter
Cupcakes	...	...	...	...	...
Pancake	...	...	...	...	...
Cookie	...	...	...	...	...
Bread	...	...	...	...	...

5.

The 3rd column represents the number of eggs that each recipe needs. Select all elements from the 3rd column and save them to the variable `eggs`.

Hint

Remember that columns are zero-indexed, so the 3rd column is selected with `2`.

6.

Which recipes require exactly 1 egg? Use a logical statement to get `True` or `False` for each value of `eggs`.

7.

Betty is going to make 2 batches of cupcakes (1st row) and 1 batch of cookies (3rd row). You already have a variable for `cupcakes`. Create a variable for `cookies` with the data from the 3rd row.

Hint

Remember that rows are zero-indexed, so the 3rd row is selected with `2`.

8.

Get the number of ingredients for a double batch of cupcakes by using multiplication on `cupcakes`. Save your new variable to `double_batch`.

9.

Create a new variable called `grocery_list` by adding `cookies` and `double_batch`.

10.

[Click here for a video walkthrough from our experts to help you check your work!](#)

US Financial Health

In this project, we'll be importing various types of financial data to try and determine the financial health and volatility of the US between 1999 and 2019.

We'll use the techniques we've learned for importing financial data, to import stock and commodity pricing data from csv files and the FRED API. Then grab GDP and goods and services export data from the World Bank API.

Finally, we'll find the log returns of the imported data, and use that to determine the volatility of the data over the 20 year period.

Let's get started!

Tasks

15/15 complete

[Mark the tasks as complete by checking them off](#)

Importing Commodity Prices

1.

In the workspace there are two csv files with historical commodity data for gold and crude oil.

This is the commodity data we'll be importing and operating on.

In order to import csv files, we'll need the pandas library imported into our program.

Import pandas in a variable called `pd`.

```
import pandas as pd
```

Copy to Clipboard

2.

Now that pandas is imported, use its `read_csv` function to import data from the `gold_prices.csv` file into a variable called `gold_prices`.

Then print the gold prices DataFrame and look it over.

```
pd.read_csv('filename.csv')
```

Copy to Clipboard

3.

Now let's do the same for the crude oil data.

Import the historical data in `crude_oil_prices.csv` into a variable called `crude_oil_prices`.

Then print it out and look it over as well.

```
pd.read_csv('filename.csv')
```

Copy to Clipboard

Importing Stock Prices

4.

We've imported the commodity prices from their csv files, now we'll focus on historical stock prices.

Pandas datareader is able to import stock pricing data from the FRED API using the `pandas_datareader.data` library.

Import `pandas_datareader.data` as `web`.

```
import pandas_datareader.data as web
```

Copy to Clipboard

5.

Since we only want data between 1999 and 2019, we'll also want to create some start and end variables.

Import the `datetime` module and create two datetimes, `start` and `end`, which represent January 1st of 1999 and 2019 respectively.

```
from datetime import datetime
```

```
start = datetime(year, month, day)
```

```
end = datetime(year, month, day)
```

Copy to Clipboard

6.

We can use the `web.DataReader` function to get historical prices for the NASDAQ 100 from the FRED API.

`web.DataReader` takes 4 arguments:

Data id code (`NASDAQ100`)

The name of the API we want to call (`fred`)

Start and end date times

Call `web.DataReader` with the appropriate arguments, and store the resulting DataFrame in a variable called `nasdaq_data`. Then print it out and look at the results.

```
nasdaq_data = web.DataReader("NASDAQ100", "fred", start, end)
```

Copy to Clipboard

7.

The FRED API also stores data from the S&P 500 Index. Let's import that as well.

Call `web.DataReader` just like in the previous step, except change the data id code from `NASDAQ100` to `SP500`.

Store the results in a variable called `sap_data` and print it out.

```
sap_data = web.DataReader("SP500", "fred", start, end)``
```

Importing World Bank Data

8.

In addition to stock and commodity prices, we also want to import more high level economic data like GDP and the total value of goods and services exported in a given year.

Luckily for us, the World Bank API tracks exactly these things.
First things first, let's import the World Bank sub-module from pandas datareader.

```
import pandas_datareader.wb as wb.  
import pandas_datareader.wb as wb
```

Copy to Clipboard

9.

We can use the `wb.download` function to get GDP data from the World Bank API.

`wb.download` takes 4 arguments:

A data indicator (`NY.GDP.MKTP.CD`)

A list of countries to get data for

Start and end datetimes

A call would look something like this:

```
wb.download(indicator='INDICATOR', country=['US'], start=start, end=end)
```

Copy to Clipboard

Call `wb.download` with the appropriate arguments, and store the resulting DataFrame in a variable called `gdp_data`.

```
wb.download(indicator='NY.GDP.MKTP.CD', country=['US'], start=start, end=end)
```

Copy to Clipboard

10.

The World Bank API also stores data about the value of goods and services exported in a given year. Let's import that as well.

Call `wb.download` just like in the previous step, except change the indicator from `NY.GDP.MKTP.CD` to `NE.EXP.GNFS.CN`.

Store the results in a variable called `export_data` and print it out.

```
wb.download(indicator='NE.EXP.GNFS.CN', country=['US'], start=start, end=end)
```

Copy to Clipboard

Calculating Log Return

11.

At this point, we've imported all the data we need. But it's all stored as either daily or yearly dollar amounts.

Pricing data is useful, but in this case, since we want to compare each data set, it would be even better if instead of daily/yearly pricing, we had information on the log returns from the daily/yearly prices.

As a first step, let's define a function called `log_return`, which should accept one parameter, `prices`. You can leave the function body empty for now.

```
def log_return(prices):  
    # code
```

Copy to Clipboard

12.

The equation for calculating the log return between two prices is as follows:

natural_log(current price/previous price)

In our case we want to run this equation for each day/year of pricing data in our imported DataFrame Series (A Series is a single column in a DataFrame).

The pandas `shift` function can be used to divide each current price by its previous price in the Series.

```
prices / prices.shift(1)
```

Copy to Clipboard

And we can use numpy's natural log function to get the log return for each entry in the new Series.

```
import numpy as np  
np.log(Series)
```

Copy to Clipboard

Using this information, fill in the code in the `log_return` function (be sure to import numpy at the top of the file).

```
import numpy as np  
...  
def log_return(prices):  
    return np.log(prices / prices.shift(1))
```

Copy to Clipboard

13.

Let's use our new `log_return` function to calculate the log return of the `gold_prices` DataFrame we imported.

Create a variable called `gold_returns`, which stores the result of calling `log_return` on the `Gold_Price` column of the `gold_prices` DataFrame.

```
gold_returns = log_return(gold_prices['Gold_Price'])
```

Copy to Clipboard

14.

Now create log return variables for each additional dataset (`crudeoil_returns`, `sap_returns`, etc.).

Remember that you only need to pass in the column of the DataFrame that contains the pricing data. In the case of gold it was `gold_prices['Gold Price']`, but the column name will vary for each dataset.

```
crude_oil_returns = log_return(crudeoil_prices['Crude_Oil_Price'])
nasdaq_returns = log_return(nasdaq_data['NASDAQ100'])
sap_returns = log_return(sap_data['SP500'])
gdp_returns = log_return(gdp_data['NY.GDP.MKTP.CD'])
export_returns = log_return(export_data['NE.EXP.GNFS.CN'])
```

Copy to Clipboard

Comparing Return Volatility

15.

We're now ready to compare the volatility of each type of data.

Variance, in the context of financial data, tells us how volatile an investment is. Use Panda's `var()` function to calculate the variance of the commodities, stocks and World Bank data returns, and print the results.

The results can be interpreted in a number of ways, but generally, the higher the variance the more volatile the data. What conclusions can you draw from this data? Which dataset was the most volatile? Did any datasets have similar variances?

```
print('gold:', gold_returns.var())
...
```

Copy to Clipboard

Conclusions

```
gold: 0.00011375928671558508
oil: 0.0005563207795629881
nasdaq: 0.0003178379833057229
sap: 8.860342194008153e-05 which is equivalent to 0.00008860342194008153
gdp: 0.0003576341131987123
export: 0.006775724487950144
```

Copy to Clipboard

You should have gotten something similar to what we have above, and this is generally in line with what we would expect. The S&P 500, a collection of 500 large companies listed on stock exchanges in the United States, has the smallest variance, and thus is the least volatile. Given that the S&P 500 index is a weighted measurement of many stocks across a variety of industries, it is seen as a safer, diversified investment.

Gold, notorious for being a stable investment has the second smallest variance.

Crude oil is the most volatile, which makes sense as gas prices are often unpredictable, especially in the last 20 years.

The stocks are interesting. The NASDAQ 100 is more volatile than the S&P 500, which, when you think about it makes sense, as the S&P 500 is far more diversified and tracks more of the market.

Then finally we have GDP and exports.

Exports are very volatile, which could have to do with industries moving overseas in the last 20 years, and global competition for the production of goods.

GDP is actually fairly similar to the NASDAQ 100 in terms of volatility, which is perhaps an interesting correlation.

Sublime Limes' Line Graphs

In this project, you will be acting as a data analyst for an online lime retailer called Sublime Limes. People all over the world can use this product to get the freshest, best-quality limes delivered to their door. One of the product managers at Sublime Limes would like to gain insight into the customers and their sales habits. Using Matplotlib, you'll create some different line graphs (or maybe, lime graphs) to communicate this information effectively.

If you get stuck during this project or would like to see an experienced developer work through it, click "**Get Unstuck**" to see a **project walkthrough video**.

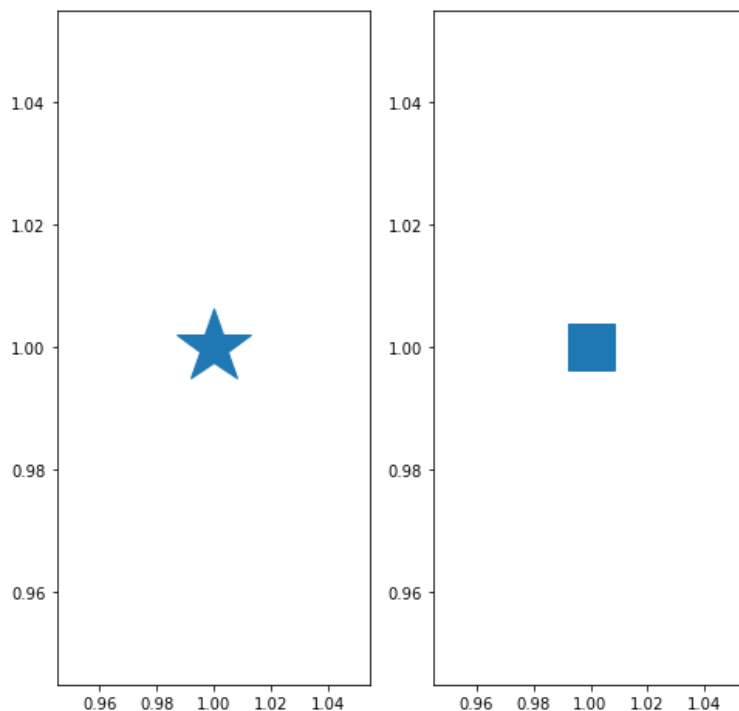
Tasks

17/17 complete

[Mark the tasks as complete by checking them off](#)

Understand the data and set up subplots

1. Add `import codecademylib` to the top of `script.py`. This will allow for your plots to render in our browser.
2. We have provided some data in different lists in `script.py`. Look through these lists and try to understand what each one represents.
3. Create a figure of width `12` and height `8`.
4. We are going to make two charts in one figure, laid out side-by-side. In other words, the figure will have one row and two columns, like this:



Write the command to create the left subplot (the one that would correspond to the plot with a star in our example figure). Save this subplot in a variable called `ax1`.

5. Write the command to create the right subplot (the one that would correspond to the plot with a square in our example figure). Save this subplot in a variable called `ax2`.

Page visits over time

6.

In the left subplot, we are going to plot the total page visits over the past year as a line.

First, let's create the list of x-values, which is `range(len(months))`. Store this in a variable called `x_values`.

Make sure this happens *after* the line where you created `ax1`, but before the line where you've created `ax2`, so that the plot goes in the subplot on the left.

7.

Plot the total page visits against these `x_values` as a line.

8.

Give the line markers that will help show each month as a distinct value.

9.

Label the x-axis and y-axis with descriptive titles of what they measure.

10.

Set the x-axis ticks to be the `x_values`.

11.

Label the x-axis tick labels to be the names stored in the `months` list.

Plotting multiple lime species

12.

In the subplot on the right, we are going to plot three lines on the same set of axes. The x-values for all three lines will correspond to the months, so we can use the list of `x_values` we used for the last plot.

On one plot, create the three lines:

number of key limes sold vs `x_values`

number of Persian limes sold vs `x_values`

number of blood limes sold vs `x_values`

Make sure this happens *after* the line where you created `ax2`, so that it goes in the subplot on the right.

13.

Give each line a specific color of your choosing.

14.

Add a legend to differentiate the lines, labeling each lime species.

15.

Set the x-axis ticks to be the `x_values`, and the tick labels to be the `months` list.

Labeling and saving

16.

Add a title to each of the two plots you've created, and adjust the margins to make the text you've added look better.

17.

Now, save your figure as a `png` with a descriptive file name.

Line with Shaded Error

Now, we are going to look at the chart called `line_graph.png`. This displays the relationship between the time students say they studied and the scores they received on their final exams.



The data you will need to recreate this chart is in the lists `hours_reported` and `exam_scores`.

Save your recreated chart to a file called `my_line_graph.png`.

Instructions

Checkpoint 1 Passed

1.

Create a figure of width 10 and height 8.

Checkpoint 2 Passed

2.

Plot the `exam_scores` on the x-axis and the `hours_reported` on the y-axis, with a `linewidth` of 2.

Checkpoint 3 Passed

3.

Let's assume the error on students' reporting of their hours studying is 20%.

Create a list `hours_lower_bound`, which has the elements of `hours_reported` minus 20%, and a list `hours_upper_bound`, which has the elements of `hours_reported` plus 20%.

You can do this with a list comprehension that looks like:

```
y_lower_bound = [element - (element * error_in_decimal) for element in original_list_of_y_values]
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Shade the area between `hours_lower_bound` and `hours_upper_bound` with an alpha of 0.2.

Checkpoint 5 Passed

5.

Give the plot the title 'Time spent studying vs final exam scores', x-axis label 'Score', and y-axis label 'Hours studying (self-reported)'.

Checkpoint 6 Passed

6.

Save your figure to a file called `my_line_graph.png`.

```
from matplotlib import pyplot as plt

hours_reported = [3, 2.5, 2.75, 2.5, 2.75, 3.0, 3.5, 3.25, 3.25, 3.5, 3.5, 3.75, 3.75, 4,
4.0, 3.75, 4.0, 4.25, 4.25, 4.5, 4.5, 5.0, 5.25, 5, 5.25, 5.5, 5.5, 5.75, 5.25, 4.75]
exam_scores = [52.53, 59.05, 61.15, 61.72, 62.58, 62.98, 64.99, 67.63, 68.52, 70.29, 71.33,
72.15, 72.67, 73.85, 74.44, 75.62, 76.81, 77.82, 78.16, 78.94, 79.08, 80.31, 80.77, 81.37,
85.13, 85.38, 89.34, 90.75, 97.24, 98.31]

# Create your figure here

# Create your hours_lower_bound and hours_upper_bound lists here

# Make your graph here

plt.show()
```

Mean Variance Optimization

<https://www.codecademy.com/paths/finance-python/tracks/visualizing-finance-data/modules/portfolio-optimization/projects/mean-variance-optimization>

You're an investment analyst at a large fund. You specialize in calculating efficient frontiers and selecting assets for portfolios.

This project is split into three tasks, each with its own Python file:

Follow the steps to calculate the efficient frontier of a portfolio with five assets (**script1.py**)
See how two additional assets affect the efficient frontier from part one (**script2.py**)
Use the covariance matrix and expected returns from ten assets to select the five assets that produce the best efficient frontier (**script3.py**)

You can select the tabs at the top of the screen to switch between files.

Setup Instructions

You have two options of completing this assignment. Either here, within Codecademy's output terminal, or on your own, in case you're more comfortable using a Jupyter notebook. If you choose to do this project on your computer instead of Codecademy, you can download what you'll need by clicking the "Download" button below. If you need help setting up your computer, be sure to check out our setup guides:

[Command Line Interface Setup](#)
[Introducing Jupyter Notebook](#)
[Setting up Jupyter Notebook](#)
[Getting Started with Jupyter](#)
[Getting More out of Jupyter Notebook](#)

Open **Mean Variance Optimization.ipynb** and follow the steps in the Jupyter Notebook. If you get stuck, you can look at **Mean Variance Optimization_Solution.ipynb** for the answer.

Download

Tasks

3/13 complete

[Mark the tasks as complete by checking them off](#)

Task One: script1.py

1.
Load **stock_data_weak.csv** file as a DataFrame and save it to a variable called `stock_data`.
Use `print(stock_data)` to visualize the data.
Use `pd.read_csv()` to load in the file.
2.
Find the quarterly returns over each period for all assets in `stock_data`. Save the result to `returns_quarterly`.
This step requires that you first select the correct columns, using:

```
selected = list(stock_data.columns[1:])
```

Copy to Clipboard

Use the `.pct_change()` method to find the quarterly return for each asset in `stock_data[selected]`.

3.
Find the expected return for each asset and save the result to `expected_returns`.
Call the `.mean()` function on `returns_quarterly` to calculate the expected return values.

4.
Find the covariance of the quarterly returns over each period and save the result to `cov_quarterly`.
Call the `.cov()` function on `returns_quarterly` to calculate the expected return values.

5.
We loaded the function `return_portfolios()` from **rf.py**, which accepts the expected return and quarterly covariance of a set of assets and returns a random collection of 5,000 portfolios. The return is formatted as a DataFrame with columns: 'Volatility' and 'Returns'.
Use the expected returns and covariances to find a set of random portfolios. Save the result to `random_portfolios`.
Set `random_portfolios` equal to `return_portfolios(expected_returns, cov_quarterly)`.

6.

Plot the random portfolios.

Use the `plot.scatter()` method, as follows, to plot the risks and returns of the portfolios.

```
df.plot.scatter(x='Volatility', y='Returns', fontsize=12)
```

Copy to Clipboard

7.

We loaded the function `optimal_portfolio()` from `rf.py`, which accepts the quarterly returns of a set of assets. The function returns the `weights`, `returns`, and `risks` of portfolios on the efficient frontiers. The returns are formatted as NumPy arrays.

Use the `optimal_portfolio()` function to calculate the `weights`, `returns`, and `risks` of portfolios on the efficient frontier.

Use the following code to calculate the weights, returns, and risks of portfolios on the efficient frontier.

```
weights, returns, risks = optimal_portfolio(returns_quarterly[1:])
```

Copy to Clipboard

We index from position 1 because the first column is filled with nans.

8.

Plot the efficient frontier as a yellow line, with dots on the plot.

Use the following code to set the x-label, y-label, and title:

```
plt.ylabel('Expected Returns', fontsize=14)
plt.xlabel('Volatility (Std. Deviation)', fontsize=14)
plt.title('Efficient Frontier', fontsize=24)
```

Copy to Clipboard

Use the following code to plot the efficient frontier:

```
plt.plot(risks, returns, 'y-o')
```

Copy to Clipboard

Task Two: script2.py

9.

You inherited a portfolio with five assets, stored in `stock_data_weak.csv`:

Asset	Expected Return
General Electric	-0.0120
Chesapeake Energy	-0.0306
AT&T	-0.0014
Cisco Systems	0.0184
Campbell Soup	0.0084

Run `script2.py` to visualize the efficient frontier for these assets.

10.

You decide to diversify the portfolio with a couple of assets to increase the expected return of portfolios on the efficient frontier.

You choose to add Bank of America and Marvel, so your assets are:

Asset	Expected Return
General Electric	-0.0120
Chesapeake Energy	-0.0306
AT&T	-0.0014
Cisco Systems	0.0184
Campbell Soup	0.0084
Bank of America	0.0341
Marvel	0.0369

Change the `path` variable to `stock_data2.csv` to visualize this new efficient frontier.

11.

It's hard to visualize how much the efficient frontier improved with these assets. Let's plot the original efficient frontier on the same figure. Use the following code to do this:

```
weak_EF = pd.read_csv('weak_risks_returns.csv')
plt.plot(weak_EF['Risks'], weak_EF['Returns'], 'g-o')
```

Copy to Clipboard

To get a better view of the efficient frontier, you may want to comment out the following line:

```
random_portfolios.plot.scatter(x='Volatility', y='Returns', fontsize=12)
```

Copy to Clipboard

By adding the two high-return assets, we increased the expected return of portfolios on the efficient frontier.

12.

If you look at the expected returns of assets in your portfolio, you'll notice that General Electric and Chesapeake Energy have a large negative return (i.e. you expect to lose money).

Asset	Expected Return
General Electric	-0.0120

Chesapeake Energy	-0.0306
AT&T	-0.0014
Cisco Systems	0.0184
Campbell Soup	0.0084
Bank of America	0.0341
Marvel	0.0369

This begs the questions: do we need these assets in our portfolio? Given their poor expected return, we should only keep them if they are uniquely uncorrelated to the rest of the assets in the portfolio.

To check the effects of these assets, we calculated the efficient frontier for a portfolio **without** General Electric and Chesapeake Energy. Add the following code to the bottom of **script2.py** to visualize this efficient frontier on the same plot:

```
strong_EF = pd.read_csv('strong_risks_returns.csv')
plt.plot(strong_EF['Risks'], strong_EF['Returns'], 'k-x')
```

Copy to Clipboard

What do you notice when you run the code?

The efficient frontier did not change when you removed the two low return assets. Because these assets had low returns and were correlated to other assets in the portfolio, they didn't improve the possible portfolios.

Task Three: script3.py

13.

Next, you will use your understanding of expected returns and covariances to inform portfolio decisions. We have asset data for 10 stocks, stored in **stock_data3.csv**.

Open **script3.py** and run the code.

Scroll to the bottom of the web browser to the right – you should see two efficient frontiers. We calculate the better efficient frontier (green dots) using all ten assets. We calculate the other efficient frontier using only five assets from the portfolio.

Your job is to choose five assets from the following list to create the best efficient frontier.

```
['PFE', 'TGT', 'M', 'VZ', 'JPM', 'MRO', 'KO', 'PG', 'CVS', 'HPQ']
```

Copy to Clipboard

Use the expected returns and covariance table to the right, and set the `selected` variable equal to a list with five of these assets.

In the hint, we provide our best solution and explain how we arrived there.

Using the following five assets will result in an efficient frontier that closely resembles a portfolio with all ten assets.

```
selected = ['TGT', 'CVS', 'M', 'VZ', 'JPM']
```

Copy to Clipboard

We used the following approach to select these assets:

`JPM` has the largest expected return, so we knew to include it.

We added the asset that was the least correlated to `JPM` – `VZ` has a small negative covariance.

We selected the assets with the next three highest expected returns: `M`, `CVS`, and `TGT`.

Honey Production

Now that you have learned how linear regression works, let's try it on an example of real-world data.

As you may have already heard, the honeybees are in a precarious state right now. You may have seen articles about the decline of the honeybee population for various reasons. You want to investigate this decline and how the trends of the past predict the future for the honeybees.

Note: All the tasks can be completed using Pandas or NumPy. Pick whichever one you prefer.

If you get stuck during this project or would like to see an experienced developer work through it, click **"Get Unstuck"** to see a **project walkthrough video**.

Tasks

0/13 complete

Mark the tasks as complete by checking them off

Check out the Data

1.

We have loaded in a DataFrame for you about honey production in the United States from [Kaggle](#). It is called `df` and has the following columns:

```
state
numcol
yieldpercol
totalprod
stocks
priceperlb
prodvalue
year
```

Use `.head()` to get a sense of how this DataFrame is structured.

A DataFrame is a 2-dimensional labeled data structure with columns of potentially different types. you can think of it like a spreadsheet or a SQL table. it is generally the most commonly used `pandas` object.

The `.head()` function return the first `n` rows:

```
print(df.head())
```

Copy to Clipboard

2.

For now, we care about the total production of honey per year. Use the `.groupby()` method provided by `pandas` to get the mean of `totalprod` per year.

Store this in a variable called `prod_per_year`.

To group the years, you might use a command like this:

```
prod_per_year = df.groupby('year').totalprod.mean().reset_index()
```

Copy to Clipboard

This will group the `year` column.

3.

Create a variable called `x` that is the column of `years` in this `prod_per_year` DataFrame.

After creating `x`, we will need to reshape it to get it into the right format, using this command:

```
x = X.values.reshape(-1, 1)
```

Copy to Clipboard

You can select a column from a DataFrame by using bracket notation:

```
column_of_interest = df["column name"]
```

Copy to Clipboard

4.

Create a variable called `y` that is the `totalprod` column in the `prod_per_year` dataset.

You can select a column from a DataFrame by using bracket notation:

```
column_of_interest = df["column name"]
```

Copy to Clipboard

5.

Using `plt.scatter()`, plot `y` vs `x` as a scatterplot.

Display the plot using `plt.show()`.

Can you see a vaguely linear relationship between these variables?

Note the uppercase `x`.

To plot a scatterplot, you can enter the `x`-values and then the `y`-values into `plt.scatter()`:

```
plt.scatter(x, y)
```

Copy to Clipboard

Create and Fit a Linear Regression Model

6.

Create a linear regression model from scikit-learn and call it `regr`.

Use the `LinearRegression()` constructor from the `linear_model` module to do this.

The `LinearRegression` class is from the `linear_model` module.

To create a new `LinearRegression` object, just call `linear_model.LinearRegression()` and assign it to your variable name:

```
my_regression_model = linear_model.LinearRegression()
```

Copy to Clipboard

7.

Fit the model to the data by using `.fit()`. You can feed `x` into your `regr` model by passing it in as a parameter of `.fit()`.

8.

After you have fit the model, print out the slope of the line (stored in a list called `regr.coef_`) and the intercept of the line (`regr.intercept_`).

9.

Create a list called `y_predict` that is the predictions your `regr` model would make on the `x` data.

You can use the `.predict()` method, using `x` as a parameter, to get the predictions from the `regr` object.

10.

Plot `y_predict` vs `x` as a line, on top of your scatterplot using `plt.plot()`.

Make sure to call `plt.show()` after plotting the line.

After the first call to `plt.plot()`, but before you call `plt.show()`, you can plot a line using:

```
plt.plot(X, y_predict)
```

Copy to Clipboard

Predict the Honey Decline

11.

So, it looks like the production of honey has been in decline, according to this linear model. Let's predict what the year 2050 may look like in terms of honey production.

Our known dataset stops at the year 2013, so let's create a NumPy array called `x_future` that is the range from 2013 to 2050. The code below makes a NumPy array with the numbers 1 through 10

```
nums = np.array(range(1, 11))
```

Copy to Clipboard

After creating that array, we need to reshape it for scikit-learn.

```
X_future = X_future.reshape(-1, 1)
```

Copy to Clipboard

You can think of `reshape()` as rotating this array. Rather than one big row of numbers, `x_future` is now a big column of numbers – there's one number in each row.

`reshape()` is a little tricky! It might help to print out `x_future` before and after reshaping.

We want to make a list out of a range object:

```
X_future = np.array(range(start_year, end_year))
```

Copy to Clipboard

12.

Create a list called `future_predict` that is the `y`-values that your `regr` model would predict for the values of `x_future`.

You can use the `.predict()` method, using `x_future` as a parameter, to get the predictions from the `regr` object.

13.

Plot `future_predict` vs `x_future` on a different plot.

How much honey will be produced in the year 2050, according to this?

Complete the following line of code:

```
plt.plot(X_future, _____)
```

```
import codecademylib3_seaborn
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn import linear_model

df = pd.read_csv("https://content.codecademy.com/programs/data-science-
path/linear_regression/honeyproduction.csv")
```

Python Classes: Medical Insurance Project

You have been asked to develop a system that makes it easier to organize patient data. You will create a `class` that does the following:

- Takes in patient parameters regarding their personal information
- Contains methods that allow users to update their information
- Gives patients insight into their potential medical fees.

Let's get started!

Setup Instructions

You have two options of completing this assignment. Either here, within Codecademy's output terminal, or on your own, in case you're more comfortable using a Jupyter notebook. If you choose to do this project on your computer instead of Codecademy, you can download what you'll need by clicking the "Download" button below. If you need help setting up your computer, be sure to check out our setup guides:

- [Command Line Interface Setup](#)
- [Introducing Jupyter Notebook](#)
- [Setting up Jupyter Notebook](#)
- [Getting Started with Jupyter](#)
- [Getting More out of Jupyter Notebook](#)

Open **Python Classes Medical Insurance.ipynb** and follow the steps in the Jupyter Notebook. If you get stuck, you can look at **Python Classes Medical Insurance_Solution.ipynb** for the answer.

Download

Tasks

0/14 complete

[Mark the tasks as complete by checking them off](#)

Building our Constructor

1.

If you look at **script.py**, you will see that we have started a `class` called `Patient`. It currently has an `__init__` method with two class variables: `self.name` and `self.age`.

Let's start by adding in some more patient parameters:

- `sex`: patient's biological identification, 0 for male and 1 for female
- `bmi`: patient BMI
- `num_of_children`: number of children patient has
- `smoker`: patient smoking status, 0 for a non-smoker and 1 for a smoker

Add these into the `__init__` method so that we can use them as we create our class methods.

Remember to add these parameters as arguments in the `__init__` method, and use the keyword `self` when defining your class variables.

When you are finished, your `__init__` method should look like this:

```
def __init__(self, name, age, sex, bmi, num_of_children, smoker):
    self.name = name
    self.age = age
    self.sex = sex
    self.bmi = bmi
    self.num_of_children = num_of_children
    self.smoker = smoker
```

[Copy to Clipboard](#)

2.

Let's test out our `__init__` method and create our first instance variable.
Create an instance variable outside of our class called `patient1`.

```
patient1 = Patient("John Doe", 25, 1, 22.2, 0, 0)
```

Copy to Clipboard

Next, print out the name of `patient1` using the following line of code:

```
print(patient1.name)
```

Copy to Clipboard

Print out the rest of `patient1`'s information to ensure the `__init__` method is functioning properly.

To test out each class variable in our `__init__`, use the following code:

```
print(patient1.name)
print(patient1.age)
print(patient1.sex)
print(patient1.bmi)
print(patient1.num_of_children)
print(patient1.smoker)
```

Copy to Clipboard

When you run the code, you should see the following string pop up in the terminal:

```
John Doe
25
1
22.2
0
0
```

Copy to Clipboard

Adding Functionality with Methods

3.

Now that our constructor is built out and ready to go, let's start creating some methods! Our first method will be `estimated_insurance_cost()`, which takes our instance's parameters (representing our patient's information) and returns their expected yearly medical fees.

Below the `__init__` constructor, define our `estimated_insurance_cost()` constructor which only takes `self` as an argument. Inside of this method, add the following formula:

```
estimated_cost=250* age-128* sex+370* bmi+425* num_of_children+24000* smoker-12500
estimated_cost=250* age-128* sex+370* bmi+425* num_of_children+24000* smoker-12500
```

Note that we are using class variables in our formula here, so be sure to remember to use the `self` keyword.

When you finish writing up your method, it should look like this:

```
def estimated_insurance_cost(self):
    estimated_cost = 250 * self.age - 128 * self.sex + 370 * self.bmi + 425 *
self.num_of_children + 24000 * self.smoker - 12500
```

Copy to Clipboard

Notice that we use `self.age` and not `age` when writing up this formula because our method uses our class variables.

Make sure that this method is defined within the `Patient` class.

4.

Inside of our `estimated_insurance_cost()` method, let's add a print statement that displays the following in the terminal:
{Patient Name}'s estimated insurance costs is {estimated cost} dollars.

Copy to Clipboard

Then, test out this method using the `patient1` instance variable.

You should use the following print statement:

```
print(self.name + "'s estimated insurance cost is " + str(estimated_cost) + "
dollars.")
```

Copy to Clipboard

To test out the method, use the following code snippet:

```
patient1.estimated_insurance_cost()
```

```
# Prints: John Doe's estimated insurance cost is 1836.0 dollars.
```

Copy to Clipboard

5.

We already have one super useful method in our class! Let's add some more and make our `Patient` class even more powerful.

What if our patient recently had a birthday? Or had a fluctuation in weight? Or had a kid? Let's add some methods that allow us to update these parameters and recalculate the estimated medical fees in one swing.

First, create an `update_age()` method. It should take in two arguments: `self` and `new_age`. In this method reassign `self.age` to `new_age`.

Your `update_age()` method should look like this so far:

```
def update_age(self, new_age):  
    self.age = new_age
```

Copy to Clipboard

6.

Let's flesh out this method some more!

Add a print statement that outputs the following statement in the terminal:

```
{Patient Name} is now {Patient Age} years old.
```

Copy to Clipboard

Test out your method using the `patient1` instance variable.

You should have the following print statement:

```
print(self.name + " is now " + str(self.age) + " years old.")
```

Copy to Clipboard

To test out your method, call it using the `patient1` variable:

```
patient1.update_age(26)
```

Copy to Clipboard

7.

We also want to see what the new insurance expenses are. Call the `estimated_insurance_cost()` method in `update_age()` using this line of code:

```
self.estimated_insurance_cost()
```

Copy to Clipboard

Test out your method with `patient1`.

We have to use `self.estimated_insurance_cost()` here instead of just saying `estimated_insurance_cost()` because `self` is our placeholder for whatever instance variable we are working with.

When you test out the method with `patient1`, you should see the following output:

```
John Doe's estimated insurance cost is 2086.0 dollars.
```

Copy to Clipboard

8.

Let's make another update method that modifies the `num_of_children` parameter.

Below the `update_age()` method, define a new one called `update_num_children()`. This method should have two arguments, `self` and `new_num_children`. Inside the method, `self.num_of_children` should be set equal to `new_num_children`.

Your `update_num_children()` method should look like this so far:

```
def update_num_children(self, new_num_children):  
    self.num_of_children = new_num_children
```

Copy to Clipboard

9.

Similarly to the method we wrote before, let's add in a print statement that clarifies the information that is being updated.

Your print statement should output the following in the terminal:

```
{Patient Name} has {Patient's Number of Children} children.
```

Copy to Clipboard

Use the `patient1` instance variable to test out this method. Set the `new_num_children` argument to `1`. Do you notice anything strange about the output?

Your print statement should look like this:

```
print(self.name + " has " + str(self.num_of_children) + " children.")
```

Copy to Clipboard

To test out the method, use the following line of code:

```
patient1.update_num_children(1)
```

Copy to Clipboard

You should see this in your output terminal:

```
John Doe has 1 children.
```

Copy to Clipboard

This is not grammatically correct, so we may want to update our method to combat this.

10.

You may have noticed our terminal output is grammatically incorrect because John Doe only has 1 child. Let's update our method to accurately convey when we should use the noun "children" versus when we should use "child".

To do this we can use control flow.

If the patient has 1 offspring, we should see the following output:

```
{Patient Name} has {Patient Number of Children} child.
```

Copy to Clipboard

Otherwise, we should see this output:

```
{Patient Name} has {Patient Number of Children} children.
```

Copy to Clipboard

Write out your control flow program, and test it out using `patient1`.

Your control flow program should look something like the following:

```
if self.num_of_children == 1:
    print(self.name + " has " + str(self.num_of_children) + "child.")
else:
    print(self.name + " has " + str(self.num_of_children) + "children.")
```

Copy to Clipboard

Your control flow program might look different than the example above, but that is no problem as long as it is functioning properly.

11.

To finish off the `update_num_children()` method, let's call our `estimated_insurance_cost()` method at the end.

Use `patient1` to ensure that everything is functioning as expected!

Reminder that to call `estimated_insurance_cost()` in your `update_num_children()` method use the following line of code:

```
self.estimated_insurance_cost()
```

Copy to Clipboard

To call the `update_num_children()` with `patient1` use the following line of code:

```
# note: you can try any number when calling the method
patient1.update_num_children(1)
```

Copy to Clipboard

In your output terminal you should see the following:

```
John Doe has 1 child
John Doe's estimated insurance cost is 2511.0
```

Copy to Clipboard

Storing Patient Information

12.

Let's create one last method that uses a dictionary to store a patient's information in one convenient variable. We can use our parameters as the keys and their specific data as the values.

Define a method called `patient_profile()` that builds a dictionary called `patient_information` to hold all of our patient's information.

Note: See the **Hint** to find out how to use a dictionary to store this data.

This dictionary should hold all six class variables in `Profile` and return the `patient_info` dictionary at the end. The following code shows you how to initialize the dictionary as well as how to create the first key and value pair for the class variable, `self.name`:

```
# initialize empty dictionary
patient_information = {}
# set name as a key tied with the name value
patient_information["name"] = self.name
```

Copy to Clipboard

Your `patient_profile()` method should look similar to the following example:

```
def patient_profile(self):
    patient_information = {}
    patient_information["Name"] = self.name
    patient_information["Age"] = self.age
    patient_information["Sex"] = self.sex
    patient_information["BMI"] = self.bmi
    patient_information["Number of Children"] = self.num_of_children
    patient_information["Smoker"] = self.smoker
    return patient_information
```

Copy to Clipboard

13.

Let's test out our final method! Use `patient1` to call the method `patient_profile()`.

Remember that in `patient_profile()` we used a return statement rather than a print statement. In order to see our dictionary outputted to the terminal, we must wrap a print statement around our method call. Use `patient1` to call the method `patient_profile()`.

To call `patient_profile()` you can use the following code:

```
print(patient1.patient_profile())
```

Copy to Clipboard

In the output terminal you should see the following information:

```
{'Name': 'John Doe', 'Age': 26, 'Sex': 1, 'BMI': 22.2, 'Number of Children': 1, 'Smoker': 0}
```

Copy to Clipboard

Extensions

14.

Congratulations! You have successfully made a powerful `Patient` class! Classes are an incredibly useful programming tool because they allow you to create a blueprint that can be used to build many objects off of. In this case, you can organize any patient's information and apply all methods from `Patient` to update and arrange their data.

There are endless possibilities for extending the capabilities of this class. If you would like to continue to work on this `Patient` class, take a look at the following challenges:

- Build out more methods that allow users to update more patient parameters, such as `update_bmi()` or `update_smoking_status()`.

- Use `try` and `except` statements to ensure that patient data is uploaded using numerical values.

- Update the class so that users can upload lists of patient data rather than just individual numbers.

Data Scientist:

Machine Learning Specialist

<https://www.codecademy.com/paths/data-science/tracks/dsml-math-for-machine-learning/modules/math-ds-linear-algebra/projects/image-transformations-with-numpy>

Limit Definition of the Derivative Exploration

<https://www.codecademy.com/paths/data-science/tracks/dsml-math-for-machine-learning/modules/math-ds-differential-calculus/projects/differential-calculus-project>

Heart Disease Research Part II

- Data Scientist: Inference Specialist

In this project, you'll investigate some data from a sample patients who were evaluated for heart disease at the Cleveland Clinic Foundation. The data was downloaded from the [UCI Machine Learning Repository](#) and then cleaned for analysis. The principal investigators responsible for data collection were:

Hungarian Institute of Cardiology, Budapest: Andras Janosi, M.D.
University Hospital, Zurich, Switzerland: William Steinbrunn, M.D.
University Hospital, Basel, Switzerland: Matthias Pfisterer, M.D.
V.A. Medical Center, Long Beach and Cleveland Clinic Foundation: Robert Detrano, M.D., Ph.D.

Note that a **solution.py** file is loaded for you in the workspace, which contains solution code for this project. We highly recommend that you complete the project on your own without checking the solution, but feel free to take a look if you get stuck or want to check your answers when you're done!

Finally, we ask you to both print results and graph results in this project. Sometimes the printouts are formatted in a way that is difficult to read — feel free to expand the middle segment to make your printouts more readable.

Tasks

1/14 complete

[Mark the tasks as complete by checking them off](#)

Inspect the Data

1.

The data has been saved as a dataframe named `heart` in **script.py**. It contains the following variables:

`age`: age in years
`sex`: sex assigned at birth; 'male' or 'female'
`trestbps`: resting blood pressure in mm Hg
`chol`: serum cholesterol in mg/dl
`cp`: chest pain type ('typical angina', 'atypical angina', 'non-anginal pain', or 'asymptomatic')
`exang`: whether the patient experiences exercise-induced angina (1: yes; 0: no)
`fbs`: whether the patient's fasting blood sugar is >120 mg/dl (1: yes; 0: no)
`thalach`: maximum heart rate achieved in exercise test
`heart_disease`: whether the patient is found to have heart disease ('presence': diagnosed with heart disease; 'absence': no heart disease)

Inspect the first few rows of data using the `.head()` method.

```
print heart.head()
```

Predictors of Heart Disease

2.

Each of the patients in this dataset underwent an exercise test, during which their heart rate was monitored. For each patient, `thalach` gives us the highest heart rate that the patient achieved during this test.

Is `thalach` associated with whether or not a patient will ultimately be diagnosed with heart disease? Use `sns.boxplot()` to plot side by side box plots of `thalach` for patients who were and were not diagnosed with heart disease (indicated by the `heart_disease` variable). Do you think there is a relationship between these variables?

Fill in the following code:

```
sns.boxplot(x = heart.___, y = heart.___)
plt.show()
```

Copy to Clipboard

Based on this plot, patients diagnosed with heart disease generally had a lower maximum heart rate during their exercise test.

3.

In order to investigate this question further, save the values for `thalach` among patients who were diagnosed with heart disease as a variable named `thalach_hd`. Then save the values of `thalach` among patients who were not diagnosed with heart disease as `thalach_no_hd`.

Here is the code for `thalach_hd`. Use similar syntax to create `thalach_no_hd`:

```
thalach_hd = heart.thalach[heart.heart_disease == 'presence']
```

Copy to Clipboard

4.

Calculate and print the difference in mean `thalach` for patients diagnosed with heart disease compared to patients without heart disease. Then do the same for the median difference.

Here is code for the mean difference. Use similar syntax with the `np.median()` function to calculate the median difference.

```
mean_diff = np.mean(thalach_no_hd) - np.mean(thalach_hd)
print('`thalach` mean Difference: ', mean_diff)
```

Copy to Clipboard

5.

We'd like to find out if the average `thalach` of a heart disease patient is *significantly different* from the average `thalach` for a person without heart disease.

Import the statistical test from `scipy.stats` that we would use to test the following null and alternative hypotheses:

Null: The average `thalach` for a person with heart disease is equal to the average `thalach` for a person without heart disease.

Alternative: The average `thalach` for a person with heart disease is NOT equal to the average `thalach` for a person without heart disease.

This requires a two-sample t-test, which can be implemented using the `ttest_ind()` function from `scipy.stats`. Import the function as follows:

```
from scipy.stats import ttest_ind
```

Copy to Clipboard

6.

Run the hypothesis test from task 5 and print out the p-value. Using a significance threshold of 0.05, is there a significant difference in average `thalach` for people with heart disease compared to people with no heart disease?

The inputs to `ttest_ind()` are the values of `thalach` for each group (no heart disease and heart disease). There are two outputs: the first output is the t-statistic and the second output is the p-value.

If you run the hypothesis test correctly, you should get a p-value of `3.456964908430172e-14`, which is equivalent to `0.000000000000003457`. This is less than 0.05, so we "reject the null hypothesis" and conclude that there is a significant difference in `thalach` for people with heart disease compared to people without heart disease.

7.

Using the same process, investigate at least one other quantitative variable. Options include `age`, `trestbps` (resting blood pressure), and `chol` (cholesterol). Are any of these variables also significantly associated with heart disease?

Note: before every new plot that you make, be sure to use `plt.clf()` to clear the previous plot first, so that plots don't get layered on top of each other. For example:

```
# first box plot:
sns.boxplot(x=heart.heart_disease, y=heart.thalach)
plt.show()

# second box plot:
plt.clf()
sns.boxplot(x=heart.heart_disease, y=heart.age)
plt.show()
```

Copy to Clipboard

Using a 0.05 significance threshold, both `age` ($p = 8.955636917529706e-05$) and `trestbps` ($p = 0.008548268928594928$) are significantly associated with heart disease. `chol` is not significantly associated with heart disease ($p = 0.13914167020436527$).

Chest Pain and Max Heart Rate

8.

Next, let's investigate the relationship between `thalach` (maximum heart rate achieved during exercise) and the type of heart pain a person experiences. Create a set of side-by-side box plots of `thalach` for each chest pain type in the data. Make sure to use `plt.clf()` to clear the previous plots first!

Are there any chest pain types for which average `thalach` is significantly higher or lower (compared to other chest pain types)?

Fill in the following code:

```
plt.clf()
sns.boxplot(x = heart. , y = heart. )
```

```
plt.show()
```

Copy to Clipboard

9.

To investigate this further, save the values of `thalach` for patients who experienced each type of chest pain as `thalach_typical`, `thalach_asymptom`, `thalach_nonangin`, and `thalach_atypical`, respectively.

Below is the syntax to create `thalach_typical`. The other variables can be created using similar syntax (the other values of `cp` are 'asymptomatic', 'non-anginal pain', and 'atypical angina').

```
thalach_typical = heart.thalach[heart.cp == 'typical angina']
```

Copy to Clipboard

10.

Run a single hypothesis test to address the following null and alternative hypotheses:

Null: People with typical angina, non-anginal pain, atypical angina, and asymptomatic people all have the same average `thalach`.

Alternative: People with typical angina, non-anginal pain, atypical angina, and asymptomatic people **do not** all have the same average `thalach`.

Save the resulting p-value as `pval` and print it out. Using a significance threshold of 0.05, is there at least one pair of chest pain categories for which people in those categories have significantly different `thalach`?

This test addresses an association between two variables: a non-binary categorical variable (`cp`, with four possible options) and a quantitative variable (`thalach`). We *could* use six separate two-sample t-tests here, but since we've been asked to use a single hypothesis test, we should use an ANOVA.

To do this, we first need to import the `f_oneway()` function:

```
from scipy.stats import f_oneway
```

Copy to Clipboard

Then, we need to input the `thalach` values for each `cp` type (fill in the code below):

```
Fstat, pval = f_oneway(thalach_typical, ___, ___, ___)  
print(pval)
```

Copy to Clipboard

11.

If you completed the previous step correctly, you should have concluded that there is at least one pair of chest pain types (`cp`) for which people with those pain types have significantly different average max heart rates during exercise (`thalach`). Run another hypothesis test to determine which of those pairs are significantly different. Use an overall type I error rate of 0.05 for all six comparisons.

For this test, we need Tukey's range test, which can be implemented with `pairwise_tukeyhsd`. First we need to import the function:

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd
```

Copy to Clipboard

The inputs to this function are our two variables of interest: the max heart rate during exercise (`thalach`) and chest pain type (`cp`). Fill in the following code:

```
results = pairwise_tukeyhsd(endog = ___, groups = ___)  
print(results)
```

Copy to Clipboard

For any pair where "Reject" is "True", we conclude that people with those chest pain types have significantly different maximum heart rates during exercise. Note: you may need to expand the output window to see the full chart.

Maybe surprisingly, people who are 'asymptomatic' seem to have a lower maximum heart rate (associated with heart disease) than people who have other kinds of chest pain.

Heart Disease and Chest Pain

12.

Finally, let's investigate the relationship between the kind of chest pain a person experiences and whether or not they have heart disease. Create a contingency table of `cp` and `heart_disease` and save it as `Xtab`, then print it out.

Fill in the following code:

```
Xtab = pd.crosstab(heart.___, heart.___)  
print(Xtab)
```

Copy to Clipboard

Based on this table, it seems like people who are diagnosed with heart disease tend to be asymptomatic, while people with who are not diagnosed with heart disease tend to have different kinds of chest pain. This helps explain our findings on the relationship between chest pain and max heart rate above. However, we'd have to do more research to try to understand why this is the case. It seems a little counter-intuitive!

13.

Run a hypothesis test for the following null and alternative hypotheses:

Null: There is NOT an association between chest pain type and whether or not someone is diagnosed with heart disease.

Alternative: There is an association between chest pain type and whether or not someone is diagnosed with heart disease.

Save the p-value as `pval` and print it out. Using a significance threshold of 0.05, is there a significant association between chest pain type and whether or not someone is diagnosed with heart disease?

This test investigates an association between two categorical variables, so we need to use a Chi-Square test. First, we can import the `chi2_contingency()` function as follows:

```
from scipy.stats import chi2_contingency
```

Copy to Clipboard

Then we can use that function to run our test. The `chi2_contingency()` function has four outputs (the p-value is the second output). It takes one input that you'll need to fill in below — the contingency table:

```
chi2, pval, dof, exp = chi2_contingency(____)
```

Copy to Clipboard

If you run the test correctly, you should calculate a p-value of `1.2517106007837527e-17`, which is equivalent to `0.0000000000000000125`. This is less than 0.05, so we can conclude that there is a significant association between these variables.

Further Exploration

14.

Congratulations! You've used a sample of data to understand how health related outcomes are associated with heart disease. Now that you've gotten your feet wet, there are a number of additional variables in this dataset that we haven't looked at yet! If you want additional practice, use this space to continue your investigation. What other variables might be predictive of heart disease? Can you identify which hypothesis test to use for each variable?

As a reminder, the variables in this data are:

age: age in years

sex: sex assigned at birth; 'male' or 'female'

trestbps: resting blood pressure in mm Hg

chol: serum cholesterol in mg/dl

cp: chest pain type ('typical angina', 'atypical angina', 'non-anginal pain', or 'asymptomatic')

exang: whether the patient experiences exercise-induced angina (1: yes; 0: no)

fbs: whether the patient's fasting blood sugar is >120 mg/dl (1: yes; 0: no)

thalach: maximum heart rate achieved in exercise test

heart_disease: whether the patient is found to have heart disease ('presence': diagnosed with heart disease; 'absence': no heart disease)

SOLUTION

```
# import libraries
import codecademylib3
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# load data
heart = pd.read_csv('heart_disease.csv')

# inspect data
print(heart.head())

# box plot of `thalach` based on heart disease
```

```

sns.boxplot(x=heart.heart_disease, y=heart.thalach)
plt.show()

# save `thalach` for hd patients and non-hd patients
thalach_hd = heart.thalach[heart.heart_disease == 'presence']
thalach_no_hd = heart.thalach[heart.heart_disease == 'absence']

# calculate and print mean difference
mean_diff = np.mean(thalach_no_hd) - np.mean(thalach_hd)
print('\`thalach` mean Difference: ', mean_diff)

# calculate and print median difference
med_diff = np.median(thalach_no_hd) - np.median(thalach_hd)
print('\`thalach` median Difference: ', med_diff)

# run two-sample t-test
from scipy.stats import ttest_ind
tstat, pval = ttest_ind(thalach_hd, thalach_no_hd)
print('p-value for `thalach` two-sample t-test: ', pval)

# investigating other quantitative variables
#age
plt.clf()
sns.boxplot(x=heart.heart_disease, y=heart.age)
plt.show()
age_hd = heart.age[heart.heart_disease == 'presence']
age_no_hd = heart.age[heart.heart_disease == 'absence']
mean_diff = np.mean(age_hd) - np.mean(age_no_hd)
print('\`age` mean Difference: ', mean_diff)
med_diff = np.median(age_hd) - np.median(age_no_hd)
print('\`age` median Difference: ', med_diff)
tstat, pval = ttest_ind(age_hd, age_no_hd)
print('p-value for `age` two-sample t-test: ', pval)

#trestbps
plt.clf()
sns.boxplot(x=heart.heart_disease, y=heart.trestbps)
plt.show()
trestbps_hd = heart.trestbps[heart.heart_disease == 'presence']
trestbps_no_hd = heart.trestbps[heart.heart_disease == 'absence']
mean_diff = np.mean(trestbps_hd) - np.mean(trestbps_no_hd)
print('\`trestbps` mean Difference: ', mean_diff)
med_diff = np.median(trestbps_hd) - np.median(trestbps_no_hd)
print('\`trestbps` median Difference: ', med_diff)
tstat, pval = ttest_ind(trestbps_hd, trestbps_no_hd)
print('p-value for `trestbps` two-sample t-test: ', pval)

#chol
plt.clf()
sns.boxplot(x=heart.heart_disease, y=heart.chol)
plt.show()
chol_hd = heart.chol[heart.heart_disease == 'presence']
chol_no_hd = heart.chol[heart.heart_disease == 'absence']
mean_diff = np.mean(chol_hd) - np.mean(chol_no_hd)
print('\`chol` mean Difference: ', mean_diff)
med_diff = np.median(chol_hd) - np.median(chol_no_hd)
print('\`chol` median Difference: ', med_diff)
tstat, pval = ttest_ind(chol_hd, chol_no_hd)
print('p-value for `chol` two-sample t-test: ', pval)

```

```

# box plot of `thalach` based on `cp`
plt.clf()
sns.boxplot(x=heart.cp, y=heart.thalach)
plt.show()

# save `thalach` based on `cp`
thalach_typical = heart.thalach[heart.cp == 'typical angina']
thalach_asymptom = heart.thalach[heart.cp == 'asymptomatic']
thalach_nonangin = heart.thalach[heart.cp == 'non-anginal pain']
thalach_atypical = heart.thalach[heart.cp == 'atypical angina']

# run ANOVA
from scipy.stats import f_oneway
Fstat, pval = f_oneway(thalach_typical, thalach_asymptom, thalach_nonangin,
thalach_atypical)
print('p-value for ANOVA: ', pval)

# run Tukey's range test
from statsmodels.stats.multicomp import pairwise_tukeyhsd
output = pairwise_tukeyhsd(heart.thalach, heart.cp)
print(output)

# contingency table of heart disease vs cp
Xtab = pd.crosstab(heart.cp, heart.heart_disease)
print(Xtab)

# run chi-square test
from scipy.stats import chi2_contingency
chi2, pval, dof, exp = chi2_contingency(Xtab)
print('p-value for chi-square test: ', pval)

```

A/B Testing at Nosh Mish Mosh

The Nosh Mish Mosh is a recipe and ingredient meal delivery service. They ship the raw materials and you get to cook them at your home! They've decided to hire a data analyst to help make product and interface decisions. Get started to help them figure out the amount of data they'll need to make meaningful decisions.

Note that a **solution.py** file is loaded for you in the workspace, which contains solution code for this project. We highly recommend that you complete the project on your own without checking the solution, but feel free to take a look if you get stuck or want to check your answers when you're done!`

Tasks

0/16 complete

[Mark the tasks as complete by checking them off](#)

Nosh Mish Mosh: An Assortment of Edible Aliments

1.

We've collected customer data for the past week and exposed it through a Python library, so first import `noshmishmosh`.

Hint

Importing a module in Python is as simple as

```
import noshmishmosh
```

Copy to Clipboard

2.

Next, we'll need to do a little bit of data analysis — let's use `numpy` to help. Import `numpy` into your workspace as `np`.

Hint

In the pursuit of brevity, we like to rename the NumPy to something a little more manageable.

```
import numpy as np
```

Copy to Clipboard

A/B Testing at Nosh Mish Mosh

3.

Nosh Mish Mosh wants to run an experiment to see if we can convince more people to purchase meal plans if we use a more artisanal-looking vegetable selection. We've photographed these modern meals with blush tomatoes and graffiti eggplants, but aren't sure if this strategy will sell enough units to benefit from establishing a business relationship with a new provider.

Before running this experiment, of course, we need to know the sample size that will be required to detect the difference we are hoping for. There are three things we need to know before we can determine that number.

the **Baseline Conversion Rate**

Minimum Detectable Effect (desired lift)

and the **Statistical Significance Threshold**

4.

Let's get the ball rolling on finding those numbers! In order to get our baseline, we need to first know how many users visit the site in a typical week. Let's grab that logged information, which is stored in `noshmishmosh.customer_visits`. Assign that to a new variable called `all_visitors`.

Hint

```
all_visitors= noshmishmosh.customer_visits
```

Copy to Clipboard

5.

Next we need to know how many visitors to the site ultimately end up buying a meal or set of meals in a typical week. We have that information saved into `purchasing_customers` field on `noshmishmosh`. Save that information into a variable called `paying_visitors`.

Hint

```
paying_visitors = noshmishmosh.purchasing_customers
```

Copy to Clipboard

6.

Calculate the lengths of the two lists, saving the results into variables called `total_visitor_count` and `paying_visitor_count`, respectively.

Hint

Calculate length using the `len` function:

```
total_visitor_count = len(all_visitors)
paying_visitor_count = len(paying_visitors)
```

Copy to Clipboard

7.

Now to get the baseline: Divide the number of purchasing visitors by the number of total visitors. Save the result in a variable called `baseline_percent`. Since we want a percentage as our answer, multiply the result by `100.0`.

Hint

```
baseline_percent = paying_visitor_count / total_visitor_count * 100
```

Copy to Clipboard

8.

Print out the `baseline_percent` so we know what to use for our baseline percentage in the A/B Sample Size Calculator.

Hint

```
print(baseline_percent)
```

Copy to Clipboard

Mish Mosh B'Gosh: The Effect Size

9.

These rainbow fingerling potatoes don't come cheap. We'd like to know for sure that, with this change, we'll be pulling in at least \$1240 more every week. In order to figure out how many more customers we need, we'll have to investigate the average revenue generated from a given sale. Luckily we have a list of the money spent by each customer in a typical week: `noshmishmosh.money_spent`. Save that list into a variable called `payment_history`.

Hint

```
payment_history = noshmishmosh.money_spent
```

Copy to Clipboard

10.

We need to find how many purchases it would take to reach \$1240 in additional revenue using our historical data. Let's start with computing the average payment per paying customer using `np.mean`, saving it as `average_payment`.

Hint

If you shortened `numpy` to `np` this would be:

```
average_payment = np.mean(payment_history)
```

Copy to Clipboard

11.

We want to know how many of these "usual" payments it would take to clear our \$1240 mark. Round the number up using `np.ceil` (because that's how many new customers it takes to bring in more than \$1240). Save that value into a `new_customers_needed` variable.

Hint

It's possible to print out the divided value and round up ourselves but there is a function in Python's `numpy` library that can do this for us:

```
new_customers_needed = np.ceil(1240 / average_payment)
```

Copy to Clipboard

12.

Now find the additional percent of weekly visitors who must make a purchase in order to make this change worthwhile. Do this by dividing the number of customers by the total visitor count for a typical week (calculated earlier), and multiplying by 100. Save the result in a variable called `percentage_point_increase`. Print `percentage_point_increase` to see what it is.

Hint

```
percentage_point_increase = new_customers_needed / total_visitor_count * 100
```

Copy to Clipboard

13.

In order to find our minimum detectable effect/desired lift, we need to express `percentage_point_increase` as a percent of `baseline_percent`. You can do this by dividing `percentage_point_increase` by `baseline_percent` and multiplying by `100.0`.

Store the results in a variable called `mde`.

Hint

We've calculated a lot of percentages so far in this project, and the fact that these are both already percentages does not change the formula.

```
mde = percentage_point_increase / baseline_percent * 100
```

Copy to Clipboard

14.

Print out the result `mde`.

Hint

```
print(mde)
```

Copy to Clipboard

Nosh Mish Mosh: Tying It All Together

15.

The last thing we need to calculate the sample size for Nosh Mish Mosh's artisanal rebranding is our statistical significance threshold. We'd like to be fairly certain, but this isn't going to be a million dollar decision, so let's go with 10%.

16.

Now put it all together! Punch the *baseline*, the *minimum detectable effect*, and the *statistical significance threshold* into the calculator and evaluate how many people need to be shown the new assets before we can check if the results are a significant improvement. Save the results in a variable called `ab_sample_size`.

SOLUTION:

```
# Step 1
import noshmishmosh

# Step 2
import numpy as np

# Step 4
all_visitors = noshmishmosh.customer_visits

# Step 5
paying_visitors = noshmishmosh.purchasing_customers

# Step 6
total_visitor_count = len(all_visitors)
paying_visitor_count = len(paying_visitors)

# Step 7
baseline_percent = paying_visitor_count / total_visitor_count * 100

# Step 8
print("Baseline percent:")
print(baseline_percent)

# Step 9
payment_history = noshmishmosh.money_spent

# Step 10
average_payment = np.mean(payment_history)

# Step 11
new_customers_needed = np.ceil(1240 / average_payment)

# Step 12
percentage_point_increase = new_customers_needed / total_visitor_count * 100
print("Percentage point increase:")
print(percentage_point_increase)

# Step 13
mde = percentage_point_increase / baseline_percent * 100

# Step 14
print("Minimum Detectable Effect:")
print(mde)
```

```
# Step 16  
ab_sample_size = 490
```

Familiar: A Study In Data Analysis

Welcome to *Familiar*, a startup in the new market of blood transfusion! You've joined the team because you appreciate the flexible hours and extremely intelligent team, but the overeager doorman welcoming you into the office is a nice way to start your workday (well, work-evening).

Familiar has fallen into some tough times lately, so you're hoping to help them make some insights about their product and help move the needle (so to speak).

Note that a **solution.py** file is also loaded for you in the workspace, which contains solution code for this project. We highly recommend that you complete the project on your own without checking the solution, but feel free to take a look if you get stuck or want to check your answers!

Tasks

0/13 complete

Mark the tasks as complete by checking them off

What Can Familiar Do For You?

1.

The Familiar team has provided us with some data on lifespans for subscribers to two different packages, the Vein Pack and the Artery Pack! This data has been loaded for you as a dataframe named `lifespans`. Use the `.head()` method to print out the first five rows and take a look!

Hint

Print `lifespans.head()` and inspect the output.

2.

The first thing we want to know is whether Familiar's most basic package, the Vein Pack, actually has a significant impact on the subscribers. It would be a marketing goldmine if we can show that subscribers to the Vein Pack live longer than other people.

Extract the life spans of subscribers to the 'vein' pack and save the data into a variable called `vein_pack_lifespans`.

Stuck? Get a hint

3.

Next, use `np.mean()` to calculate the average lifespan for Vein Pack subscribers and print the result. Is it longer than 73 years?

Hint

Fill in the following code:

```
print(np.mean(____))
```

Copy to Clipboard

4.

We'd like to find out if the average lifespan of a Vein Pack subscriber is *significantly different* from the average life expectancy of 73 years.

Import the statistical test from `scipy.stats` that we would use to test the following null and alternative hypotheses:

Null: The average lifespan of a Vein Pack subscriber is 73 years.

Alternative: The average lifespan of a Vein Pack subscriber is NOT 73 years.

Hint

We have a single sample of lifespans and we want to compare the mean of this sample to a hypothetical population value of 73 years. Lifespans are a quantitative variable. The following table may be helpful:

	Comparing a sample statistic to a hypothesized population value for a...	Testing an association between a binary categorical variable (2 categories) and a...	Testing an association between a categorical variable with 3 or more categories and a...
Quantitative Variable	<i>One sample t-test</i>	<i>Two-Sample t-test</i>	<i>ANOVA with Tukey's Range Test</i>
Categorical Variable	<i>Binomial Test</i>	<i>Chi-Square Test</i>	

If you remember the name of the test but not the name of the function, try googling "how to run ____ test in Python" and see if you can find the function yourself!

5.

Now that you've imported the function you need, run the significance test and print out the p-value! Is the average lifespan of a Vein Pack subscriber significantly longer than 73 years? Use a significance threshold of 0.05.

Hint

If you've run your test correctly, you should get a p-value of `5.972157921433082e-07`, which is written in scientific notation and therefore equivalent to `0.000000597`. This is much smaller than 0.05, so we conclude that the average lifespan of Vein Pack subscribers IS significantly different from 73 years.

Upselling Familiar: Pumping Life Into The Company

6.

In order to differentiate Familiar's different product lines, we'd like to compare this lifespan data between our different packages. Our next step up from the Vein Pack is the Artery Pack.

Let's get the lifespans of Artery Pack subscribers. Using the same `lifespans` dataset, extract the lifespans of subscribers to the Artery Pack and save them as `artery_pack_lifespans`.

Hint

Adapt the syntax that you used to get the Vein Pack lifespans:

```
artery_pack_lifespans = lifespans.lifespan[lifespans.pack=='__']
```

Copy to Clipboard

7.

Use `np.mean()` to calculate the average lifespan for Artery Pack subscribers and print the result. Is it longer than for the Vein Pack?

Hint

Fill in the following code:

```
print(np.mean(__))
```

Copy to Clipboard

8.

We'd like to find out if the average lifespan of a Vein Pack subscriber is *significantly different* from the average life expectancy for the Artery Pack.

Import the statistical test from `scipy.stats` that we would use to test the following null and alternative hypotheses:

Null: The average lifespan of a Vein Pack subscriber is equal to the average lifespan of an Artery Pack subscriber.

Alternative: The average lifespan of a Vein Pack subscriber is NOT equal to the average lifespan of an Artery Pack subscriber.

Hint

For this test, we're really looking at an association between two variables: whether a subscriber got the Vein Pack or Artery Pack (a binary categorical variable) and their lifetime (a quantitative variable). The following table may be helpful:

	Comparing a sample statistic to a hypothesized population value for a...	Testing an association between a binary categorical variable (2 categories) and a...	Testing an association between a categorical variable with 3 or more categories and a...
Quantitative Variable	<i>One sample t-test</i>	<i>Two-Sample t-test</i>	<i>ANOVA with Tukey's Range Test</i>
Categorical Variable	<i>Binomial Test</i>	<i>Chi-Square Test</i>	

If you remember the name of the test but not the name of the function, try googling "how to run ____ test in Python" and see if you can find the function yourself!

9.

Now that you've imported the function you need, run the significance test and print out the p-value! Is the average lifespan of a Vein Pack subscriber significantly different from the average lifespan of an Artery Pack subscriber? Use a significance threshold of 0.05.

Hint

If you've run your test correctly, you should get a p-value of `0.05588883079070819`. This is larger than 0.05, so we conclude that the average lifespan of Vein Pack subscribers not significantly different from the average lifespan of an Artery Pack subscriber.

Side Effects: A Familiar Problem

10.

The Familiar team has provided us with another dataset containing survey data about iron counts for our subscribers. This data has been pre-processed to categorize iron counts as "low", "normal", and "high" for each subscriber. Familiar wants to be able to advise potential subscribers about possible side effects of these packs and whether they differ for the Vein vs. the Artery pack.

The data has been loaded for you as a dataframe named `iron`. Use the `.head()` method to print out the first five rows and take a look!

Hint

Print `iron.head()` and inspect the output.

11.

Is there an association between the pack that a subscriber gets (Vein vs. Artery) and their iron level? Use the pandas `crosstab()` function to create a contingency table of the `pack` and `iron` columns in the `iron` data. Save the result as `xtab` and print it out.

Hint

Fill in the following code:

```
Xtab = pd.crosstab(iron.___, iron.___)
print(Xtab)
```

Copy to Clipboard

Based on this table, it seems like Artery Pack subscribers tend to have high iron, whereas Vein Pack subscribers tend to have low iron.

12.

We'd like to find out if there is a significant association between which pack (Vein vs. Artery) someone subscribes to and their iron level.

Import the statistical test from `scipy.stats` that we would use to test the following null and alternative hypotheses:

Null: There is NOT an association between which pack (Vein vs. Artery) someone subscribes to and their iron level.

Alternative: There is an association between which pack (Vein vs. Artery) someone subscribes to and their iron level.

Hint

For this test, we're interested in an association between two categorical variables: whether a subscriber got the Vein Pack or Artery Pack and their iron level. The following table may be helpful:

	Comparing a sample statistic to a hypothesized population value for a...	Testing an association between a binary categorical variable (2 categories) and a...	Testing an association between a categorical variable with 3 or more categories and a...
Quantitative Variable	<i>One sample t-test</i>	<i>Two-Sample t-test</i>	<i>ANOVA with Tukey's Range Test</i>
Categorical Variable	<i>Binomial Test</i>	<i>Chi-Square Test</i>	

If you remember the name of the test but not the name of the function, try googling "how to run ___ test in Python" and see if you can find the function yourself!

13.

Now that you've imported the function you need, run the significance test and print out the p-value! Is there a significant association between which pack (Vein vs. Artery) someone subscribes to and their iron level? Use a significance threshold of 0.05.

Hint

If you've run your test correctly, you should get a p-value of $9.359749337433008 \times 10^{-25}$, which is equivalent to 0.000000000000000000000000936. This is smaller than 0.05, so we conclude that there IS a significant association between pack and iron level.

SOLUTIONS:

```
# Import libraries
import pandas as pd
import numpy as np

# Load datasets
lifespans = pd.read_csv('familiar_lifespan.csv')
iron = pd.read_csv('familiar_iron.csv')

# Print lifespan data
print(lifespans.head())

# Save lifespans for vein pack subscribers
vein_pack_lifespans = lifespans.lifespan[lifespans.pack=='vein']

# Calculate average lifespan for vein pack
print(np.mean(vein_pack_lifespans))

# Run one-sample t-test
from scipy.stats import ttest_1samp
tstat, pval = ttest_1samp(vein_pack_lifespans, 73)
print(pval)

# Save lifespans for artery pack subscribers
artery_pack_lifespans = lifespans.lifespan[lifespans.pack=='artery']

# Calculate artery pack life spans
print(np.mean(artery_pack_lifespans))

# Run two-sample t-test
from scipy.stats import ttest_ind
tstat, pval = ttest_ind(vein_pack_lifespans, artery_pack_lifespans)
print(pval)

# Inspect first 5 rows of iron dataset
print(iron.head())

# Create contingency table
Xtab = pd.crosstab(iron.pack, iron.iron)
print(Xtab)

# Run Chi-Square test
from scipy.stats import chi2_contingency
chi2, pval, dof, exp = chi2_contingency(Xtab)
print(pval)
```

FetchMaker

Congratulations! You've just started working at the hottest new tech startup, FetchMaker. FetchMaker's mission is to match up prospective dog owners with their perfect pet. FetchMaker has been collecting data on their adoptable dogs, and it's your job to analyze some of that data.

Note that a **solution.py** file is also loaded for you in the workspace, which contains solution code for this project. We highly recommend that you complete the project on your own without checking the solution, but feel free to take a look if you get stuck or want to check your answers!

Tasks

0/11 complete

[Mark the tasks as complete by checking them off](#)

Data to the Rescue

1.

FetchMaker has provided us with data for a sample of dogs from their app, including the following attributes:

- `weight`, an integer representing how heavy a dog is in pounds
- `tail_length`, a float representing tail length in inches
- `age`, in years
- `color`, a String such as "brown" or "grey"
- `is_rescue`, a boolean 0 or 1

The data has been saved for you as a pandas DataFrame named `dogs`. Use the `.head()` method to inspect the first five rows of the dataset.

Hint

Print `dogs.head()`

2.

FetchMaker estimates (based on historical data for all dogs) that 8% of dogs in their system are rescues. They would like to know if whippets are significantly more or less likely than other dogs to be a rescue. Store the `is_rescue` values for 'whippet's in a variable called `whippet_rescue`.

Hint

Fill in the following code:

```
whippet_rescue = dogs.is_rescue[dogs.breed == '___']
```

Copy to Clipboard

3.

How many whippets are rescues (remember that the value of `is_rescue` is 1 for rescues and 0 otherwise)? Save this number as `num_whippet_rescues` and print it out.

Hint

Fill in the following code:

```
num_whippet_rescues = np.sum(whippet_rescue == ___)
```

Copy to Clipboard

OR

Use `np.count_nonzero()` to count the number of non-zero values in `whippet_rescue`

4.

How many whippets are in this sample of data in total? Save this number as `num_whippets` and print it out.

Hint

The number of whippets is equal to the length of `whippet_rescue`. You can calculate length using `len()`.

5.

Use a hypothesis test to test the following null and alternative hypotheses:

- Null: 8% of whippets are rescues

- Alternative: more or less than 8% of whippets are rescues

Save the p-value from this test as `pval` and print it out. Using a significance threshold of 0.05, Is the proportion of whippets who are rescues significantly different from 8%?

Hint

For this test, we are focused on a single binary categorical variable, which indicates whether or not each whippet is a rescue. We want to compare the number of rescues in our sample to a hypothetical population-level proportion of 0.08. Therefore, we should use a binomial test.

First, we need to import the `binom_test()` function:

```
from scipy.stats import binom_test
```

Copy to Clipboard

Then, we need to use the function to run our test (fill in the code below):

```
pval = binom_test(x = __, n = __, p = __)
```

Copy to Clipboard

In general, `x` is the number of 'successes', `n` is the sample size, and `p` is the null probability of 'success'.

Mid-Sized Dog Weights

6.

Three of FetchMaker's most popular mid-sized dog breeds are 'whippet's, 'terrier's, and 'pitbull's. Is there a significant difference in the average weights of these three dog breeds?

To start answering this question, save the weights of each of these breeds in three separate series named `wt_whippets`, `wt_terriers`, and `wt_pitbulls`, respectively.

Hint

The syntax for whippets is as follows:

```
wt_whippets = dogs.weight[dogs.breed == 'whippet']
```

Copy to Clipboard

7.

Run a single hypothesis test to address the following null and alternative hypotheses:

Null: whippets, terriers, and pitbulls all weigh the same amount on average

Alternative: whippets, terriers, and pitbulls **do not** all weigh the same amount on average (at least one pair of breeds has differing average weights)

Save the resulting p-value as `pval` and print it out. Using a significance threshold of 0.05, is there at least one pair of dog breeds that have significantly different average weights?

Hint

This test addresses an association between two variables: a non-binary categorical variable (breed, with three possible options) and a quantitative variable (weight). It is not a good idea to run three separate two-sample t-tests here, because running multiple t-tests increases our chances of a type I error, or a false positive. In order to run a single hypothesis test with three categories, we should use an ANOVA.

To do this, we first need to import the `f_oneway()` function:

```
from scipy.stats import f_oneway
```

Copy to Clipboard

Then, we need to input the weights of each breed (fill in the code below):

```
Fstat, pval = f_oneway(__, __, __)
print(pval)
```

Copy to Clipboard

8.

If you completed the previous step correctly, you should have concluded that at least one pair of dog breeds have significantly different average weights.

Run another hypothesis test to determine which of those breeds (whippets, terriers, and pitbulls) weigh different amounts on average. Use an overall type I error rate of 0.05 for all three comparisons. Note that we've already provided you with code in `script.py` to subset the data to just these breeds and have saved this subset as `dogs_wtp` using the following code:

```
dogs_wtp = dogs[dogs.breed.isin(['whippet', 'terrier', 'pitbull'])]
```

Copy to Clipboard

This should make it easier for you to run the test you need!

Print out the results. Which pairs of dog breeds weigh different amounts?

Hint

For this test, we need Tukey's range test, which can be implemented with `pairwise_tukeyhsd`. First we need to import the function:

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd
```

Copy to Clipboard

The inputs to this function are our two variables of interest: the weights and breeds of the dogs. We can use the subsetted data to pull out these variables (`dogs_wtp.weight` and `dogs_wtp.breed`), then input them into the function (fill in the code):

```
results = pairwise_tukeyhsd(endog = ___, groups = ___)
print(results)
```

Copy to Clipboard

For any pair where "Reject" is "True", we conclude that those two breeds weigh significantly different amounts.

Poodle and Shih-tzu Colors

9.

FetchMaker wants to know if 'poodle's and 'shih-tzu's come in different colors. Note that we've already provided you with code in `script.py` to subset the data to just these breeds and have saved this subset as `dogs_ps` using the following code:

```
dogs_ps = dogs[dogs.breed.isin(['poodle', 'shih-tzu'])]
```

Copy to Clipboard

This should make it easier for you to investigate this question!

To start, use the subsetted data to create a contingency table of dog colors by breed (poodle vs. shih-tzu). Save the table as `xtab` and print it out.

Hint

Use the pandas `crosstab()` function (fill in the code below):

```
xtab = pd.crosstab(___, ___)
print(xtab)
```

Copy to Clipboard

Fill in the relevant columns. It does not matter in which order.

10.

Run a hypothesis test for the following null and alternative hypotheses:

Null: There is an association between breed (poodle vs. shih-tzu) and color.

Alternative: There is not an association between breed (poodle vs. shih-tzu) and color.

Save the p-value as `pval` and print it out. Do poodles and shih-tzus come in significantly different color combinations? Use a significance threshold of 0.05.

Hint

This test investigates an association between two categorical variables, so we can use a Chi-Square test. First, we can import the `chi2_contingency()` function as follows:

```
from scipy.stats import chi2_contingency
```

Copy to Clipboard

Then we can use that function to run our test. The `chi2_contingency()` function has four outputs and the p-value is the second one. It takes one input that you'll need to fill in below — the contingency table:

```
chi2, pval, dof, exp = chi2_contingency(___)
```

Copy to Clipboard

Good learner! Have a treat!

11.

Great job!

Feel free to play around with the FetchMaker data some more and run some hypothesis tests of your own.

The breeds you can explore are "poodle", "rottweiler", "whippet", "greyhound", "terrier", "chihuahua", "shih-tzu", and "pitbull".

Extra challenge: Remind yourself of your **data visualization** skills and your ability to describe the **central tendency** of the data. For example, a boxplot visualization can add a lot to your understanding of an ANOVA result.

Solutions

```
# Import libraries
import numpy as np
import pandas as pd
import codecademylib3

# Import data
dogs = pd.read_csv('dog_data.csv')
```

```

# Inspect first few rows of data
print(dogs.head())

# Save the is_rescue column for whippets
whippet_rescue = dogs.is_rescue[dogs.breed == 'whippet']

# Calculate and print the number of whippet rescues
num_whippet_rescues = np.sum(whippet_rescue == 1)
print(num_whippet_rescues)

# Calculate and print the number of whippets
num_whippets = len(whippet_rescue)
print(num_whippets)

# Run a binomial test
from scipy.stats import binom_test
pval = binom_test(num_whippet_rescues, num_whippets, .08)
print(pval)

# Save the weights of whippets, terriers, and pitbulls
wt_whippets = dogs.weight[dogs.breed == 'whippet']
wt_terriers = dogs.weight[dogs.breed == 'terrier']
wt_pitbulls = dogs.weight[dogs.breed == 'pitbull']

# Run an ANOVA
from scipy.stats import f_oneway
Fstat, pval = f_oneway(wt_whippets, wt_terriers, wt_pitbulls)
print(pval)

# Subset to just whippets, terriers, and pitbulls
dogs_wtp = dogs[dogs.breed.isin(['whippet', 'terrier', 'pitbull'])]

# Run Tukey's Range Test
from statsmodels.stats.multicomp import pairwise_tukeyhsd
output = pairwise_tukeyhsd(dogs_wtp.weight, dogs_wtp.breed)
print(output)

# Subset to just poodles and shihtzus
dogs_ps = dogs[dogs.breed.isin(['poodle', 'shihtzu'])]

# Create a contingency table of color vs. breed
Xtab = pd.crosstab(dogs_ps.color, dogs_ps.breed)
print(Xtab)

# Run a Chi-Square Test
from scipy.stats import chi2_contingency
chi2, pval, dof, exp = chi2_contingency(Xtab)
print(pval)

```


Analyzing Farmburg's A/B Test

Brian is a Product Manager at FarmBurg, a company that makes a farming simulation social network game. In the FarmBurg game, you can plow, plant, and harvest different crops. Brian has been conducting an A/B Test with three different variants, and he wants you to help him analyze the results. Using the Python modules pandas and SciPy, you will help him make some important business decisions!

Note that a `solution.py` file is also loaded for you in the workspace, which contains solution code for this project. We highly recommend that you complete the project on your own without checking the solution, but feel free to take a look if you get stuck or want to check your answers!

Tasks

0/13 complete

[Mark the tasks as complete by checking them off](#)

Project Requirements

1.

Brian ran an A/B test with three different groups: A, B, and C. He has provided us with a CSV file of his results named `clicks.csv`. It has the following columns:

`user_id`: a unique id for each visitor to the FarmBurg site

`group`: either 'A', 'B', or 'C' depending on which group the visitor was assigned to

`is_purchase`: either 'Yes' if the visitor made a purchase or 'No' if they did not.

We've already imported pandas as `pd` and loaded `clicks.csv` as `abdata`. Inspect the data using the `.head()` method.

Hint

Fill in the following code:

```
print(abdata.____())
```

Copy to Clipboard

2.

Note that we have two categorical variables: `group` and `is_purchase`. We are interested in whether visitors are more likely to make a purchase if they are in any one group compared to the others. Because we want to know if there is an association between two categorical variables, we'll start by using a Chi-Square test to address our question.

In order to run a Chi-Square test, we first need to create a contingency table of the variables `group` and `is_purchase`. Use `pd.crosstab()` to create this table and name the result `xtab`, then print it out.

Which group appears to have the highest number of purchases?

Hint

Fill in the following code:

```
xtab = pd.crosstab(abdata.____, abdata.____)
print(xtab)
```

Copy to Clipboard

Group A has the highest number of purchases, with 316 purchases.

3.

To conduct the Chi-Square Test, import `chi2_contingency` from `scipy.stats`.

Then, use the function `chi2_contingency` with the data in `xtab` to calculate the p-value. Remember that of the four values returned by `chi2_contingency`, the p-value is the second value.

Save the p-value to a variable named `pval` and print the result. Using a significance threshold of 0.05, is there a significant difference in the purchase rate for groups A, B, and C?

Note that you might see a number in scientific notation. For example, `1.234e-8` is equal to `0.00000001234` (we move the decimal to the left by 8 places and insert zeros).

To calculate the proportion of visits that would need to be purchases in order to meet the \$1,000 weekly goal for a \$0.99 price point, we divide the number of sales needed (calculated in the previous step as `num_sales_needed_099`) by the number of expected visits (`num_visits`):

```
p_sales_needed_099 = num_sales_needed_099/num_visits
```

Copy to Clipboard

7.

Repeat the steps from tasks 5 and 6 for the other price points (\$1.99 and \$4.99). Save the number of sales needed for each price point as `num_sales_needed_199` and `num_sales_needed_499`, respectively. Then, save the proportion of visits needed as `p_sales_needed_199` and `p_sales_needed_499`, respectively.

Print out the proportions. Note that for higher price points, you'll need to sell fewer upgrade packages in order to meet your minimum revenue target — so the proportions should decrease as the price points increase.

Hint

Use the syntax provided in the hints for tasks 6 and 7, but use the updated variable names.

8.

Now let's return to Brian's question. To start, we want to know if the percent of Group A (the \$0.99 price point) that purchased an upgrade package is *significantly* greater than `p_sales_needed_099` (the percent of visitors who need to buy an upgrade package at \$0.99 in order to make our minimum revenue target of \$1,000).

To answer this question, we want to focus on just the visitors in group A. Then, we want to compare the number of purchases in that group to `p_sales_needed_099`.

Since we have a single sample of categorical data and want to compare it to a hypothetical population value, a *binomial test* is appropriate. In order to run a binomial test for group A, we need to know two pieces of information:

The number of visitors in group A (the number of visitors who were offered the \$0.99 price point)

The number of visitors in Group A who made a purchase

Calculate these two numbers and save them as `samp_size_099` and `sales_099`, respectively. Note that you can use the contingency table that you printed earlier to get these numbers OR you can use Python syntax.

Hint

To do this with Python syntax, use the following code:

```
samp_size_099 = np.sum(abdata.group == 'A')
sales_099 = np.sum((abdata.group == 'A') & (abdata.is_purchase == 'Yes'))
```

Copy to Clipboard

Otherwise, you can look up these numbers in the contingency table you printed earlier:

is_purchase group	No	Yes
A	1350	316
B	1483	183
C	1583	83

The sample size for group A is the sum of the values in the first row (`1350 + 316`).

9.

Calculate the sample size and number of purchases in group B (the \$1.99 price point) and save them as `samp_size_199` and `sales_199`, respectively. Then do the same for group C (the \$4.99 price point) and save them as `samp_size_499` and `sales_499`, respectively.

Hint

Use the syntax from the hint for the previous task, but update the variable names appropriately.

10.

For Group A (\$0.99 price point), perform a binomial test using `binom_test()` to see if the observed purchase rate is significantly greater than `p_sales_needed_099`. Remember that there are four inputs to `binom_test()`:

`x` will be the number of purchases for Group A
`n` will be the total number of visitors assigned group A
`p` will be the target percent of purchases for the \$0.99 price point
`alternative` will indicate the alternative hypothesis for this test; in this case, we want to know if the observed purchase rate is significantly 'greater' than the purchase rate that results in the minimum revenue target.

Save the results to `pvalueA`, and print its value. Note that you'll first need to import the `binom_test()` function from `scipy.stats` using the following line of code:

```
from scipy.stats import binom_test
```

Copy to Clipboard

Hint

Fill in the following code:

```
from scipy.stats import binom_test
```

```
pvalueA = binom_test(x= __, n= __, p=__, alternative = 'greater')  
print(pvalueA)
```

Copy to Clipboard

Remember that the number of purchases in group A is saved as `sales_099`, the sample size for group A is saved as `samp_size_099`, and the target proportion of sales needed to meet the \$1000 goal at a \$0.99 price point is saved as `p_sales_needed_099`.

11.

For Group B (\$1.99 price point), perform a binomial test to see if the observed purchase rate is significantly greater than `p_sales_needed_199`.

Save the results to `pvalueB`, and print its value.

Hint

Use the same syntax as in task 10, but update the values to match the number of purchases, sample size, and target for group B.

12.

For Group C (\$4.99 price point), perform a binomial test to see if the observed purchase rate is significantly greater than `p_sales_needed_499`.

Save the results to `pvalueC`, and print its value.

Hint

Use the same syntax as in task 10, but update the values to match the number of purchases, sample size, and target for group C.

13.

Based on the three p-values you calculated for the binomial tests in each group and a significance threshold of 0.05, were there any groups where the purchase rate was significantly higher than the target? Based on this information, what price should Brian charge for the upgrade package?

Hint

`pvalueC` is the only p-value below the threshold of 0.05. Therefore, the C group is the only group where we would conclude that the purchase rate is significantly higher than the target needed to reach \$1000 revenue per week. Therefore, Brian should charge \$4.99 for the upgrade.

SOLUTIONS:

<https://www.codecademy.com/paths/data-science/tracks/dsml-statistics-fundamentals-part-ii/modules/stats-hypothesis-testing-projects/projects/analyzing-farmburgs-a-b-test>

```
# Import libraries  
import codecademylib3  
import pandas as pd  
import numpy as np  
  
# Read in the `clicks.csv` file as `abdata`  
abdata = pd.read_csv('clicks.csv')
```

```

# Inspect the dataframe
print(abdata.head())

# Create a contingency table with pd.crosstab
Xtab = pd.crosstab(abdata.group, abdata.is_purchase)

# Print the contingency table
print(Xtab)

# Import chi2_contingency module
from scipy.stats import chi2_contingency

# Calculate the p-value
chi2, pval, dof, expected = chi2_contingency(Xtab)

# Print the p-value
print(pval)

# Determine if the p-value is significant
is_significant = True

# Calculate and print the number of visits
num_visits = len(abdata)

# Print the number of visits
print(num_visits)

# Calculate the purchase rate needed at 0.99
num_sales_needed_099 = 1000/0.99
p_sales_needed_099 = num_sales_needed_099/num_visits

# Print the purchase rate needed at 0.99
print(p_sales_needed_099)

# Calculate the purchase rate needed at 1.99
num_sales_needed_199 = 1000/1.99
p_sales_needed_199 = num_sales_needed_199/num_visits

# Print the purchase rate needed at 1.99
print(p_sales_needed_199)

# Calculate the purchase rate needed at 4.99
num_sales_needed_499 = 1000/4.99
p_sales_needed_499 = num_sales_needed_499/num_visits

# Print the purchase rate needed at 4.99
print(p_sales_needed_499)

# Calculate samp size & sales for 0.99 price point
samp_size_099 = np.sum(abdata.group == 'A')
sales_099 = np.sum((abdata.group == 'A') & (abdata.is_purchase == 'Yes'))

# Print samp size & sales for 0.99 price point
print(samp_size_099)
print(sales_099)

# Calculate samp size & sales for 1.99 price point
samp_size_199 = np.sum(abdata.group == 'B')
sales_199 = np.sum((abdata.group == 'B') & (abdata.is_purchase == 'Yes'))

# Print samp size & sales for 1.99 price point
print(samp_size_199)
print(sales_199)

# Calculate samp size & sales for 4.99 price point
samp_size_499 = np.sum(abdata.group == 'C')
sales_499 = np.sum((abdata.group == 'C') & (abdata.is_purchase == 'Yes'))

# Print samp size & sales for 4.99 price point
print(samp_size_499)
print(sales_499)

# Import the binom_test module
from scipy.stats import binom_test

# Calculate the p-value for Group A

```

```
pvalueA = binom_test(sales_099, n=samp_size_099, p=p_sales_needed_099, alternative='greater')

# Print the p-value for Group A
print(pvalueA)

# Calculate the p-value for Group B
pvalueB = binom_test(sales_199, n=samp_size_199, p=p_sales_needed_199, alternative='greater')

# Print the p-value for Group B
print(pvalueB)

# Calculate the p-value for Group C
pvalueC = binom_test(sales_499, n=samp_size_499, p=p_sales_needed_499, alternative='greater')

# Print the p-value for Group C
print(pvalueC)

# Set the correct value for the final answer variable
final_answer = '4.99'

# Print the chosen price group
print(final_answer)
```

Transforming Data into Features

You are a data scientist at a clothing company and are working with a data set of customer reviews. This dataset is originally from [Kaggle](#) and has a lot of potential for various machine learning purposes. You are tasked with transforming some of these features to make the data more useful for analysis. To do this, you will have time to practice the following:

- Transforming categorical data
- Scaling your data
- Working with date-time features

Let's get started!

Tasks

11/16 complete

Mark the tasks as complete by checking them off

Basic Exploration

1.

Let's start with some basic exploring by performing the following:

First, import your dataset. It is stored under a file named `reviews.csv`. Save it to a variable called `reviews`.

Stuck? Get a hint

2.

Next, we want to look at the column names of our dataset along with their data types. Do the following two steps:

- Print the column names of your dataset.

- Check your features' data types by printing `.info()`.

Stuck? Get a hint

Data Transformations

3.

Transform the `recommended` feature. Start by printing the feature's `.value_counts()`.

Stuck? Get a hint

4.

Since this is a True/False feature, we want to transform it to 1 for True and 0 for False.

To do this, create a dictionary called `binary_dict` where:

- The keys are what is currently in the `recommended` feature.

- The values are what we want in the new column (0s and 1s).

Click the hint if you get stuck.

Stuck? Get a hint

5.

Using `binary_dict`, transform the `recommended` column so that it will now be binary. Print the results using `.value_counts()` to confirm the transformation.

Stuck? Get a hint

6.

Let's run through a similar process to transform the `rating` feature. This is ordinal data so our transformation should make that more clear. Again, start by printing the `.value_counts()`.

To check your output, click the hint.

Stuck? Get a hint

7.

We want to make the following changes to the values:

```
'Loved it' - 5
'Liked it' - 4
'Was okay' - 3
'Not great' - 2
'Hated it' - 1
```

Create a dictionary called `rating_dict` where the keys are what is currently in the feature and the values are what we want in the new column. You can use the hierarchy listed above to make your dictionary.

Stuck? Get a hint

8.

Using `rating_dict`, transform the `rating` column so it contains numerical values. Print the results using `.value_counts()` to confirm the transformation.

Stuck? Get a hint

9.

Let's now transform the `department_name` feature. This process will be slightly different, but start by printing the `.value_counts()` of the feature.

Use Panda's `get_dummies` to one-hot encode our feature.

Attach the results back to our original data frame.

Print the column names to see!

Hint

```
#get the number of categories in a feature
print(df['col_name'].value_counts())
```

Copy to Clipboard

10.

Use panda's `get_dummies()` method to one-hot encode our feature. Assign this to a variable called `one_hot`.

Hint

To perform one-hot encoding, use code similar to the following:

```
#perform get_dummies
one_hot = pd.get_dummies(df['col_name'])
```

Copy to Clipboard

11.

Join the results from `one_hot` back to our original data frame. Then print out the column names. What has been added?

Hint

Use similar code to the following:

```
#join the new columns back onto the original
df = df.join(one_hot)

#print column names
print(df.columns)
```

Copy to Clipboard

We now have the following new columns:

```
'Bottoms'
'Dresses'
'Intimate'
'Jackets'
'Tops'
'Trend'
```

12.

Let's make one more feature transformation!

Transform the `review_date` feature.

This feature is listed as an `object` type, but we want this to be transformed into a date-time feature.

Transform `review_date` into a date-time feature.

Print the feature type to confirm the transformation.

Click the hint if you get stuck.

Hint

Your code should be similar to the following:

```
df['new_col'] = pd.to_datetime(df['col'])

print(df['new_col'].dtype)
```

Copy to Clipboard

Scaling the Data

13.

The final step we will take in our transformation project is scaling our data. We notice that we have a wide range of numbers thus far, so it is best to put everything on the same scale.

Let's get our data frame to only have the numerical features we created. If you get stuck, click the hint.

Hint

Use the following line of code:

```
#get numerical columns
reviews = reviews[['clothing_id', 'age', 'recommended', 'rating', 'Bottoms',
' Dresses', 'Intimate', 'Jackets', 'Tops', 'Trend']].copy()
```

Copy to Clipboard

14.

Reset the index to be our `clothing_id` feature.

Hint

Use the `.set_index()` method:

```
#reset index
df = df.set_index(col)
```

Copy to Clipboard

15.

We are ready to scale our data! Perform a `.fit_transform()` on our data set, and print the results to see how the features have changed.

Hint

Create a `StandardScaler()` and then use `.fit_transform()` on `reviews`. Your code should be similar to the following two lines of code:

```
scaler = StandardScaler()
scaler.fit_transform(df)
```

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

#import data
reviews = pd.read_csv('reviews.csv')

#print column names
print(reviews.info())

#print .info
```

```

#look at the counts of recommended
print(reviews.recommended.value_counts())

#create binary dictionary
binary_dict = {True: 1, False: 0}

#transform column
reviews.recommended = reviews.recommended.map(binary_dict)

#print your transformed column
print(reviews.recommended.value_counts())

#look at the counts of rating
print(reviews.rating.value_counts())

#create dictionary
rating_dict = {
    'Loved it': 5,
    'Liked it' : 4,
    'Was okay' : 3,
    'Not great' : 2,
    'Hated it' : 1
}

#transform rating column
reviews.rating = reviews.rating.map(rating_dict)

#print your transformed column values
reviews.rating.value_counts()

#get the number of categories in a feature
print(reviews.department_name.value_counts())

#perform get dummies
one_hot = pd.get_dummies(reviews.department_name)

#join the new columns back onto the original
reviews = reviews.join(one_hot)

#print column names
print(reviews.columns)

#transform review_date to date-time data
reviews.review_date = pd.to_datetime(reviews.review_date)

#print review date data type
print(reviews.review_date.dtype)

#get numerical columns
#get numerical columns
reviews = reviews[['clothing_id', 'age', 'recommended', 'rating', 'Bottoms', 'Dresses', 'Intimate',
'Jackets', 'Tops', 'Trend']].copy()

#reset index
reviews = reviews.set_index('clothing_id')

#instantiate standard scaler
scaler = StandardScaler()

#fit transform data
reviews_scaled = scaler.fit_transform(reviews)
print(pd.DataFrame(data=reviews_scaled, columns=reviews.columns).head())

```

Honey Production

Now that you have learned how linear regression works, let's try it on an example of real-world data.

As you may have already heard, the honeybees are in a precarious state right now. You may have seen articles about the decline of the honeybee population for various reasons. You want to investigate this decline and how the trends of the past predict the future for the honeybees.

Note: All the tasks can be completed using Pandas or NumPy. Pick whichever one you prefer.

If you get stuck during this project or would like to see an experienced developer work through it, click **"Get Unstuck"** to see a **project walkthrough video**.

Tasks

13/13 complete

Mark the tasks as complete by checking them off

Check out the Data

1.

We have loaded in a DataFrame for you about honey production in the United States from [Kaggle](#). It is called `df` and has the following columns:

```
state
numcol
yieldpercol
totalprod
stocks
priceperlb
prodvalue
year
```

Use `.head()` to get a sense of how this DataFrame is structured.

Hint

A DataFrame is a 2-dimensional labeled data structure with columns of potentially different types. you can think of it like a spreadsheet or a SQL table. it is generally the most commonly used `pandas` object.

The `.head()` function return the first `n` rows:

```
print(df.head())
```

Copy to Clipboard

2.

For now, we care about the total production of honey per year. Use the `.groupby()` method provided by `pandas` to get the mean of `totalprod` per year.

Store this in a variable called `prod_per_year`.

Hint

To group the years, you might use a command like this:

```
prod_per_year = df.groupby('year').totalprod.mean().reset_index()
```

Copy to Clipboard

This will group the `year` column.

3.

Create a variable called `x` that is the column of `years` in this `prod_per_year` DataFrame.

After creating `x`, we will need to reshape it to get it into the right format, using this command:

```
X = X.values.reshape(-1, 1)
```

Copy to Clipboard

Hint

You can select a column from a DataFrame by using bracket notation:

```
column_of_interest = df["column name"]
```

Copy to Clipboard

4.

Create a variable called `y` that is the `totalprod` column in the `prod_per_year` dataset.

Hint

You can select a column from a DataFrame by using bracket notation:

```
column_of_interest = df["column name"]
```

Copy to Clipboard

5.

Using `plt.scatter()`, plot `y` vs `x` as a scatterplot.

Display the plot using `plt.show()`.

Can you see a vaguely linear relationship between these variables?

Hint

Note the uppercase `x`.

To plot a scatterplot, you can enter the x-values and then the y-values into `plt.scatter()`:

```
plt.scatter(x, y)
```

Copy to Clipboard

Create and Fit a Linear Regression Model

6.

Create a linear regression model from scikit-learn and call it `regr`.

Use the `LinearRegression()` constructor from the `linear_model` module to do this.

Hint

The `LinearRegression` class is from the `linear_model` module.

To create a new `LinearRegression` object, just call `linear_model.LinearRegression()` and assign it to your variable name:

```
my_regression_model = linear_model.LinearRegression()
```

Copy to Clipboard

7.

Fit the model to the data by using `.fit()`. You can feed `x` into your `regr` model by passing it in as a parameter of `.fit()`.

8.

After you have fit the model, print out the slope of the line (stored in a list called `regr.coef_`) and the intercept of the line (`regr.intercept_`).

Hint

The slope of the line will be the first (and only) element of the `regr.coef_` list. So using:

```
print(regr.coef_[0])
```

Copy to Clipboard

should print the slope.

9.

Create a list called `y_predict` that is the predictions your `regr` model would make on the `x` data.

Hint

You can use the `.predict()` method, using `x` as a parameter, to get the predictions from the `regr` object.

10.

Plot `y_predict` vs `x` as a line, on top of your scatterplot using `plt.plot()`.

Make sure to call `plt.show()` after plotting the line.

Hint

After the first call to `plt.plot()`, but before you call `plt.show()`, you can plot a line using:

```
plt.plot(X, y_predict)
```

Copy to Clipboard

Predict the Honey Decline

11.

So, it looks like the production of honey has been in decline, according to this linear model. Let's predict what the year 2050 may look like in terms of honey production.

Our known dataset stops at the year 2013, so let's create a NumPy array called `x_future` that is the range from 2013 to 2050. The code below makes a NumPy array with the numbers 1 through 10

```
nums = np.array(range(1, 11))
```

Copy to Clipboard

After creating that array, we need to reshape it for scikit-learn.

```
X_future = X_future.reshape(-1, 1)
```

Copy to Clipboard

You can think of `reshape()` as rotating this array. Rather than one big row of numbers, `x_future` is now a big column of numbers — there's one number in each row.

`reshape()` is a little tricky! It might help to print out `x_future` before and after reshaping.

Hint

We want to make a list out of a range object:

```
X_future = np.array(range(start_year, end_year))
```

Copy to Clipboard

12.

Create a list called `future_predict` that is the y-values that your `regr` model would predict for the values of `x_future`.

Hint

You can use the `.predict()` method, using `x_future` as a parameter, to get the predictions from the `regr` object.

13.

Plot `future_predict` vs `x_future` on a different plot.

How much honey will be produced in the year 2050, according to this?

Hint

Complete the following line of code:

```
plt.plot(X_future, _____)
```

Tennis Ace

Overview

This project contains a series of open-ended requirements which describe the project you'll be building. There are many possible ways to correctly fulfill all of these requirements, and you should expect to use the internet, Codecademy, and other resources when you encounter a problem.

Project Goals

You will create a linear regression model that predicts the outcome for a tennis player based on their playing habits. By analyzing and modeling the [Association of Tennis Professionals \(ATP\)](#) data, you will determine what it takes to be one of the best tennis players in the world.

Setup Instructions

If you choose to do this project on your computer instead of Codecademy, you can download what you'll need by clicking the "Download" button below. If you need help setting up your computer, be sure to check out our [setup guide](#).

Download

Tasks

0/8 complete

[Mark the tasks as complete by checking them off](#)

Prerequisites

1.

In order to complete this project, you should have completed the Linear Regression and Multiple Linear Regression lessons in the [Machine Learning Course](#). This content is also covered in the [Data Scientist Career Path](#).

Project Requirements

2.

"Game, Set, Match!"

No three words are sweeter to hear as a tennis player than those, which indicate that a player has beaten their opponent. While you can head down to your nearest court and aim to overcome your challenger across the net without much practice, a league of professionals spends day and night, month after month practicing to be among the best in the world. Today you will put your linear regression knowledge to the test to better understand what it takes to be an all-star tennis player. Provided in **tennis_stats.csv** is data from the men's professional tennis league, which is called the [ATP \(Association of Tennis Professionals\)](#). Data from the top 1500 ranked players in the ATP over the span of 2009 to 2017 are provided in file. The statistics recorded for each player in each year include service game (offensive) statistics, return game (defensive) statistics and outcomes. Load the csv into a DataFrame and investigate it to gain familiarity with the data. Open the hint for more information about each column of the dataset.

Stuck? Get a hint

3.

Perform exploratory analysis on the data by plotting different features against the different outcomes. What relationships do you find between the features and outcomes? Do any of the features seem to predict the outcomes?

Stuck? Get a hint

4.

Use one feature from the dataset to build a single feature linear regression model on the data. Your model, at this point, should use *only* one feature and predict one of the outcome columns. Before training the model, split your data into training and test datasets so that you can evaluate your model on the test set. How does your model perform? Plot your model's predictions on the test set against the actual outcome variable to visualize the performance.

Stuck? Get a hint

5.

Create a few more linear regression models that use one feature to predict one of the outcomes. Which model that you create is the best?

Stuck? Get a hint

6.

Create a few linear regression models that use two features to predict yearly earnings. Which set of two features results in the best model?

Stuck? Get a hint

7.

Create a few linear regression models that use multiple features to predict yearly earnings. Which set of features results in the best model?

Head to the Codecademy forums and share your set of features that resulted in the highest test score for predicting your outcome. What features are most important for being a successful tennis player?

Stuck? Get a hint

Solution

8.

Great work! Visit [our forums](#) to compare your project to our sample solution code. You can also learn how to host your own solution on GitHub so you can share it with other learners! Your solution might look different from ours, and that's okay! There are multiple ways to solve these projects, and you'll learn more by seeing others' code.

Predict Credit Card Fraud

<https://www.codecademy.com/paths/data-science/tracks/dsml-supervised-learning-i/modules/mle-logistic-regression-66e97567-68e4-431c-895f-c3c64abc5547/projects/logreg-credit-card-fraud>

Inference specialist

<https://www.codecademy.com/paths/data-science-inf/tracks/dsinf-regression-for-inference-data-science/modules/dsinf-logistic-regression/projects/logreg-credit-card-fraud>

Credit card fraud is one of the leading causes of identify theft around the world. In 2018 alone, over [\\$24 billion](#) were stolen through fraudulent credit card transactions. Financial institutions employ a wide variety of different techniques to prevent fraud, one of the most common being Logistic Regression.

In this project, you are a Data Scientist working for a credit card company. You have access to a dataset (based on a [synthetic financial dataset](#)), that represents a typical set of credit card transactions. `transactions.csv` is the original dataset containing 200k transactions. For starters, we're *going to be working with a small portion of this dataset*, `transactions_modified.csv`, which contains one thousand transactions. Your task is to use Logistic Regression and create a predictive model to determine if a transaction is fraudulent or not.

Note that a **solution.py** file is loaded for you in the workspace, which contains solution code for this project. We highly recommend that you complete the project on your own without checking the solution, but feel free to take a look if you get stuck or want to check your answers when you're done!

Tasks

0/17 complete

[Mark the tasks as complete by checking them off](#)

Load the Data

1.
The file `transactions_modified.csv` contains data on 1000 simulated credit card transactions. Let's begin by loading the data into a pandas DataFrame named `transactions`. Take a peek at the dataset using `.head()` and you can use `.info()` to examine how many rows are there and what datatypes the are. How many transactions are fraudulent? Print your answer.

Stuck? Get a hint

Clean the Data

2.
Looking at the dataset, combined with our knowledge of credit card transactions in general, we can see that there are a few interesting columns to look at. We know that the `amount` of a given transaction is going to be important. Calculate summary statistics for this column. What does the distribution look like?

Stuck? Get a hint

3.
We have a lot of information about the `type` of transaction we are looking at. Let's create a new column called `isPayment` that assigns a `1` when `type` is "PAYMENT" or "DEBIT", and a `0` otherwise.

Stuck? Get a hint

4.
Similarly, create a column called `isMovement`, which will capture if money moved out of the origin account. This column will have a value of `1` when `type` is either "CASH_OUT" or "TRANSFER", and a `0` otherwise.

Stuck? Get a hint

5.
With financial fraud, another key factor to investigate would be the difference in value between the origin and destination account. Our theory, in this case, being that destination accounts with a significantly different value could be suspect of fraud. Let's create a column called `accountDiff` with the absolute difference of the `oldbalanceOrg` and `oldbalanceDest` columns.

Stuck? Get a hint

Select and Split the Data

6.

Before we can start training our model, we need to define our features and label columns. Our label column in this dataset is the `isFraud` field. Create a variable called `features` which will be an array consisting of the following fields:

```
amount
isPayment
isMovement
accountDiff
```

Also create a variable called `label` with the column `isFraud`.

Stuck? Get a hint

7.

Split the data into training and test sets using `sklearn's train_test_split()` method. We'll use the training set to train the model and the test set to evaluate the model. Use a `test_size` value of 0.3.

Stuck? Get a hint

Normalize the Data

8.

Since `sklearn's` Logistic Regression implementation uses Regularization, we need to scale our feature data. Create a `StandardScaler` object, `.fit_transform()` it on the training features, and `.transform()` the test features.

Stuck? Get a hint

Create and Evaluate the Model

9.

Create a `LogisticRegression` model with `sklearn` and `.fit()` it on the training data.

Fitting the model find the best coefficients for our selected features so it can more accurately predict our label. We will start with the default threshold of 0.5.

Stuck? Get a hint

10.

Run the model's `.score()` method on the training data and print the training score.

Scoring the model on the training data will process the training data through the trained model and will predict which transactions are fraudulent. The score returned is the percentage of correct classifications, or the accuracy.

Stuck? Get a hint

11.

Run the model's `.score()` method on the test data and print the test score.

Scoring the model on the test data will process the test data through the trained model and will predict which transactions are fraudulent. The score returned is the percentage of correct classifications, or the accuracy, and will be an indicator for the success of your model.

How did your model perform?

Stuck? Get a hint

12.

Print the coefficients for our model to see how important each feature column was for prediction. Which feature was most important? Least important?

Stuck? Get a hint

Predict With the Model

13.

Let's use our model to process more transactions that have gone through our systems. There are three numpy arrays pre-loaded in the workspace with information on new sample transactions under "New transaction data"

```
# New transaction data
transaction1 = np.array([123456.78, 0.0, 1.0, 54670.1])
transaction2 = np.array([98765.43, 1.0, 0.0, 8524.75])
transaction3 = np.array([543678.31, 1.0, 0.0, 510025.5])
```

Copy to Clipboard

Create a fourth array, `your_transaction`, and add any transaction information you'd like. Make sure to enter all values as floats with a `.`!

Stuck? Get a hint

14.

Combine the new transactions and `your_transaction` into a single numpy array called `sample_transactions`.

Stuck? Get a hint

15.

Since our Logistic Regression model was trained on scaled feature data, we must also scale the feature data we are making predictions on. Using the `StandardScaler` object created earlier, apply its `.transform()` method to `sample_transactions` and save the result to `sample_transactions`.

Stuck? Get a hint

16.

Which transactions are fraudulent? Use your model's `.predict()` method on `sample_transactions` and print the result to find out.

Want to see the probabilities that led to these predictions? Call your model's `.predict_proba()` method on `sample_transactions` and print the result. The 1st column is the probability of a transaction not being fraudulent, and the 2nd column is the probability of a transaction being fraudulent (which was calculated by our model to make the final classification decision).

Stuck? Get a hint

17.

Congratulations on completing the project!

Note that we'd used a modified version of the dataset. You can now try to re-run the project using the original dataset, `transactions.csv`. Examine how the results change. If you notice something weird, you're totally on to something! That "something" is what is known as an *imbalanced class classification problem*. We will cover this very relevant topic (among many other things) in the Logistic Regression II module!

Stuck? Get a hint

Income Classification using Logistic Regression

<https://www.codecademy.com/paths/data-science/tracks/dsml-supervised-learning-i/modules/logistic-regression-ii/projects/logistic-regression-ii-project>

In this project, we will be using a dataset containing census information from the 1994 Census database to create a logistic regression model that predicts whether or not a person makes more than \$50,000.

Let's get started!

Datasets

The original data set is available at the UCI Machine Learning Repository [here](#).

The dataset has been loaded for you in *script.py* and saved as a DataFrame named `df`. Some of the input and output features of interest are:

age: continuous
workclass: Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, Local-gov, State-gov, Without-pay, Never-worked.
education: Bachelors, Some-college, 11th, HS-grad, Prof-school, Assoc-acdm, Assoc-voc, 9th, 7th-8th, 12th, Masters, 1st-4th, 10th, Doctorate, 5th-6th, Preschool
race: White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black
sex: Female, Male
capital-gain: continuous
capital-loss: continuous
hours-per-week: continuous
native country: discrete
income: discrete, >50K, <=50K

Tasks

0/10 complete

[Mark the tasks as complete by checking them off](#)

EDA and Logistic Regression Assumptions

1.
The dataset has been loaded for you in **script.py** and saved as a dataframe named `df`. The outcome variable here is income. Check if the dataset is imbalanced.
2.
Notice we have created a variable named `feature_cols`. This contains a list of the variables we will use as our predictor variables.
Transform the dataset of predictor variables to dummy variables and save this in a new DataFrame called `x` (upper case "X").
3.
Using `x`, create a heatmap of the correlation values.
4.
Determine if scaling is needed for `x` prior to modeling.
Then create the `y` output variable which is binary, 0 when income is less than \$50K, 1 when greater than \$50K.

Logistic Regression Models and Evaluation

5. Split the data into a training and testing set. Set the `random_state` to 1 and `test_size` to .2. Then using `x_train`, `x_test`, `y_train`, `y_test`, fit a logistic regression model in scikit-learn on the training set with parameters `C=0.05`, `penalty='l1'`, `solver='liblinear'`. Lastly, use `.predict()` to create the y predictions and save this as `y_pred`.
6. Print the model parameters (intercept and coefficients).
7. Evaluate the predictions of the model on the test set. Print the confusion matrix and accuracy score.
8. Create a new DataFrame of the model coefficients and variable names. Sort values based on coefficient and exclude any that are equal to zero. Print the values of the DataFrame.
9. Create a barplot of the coefficients sorted in ascending order.
10. Plot the ROC curve and print the AUC value.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score,
precision_score, recall_score, f1_score, roc_curve, roc_auc_score
from sklearn.preprocessing import StandardScaler

import codecademylib3
import matplotlib.pyplot as plt
import seaborn as sns

col_names = ['age', 'workclass', 'fnlwgt', 'education', 'education-num',
'marital-status', 'occupation', 'relationship', 'race', 'sex',
'capital-gain', 'capital-loss', 'hours-per-week', 'native-country', 'income']
df = pd.read_csv('adult.data', header = None, names = col_names)

#Clean columns by stripping extra whitespace for columns of type "object"
for c in df.select_dtypes(include=['object']).columns:
    df[c] = df[c].str.strip()
print(df.head())

#1. Check Class Imbalance
print(df.income.value_counts(normalize=True))

#2. Create feature dataframe X with feature columns and dummy variables for categorical
features
feature_cols = ['age', 'capital-gain', 'capital-loss', 'hours-per-week', 'sex', 'race',
'hours-per-week', 'education']
X = pd.get_dummies(df[feature_cols], drop_first=True)

#3. Create a heatmap of X data to see feature correlation
plt.figure()
sns.heatmap(X.corr())#, annot=True)
plt.show()
plt.close()

#4. Create output variable y which is binary, 0 when income is less than 50k, 1 when it is
greater than 50k
y = np.where(df.income=='<=50K', 0, 1)

#5a. Split data into a train and test set
x_train, x_test, y_train, y_test = train_test_split(X, y, random_state=1, test_size=.2)
```

```

#5b. Fit LR model with sklearn on train set, and predicting on the test set
log_reg = LogisticRegression(C=0.05, penalty='l1', solver='liblinear')
log_reg.fit(x_train, y_train)
y_pred = log_reg.predict(x_test)

#6. Print model parameters (intercept and coefficients)
print('Model Parameters, Intercept:')
print(log_reg.intercept_[0])
print('Model Parameters, Coeff:')
print(log_reg.coef_)

#7. Evaluate the predictions of the model on the test set. Print the confusion matrix and
accuracy score.
print('Confusion Matrix on test set:')
print(confusion_matrix(y_test, y_pred))
print(f'Accuracy Score on test set: {log_reg.score(x_test, y_test)}')

# 8.Create new DataFrame of the model coefficients and variable names; sort values based on
coefficient
coef_df = pd.DataFrame(zip(x_train.columns, log_reg.coef_[0]), columns=['var',
'coef']).sort_values('coef')
coef_df = coef_df[coef_df.coef.abs()>0].sort_values('coef')
print(coef_df)

#9. barplot of the coefficients sorted in ascending order
plt.figure(figsize = (10,7))
sns.barplot(data=coef_df, x='var', y='coef')
plt.xticks(rotation=90);
plt.title('LR Coefficient Values')
plt.show()
plt.close()

#10. Plot the ROC curve and print the AUC value.
y_pred_prob = log_reg.predict_proba(x_test)
roc_auc = roc_auc_score(y_test, y_pred_prob[:,1])
print(f'ROC AUC score: {roc_auc}')

fpr, tpr, thresholds = roc_curve(y_test,y_pred_prob[:,1])
plt.figure()
plt.plot(fpr, tpr, color='darkorange',
        label='ROC curve (area = %0.2f)' % roc_auc)

plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.plot([0,1],[0,1], color='navy',linestyle='--')
plt.title('ROC Curve')
plt.grid()
plt.show()

```

Cancer Classifier

In this project, we will be using several Python libraries to make a K-Nearest Neighbor classifier that is trained to predict whether a patient has breast cancer.

If you get stuck during this project or would like to see an experienced developer work through it, click **"Get Unstuck"** to see a **project walkthrough video**.

Tasks

0/18 complete

[Mark the tasks as complete by checking them off](#)

Explore the data

1.

Let's begin by importing the breast cancer data from `sklearn`. We want to import the function `load_breast_cancer` from `sklearn.datasets`.

Once we've imported the dataset, let's load the data into a variable called `breast_cancer_data`. Do this by setting `breast_cancer_data` equal to the function `load_breast_cancer()`.

2.

Before jumping into creating our classifier, let's take a look at the data. Begin by printing `breast_cancer_data.data[0]`. That's the first datapoint in our set. But what do all of those numbers represent? Let's also print `breast_cancer_data.feature_names`.

3.

We now have a sense of what the data looks like, but what are we trying to classify? Let's print both `breast_cancer_data.target` and `breast_cancer_data.target_names`. Was the very first data point tagged as malignant or benign?

Splitting the data into Training and Validation Sets

4.

We have our data, but now it needs to be split into training and validation sets. Luckily, `sklearn` has a function that does that for us. Begin by importing the `train_test_split` function from `sklearn.model_selection`.

5.

Call the `train_test_split` function. It takes several parameters:

The data you want to split (for us `breast_cancer_data.data`)

The labels associated with that data (for us, `breast_cancer_data.target`).

The `test_size`. This is what percentage of your data you want to be in your testing set.

Let's use `test_size = 0.2`

`random_state`. This will ensure that every time you run your code, the data is split in the same way. This can be any number. We used `random_state = 100`.

6.

Right now we're not storing the return value of `train_test_split`. `train_test_split` returns four values in the following order:

The training set

The validation set

The training labels

The validation labels

Store those values in variables named `training_data`, `validation_data`, `training_labels`, and `validation_labels`.

7.

Let's confirm that worked correctly. Print out the length of `training_data` and `training_labels`. They should be the same size - one label for every piece of data!

Running the classifier

8.

Now that we've created training and validation sets, we can create a `KNeighborsClassifier` and test its accuracy. Begin by importing `KNeighborsClassifier` from `sklearn.neighbors`.

9.

Create a `KNeighborsClassifier` where `n_neighbors = 3`. Name the classifier `classifier`.

10.

Train your classifier using the `fit` function. This function takes two parameters: the training set and the training labels.

11.

Now that the classifier has been trained, let's find how accurate it is on the validation set. Call the classifier's `score` function. `score` takes two parameters: the validation set and the validation labels. Print the result!

12.

The classifier does pretty well when `k = 3`. But maybe there's a better `k`! Put the previous 3 lines of code inside a for loop. The loop should have a variable named `k` that starts at 1 and increases to 100. Rather than `n_neighbors` always being 3, it should be this new variable `k`.

You should now see 100 different validation accuracies print out. Which `k` seems the best?

Graphing the results

13.

We now have the validation accuracy for 100 different `ks`. Rather than just printing it out, let's make a graph using `matplotlib`. Begin by importing `matplotlib.pyplot` as `plt`.

14.

The x-axis should be the values of `k` that we tested. This should be a list of numbers between 1 and 100. You can use the `range` function to make this list. Store it in a variable named `k_list`.

15.

The y-axis of our graph should be the validation accuracy. Instead of printing the validation accuracies, we want to add them to a list. Outside of the for loop, create an empty list named `accuracies`. Inside the for loop, instead of printing each accuracy, append it to `accuracies`.

16.

We can now plot our data! Call `plt.plot()`. The first parameter should be `k_list` and the second parameter should be `accuracies`.

After plotting the graph, show it using `plt.show()`.

17.

Let's add some labels and a title. Set the x-axis label to `"k"` using `plt.xlabel()`. Set the y-axis label to `"Validation Accuracy"`. Set the title to `"Breast Cancer Classifier Accuracy"`.

18.

Great work! If you want to play around with this more, try changing the `random_state` parameter when making the training set and validation set. This will change which points are in the training set and which are in the validation set.

Ideally, the graph will look the same no matter how you split up the training set and test set. This data set is fairly small, so there is slightly more variance than usual.

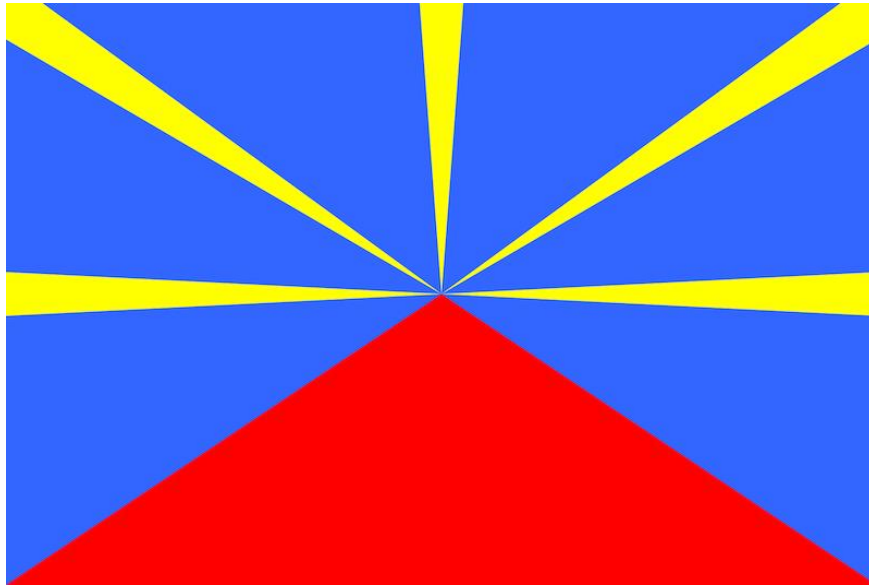
Code Review

Machine Learning Specialist

Find the flag!

<https://www.codecademy.com/paths/data-science/tracks/dsml-supervised-learning-i/modules/mle-decision-trees-8b27e5b3-0352-4887-a8e5-e4a507597ad5/projects/mlfun-decision-trees-project>

Can you guess which continent this flag comes from?



What are some of the features that would clue you in? Maybe some of the colors are good indicators. The presence or absence of certain shapes could give you a hint. In this project, we'll use decision trees to try to predict the continent of flags based on several of these features.

We'll explore which features are the best to use and the best way to create your decision tree.

Datasets

The original data set is available at the UCI Machine Learning Repository:

<https://archive.ics.uci.edu/ml/datasets/Flags>

Tasks

0/15 complete

[Mark the tasks as complete by checking them off](#)

Investigate the data

1.

The dataset has been loaded for you in **script.py** and saved as a dataframe named `df`. Some of the input and output features of interest are:

`name`: Name of the country concerned

`landmass`: 1=N.America, 2=S.America, 3=Europe, 4=Africa, 5=Asia, 6=Oceania

`bars`: Number of vertical bars in the flag

`stripes`: Number of horizontal stripes in the flag

`colours`: Number of different colours in the flag

`red`: 0 if red absent, 1 if red present in the flag

...

`mainhue`: predominant colour in the flag (tie-breaks decided by taking the topmost hue, if that fails then the most central hue, and if that fails the leftmost hue)


```
circles: Number of circles in the flag
crosses: Number of (upright) crosses
saltires: Number of diagonal crosses
quarters: Number of quartered sections
sunstars: Number of sun or star symbols
```

We will build a decision tree classifier to predict what continent a particular flag comes from. Before that, we want to understand the distribution of flags by continent. Calculate the count of flags by `landmass` value.

Use `df['landmass'].value_counts()`.

2.

Rather than looking at all six continents, we will focus on just two, Europe and Oceania. Create a new dataframe with only flags from Europe and Oceania.

Europe and Oceania correspond to `landmass` values of 3 and 6 respectively. Consider using

`df["landmass"].isin([3,6])` to subset the original DataFrame.

3.

Given the list of predictors in the list `var`, print the average values of each for these two continents. Note which predictors have very different averages.

Use `.groupby('landmass')[var].mean()`.

4.

We will build a classifier to distinguish flags for these two continents – but first, inspect the variable types for each of the predictors.

```
labels = (df["landmass"].isin([3,6]))*1
```

Copy to Clipboard

Try using `df_36[var].dtypes`.

5.

Note that all the predictor variables are numeric except for `mainhue`. Transform the dataset of predictor variables to dummy variables and save this in a new dataframe called `data`.

```
data = pd.get_dummies(df[var])
```

Copy to Clipboard

6.

Split the data into a train and test set.

Use `train_test_split(X,y, random_state=1, test_size=.4)`.

Tune Decision Tree Classifiers by Depth

7.

We will explore tuning the decision tree model by testing the performance over a range of `max_depth` values. Fit a decision tree classifier for `max_depth` values from 1-20. Save the accuracy score in for each depth in the list `acc_depth`.

Inside the loop skeleton, each decision tree should be fit on the training data, i.e. `train_data` and `train_labels`. The accuracy score is then computed using `test_data` and `test_labels`.

8.

Plot the accuracy of the decision tree models versus the `max_depth`.

The data was saved in the previous loop, use `plt.plot(depths, acc_depth)`.

9.

Find the largest accuracy and the depth this occurs.

10.

Refit the decision tree model using the `max_depth` from above; plot the decision tree.

Use `tree.plot_tree` to plot the decision tree after it is refit.

Tune Decision Tree Classifiers by Pruning

11.

Like we did with `max_depth`, we will now tune the tree by using the hyperparameter `ccp_alpha`, which is a pruning parameter. Fit a decision tree classifier for each value in `ccp`. Save the accuracy score in the list `acc_pruned`.

Inside the loop skeleton, each decision tree should be fit on the training data, i.e. `train_data` and `train_labels`. The accuracy score is then computed using `test_data` and `test_labels`.

12.

Plot the accuracy of the decision tree models versus the `ccp_alpha`.

The data was saved in the previous loop, use `plt.plot(ccp, acc_pruned)`.

13.

Find the largest accuracy and the `ccp_alpha` value this occurs.

The accuracy values are saved in `acc_pruned`, so the maximum can be found with

```
np.max(acc_pruned).
```

14.

Fit a decision tree model with the values for `max_depth` and `ccp_alpha` found above. Plot the final decision tree.

Use `tree.plot_tree` to plot the decision tree after it is refit.

15.

Nice work! Note that the accuracy of our final model increased and the structure of the tree was simpler – many unnecessary branches were removed in the pruning process making for a much easier interpretation.

There are a few different ways to extend this project:

- Try to classify something else! Rather than predicting the `"Landmass"` feature, could predict something like the `"Language"`?

- Find a subset of features that work better than what we're currently using. An important note is that a feature that has categorical data won't work very well as a feature. For example, we don't want a decision node to split nodes based on whether the value for `"Language"` is above or below 5.

- Tune more parameters of the model. You can find a description of all the parameters you can tune in the [Decision Tree Classifier documentation](#). For example, see what happens if you tune `max_leaf_nodes`. Think about whether you would be overfitting or underfitting the data based on how many leaf nodes you allow.

Wrapper Methods

In this project, you'll analyze data from a survey conducted by Fabio Mendoza Palechor and Alexis de la Hoz Manotas that asked people about their eating habits and weight. The data was obtained from the [UCI Machine Learning Repository](#). Categorical variables were changed to numerical ones in order to facilitate analysis.

First, you'll fit a logistic regression model to try to predict whether survey respondents are obese based on their answers to questions in the survey. After that, you'll use three different wrapper methods to choose a smaller feature subset.

You'll use sequential forward selection, sequential backward floating selection, and recursive feature elimination. After implementing each wrapper method, you'll evaluate the model accuracy on the resulting smaller feature subsets and compare that with the model accuracy using all available features.

Setup Instructions

You have two options of completing this assignment. Either here, within Codecademy's output terminal, or on your own, in case you're more comfortable using a Jupyter notebook. For this assignment, we strongly recommend using your own Jupyter notebook if you are comfortable as you will be able to run each model individually.

If you choose to do this project on your computer instead of Codecademy, you can download what you'll need by clicking the "Download" button below. If you need help setting up your computer, be sure to check out our setup guides:

[Command Line Interface Setup](#)
[Introducing Jupyter Notebook](#)
[Setting up Jupyter Notebook](#)
[Getting Started with Jupyter](#)
[Getting More out of Jupyter Notebook](#)

Click the download button below, and open **wrapper_method_starter.ipynb** and follow the steps in the Jupyter Notebook. If you get stuck, you can look at **wrapper_method_solution.ipynb** for the answer. The **obesity.csv** file is also included in this zip file, which you will need to complete this project.

Download

Tasks

21/21 complete

[Mark the tasks as complete by checking them off](#)

Evaluating a Logistic Regression Model

1.

The data set `obesity` contains 18 predictor variables. Here's a brief description of them.

`Gender` is `1` if a respondent is male and `0` if a respondent is female.

`Age` is a respondent's age in years.

`family_history_with_overweight` is `1` if a respondent has family member who is or was overweight, `0` if not.

`FAVC` is `1` if a respondent eats high caloric food frequently, `0` if not.

`FCVC` is `1` if a respondent usually eats vegetables in their meals, `0` if not.

`NCP` represents how many main meals a respondent has daily (`0` for 1-2 meals, `1` for 3 meals, and `2` for more than 3 meals).

`CAEC` represents how much food a respondent eats between meals on a scale of `0` to `3`.

SMOKE is 1 if a respondent smokes, 0 if not.

CH2O represents how much water a respondent drinks on a scale of 0 to 2.

SCC is 1 if a respondent monitors their caloric intake, 0 if not.

FAF represents how much physical activity a respondent does on a scale of 0 to 3.

TUE represents how much time a respondent spends looking at devices with screens on a scale of 0 to 2.

CALC represents how often a respondent drinks alcohol on a scale of 0 to 3.

Automobile, Bike, Motorbike, Public_Transportation, and Walking indicate a respondent's primary mode of transportation. Their primary mode of transportation is indicated by a 1 and the other columns will contain a 0.

The outcome variable, NObesyedad, is a 1 if a patient is obese and a 0 if not.

Use the `.head()` method and inspect the data.

Use the `print()` function and `obesity.head()`.

2.

In order to use a logistic regression model, you'll need to split the data into two parts: the predictor variables and an outcome variable. Do this by splitting the data into a DataFrame of predictor variables called `x` and a Series of outcome variables `y`.

Fill in the appropriate column name in the following code.

```
x = obesity.drop(["___"], axis=1)
y = obesity.____
```

Copy to Clipboard

3.

Create a logistic regression model called `lr`. Include the parameter `max_iter=1000` to make sure that the model will converge when you try to fit it.

Use a `LogisticRegression()` object.

4.

Use the `.fit()` method on `lr` to fit the model to `x` and `y`.

Fill in the blanks in the code `lr.fit(_, _)`.

5.

A model's *accuracy* is the proportion of classes that the model correctly predicts. is Compute and print the accuracy of `lr` by using the `.score()` method. What percentage of respondents did the model correctly predict as being either obese or not obese? You may want to write this number down so that you can refer to it during future tasks.

Fill in the blanks in the code `print(lr.score(_, _))`. The accuracy should be about 0.7659, meaning that the model correctly predicts the class of about 76% of respondents.

Sequential Forward Selection

6.

Now that you've created a logistic regression model and evaluated its performance, you're ready to do some feature selection.

Create a sequential forward selection model called `sfs`.

Be sure to set the `estimator` parameter to `lr` and set the `forward` and `floating` parameters to the appropriate values.

Also use the parameters `k_features=9`, `scoring='accuracy'`, and `cv=0`.

Use something similar to the following. You'll need to change the name of the model and some of the parameters.

```
sffs = SFS(model,
            k_features=5,
            forward=True,
            floating=True,
            scoring='accuracy',
            cv=0)
```

Copy to Clipboard

7.

Use the `.fit()` method on `sfs` to fit the model to `x` and `y`. This step will take some time (not more than a minute) to run.

Fill in the blanks in the code `sfs.fit(, )`.

8.

Now that you've run the sequential forward selection algorithm on the logistic regression model with `x` and `y` you can see what features were chosen and check the model accuracy on the smaller feature set. Print `sfs.subsets_[9]` to inspect the results of sequential forward selection.

Use the `print()` function.

9.

Use the dictionary `sfs.subsets_[9]` to print a tuple of chosen feature names. Then use it to print the accuracy of the model after doing sequential forward selection. How does this compare to the model's accuracy on all available features? You may want to write this number down so that you can compare it with the accuracies of other models.

Fill in the keys `'feature_names'` and `'avg_score'` to access the feature names and accuracy.

```
print(sfs.subsets_[9][    ])  
print(sfs.subsets_[9][    ])
```

Copy to Clipboard

The accuracy should be about 0.7693.

10.

It can be helpful to visualize the results of sequential forward selection and see how accuracy is affected as each feature is added. Use the code `plot_sfs(sfs.get_metric_dict())` to plot the model accuracy as a function of the number of features used. Make sure to show your plot as well.

Use the the given code as well as `plt.show()`.

Sequential Backward Selection

11.

Before completing this task group, you should comment out all the code you wrote for the previous task group (Sequential Forward Selection). Your code will take a long time to run and this will ensure that the learning environment won't time out.

Sequential forward selection was able to find a feature subset that performed marginally better than the full feature set. Let's use a different sequential method and see how it compares.

Create a sequential backward selection model called `sbs`.

Be sure to set the `estimator` parameter to `lr` and set the `forward` and `floating` parameters to the appropriate values.

Also use the parameters `k_features=7`, `scoring='accuracy'`, and `cv=0`.

Use something similar to the following. You'll need to change the name of the model and some of the parameters.

```
sffs = SFS(model,  
           k_features=5,  
           forward=True,  
           floating=True,  
           scoring='accuracy',  
           cv=0)
```

Copy to Clipboard

12.

Use the `.fit()` method on `sbs` to fit the model to `x` and `y`.

Fill in the blanks in the code `sbfs.fit(, )`.

13.

Now that you've run the sequential backward selection algorithm on the logistic regression model with `x` and `y` you can see what features were chosen and check the model accuracy on the smaller feature set. Print `sbs.subsets_[7]` to inspect the results of sequential backward selection. Use the `print()` function.

14.

Use the dictionary `sbs.subsets_[7]` to print a tuple of chosen feature names. Then use it to print the accuracy of the model after doing sequential backward selection. How does this compare to the model's accuracy on all available features? You may want to write this number down so that you can compare it with the accuracies of other models.

Fill in the keys `'feature_names'` and `'avg_score'` to access the feature names and accuracy.

```
print(sbs.subsets_[7][____])
print(sbs.subsets_[7][____])
```

Copy to Clipboard

The accuracy should be about 0.7821.

15.

You can visualize the results of sequential backward floating selection just as you did with sequential forward selection. Use the code `plot_sfs(sbs.get_metric_dict())` to plot the model accuracy as a function of the number of features used. Be sure to include `plt.clf()` in order to clear the previous graph before trying to show the new graph.

Use the given code as well as `plt.show()`.

Recursive Feature Elimination

16.

Before completing this task group, you should uncomment all the code you wrote for the previous task groups (Sequential Forward Selection and Sequential Backward Selection). Your code will take a long time to run and this will ensure that the learning environment won't time out. So far you've tried two different sequential feature selection methods. Let's try one more: recursive feature elimination. First you'll standardize the data, then you'll fit the RFE model and inspect the results.

At a later step of this project, you'll need to be able to access feature names. Enter the code

```
features = X.columns
```

 for use later.

Type `features = X.columns`.

17.

Before doing applying recursive feature elimination it is necessary to standardize the data. Standardize `x` and save it as a DataFrame by creating a `StandardScaler()` object and using the `.fit_transform()` method.

Use the code `X = pd.DataFrame(StandardScaler().fit_transform(X))`.

18.

Create an `RFE()` object that selects 8 features. Be sure to set the `estimator` parameter to `lr`.

Use something similar to the following. You'll need to change the parameters.

```
rfe = RFE(estimator=model, n_features_to_select=5)
```

Copy to Clipboard

19.

Fit the recursive feature elimination model to `x` and `y`.

Fill in the blanks in the code `rfe.fit(____, ____)`.

20.

Now that you've fit the RFE model you can evaluate the results. Create a list of chosen feature names and call it `rfe_features`. You can use a list comprehension and filter the features in `zip(features, rfe.support_)` based on whether their support is `True` (meaning the model kept them) or `False` (meaning the model eliminated them).

`[f for (f, support) in zip(features, rfe.support_) if support]` will produce the desired list of feature names.

21.

Use the `.score()` method on `rfe` and print the model accuracy after doing recursive feature elimination. How does this compare to the model's accuracy on all available features?

Fill in the blanks in the code `print(rfe.score(_, _))`.

The accuracy should be about 0.7579.

Predict Wine Quality with Regularization

The data you're going to be working with is from the [Wine Quality Dataset](#) in the UCI Machine Learning Repository. We're looking at the red wine data in particular and while the original dataset has a 1-10 rating for each wine, we've made it a classification problem with a wine quality of good (>5 rating) or bad (≤ 5 rating). The goals of this project are to:

- implement different logistic regression classifiers
- find the best ridge-regularized classifier using hyperparameter tuning
- implement a tuned lasso-regularized feature selection method

What we're working with:

11 input variables (based on physicochemical tests): 'fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'pH', 'sulphates' and 'alcohol'.

An output variable, 'quality' (0 for bad and 1 for good)

Tasks

0/17 complete

[Mark the tasks as complete by checking them off](#)

Logistic Regression Classifier without Regularization

1.
Before we begin modeling, let's scale our data using `StandardScaler()`. Use `StandardScaler().fit()` to fit the variable `features` and then use `transform()` to get `x` to get the transformed input to our model.
2.
Perform an 80:20 train-test split on the data. Set the `random_state` to `99` for reproducibility.
3.
Define a classifier, `clf_no_reg`, a logistic regression model without regularization and fit it to the training data.
4.
We're now going to plot the coefficients obtained from fitting the Logistic Regression model. Copy-paste the following code to get the ordered coefficients as a bar plot:

```
predictors = features.columns
coefficients = clf_no_reg.coef_.ravel()
coef = pd.Series(coefficients, predictors).sort_values()
coef.plot(kind='bar', title = 'Coefficients (no regularization)')
plt.tight_layout()
plt.show()
plt.clf()
```

[Copy to Clipboard](#)

5.
You're now ready to evaluate this classifier! In the case of linear regression, we evaluated our models using mean-squared-error. For classifiers, it is important that the classifier not only has high accuracy, but also high *precision* and *recall*, i.e., a low false positive and false negative rate. A metric known as *f1 score*, which is the weighted mean of precision and recall, captures the performance of a classifier holistically. It takes values between 0 and 1 and the closer it is to 1, the better the classifier. Use `f1_score()` to calculate the f1 score for the training and test data.

Logistic Regression with L2 Regularization

6.

We've seen in the previous article that the default implementation of logistic regression in `scikit-learn` is ridge-regularized! Use the default implementation to implement a classifier `clf_default` that is L2-regularized.

7.

Obtain the training and test `f1_score` for the ridge-regularized classifier using code similar to what we have in Task 5. Notice if either score goes up or down.

8.

The scores remain the same! Does this mean that regularization did nothing? Indeed! This means that the constraint boundary for the regularization we performed is large enough to hold the original loss function minimum, thus rendering our model the same as the unregularized one. How can we tune up the regularization? Recall that C is the inverse of the regularization strength (alpha), meaning that *smaller* values of C correspond to *more* regularization. The `scikit-learn` default for C is 1; therefore, in order to increase the amount of regularization, we need to consider values of C that are less than 1. But how far do we need to go? Let's try a coarse-grained search before performing a fine-grained one.

Define an array, `C_array` that takes the values `C_array = [0.0001, 0.001, 0.01, 0.1, 1]`. Get an array each for the training and test scores corresponding to these values of C .

9.

Use the following plotting code to plot the training and test scores as a function of C . Does this clarify the range of C 's we need to be doing a fine-grained search for?

```
plt.plot(C_array, training_array)
plt.plot(C_array, test_array)
plt.xscale('log')
plt.show()
plt.clf()
```

Copy to Clipboard

Hyperparameter Tuning for L2 Regularization

10.

We're now ready to perform hyperparameter tuning using `GridSearchCV`! Looking at the plot, the optimal C seems to be somewhere around 0.001 so a search window between 0.0001 and 0.01 is not a bad idea here.

Let's first get setup with the right inputs for this. Use `np.logspace()` to obtain 100 values between 10^{-4} and 10^{-2} and define a dictionary of C values named `tuning_C` that can function as an input to `GridSearchCV`'s parameter grid.

11.

Define a grid search model on the parameter grid defined above for a logistic regression model with ridge regularization. Set the `scoring` metric to 'f1' and the number of folds to 5. Fit this to the training data.

12.

Obtain the best C value from this search and the score corresponding to it using the `best_params_` and `best_score` attributes respectively.

13.

The score you got above reflects the mean f1-score on the 5 folds corresponding to the best classifier. Notice however that we haven't yet used the test data, `x_test`, `y_test` from our original train-test split! This was done with good reason: the original test data can now be used as our validation dataset to validate whether our "best classifier" is doing as well as we'd like it to on essentially unknown data.

Define a new classifier `clf_best_ridge` that corresponds to the best C value you obtained in the previous task. Fit it to the training data and obtain the `f1_score` on the test data to validate the model.

Feature Selection using L1 Regularization

14.

We're now going to use a grid search cross-validation method to regularize the classifier, but with L1 regularization instead. Instead of using `GridSearchCV`, we're going to use `LogisticRegressionCV`.

The syntax here is a little different. The arguments to `LogisticRegressionCV` that are relevant to us:

`Cs`: A list/array of `C` values to check; choose values between 0.01 and 100 here.

`cv`: Number of folds (5 is a good choice here!)

`penalty`: Remember to choose `'l1'` for this!

`solver`: Recall that L1 penalty requires that we specify the solver to be `'liblinear'`.

`scoring`: `'f1'` is still a great choice for a classifier.

Using the above, define a cross-validated classifier, `clf_l1` and fit `(X, y)` here. (Note that we're not doing a train-test-validation split like last time!)

15.

The classifier has the attribute `c_` which prints the optimal `C` value. The attribute `coef_` gives us the coefficients of the best lasso-regularized classifier. Print both of these.

16.

We can now reproduce the coefficient plot we'd produced for the unregularized scenario. Use the following lines of code to plot the sorted values of the coefficients as a bar plot:

```
coefficients = clf_l1.coef_.ravel()
coef = pd.Series(coefficients, predictors).sort_values()

plt.figure(figsize = (12,8))
coef.plot(kind='bar', title = 'Coefficients for tuned L1')
plt.tight_layout()
plt.show()
plt.clf()
```

Copy to Clipboard

17.

Notice how our L1 classifier has set one of the coefficients to zero! We've effectively eliminated one feature, `density`, from the model, thus using Lasso regularization as a feature selection method here.

SOLUTION

```
import numpy as np
import pandas as pd
import codecademylib3
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('wine_quality.csv')
print(df.columns)
y = df['quality']
features = df.drop(columns = ['quality'])

## 1. Data transformation
from sklearn.preprocessing import StandardScaler
standard_scaler_fit = StandardScaler().fit(features)
X = standard_scaler_fit.transform(features)

## 2. Train-test split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=99)

## 3. Fit a logistic regression classifier without regularization
from sklearn.linear_model import LogisticRegression
```

```

clf_no_reg = LogisticRegression(penalty = 'none') #(Remember to set the penalty parameter
to 'none'!)clf_no_reg.fit(X_train, y_train)
clf_no_reg.fit(X_train, y_train)

## 4. Plot the coefficients
predictors = features.columns
coefficients = clf_no_reg.coef_.ravel()
coef = pd.Series(coefficients,predictors).sort_values()
coef.plot(kind='bar', title = 'Coefficients (no regularization)')
plt.tight_layout()
plt.show()
plt.clf()

## 5. Training and test performance
from sklearn.metrics import f1_score
y_pred_test = clf_no_reg.predict(X_test)
y_pred_train = clf_no_reg.predict(X_train)
print('Training Score', f1_score(y_train, y_pred_train))
print('Testing Score', f1_score(y_test, y_pred_test))

## 6. Default Implementation (L2-regularized!)
clf_default = LogisticRegression()
clf_default.fit(X_train, y_train)

## 7. Ridge Scores
y_pred_train_ridge = clf_default.predict(X_train)
y_pred_test_ridge = clf_default.predict(X_test)
print(' Ridge-regularized Training Score', f1_score(y_train, y_pred_train_ridge))
print('Ridge-regularized Testing Score', f1_score(y_test, y_pred_test_ridge))

## 8. Coarse-grained hyperparameter tuning
training_array = []
test_array = []
C_array = [0.0001, 0.001, 0.01, 0.1, 1]
for x in C_array:
    clf = LogisticRegression(C = x )
    clf.fit(X_train, y_train)
    y_pred_test = clf.predict(X_test)
    y_pred_train = clf.predict(X_train)
    training_array.append(f1_score(y_train, y_pred_train))
    test_array.append(f1_score(y_test, y_pred_test))

## 9. Plot training and test scores as a function of C
plt.plot(C_array,training_array, label = 'Training Score')
plt.plot(C_array,test_array, label = 'Test Score')
plt.xscale('log')
plt.xlabel('C')
plt.legend()
plt.show()
plt.clf()

## 10. Making a parameter grid for GridSearchCV
C_array = np.logspace(-4, -2, 100)
#Making a dict to enter as an input to param_grid
tuning_C = {'C':C_array}

## 11. Implementing GridSearchCV with l2 penalty
from sklearn.model_selection import GridSearchCV
clf_gs = LogisticRegression()
gs = GridSearchCV(clf_gs, param_grid = tuning_C, scoring = 'f1', cv = 5)
gs.fit(X_train,y_train)

## 12. Optimal C value and the score corresponding to it
print(gs.best_params_, gs.best_score_)

```

```
## 13. Validating the "best classifier"
clf_best = LogisticRegression(C = gs.best_params_['C'])
clf_best.fit(X_train,y_train)
y_pred_best = clf_best.predict(X_test)
print(f1_score(y_test,y_pred_best))

## 14. Implement L1 hyperparameter tuning with LogisticRegressionCV
from sklearn.linear_model import LogisticRegressionCV
C_array = np.logspace(-2,2,100)
clf_l1 = LogisticRegressionCV(Cs=C_array, cv = 5, penalty = 'l1', scoring = 'f1', solver =
'liblinear')
clf_l1.fit(X,y)

## 15. Optimal C value and corresponding coefficients
print('Best C value', clf_l1.C_)
print('Best fit coefficients', clf_l1.coef_)

## 16. Plotting the tuned L1 coefficients
coefficients = clf_l1.coef_.ravel()
coef = pd.Series(coefficients,predictors).sort_values()
coef.plot(kind='bar', title = 'Coefficients for tuned L1')
plt.tight_layout()
plt.show()
plt.clf()
```

Handwriting Recognition using K-Means

The [U.S. Postal Service](#) has been using machine learning and scanning technologies since 1999. Because its postal offices have to look at roughly half a billion pieces of mail every day, they have done extensive research and developed very efficient algorithms for reading and understanding addresses. And not only the post office:

ATMs can recognize handwritten bank checks
Evernote can recognize handwritten task lists
Expensify can recognize handwritten receipts

But how do they do it?

In this project, you will be using **K-means clustering** (the algorithm behind this magic) and scikit-learn to cluster images of handwritten digits.

Let's get started!

If you get stuck during this project or would like to see an experienced developer work through it, click "**Get Unstuck**" to see a **project walkthrough video**.

Tasks

0/18 complete

Mark the tasks as complete by checking them off

Getting Started with the Digits Dataset:

1.

The `sklearn` library comes with a digits dataset for practice.

In `script.py`, we have already added three lines of code:

```
import codecademylib3_seaborn
import numpy as np
from matplotlib import pyplot as plt
```

Copy to Clipboard

From `sklearn` library, import the `datasets` module.

Then, load in the `digits` data using `.load_digits()` and print `digits`.

2.

When first starting out with a dataset, it's always a good idea to go through the data description and see what you can already learn.

Instead of printing the `digits`, print `digits.DESCR`.

What is the size of an image (in pixel)?

Where is this dataset from?

3.

Let's see what the data looks like!

Print `digits.data`.

4.

Next, print out the target values in `digits.target`.

5.

To visualize the data images, we need to use Matplotlib. Let's visualize the image at index `100`:

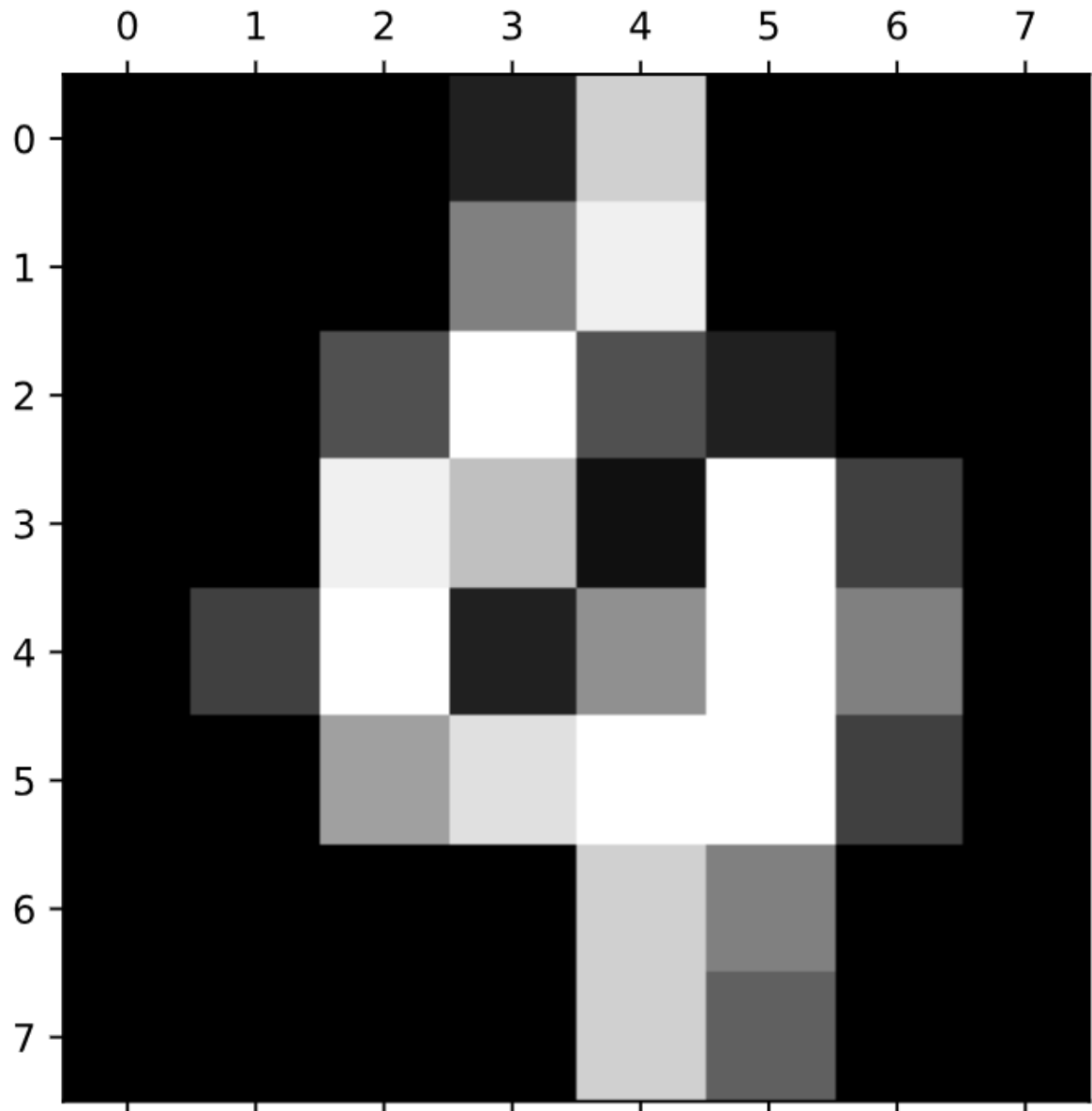
```
plt.gray()

plt.matshow(digits.images[100])

plt.show()
```

Copy to Clipboard

The image should look like:



Is it a 4? Let's print out the target label at index 100 to find out!

```
print(digits.target[100])
```

[Copy to Clipboard](#)

[Open the hint to see how you can visualize more than one image.](#)

K-Means Clustering:

6.

Now we understand what we are working with. Let's cluster the 1797 different digit images into groups.

Import `KMeans` from `sklearn.cluster`.

7.

What should be the `k`, the number of clusters, here?

Use the `KMeans()` method to build a model that finds `k` clusters.

8.

Use the `.fit()` method to fit the `digits.data` to the model.

Visualizing after K-Means:

9.

Let's visualize all the centroids! Because data samples live in a 64-dimensional space, the centroids have values so they can be images!

First, add a figure of size 8x3 using `.figure()`.

Then, add a title using `.suptitle()`.

10.

Scikit-learn sometimes calls centroids "cluster centers".

Write a `for` loop to displays each of the `cluster_centers` like so:

```
for i in range(10):
```

```
    # Initialize subplots in a grid of 2X5, at i+1th position
    ax = fig.add_subplot(2, 5, 1 + i)

    # Display images
    ax.imshow(model.cluster_centers_[i].reshape((8, 8)), cmap=plt.cm.binary)
```

Copy to Clipboard

The cluster centers should be a list with 64 values (0-16). Here, we are making each of the cluster centers into an 8x8 2D array.

11.

Outside of the `for` loop, use `.show()` to display the visualization.

Did it work?

These are the centroids of handwriting from *thirty* different people collected by Bogazici University (Istanbul, Turkey):

- Index 0 looks like 0
- Index 1 looks like 9
- Index 2 looks like 2
- Index 3 looks like 1
- Index 4 looks like 6
- Index 5 looks like 8
- Index 6 looks like 4
- Index 7 looks like 5
- Index 8 looks like 7
- Index 9 looks like 3

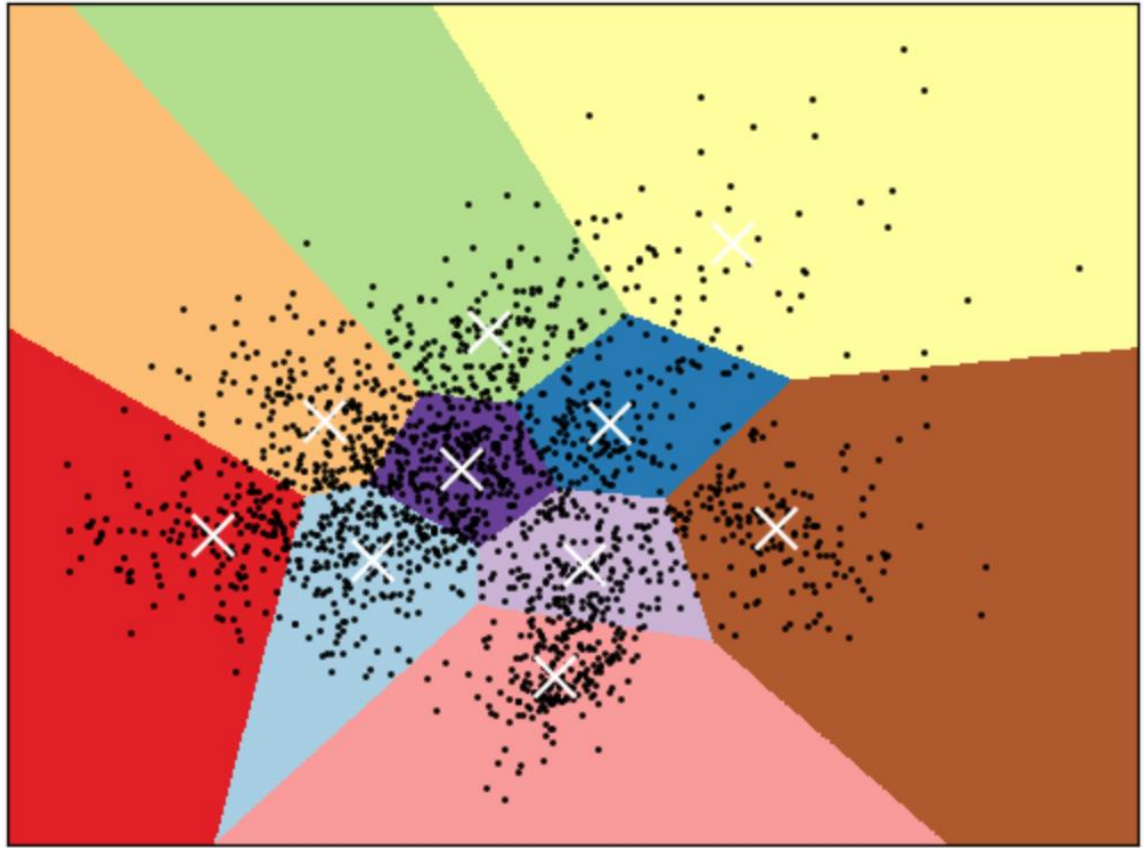
Notice how the centroids that look like 1 and 8 look very similar and 1 and 4 also look very similar.

12.

Optional:

If you want to see another example that visualizes the data clusters and their centers using K-means, check out the `sklearn`'s [own example](#).

K-means clustering on the digits dataset (PCA-reduced data) Centroids are marked with white cross



You will get a different K-means clustering map each time you run a new K-means model.

Testing Your Model:

13.

Instead of feeding new arrays into the model, let's do something cooler!
Inside the right panel, go to **test.html**.

14.

What year will robots take over the world?

Use your mouse to write a digit in each of the boxes and click **Get Array**.

15.

Back in **script.py**, create a new variable named `new_samples` and copy and paste the 2D array into it.

```
new_samples = np.array( )
```

Copy to Clipboard

16.

Use the `.predict()` function to predict new labels for these four new digits. Store those predictions in a variable named `new_labels`.

17.

But wait, because this is a clustering algorithm, we don't know which label is which.

By looking at the cluster centers, let's map out each of the labels with the digits we think it represents:

```
for i in range(len(new_labels)):
    if new_labels[i] == 0:
        print(0, end='')
    elif new_labels[i] == 1:
        print(9, end='')
```



```
elif new_labels[i] == 2:  
    print(2, end='')  
elif new_labels[i] == 3:  
    print(1, end='')  
elif new_labels[i] == 4:  
    print(6, end='')  
elif new_labels[i] == 5:  
    print(8, end='')  
elif new_labels[i] == 6:  
    print(4, end='')  
elif new_labels[i] == 7:  
    print(5, end='')  
elif new_labels[i] == 8:  
    print(7, end='')  
elif new_labels[i] == 9:  
    print(3, end='')
```

[Copy to Clipboard](#)

18.

Is the model recognizing your handwriting?

Remember, this model is trained on handwritten digits of 30 Turkish people (from the 1990's).

PCA Project

In this project, you will classify particles into gamma(signal) or hadrons(background). Given that the features are correlated, you will perform PCA to get a new set of features, and select the features that contain the most information. The data set was generated by a Monte Carlo program, Corsika, described in D. Heck et al., CORSIKA, A Monte Carlo code to simulate extensive air showers, Forschungszentrum Karlsruhe FZKA 6019 (1998).

The first part of the project 'Observing the Dataset' is in `script_1.py`, with its solution in `script_1_solution.py`. The second part of the project 'Performing PCA' is in `script_2.py`, with its solution in `script_2_solution.py`. It is highly recommend that you complete the project on your own without checking the solutions, but feel free to take a look if you get stuck or want to check your answers when you're done!

Tasks

0/14 complete

Mark the tasks as complete by checking them off

Observing the Dataset

1.
Remove any nulls from the dataset.
Use the Pandas method that `df.dropna()` to drop NAN values.
2.
Extract the numerical columns onto a variable named `data_matrix` and the classes to a variable named `classes`
Drop the class label using the pandas method of `df.drop`, as in `df.drop(columns='class')`.
3.
Create a correlation matrix of the data matrix and show it as a heatmap.
Create a correlation matrix using the Pandas method `data_matrix.corr()`.
4.
Find the eigenvectors and eigenvalues using the NumPy function `np.linalg.eig`. Here we also order the eigenvalues from greatest to smallest by ordering its indices first, and use these indices to also order the eigenvectors.
Use the NumPy function `np.linalg.eig(correlation_matrix)` to retrieve eigenvalues and eigenvectors.
5.
Find the proportions of each eigenvalue to the total sum of the eigenvalues. These proportions represent the percentages of information that each eigenvalue's associated eigenvector contains.
Divide the eigenvalues by the sum of the eigenvalues to get the normalized proportions,
`eigenvalues/eigenvalues.sum()`.
6.
Find the cumulative percentages of the ordered eigenvectors.

Use the NumPy method `np.cumsum(information_percents)` to find the cumulative percentages.

Performing PCA

7.
Recall that PCA uses the standardized matrix, which scales for the mean and standard deviation. Calculate the standardized data matrix.
Standardize the data matrix by subtracting the mean and dividing by the standard deviation as `(data_matrix - mean) / stdd`.
8.
Using the sklearn module `PCA`, perform PCA by fitting and transforming the standardized data matrix.
Perform the fit and transform step using the `pca.fit_transform(data_matrix_standardized)`.
9.
Using the properties of the PCA trained object, `pca`, calculate the eigenvalues from the singular values and extract the eigenvectors.
Recall that the singular values and eigenvectors are in the properties of `pca` as `.singular_values_` and `.components.T`, respectively.
10.
Using the properties of the `pca` trained object, extract the variance ratios, which are equivalent to the eigenvalue proportions we calculated earlier.
Recall the variance ratios are in the property of `pca` in `explained_variance_ratio_`.
11.
Perform PCA once again but with 2 principal axes. Make sure to fit and transform the standardized data matrix.

Specify the number of principal axes using the parameter `n_components` in the `PCA` module, as in `PCA(n_components=2)`.

12.

Plot the principal components and have its class as its hue to see if clustering of any kind has occurred.

Use the seaborn function `.lplot` to find the scatter plot with a hue based on the label 'class'.

13.

Once again, we will perform PCA on 2 components, but we will also use the newly transformed PCA features as input to a support vector classifier! Fit the transformed features onto the classifier and generate a score.

Use the `.fit` method of the `svc_1` object as in `svc_1.fit(X_train, y_train)`.

14.

Now, fit the classifier with the first two features of the original standardized data matrix and generate a score.

Looking at the two SVG models, the first model with the PCA features and the second model with the original data features, which model has a higher accuracy score?

Find the score of the testing data using the `svc_2.score(X_test, y_test)` from the `svc_2` object.

Feel free to play around with the features selected for the two SVG models and see what accuracy scores you get! You can change the number of PCA features used in the first SVG model or change which features are selected for the second SVG model.

Random Forests Project

In this project, we will be using a dataset containing census information from UCI's Machine Learning Repository.

By using this census data with a random forest, we will try to predict whether or not a person makes more than \$50,000.

Let's get started!

Datasets

The original data set is available at the UCI Machine Learning Repository:

<https://archive.ics.uci.edu/ml/datasets/census+income>

The dataset has been loaded for you in **script.py** and saved as a dataframe named `df`. Some of the input and output features of interest are:

age: continuous
workclass: Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, Local-gov, State-gov, Without-pay, Never-worked.
education: Bachelors, Some-college, 11th, HS-grad, Prof-school, Assoc-acdm, Assoc-voc, 9th, 7th-8th, 12th, Masters, 1st-4th, 10th, Doctorate, 5th-6th, Preschool
race: White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black
sex: Female, Male
capital-gain: continuous
capital-loss: continuous
hours-per-week: continuous
native country: discrete
income: discrete, >50K, <=50K

Tasks

0/17 complete

Mark the tasks as complete by checking them off

Investigate the data

1.
We will build a random forest classifier to predict the income category. First, take a look at the distribution of income values – what percentage of samples have incomes less than 50k and greater than 50k?
2.
There's a small problem with our data that is a little hard to catch – every string has an extra space at the start. For example, the first row's native-country is " United-States", but we want it to be "United-States". One way to fix this is to select all columns of type `object` and use the string method `.str.strip()`.
3.
Create a features dataframe `x`. This should include only features in the list `feature_cols` and convert categorical features to dummy variables using `pd.get_dummies()`. Include the paramter `drop_first=True` to eliminate redundant features.
4.
Create the output variable `y`, which is binary. It should be 0 when income is less than 50k and 1 when it is greater than 50k.
5.
Split the data into a train and test set with a test size of `20%`.

Build and Tune Random Forest Classifiers by Depth

6.

Instantiate an instance of a `RandomForestClassifier()` (with default parameters). Fit the model on the train data and print the score (accuracy) on the test data. This will act as a baseline to compare other model performances.

7.

We will explore tuning the random forest classifier model by testing the performance over a range of `max_depth` values. Fit a random forest classifier for `max_depth` values from 1-25. Save the accuracy score for the train and test sets in the lists `accuracy_train`, `accuracy_test`.

8.

Find the largest accuracy and the depth this occurs on the test data.

9.

Plot the training and test accuracy of the models versus the `max_depth`.

10.

Refit the random forest model using the `max_depth` from above; save the feature importances in a dataframe. Sort the results and print the top five features.

Create Additional Features and Re-Tune

11.

Looking at the education feature, there are 16 unique values – from preschool to professional school. Rather than adding dummy variables for each value, it makes sense to bin some of these values together. While there are many ways to do this, we will take the approach of combining the values into 3 groups: `High school and less`, `College to Bachelors` and `Masters and more`. Create a new column in `df` for this new features called `education_bin`.

12.

Like we did previously, we will now add this new feature into our feature list and recreate `x`.

13.

As we did before, we will tune the random forest classifier model by testing the performance over a range of `max_depth` values. Fit a random forest classifier for `max_depth` values from 1-25. Save the accuracy score for the train and test sets in the lists `accuracy_train`, `accuracy_test`.

14.

Find the largest accuracy and the depth this occurs on the test data. Compare the results from the previous model tuned.

15.

Plot the training and test accuracy of the models versus the `max_depth`. Compare the results from the previous model tuned.

16.

Refit the random forest model using the `max_depth` from above; save the feature importances in a dataframe. Sort the results and print the top five features. Compare the results from the previous model tuned.

17.

Nice work! Note that the accuracy of our final model increased and one of our added features is now in the top 5 based on importance!

There are a few different ways to extend this project:

- Are there other features that may lead to an even better performance? Consider creating new ones or adding additional features not part of the original feature list.

- Consider tuning hyperparameters based on a different evaluation metric – our classes are fairly imbalanced, AUC or F1 may lead to a different result

- Tune more parameters of the model. You can find a description of all the parameters you can tune in the [Random Forest Classifier documentation](#). For example, see what happens if you tune `max_features` or `n_estimators`.

```
import pandas as pd
import numpy as np
import codecademylib3
```

```

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier, BaggingClassifier,
RandomForestRegressor

from sklearn import tree

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, precision_score, recall_score, fl_score,
roc_auc_score

col_names = ['age', 'workclass', 'fnlwgt', 'education', 'education-num',
             'marital-status', 'occupation', 'relationship', 'race', 'sex',
             'capital-gain', 'capital-loss', 'hours-per-week', 'native-country', 'income']

df = pd.read_csv('adult.data', header=None, names = col_names)

#Distribution of income

print(df.income.value_counts(normalize=True))

#Clean columns by stripping extra whitespace for columns of type "object"

for c in df.select_dtypes(include=['object']).columns:

    df[c] = df[c].str.strip()

feature_cols = ['age',
                'capital-gain', 'capital-loss', 'hours-per-week', 'sex', 'race']

#Create feature dataframe X with feature columns and dummy variables for categorical
features

X = pd.get_dummies(df[feature_cols], drop_first=True)

#Create output variable y which is binary, 0 when income is less than 50k, 1 when it is
greather than 50k

y = np.where(df.income=='<=50K', 0, 1)

```

```

#Split data into a train and test set

x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=1, test_size=.2)

#Instantiate random forest classifier, fit and score with default parameters

rf = RandomForestClassifier()

rf.fit(x_train, y_train)

rf.score(x_test, y_test)

print(f'Accuracy score for default random forest: {round(rf.score(x_test,
y_test)*100,3)}%')

#Tune the hyperparameter max_depth over a range from 1-25, save scores for test and train
set

np.random.seed(0)

accuracy_train=[]

accuracy_test = []

depths = range(1,26)

for i in depths:

    rf = RandomForestClassifier(max_depth=i)

    rf.fit(x_train, y_train)

    y_pred = rf.predict(x_test)

    accuracy_test.append(accuracy_score(y_test, rf.predict(x_test)))

    accuracy_train.append(accuracy_score(y_train, rf.predict(x_train)))

#Find the best accuracy and at what depth that occurs

best_acc= np.max(accuracy_test)

best_depth = depths[np.argmax(accuracy_test)]

print(f'The highest accuracy on the test is achieved when depth: {best_depth}')

print(f'The highest accuracy on the test set is: {round(best_acc*100,3)}%')

#Plot the accuracy scores for the test and train set over the range of depth values

plt.plot(depths, accuracy_test, 'bo--', depths, accuracy_train, 'r*:')

```

```

plt.legend(['test accuracy', 'train accuracy'])

plt.xlabel('max depth')

plt.ylabel('accuracy')

plt.show()

#Save the best random forest model and save the feature importances in a dataframe

best_rf = RandomForestClassifier(max_depth=best_depth)

best_rf.fit(x_train, y_train)

feature_imp_df = pd.DataFrame(zip(x_train.columns, best_rf.feature_importances_),
                               columns=['feature', 'importance'])

print('Top 5 random forest features:')

print(feature_imp_df.sort_values('importance', ascending=False).iloc[0:5])


#Create two new features, based on education and native country

df['education_bin'] = pd.cut(df['education-num'], [0,9,13,16], labels=['HS or less',
'College to Bachelors', 'Masters or more'])

feature_cols = ['age',

                'capital-gain', 'capital-loss', 'hours-per-week', 'sex', 'race', 'education_bin']

#Use these two new additional features and recreate X and test/train split

X = pd.get_dummies(df[feature_cols], drop_first=True)

x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=1, test_size=.2)


#Find the best max depth now with the additional two features

np.random.seed(0)

accuracy_train=[]

accuracy_test = []

depths = range(1,10)

for i in depths:

```



```

    rf = RandomForestClassifier(max_depth=i)

    rf.fit(x_train, y_train)

    y_pred = rf.predict(x_test)

    accuracy_test.append(accuracy_score(y_test, rf.predict(x_test)))

    accuracy_train.append(accuracy_score(y_train, rf.predict(x_train)))

best_acc= np.max(accuracy_test)

best_depth = depths[np.argmax(accuracy_test)]

print(f'The highest accuracy on the test is achieved when depth: {best_depth}')

print(f'The highest accuracy on the test set is: {round(best_acc*100,3)}%')


plt.figure(2)

plt.plot(depths, accuracy_test, 'bo--', depths, accuracy_train, 'r*:')

plt.legend(['test accuracy', 'train accuracy'])

plt.xlabel('max depth')

plt.ylabel('accuracy')

plt.show()

#Save the best model and print the two features with the new feature set

best_rf = RandomForestClassifier(max_depth=best_depth)

best_rf.fit(x_train, y_train)

feature_imp_df = pd.DataFrame(zip(x_train.columns, best_rf.feature_importances_),
columns=['feature', 'importance'])

print('Top 5 random forest features:')

print(feature_imp_df.sort_values('importance', ascending=False).iloc[0:5])

```

Predict Baseball Strike Zones With Machine Learning

<https://www.codecademy.com/paths/data-science/tracks/dsml-supervised-learning-ii-sv-ms-rm-nb/modules/support-vector-machines-skill-path-fe1996b3-8d55-4eb8-b35b-42b0110f2011/projects/baseball>

Support Vector Machines are powerful machine learning models that can make complex decision boundaries. An SVM's decision boundary can twist and curve to accommodate the training data.

In this project, we will use an SVM trained using a baseball dataset to find the decision boundary of the strike zone.

The strike zone can be thought of as a decision boundary that determines whether or not a pitch is a strike or a ball. There is a strict definition of the strike zone — in practice, however, it will vary depending on the umpire or the player at bat.

Let's use our knowledge of SVMs to find the *real* strike zone of several baseball players.

If you get stuck during this project or would like to see an experienced developer work through it, click "**Get Unstuck**" to see a **project walkthrough video**.

Tasks

0/17 complete

[Mark the tasks as complete by checking them off](#)

Create the labels

1.

We've imported several DataFrames related to some of baseball's biggest stars. We have data on [Aaron Judge](#) and [Jose Altuve](#). Judge is one of the tallest players in the league and Altuve is one of the shortest. Their strike zones should be pretty different!

Each row in these DataFrames corresponds to a single pitch that the batter saw in the 2017 season. To begin, let's take a look at all of the features of a pitch. Print `aaron_judge.columns`.

In this project, we'll ask you to print out a lot of information. To avoid clutter, feel free to delete the print statements once you understand the data.

We used the [pybaseball](#) Python package to get the data for this project. If you're interested in getting more data, the documentation for pybaseball can help you get data that you're interested onto your own computer.

2.

Some of these features have obscure names. Let's learn what the feature `description` means.

Print `aaron_judge.description.unique()` to see the different values the `description` feature could have.

3.

We're interested in looking at whether a pitch was a ball or a strike. That information is stored in the `type` feature. Look at the unique values stored in the `type` feature to get a sense of how balls and strikes are recorded.

4.

Great! We know every row's `type` feature is either an `'S'` for a strike, a `'B'` for a ball, or an `'X'` for neither (for example, an `'X'` could be a hit or an out).

We'll want to use this feature as the label of our data points. However, instead of using strings, it will be easier if we change every `'S'` to a 1 and every `'B'` to a 0.

You can change the values of a DataFrame column using the `map()` functions. For example, in the code below, every `'A'` in `example_column` is changed to a 1, and every `'B'` is changed to a 2.

```
df['example_column'] = df['example_column'].map({'A':1, 'B':2})
```

Copy to Clipboard

5.

Let's make sure that worked. Print the `type` column from the `aaron_judge` DataFrame.

Plotting the pitches

6.

There were some `NaN`s in there. We'll take care of those in a second. For now, let's look at the other features we're interested in.

We want to predict whether a pitch is a ball or a strike based on its location over the plate. You can find the ball's location in the columns `plate_x` and `plate_z`.

Print `aaron_judge['plate_x']` to see what that column looks like.

`plate_x` measures how far left or right the pitch is from the center of home plate. If `plate_x = 0`, that means the pitch was directly in the middle of the home plate.

7.

We now have the three columns we want to work with: `'plate_x'`, `'plate_z'`, and `'type'`.

Let's remove every row that has a `NaN` in any of those columns.

You can do this by calling the `dropna` function. This function can take a parameter named `subset` which should be a list of the columns you're interested in.

For example, the following code drops all of the `NaN` values from the columns `'A'`, `'B'`, and `'C'`.

```
data_frame = data_frame.dropna(subset = ['A', 'B', 'C'])
```

Copy to Clipboard

8.

We now have points to plot using Matplotlib. Call `plt.scatter()` using five parameters:

- The parameter `x` should be the `plate_x` column.

- The parameter `y` should be the `plate_z` column.

- To color the points correctly, the parameter `c` should be the `type` column.

- To make the strikes red and the balls blue, set the `cmap` parameter to `plt.cm.coolwarm`.

- To make the points slightly transparent, set the `alpha` parameter to `0.25`.

Call `plt.show()` to see your graph.

`plate_z` measures how high off the ground the pitch was. If `plate_z = 0`, that means the pitch was at ground level when it got to the home plate.

Building the SVM

9.

Now that we've seen the location of every pitch, let's create an SVM to create a decision boundary. This decision boundary will be the *real* strike zone for that player. For this section, make sure to write all of your code below the call to the `scatter` function but above the `show` function.

To begin, we want to validate our model, so we need to split the data into a training set and a validation set.

Call the `train_test_split` function using `aaron_judge` as a parameter.

Set the parameter `random_state` equal to `1` to ensure your data is split in the same way as our solution code.

This function returns two objects. Store the return values in variables named `training_set` and `validation_set`.

10.

Next, create an `SVC` named `classifier` with `kernel = 'rbf'`. For right now, don't worry about setting the `C` or `gamma` parameters.

11.

Call `classifier.fit()` method. This method should take two parameters:

- The training data. This is the `plate_x` column and the `plate_z` column in `training_set`.

- The labels. This is the `type` column in `training_set`.

The code below shows an example of selecting two columns from a DataFrame:

```
two_columns = data_frame[['A', 'B']]
```

Copy to Clipboard

12.

To visualize the SVM, call the `draw_boundary` function. This is a function that we wrote ourselves - you won't find it in `scikit-learn`.

This function takes two parameters:

- The axes of your graph. For us, this is the `ax` variable that we defined at the top of your code.

- The trained SVM. For us, this is `classifier`. Make sure you've called `.fit()` before trying to visualize the decision boundary.

Run your code to see the predicted strike zone!

Note that the decision boundary will be drawn based on the size of the current axes. So if you call `draw_boundary` before calling `scatter` function, you will only see the boundary as a small square.

To get around this, you could manually set the size of the axes by using something like `ax.set_ylim(-2, 2)` before calling `draw_boundary`.

Optimizing the SVM

13.

Nice work! We're now able to see the strike zone. But we don't know how accurate our classifier is yet. Let's find its accuracy by calling the `.score()` method and printing the results.

`.score()` takes two parameters — the points in the validation set and the labels associated with those points.

These two parameters should be very similar to the parameters used in `.fit()`.

14.

Let's change some of the SVM's parameters to see if we can get better accuracy.

Set the parameters of the SVM to be `gamma = 100` and `C = 100`.

This will overfit the data, but it will be a good place to start. Run the code to see the overfitted decision boundary. What's the new accuracy?

15.

Try to find a configuration of `gamma` and `C` that greatly improves the accuracy. You may want to use nested for loops. Loop through different values of `gamma` and `C` and print the accuracy using those parameters. Our best SVM had an accuracy of 83.41%. Can you beat ours?

Explore Other Players

16.

Finally, let's see how different players' strike zones change. Aaron Judge is the tallest player in the MLB. Jose Altuve is the shortest player. Instead of using the `aaron_judge` variable, use `jose_altuve`.

To make this easier, you might want to consider putting all of your code inside a function and using the dataset as a parameter.

We've also imported `dauid_ortiz`.

Note that the range of the axes will change for these players. To really compare the strike zones, you may want to force the axes to be the same.

Try putting `ax.set_ylim(-2, 6)` and `ax.set_xlim(-3, 3)` right before calling `plt.show()`

17.

See if you can make an SVM that is more accurate by using more features. Perhaps the location of the ball isn't the only important feature!

You can see the columns available to you by printing `aaron_judge.columns`.

For example, try adding the `strikes` column to your SVM — the number of strikes the batter already has might have an impact on whether the next pitch is a strike or a ball.

Note that our `draw_boundary` function won't work if you have more than two features. If you add more features, make sure to comment that out!

Try to make the best SVM possible and share your results with us!

Build a Book Recommender System

<https://www.codecademy.com/paths/data-science/tracks/dsml-supervised-learning-ii-sv-ms-rm-nb/modules/learn-recommender-systems/projects/build-a-book-recommender-system>

Recommender systems are used in all sorts of organizations to help users make decisions and, for many companies, earn more revenue. Prototyping simple recommender systems also does not need to take a lot of time. In this project, we will build a book recommender system for Books'R'Us using Surprise.

Books'R'Us is a national bookstore chain that sells books of all sorts to people all over the country. They recently have built their website, and now want to add a book recommender system to their site. We will prepare and train the recommender system using book review data left on their site. This data has been put together in a Pandas DataFrame called `book_ratings`.

Tasks

0/10 complete

Mark the tasks as complete by checking them off

Prepare your data for recommender implementation

1. Take a sneak peek into the dataset by printing the first five rows. How big is the dataset? What are the data types of the different fields?
2. In order to understand these ratings, let's look at a count of all the ratings in the data. Examine the distribution of the ratings using `value_counts`.
3. Unfortunately, it appears we have some data where the ratings are 0. The ratings on the website only go from 1 to 5 inclusive. Filter out all ratings that are not in this range.
4. We need to prepare this data for use in Surprise. First, build a Surprise `reader` Object that utilizes the rating scale established above. Look at the [Surprise documentation](#) to help you out.

Build a recommender system using `Surprise`

5. Load `book_ratings` into a Surprise Dataset so it can be used with Surprise's algorithms.
6. We have a dataset that is ready for use in Surprise. Split the data, and put 80% of the data into a training set, and 20% into a test set. Set a `random_state` of 7 to improve reproducibility.
7. We can finally train a recommender system. Use the `KNNBasic` from Surprise to train a collaborative filter using the training set.
8. How good is the algorithm we trained? Calculate the RMSE of the recommender system using the `testset` data.
9. Time to get a recommendation! User `8842281e1d1347389f2ab93d60773d4d` gave the science-fiction book "The Three-Body Problem" (`book_id=18245960`) a 5. What rating does the algorithm predict this user will give the science-fiction book "The Martian" (`book_id=18007564`)?
- 10.

We have successfully built a working prototype for a recommender system. Try adjusting the hyperparameters of the collaborative filter you built, and see if you can reduce the RMSE of the collaborative filter you built above.

SOLUTIONS

```
import pandas as pd
import codecademylib3
from surprise import Reader

book_ratings = pd.read_csv('goodreads_ratings.csv')
print(book_ratings.head())

#1. Print dataset size and examine column data types
print(len(book_ratings))
print(book_ratings.info())

#2. Distribution of ratings
print(book_ratings['rating'].value_counts())

#3. Filter ratings that are out of range
book_ratings = book_ratings[book_ratings['rating']!=0]

#4. Prepare data for surprise: build a Surprise reader object
from surprise import Reader
reader = Reader(rating_scale=(1, 5))

#5. Load `book_ratings` into a Surprise Dataset
from surprise import Dataset
rec_data = Dataset.load_from_df(book_ratings[['user_id',
                                              'book_id',
                                              'rating']],
                                reader)

#6. Create a 80:20 train-test split and set the random state to 7
from surprise.model_selection import train_test_split
trainset, testset = train_test_split(rec_data, test_size=.2, random_state=7)

#7. Use KNNBasic from Surprise to train a collaborative filter
from surprise import KNNBasic
nn_algo = KNNBasic()
nn_algo.fit(trainset)

#8. Evaluate the recommender system
from surprise import accuracy
predictions = nn_algo.test(testset)
accuracy.rmse(predictions)

#9. Prediction on a user who gave the "The Three-Body Problem" a rating of 5
print(nn_algo.predict('8842281e1d1347389f2ab93d60773d4d', '18007564').est)
```

Email Similarity

<https://www.codecademy.com/paths/data-science/tracks/dsml-supervised-learning-ii-sv-ms-rm-nb/modules/naive-bayes-classifier-skill-path-3ca45f60-810b-4e2a-b234-149f1c5d88f5/projects/naive-bayes-project>

In this project, you will use scikit-learn's **Naive Bayes** implementation on several different datasets. By reporting the accuracy of the classifier, we can find which datasets are harder to distinguish. For example, how difficult do you think it is to distinguish the difference between emails about hockey and emails about soccer? How hard is it to tell the difference between emails about hockey and emails about tech? In this project, we'll find out exactly how difficult those two tasks are.

If you get stuck during this project or would like to see an experienced developer work through it, click "**Get Unstuck**" to see a **project walkthrough video**.

Tasks

0/15 complete

[Mark the tasks as complete by checking them off](#)

Exploring the Data

1.
We've imported a dataset of emails from scikit-learn's datasets. All of these emails are tagged based on their content.
Print `emails.target_names` to see the different categories.
2.
We're interested in seeing how effective our Naive Bayes classifier is at telling the difference between a baseball email and a hockey email. We can select the categories of articles we want from `fetch_20newsgroups` by adding the parameter `categories`.
In the function call, set `categories` equal to the list `['rec.sport.baseball', 'rec.sport.hockey']`
3.
Let's take a look at one of these emails.
All of the emails are stored in a list called `emails.data`. Print the email at index `5` in the list.
4.
All of the labels can be found in the list `emails.target`. Print the label of the email at index `5`.
The labels themselves are numbers, but those numbers correspond to the label names found at `emails.target_names`.
Is this a baseball email or a hockey email?

Making the Training and Test Sets

5.
We now want to split our data into training and test sets. Change the name of your variable from `emails` to `train_emails`. Add these three parameters to the function call:

```
subset='train'
shuffle = True
random_state = 108
```


Adding the `random_state` parameter will make sure that every time you run the code, your dataset is split in the same way.
6.
Create another variable named `test_emails` and set it equal to `fetch_20newsgroups`. The parameters of the function should be the same as before except `subset` should now be `'test'`.

Counting Words

7.

We want to transform these emails into lists of word counts. The `CountVectorizer` class makes this easy for us.

Create a `CountVectorizer` object and name it `counter`.

8.

We need to tell `counter` what possible words can exist in our emails. `counter` has a `.fit()` a function that takes a list of all your data.

Call `.fit()` with `test_emails.data + train_emails.data` as a parameter.

9.

We can now make a list of the counts of our words in our training set.

Create a variable named `train_counts`. Set it equal to `counter`'s `transform` function using `train_emails.data` as a parameter.

10.

Let's also make a variable named `test_counts`. This should be the same function call as before, but use `test_emails.data` as the parameter of `transform`.

Making a Naive Bayes Classifier

11.

Let's now make a Naive Bayes classifier that we can train and test on. Create a `MultinomialNB` object named `classifier`.

12.

Call `classifier`'s `.fit()` function. `.fit()` takes two parameters. The first should be our training set, which for us is `train_counts`. The second should be the labels associated with the training emails. Those are found in `train_emails.target`.

13.

Test the Naive Bayes Classifier by printing `classifier`'s `.score()` function. `.score()` takes the test set and the test labels as parameters.

`.score()` returns the accuracy of the classifier on the test data. Accuracy measures the percentage of classifications a classifier correctly made.

Testing Other Datasets

14.

Our classifier does a pretty good job distinguishing between soccer emails and hockey emails. But let's see how it does with emails about really different topics.

Find where you create `train_emails` and `test_emails`. Change the categories to be

`['comp.sys.ibm.pc.hardware', 'rec.sport.hockey']`.

Did your classifier do a better or worse job on these two datasets?

15.

Play around with different sets of data. Can you find a set that's incredibly accurate or incredibly inaccurate? The possible categories are listed below.

```
'alt.atheism'
'comp.graphics'
'comp.os.ms-windows.misc'
'comp.sys.ibm.pc.hardware'
'comp.sys.mac.hardware'
'comp.windows.x'
'misc.forsale'
'rec.autos'
'rec.motorcycles'
'rec.sport.baseball'
'rec.sport.hockey'
'sci.crypt'
'sci.electronics'
'sci.med'
'sci.space'
'soc.religion.christian'
'talk.politics.guns'
'talk.politics.mideast'
```



```
'talk.politics.misc'  
'talk.religion.misc'
```

```
from sklearn.datasets import fetch_20newsgroups  
  
from sklearn.naive_bayes import MultinomialNB  
  
from sklearn.feature_extraction.text import CountVectorizer  
  
emails = fetch_20newsgroups()
```

Perceptron Logic Gates

In this project, we will use perceptrons to model the fundamental building blocks of computers — logic gates.

Name	AND	OR	XOR																																													
Symbol																																																
Truth Table	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	X	0	0	0	1	0	0	0	1	0	1	1	1	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	X	0	0	0	1	0	1	0	1	1	1	1	1	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	X	0	0	0	1	0	1	0	1	1	1	1	0
A	B	X																																														
0	0	0																																														
1	0	0																																														
0	1	0																																														
1	1	1																																														
A	B	X																																														
0	0	0																																														
1	0	1																																														
0	1	1																																														
1	1	1																																														
A	B	X																																														
0	0	0																																														
1	0	1																																														
0	1	1																																														
1	1	0																																														

For example, the table below shows the results of an AND gate. Given two inputs, an AND gate will output a 1 only if both inputs are a 1:

Input 1	Input 2	Output
0	0	0
0	1	0
1	0	0
1	1	1

We'll discuss how an AND gate can be thought of as linearly separable data and train a perceptron to perform AND.

We'll also investigate an XOR gate — a gate that outputs a 1 only if one of the inputs is a 1:

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0

We'll think about why an XOR gate isn't linearly separable and show how a perceptron fails to learn XOR.

If you get stuck during this project or would like to see an experienced developer work through it, click ["Get Unstuck"](#) to see a **project walkthrough video**.

Tasks

0/16 complete

[Mark the tasks as complete by checking them off](#)

Creating and visualizing AND Data

1.

To begin, let's think of an AND gate as a dataset of four points. The four points should be the four possible inputs to the AND gate. For example, the first point in the dataset should be `[0, 0]`.

Create a variable named `data` that is a list that contains the four possible inputs to an AND gate.

2.

Each data point should have a label associated with it. The label will be the result of the AND gate given the input.

Create a variable named `labels`. This should be a list where each label corresponds to a point in `data`. For example, if the last item in `data` is `[1, 1]`, the last label should be `1`.

3.

Let's plot these four points on a graph.

Call `plt.scatter()` using three parameters:

The first parameter should be a list of the x values of each point. You can get this using list comprehension — `[point[0] for point in data]`.

The second parameter should be a list of the y values of each point. The third parameter should be `c = labels`.

This will make the points with label `1` a different color than points with label `0`.

After calling `plt.scatter()` call `plt.show()`. As you continue to write your code, make sure `plt.show()` is always the last line of code.

Why is this data linearly separable?

Building the Perceptron

4.

Now that we have our data, let's build a perceptron to learn AND. Create a `Perceptron` object named `classifier`. For now, set the parameter `max_iter` to 40, and set `random_state` to 22.

`max_iter` sets the number of times the perceptron loops through the training data. The default is 1000, so we're cutting the training pretty short! Let's see if our algorithm learns AND, even with very little training.

5.

We'll now train the model. Call the `.fit()` method using `data` and `labels` as parameters.

6.

Let's see if the algorithm learned AND. Call `classifier's .score()` method using `data` and `labels` as parameters. Print the results. This will print the accuracy of the model on the data points.

Note that it is pretty unusual to train and test on the same dataset. In this case, since there are only four possible inputs to AND, we're stuck training on every possible input and testing on those same points.

7.

Your perceptron should have 100% accuracy! You just taught it an AND gate!

Let's change the labels so your data now represents an XOR gate. The label should be a `1` only if one of the inputs is a `1`.

What is the accuracy of the perceptron now? Is the data linearly separable?

8.

Try changing the data to represent an OR gate. The label of a point should be a `1` if *any* of the input values are a `1`.

Before running your code, predict what will happen. Is the data linearly separable? What do you expect the accuracy of the perceptron to be?

Visualizing the Perceptron

9.

We know the perceptron has been trained correctly, but let's try to visualize what decision boundary it is making. Reset your labels to be representing an AND gate.

Let's first investigate the classifier's `.decision_function()` method. Given a list of points, this method returns the distance those points are from the decision boundary. The closer the number is to `0`, the closer that point is to the decision boundary.

Try calling `classifier's .decision_function()` method using `[[0, 0], [1, 1], [0.5, 0.5]]` as a parameter. Print the results.

Is the point `[0, 0]` or the point `[1, 1]` closer to the decision boundary?

10.

Even though an input like `[0.5, 0.5]` isn't a *real* input to an AND logic gate, we can still check to see how far it is from the decision boundary.

We could also do this to the point `[0, 0.1]`, `[0, 0.2]` and so on. If we do this for a grid of points, we can make a heat map that reveals the decision boundary.

To begin, we need to create a list of the points we want to input to `.decision_function()`.

Begin by creating a list named `x_values`. `x_values` should be a list of 100 evenly spaced decimals between `0` and `1`.

`np.linspace(0, 1, 100)` will do this.

Do the same for `y_values`.

11.

We have a list of 100 x values and 100 y values. We now want to find every possible combination of those x and y values.

The function product will do this for you. For example, consider the following code:

```
list(product([1, 2, 3], [4, 5, 6]))
```

Copy to Clipboard

This code will produce the following list:

```
[(1, 4), (1, 5), (1, 6), (2, 4), (2, 5), (2, 6), (3, 4), (3, 5), (3, 6)]
```

Copy to Clipboard

Call `product()` using `x_values` and `y_values` as parameters. Don't forget to put `list()` around the call to `product()`.

Store the result in a variable named `point_grid`.

12.

Call `classifier's .decision_function()` method using `point_grid` as a parameter. Store the results in a variable named `distances`.

13.

Right now `distances` stores positive and negative values. We only care about how far away a point is from the boundary — we don't care about the sign.

Take the absolute value of every distance. Use list comprehension to call `abs()` on every point in the list and store it in a new variable called `abs_distances`.

14.

We're almost ready to draw the heat map. We're going to be using Matplotlib's `pcolormesh()` function.

Right now, `abs_distances` is a list of 10000 numbers. `pcolormesh` needs a two dimensional list. We need to turn `abs_distances` into a 100 by 100 two dimensional array.

Numpy's `reshape` function does this for us. The code below turns list `lst` into a 2 by 2 list.

```
lst = [1, 2, 3, 4]
new_lst = np.reshape(lst, (2, 2))
```

Copy to Clipboard

`new_lst` now looks like this:

```
[[1, 2], [3, 4]]
```

Copy to Clipboard

Turn `abs_distances` into a 100 by 100 list and name it `distances_matrix`.

15.

It's finally time to draw the [heat map](#)! Call `plt.pcolormesh()` with the following three parameters:

```
x_values
y_values
distances_matrix
```

Save the result in a variable named `heatmap`.

Then call `plt.colorbar()` using `heatmap` as a parameter. This will put a legend on the heat map.

Make sure `plt.show()` is still below these function calls.

16.

Great work! You now have a great visualization of what the perceptron is doing. You should see a purple line where the distances are 0. That's the decision boundary!

Change your labels back to representing an OR gate. Where does the decision boundary go?

Change your labels to represent an XOR gate. Remember, this data is not linearly separable. Where does the decision boundary go?

Perceptrons can't solve problems that aren't linearly separable. However, if you combine multiple perceptrons together, you now have a neural net that can solve these problems!

This is incredibly similar to logic gates. AND gates and OR gates can't produce the output of XOR gates, but when you combine a few ANDs and ORs, you can make an XOR!

Data Scientist: Inference Specialist

Life Expectancy By Country

Over the course of the past few centuries, technological and medical advancements have helped increase the life expectancy of humans. However, as of now, the average life expectancy of humans varies depending on what country you live in.

In this project, we will investigate a dataset containing information about the average life expectancy in 158 different countries. We will specifically look at how a country's economic success might impact the life expectancy in that area.

Setup Instructions

You have two options of completing this assignment. Either here, within Codecademy's output terminal, or on your own, in case you're more comfortable using a Jupyter notebook. If you choose to do this project on your computer instead of Codecademy, you can download what you'll need by clicking the "Download" button below. If you need help setting up your computer, be sure to check out our setup guides:

[Command Line Interface Setup](#)
[Introducing Jupyter Notebook](#)
[Setting up Jupyter Notebook](#)
[Getting Started with Jupyter](#)
[Getting More out of Jupyter Notebook](#)

Open **Life Expectancy By Country.ipynb** and follow the steps in the Jupyter Notebook. If you get stuck, you can look at **Life Expectancy By Country_Solution.ipynb** for the answer.

Download

Tasks

12/12 complete

Mark the tasks as complete by checking them off

Access the Data

1.

We've imported a dataset containing the life expectancy in different countries. The data can be found in the variable named `data`.

To begin, let's get a sense of what this data looks like. Print `data.head()` to see the first 5 rows of the dataset.

Look at the names of the columns. What other pieces of information does this dataset contain?

You may want to comment out this print statement after looking at the data.

2.

Let's isolate the column that contains the life expectancy and store it in a variable named `life_expectancy`. To get a single column from a Panda DataFrame, use this syntax:

```
single_column = dataframeName["columnName"]
```

Copy to Clipboard

Make sure to pay attention to capitalization and spaces when using the column name!

Find the Quantiles

3.

We can now use NumPy functions on that column! Let's use the `np.quantile()` function to find the quartiles of `life_expectancy`. Store the result in a variable named `life_expectancy_quartiles` and print the results.

4.

Nice work! By looking at those three values you can get a sense of the spread of the data. For example, it seems like some of the data is fairly close together — a quarter of the data is between 72.5 years and 75.4 years.

Could you predict what the histogram might look like from those three numbers? Plot the histogram by using the following two lines of code. Does it look how you expected?

```
plt.hist(life_expectancy)
plt.show()
```

Copy to Clipboard

5.

Let's take a moment to think about the meaning of these quartiles. If your country has a life expectancy of 70 years, does that fall in the first, second, third, or final quarter of the data?

Click on the hint to see the answer!

Splitting the Data by GDP

6.

GDP is a measure of a country's wealth. Let's now use the GDP data to see if life expectancy is affected by this value.

Let's split the data into two groups based on GDP. If we find the median GDP, we can create two datasets for "low GDP countries" and "high GDP countries".

To start, let's isolate the GDP column and store it in a variable named `gdp`. This should be similar to how you isolated the life expectancy column.

7.

We now want to find the median GDP. You can use NumPy's `np.median()` function, but since the median is also a quantile, we can call `np.quantile()` using 0.5 as the second parameter.

Store the median in a variable named `median_gdp`. Print that variable to see the median.

8.

Let's now grab all of the rows from our original dataset that have a GDP less than or equal to the median. The following code will do that for you

```
low_gdp = data[data['GDP'] <= median_gdp]
```

Copy to Clipboard

Do the same for all of the rows that have a GDP higher than the median. Store those rows in a variable named `high_gdp`.

The line of code should look almost identical to the one above, but you should change the `<=` to `>`.

Remember to change the name of the variable!

9.

Now that we've split the data based on the GDP, let's see how the life expectancy of each group compares to each other.

Find the quartiles of the "Life Expectancy" column of `low_gdp`. Store the quartiles in a variable named `low_gdp_quartiles`. Print the results.

10.

Find the quartiles of the high GDP countries and store them in a variable named `high_gdp_quartiles`. This should look very similar to the last line of code you wrote. Print the results.

Histogram and Conclusions

11.

By looking at the quartiles, you should get a sense of the spread and central tendency of these two datasets. But let's plot a histogram of each dataset to really compare them.

Remove the two lines of code that are currently plotting the histogram of the original dataset. At the bottom of your code, add these four lines:

```
plt.hist(high_gdp["Life Expectancy"], alpha = 0.5, label = "High GDP")
plt.hist(low_gdp["Life Expectancy"], alpha = 0.5, label = "Low GDP")
```

```
plt.legend()
plt.show()
```

Copy to Clipboard

12.

We can now truly see the impact GDP has on life expectancy.

Once again, consider a country that has a life expectancy of 70 years. If that country is in the top half of GDP countries, is it in the first, second, third, or fourth quarter of the data with respect to life expectancy? What if the country is in the bottom half of GDP countries? Check the hint to see our thoughts.

```
import codecademylib3_seaborn
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("country_data.csv")

# q1
print(data.head())

# q2
life_expectancy = data['Life Expectancy']

# q3
life_expectancy_quartiles = tuple(np.quantile(life_expectancy, (0.25, 0.50, 0.75)))
print(life_expectancy_quartiles)

plt.hist(life_expectancy) # , edgecolor='white', bins=7)
plt.xlim(45, 85)
plt.show()

q1, q2, q3 = life_expectancy_quartiles

age = 70
for q in life_expectancy_quartiles:
    if q <= q1:
        print(f"Age of {age} belongs to FIRST quartile.")
    elif q <= q2:
        print(f"Age of {age} belongs to SECOND quartile.")
    elif q <= q3:
        print(f"Age of {age} belongs to THIRD quartile.")
    else:
        print(f"Age of {age} belongs to THIRD quartile.")

print(q1, q2, q3)

gdp = data['GDP']

np.quantile(gdp, .5)
```


Data Scientist: Inference Specialist

Healthcare in Different States

In this project, we will use boxplots to investigate the way hospitals in different states across the United States charge their patients for medical procedures.

This dataset might look a bit familiar to you — we used it in our lesson about describing histograms. The data originally came from the [United State Health and Human Services Department](#).

Let's use boxplots to find more meaning in this data!

Tasks

12/12 complete

Mark the tasks as complete by checking them off

Investigate the Data

1.

We've imported the dataset into a variable named `healthcare`. Let's take a look at what data we have to work with. Print `healthcare.head()`. This will print the first five rows of the dataset.

Scroll through the table to see what information we have. We know that we want to eventually look at the way healthcare works in different states. What column will be useful to do that?

Note that we're going to ask you to print out multiple values over the course of the project. Feel free to comment out old print statements using `#` to avoid clutter:

```
# This is a comment
```

Copy to Clipboard

2.

We're going to focus on only the ways in which hospitals charge patients for chest pain. But you could do this for any of the diagnoses in our dataset. Print `healthcare["DRG Definition"].unique()` to see all of the different diagnoses in our dataset.

Can you find the official name of the diagnosis related to chest pain?

3.

Let's grab only the rows in the dataset that are about chest pain. Use this line of code to do that.

```
chest_pain = healthcare[healthcare['DRG Definition'] == '313 - CHEST PAIN']
```

Copy to Clipboard

If you're interested in investigating a different diagnosis, replace `'313 - CHEST PAIN'` with the name of the other diagnosis. You might want to change the new variable name to be something other than `chest_pain` if you do this!

Separating By State

4.

We now want to separate the dataset by state. Eventually, we'll use a `for` loop to do this for every state, but for now, let's prove to ourselves that we can make a boxplot for one state.

When we printed the head, we saw the first few rows had a `'Provider State'` of `"AL"`. Those were hospitals in Alabama.

To get every chest pain diagnosis in Alabama, use this line of code:

```
alabama_chest_pain = chest_pain[chest_pain['Provider State'] == "AL"]
```

Copy to Clipboard

Make sure to use the variable that you created in the previous step. We called it `chest_pain`, but you might have named it something else if you used a different diagnosis.

Again, feel free to use a state of your choosing instead of Alabama. Different state abbreviations should work, like `"CO"` or `"NY"`. Name your variable appropriately!

5.

We're almost there! We now have all of the hospitals in Alabama that have a diagnosed chest pain. We now want to find the average cost of those diagnoses. These value is stored in the column `' Average Covered Charges '` (Note the spaces at the start and the end of the string!)

To get only these values, call this line of code:

```
costs = alabama_chest_pain[' Average Covered Charges '].values
```

Copy to Clipboard

Again, make sure that you're using the correct variable names – yours might be different.

6.

Let's now make a boxplot of those values! Call `plt.boxplot()` using `costs` as the first parameter.

Then call `plt.show()` to see your boxplot!

Making a Boxplot for All States

7.

Nice work! We've made a boxplot for one state. But how does that state compare to the others? Let's make a boxplot for every state! Comment out (or delete) the two lines of code you just wrote to make the boxplot. To begin, we first need to create a list of all the states in our dataset. We can once again use the `unique()` function. We used this function when we looked at all of the diagnoses.

Find all of the unique states from the dataset `chest_pain` and store it in a variable named `states`.

8.

We'll now use a `for` loop to separate the dataset into a dataset for each state:

```
datasets = []
for state in states:
    datasets.append(chest_pain[chest_pain['Provider State'] == state]['
Average Covered Charges '].values)
```

Copy to Clipboard

`datasets` now contains 50 datasets – one for each state.

9.

We're about to draw 50 boxplots. Before we draw them, let's make sure there's enough room. Call `plt.figure(figsize=(20,6))`. This will make your figure long to allow room for so many boxplots!

10.

Draw the boxplot using `datasets` as the first parameter. Add the second parameter `labels = states` to label your boxplots.

Finally, make sure to call `plt.show()`.

Interpret and Explore

11.

Nice work! You should now see 50 boxplots. If the graph is too small to see, you can expand it by clicking on the arrows in the top right corner of the right panel. You can also take a look at [this image](#) to see what your final results should look like.

What information have you learned by looking at these boxplots side by side? What state has the largest spread? What state has the largest median? Which states have the most outliers?

Check the hint to see our thoughts.

12.

Take some time to explore more from here. Here are some ways in which you can investigate the data more:

- Look at diagnoses other than `'313 - CHEST PAIN'`.

- Group states by regions. Maybe hospitals in the Northeast charge patients differently than hospitals in the South.

Plot something other than 'Average Covered Charges'. You have data about how much Medicare pays in the 'Average Medicare Payments' column.

```
import codecademylib3_seaborn

import pandas as pd

from matplotlib import pyplot as plt

healthcare = pd.read_csv("healthcare.csv")
```

Data Scientist: Inference Specialist

Summarizing Automobile Evaluation Data

In the following project you'll use what you've learned about summarizing categorical data to analyze a sample from a popular open source dataset. This dataset contains information on the cost and physical attributes of several thousand cars. Originally, this dataset was used for to train a classification model that assigned an acceptability score/category to cars based on these attributes.

The car evaluation dataset has been sourced from the UCI Machine Learning Repository and has been slightly modified for this project. Specifically, one additional field `manufacturer_country` has been simulated for illustrative purposes. You can read more about the details, features, and original uses of this dataset in research on the [UCI data description page](#).

Setup Instructions

You have two options of completing this assignment. Either here, within Codecademy's output terminal, or on your own, in case you're more comfortable using a Jupyter notebook. If you choose to do this project on your computer instead of Codecademy, you can download what you'll need by clicking the "Download" button below. If you need help setting up your computer, be sure to check out our setup guides:

[Command Line Interface Setup](#)
[Introducing Jupyter Notebook](#)
[Setting up Jupyter Notebook](#)
[Getting Started with Jupyter](#)
[Getting More out of Jupyter Notebook](#)

Open **Summarizing Automobile Evaluation Data.ipynb** and follow the steps in the Jupyter Notebook. If you get stuck, you can look at **Summarizing Automobile Evaluation Data_Solution.ipynb** for the answer.

Download

Tasks

11/11 complete

Mark the tasks as complete by checking them off

Summarizing Manufacturing Country

1.
`manufacturer_country` is a *nominal categorical variable* that indicates the country of the manufacturer of each car reviewed. Create a table of frequencies of all the cars reviewed by `manufacturer_country`. What is the modal category? Which country appears 4th most frequently? Print out your results.
2.
Calculate a table of proportions for countries that appear in `manufacturer_country` in the dataset. What percentage of cars were manufactured in Japan?

Summarizing Buying Costs

3.
`buying_cost` is a categorical variable which describes the cost of buying any car in the dataset. Print out a list of the possible values for this variable.
4.
`buying_cost` is an *ordinal categorical variable*, which means we can create an order associated with the values in the data and perform additional numeric operations on the variable. Create a new list, `buying_cost_categories`, that contains the unique values in `buying_cost`, ordered from lowest to highest.
5.
Convert `buying_cost` to type `'category'` using the list you created in the previous exercise.
6.
Calculate the median category of the `buying_cost` variable and print the result.

Summarizing Luggage Capacity

7.

`luggage` is a categorical variable in the car evaluations dataset that records the luggage capacity for each reviewed car. Calculate a table of proportions for this variable and print the result.

8.

Are there any missing values in this column? Replicate the table of proportions from the previous exercise, but do not drop any missing values from the count. Print your result.

9.

Without passing `normalize = True` to `.value_counts()`, can you replicate the result you got in the previous exercises?

Summarizing Passenger Capacity

10.

`doors` is a categorical variable in the car evaluations dataset that records the count of doors for each reviewed car. Find the count of cars that have 5 or more doors. You can identify cars with 5+ doors by looking for cars that have a value of '5more' in the `doors` column. Print your result.

11.

Find the proportion of cars that have 5+ doors and print the result.

Code Review

Get targeted feedback about your code and how well it meets the objective of the project. You can request 1 AI-powered review per project when you complete all tasks.

SOLUTION:

```
import pandas as pd
import numpy as np

car_eval = pd.read_csv('car_eval_dataset.csv')

# Task Group 1

# 1. Create a table of frequencies of all the cars reviewed by
"manufacturer_country".
# What is the modal category? Which country appears 4th most frequently?

# Note: `.value_counts()` produces a table of frequencies in order, you
can reference the
# N-th row of this table to find the Nth most common value in the data. In
this case, it's Germany and the US...
print(car_eval.manufacturer_country.value_counts())

# 2. Calculate a table of proportions for countries that appear in
"manufacturer_country" in the dataset.

# Note: Using normalize w. `.value_counts()` normalizes the table of
frequencies that `.value_counts()` produces by default to a table of
```

```

proportions. A table of frequencies takes the count of observations, a
table of proportions takes the proportion each value represents of the
total.
print(car_eval.manufacturer_country.value_counts(normalize=True))

# Japan          0.228

# Task Group 2

# 3. "buying_cost" is a categorical variable which describes the cost of
buying any car in the dataset.
# Note: The `.unique()` method strips all unique values from a column.

print(car_eval["buying_cost"].unique())

# 4. Create a list of the unique categories in from lowest to highest
# cost in the "buying_cost" variable.

# NOTE: Learner just needs to create their own list and save it for use in
# subsequent steps
buying_cost_categories = ['low', 'med', 'high', 'vhigh']
print(buying_cost_categories)

# 5. Convert `buying_cost` to type `category` using the order created in
the previous
# exercise.

# Note: You can convert a field to type category using the function
`pandas.Categorical()`. The pandas categorical type allows us to perform
numeric operations on categorical data.

# You can also check the column has type category by checking
`print(car_eval.buying_cost)` The output out that should note the column
type (see below)

car_eval["buying_cost"] = pd.Categorical(
    car_eval["buying_cost"],
    buying_cost_categories,
    ordered=True
)

print(car_eval.buying_cost)

# Name: buying_cost, Length: 1000, dtype: category
# Categories (4, object): [low < med < high < vhigh]

# 6. Calculate the median category of the `buying_cost` variable.

# Note: In Python, you can use `np.median()` to calculate the median value
of a numerical series. In this case, you also must access the numerical

```

values of the categories. This can be done with the ``.cat.codes`` attribute.

```
median_category_num = np.median(car_eval['buying_cost'].cat.codes)
print(median_category_num)
```

```
median_category = buying_cost_categories[int(median_category_num)]
print(median_category)
```

Task Group 3

7. Calculate a table of proportions for 'luggage'.

Note: Using `normalize w. `.value_counts()`` normalizes the
table of frequencies that ``.value_counts()`` produces by default
to a table of proportions

```
print(car_eval.luggage.value_counts(normalize=True))
```

8. Are there any missing values in this column? Replicate the table of
proportions from the previous exercise, but do not exclude any missing
values from the count.

Note: Using ``.value_counts()``, missing values are removed by default. To
keep missing values in the summary you can pass ``.dropna = False`` to
``.value_counts()``. If the result using ``.dropna = False`` is the same, then
you can conclude there are no missing values. In this case, we see they
are the same.

```
print(car_eval.luggage.value_counts(dropna=False, normalize=True))
```

9. Without passing ``.normalize = True`` to ``.value_counts()``,
can you replicate the result you got in the previous exercise?

Note: This method relies on ``.luggage`` having no null values.
If a field does have nulls The below is a more robust solution,
using the ``.count()`` method excludes NULLs in the denominator just
as ``.value_counts(normalize=True)`` does

```
print(car_eval.luggage.value_counts()/len(car_eval.luggage))
```

Safe alternative if there are Nulls:

```
car_eval.luggage.value_counts()/car_eval.luggage.count()
```

Task Group 4

10/11. Find the frequency and proportion of cars that have 5 or more
doors.

```
# Note: You must first create a series that evaluates to true/false
values.
# Then, by calling .sum() and .mean() on this series the value and count
of
# `True`/1 values are calculated

frequency = (car_eval.doors == '5more').sum()
proportion = (car_eval.doors == '5more').mean()
print(frequency)
print(proportion)
```


Data Scientist: Inference Specialist

Startup Transformation

In this project, you'll work as a data analyst for a tech startup that is looking to improve its operations after a global pandemic has taken the world by storm.

You will apply data transformation techniques to make better sense of the company's data and help answer important questions such as:

- Is the company in good financial health?
- Does the company need to let go of any employees?
- Should the company allow employees to work from home permanently?

Let's get started!

Tasks

21/21 complete

[Mark the tasks as complete by checking them off](#)

Analyzing Revenue and Expenses

1.
The management team of the company you work for is concerned about the status of the company after a global pandemic.
The CFO (Chief Financial Officer) asks you to perform some data analysis on the past six months of the company's financial data, which has been loaded in the variable `financial_data`.
First, examine the first few rows of the data using `print()` and `.head()`.

2.
Notice that `financial_data` has three columns – `Month`, `Revenue`, and `Expenses`.
Store each column in three separate variables called `month`, `revenue`, and `expenses`.

3.
Next, use the following code to create a plot of revenue over the past six months:

```
plt.plot(month, revenue)
plt.show()
```

Copy to Clipboard

4.
On the right, you should now see a plot of revenue over time. You can label and format the figure using the following functions:

```
plt.xlabel('Month')
plt.ylabel('Amount ($)')
plt.title('Revenue')
```

Copy to Clipboard

These should be added before `plt.show()`. Add the labels to your plot.

5.
Repeat steps 3 and 4 for monthly expenses. In other words, create a second plot of monthly expenses over the past 6 months. Note that you'll need to use the function `plt.clf()` prior to creating this new plot. Otherwise, it will be plotted on-top of the revenue plot. The code should look something like this:

```
plt.clf()
#insert code to create plot here
#add labels to the plot here
plt.show()
```

Copy to Clipboard

How are monthly expenses changing over time?

6.

As shown, revenue seems to be quickly decreasing while expenses are increasing. If the current trend continues, expenses will soon surpass revenues, putting the company at risk.

After you show this chart to the management team, they are alarmed. They conclude that expenses must be cut immediately and give you a new file to analyze called `expenses.csv`.

Use `pandas` to read in `expenses.csv` and store it in a variable called `expense_overview`.

Print the first seven rows of the data.

Pie Chart and Collapsing Categories

7.

Notice that there are two columns:

`Expense`: indicates the expense category

`Proportion`: indicates how much of the overall expenses a specific category takes up

Store the `Expense` column in a variable called `expense_categories` and the `Proportion` column in a variable called `proportions`.

8.

Next, we want to create a pie chart of the different expense categories. Use `plt.clf()` again to clear the previous plot, then create a pie chart using the `plt.pie()` method, passing in two arguments:

```
proportions
labels = expense_categories
```

Give your pie chart a title using `plt.title()`, then use `plt.show()` at the end to show the plot.

9.

Notice that the pie chart currently looks deformed.

Above `plt.show()`, add in the following two lines of code to set the axis and adjust the spacing:

```
plt.axis('Equal')
plt.tight_layout()
```

Copy to Clipboard

Take a moment to look at the pie chart. Which expense categories make up most of the data, and which ones aren't so significant?

10.

It seems that `Salaries`, `Advertising`, and `Office Rent` make up most of the expenses, while the rest of the categories make up a small percentage.

Before you hand this pie chart back to management, you would like to update the pie chart so that all categories making up less than 5% of the overall expenses (`Equipment`, `Utilities`, `Supplies`, and `Food`) are collapsed into an "Other" category.

Update the pie chart accordingly.

11.

You should now see four categories in your updated pie chart:

```
Salaries
Advertising
Office Rent
Other
```

This simplified pie chart helps the management team see a big picture view of the company's expenses without getting distracted by noisy data.

If the company wants to cut costs in a big way, which category do you think they should focus on? Put your answer in a string variable called `expense_cut`.

Employee Productivity

12.

Salaries make up 62% of expenses. The management team determines that to cut costs in a meaningful way, they must let go of some employees.

Each employee at the company is assigned a productivity score based on their work. The management would like to keep the most highly productive employees and let go of the least productive employees.

First, use `pandas` to load in `employees.csv` and store it in a variable called `employees`.

Print the first few rows of the data.

13.

Notice that there is a `Productivity` column, which indicates the productivity score assigned to that employee.

Sort the `employees` data frame (in ascending order) by the `Productivity` column and store the result in a variable called `sorted_productivity`.

To sort a data frame, you can do the following:

```
sorted_data = dataframe_name.sort_values(by=['Column Name'])
```

Copy to Clipboard

Print `sorted_productivity`.

14.

You should now see the employees with the lowest productivity scores at the top of the data frame.

The company decides to let go of the 100 least productive employees.

Store the first 100 rows of `sorted_productivity` in a new variable called `employees_cut` and print out the result.

Unfortunately, this batch of employees won't be so lucky.

15.

Your colleague Sarah, a data scientist at the company, would like to explore the relationship between `Income` and `Productivity` more in depth, but she points out that these two features are on vastly different scales. For example, productivity is a feature that ranges from 0-100, but income is measured in the thousands of dollars.

Moreover, there are outliers in the data that add an additional layer of complexity.

She asks you for advice on how she should transform the data. Should she perform normalization, standardization, log transformation, or something else?

Put your answer in a string in a variable called `transformation`.

Commute Times and Log Transformation

16.

The COO (Chief Operating Officer) is debating whether to allow employees to continue to work from home post-pandemic.

He first wants to take a look at roughly how long the average commute time is for employees at the company. He asks for your help to analyze this data.

The `employees` data frame has a column called `Commute Time` that stores the commute time (in minutes) for each employee.

Create a variable called `commute_times` that stores the `Commute Time` column.

17.

Let's do some quick analysis on the commute times of employees.

Use `print()` and `.describe()` to print out descriptive statistics for `commute_times`.

What are the average and median commute times? Might it be worth it for the company to explore allowing remote work indefinitely so employees can save time during the day?

18.

Let's explore the shape of the commute time data using a histogram.

First, use `plt.clf()` to clear the previous plots. Then use `plt.hist()` to plot the histogram of `commute_times`. Finally, use `plt.show()` to show the plot. Feel free to add labels above `plt.show()` if you would like to practice!

What do you notice about the shape of the data? Is it symmetric, left skewed, or right skewed?

19.

The data seems to be skewed to the right. To make it more symmetrical, we might try applying a log transformation.

Right under the `commute_times` variable, create a variable called `commute_times_log` that stores a log-transformed version of `commute_times`.

To apply log-transform, you can use `numpy's log()` function.

20.

Replace the histogram for `commute_times` with one for `commute_times_log`.

Notice how the shape of the data changes from being right skewed to a more symmetrical (and even slightly left-skewed) in shape. After applying log transformation, the transformed data is more "normal" than before.

Extra

21.

Congratulations! In this project, you performed data analysis to help a management team answer important questions about the status of the company during a difficult time.

You did this by analyzing data sets and applying common data transformation techniques. These are important skills to have as a data analyst.

If you'd like extra practice with data transformations, here are some suggestions to extend this project:

- Apply standardization to the employees data using `StandardScaler()` from `sklearn`. Refer to [this](#) article if you need help.

- Explore the income and productivity features in more detail. Can you find a relationship between productivity and income?

Happy coding!

Data Scientist: Inference Specialist

Image Transformations with NumPy

In this project, we are going to create and transform images using the power of NumPy arrays and linear algebra. We will see how we can represent NumPy arrays as images and transform those images by treating them as matrices.

Note that A **solution.py** file is loaded for you in the workspace, which contains the solution code for this project. We highly recommend that you complete the project on your own without checking the solution, but feel free to take a look if you get stuck or want to check your answers when you're done!

Tasks

9/9 complete

Mark the tasks as complete by checking them off

Using the Helper Function

1.

In **script.py**, there are two things preloaded for you:

`heart_img`: a NumPy array representing a 7x7 matrix of RGB values

`show_image()`: a helper function that will allow you to show images easily throughout your project

We are going to use `show_image()` to reveal the photo `heart_img` represents. Before we do so, let's go over the functionality of `show_image()`.

`show_image()` takes in two parameters:

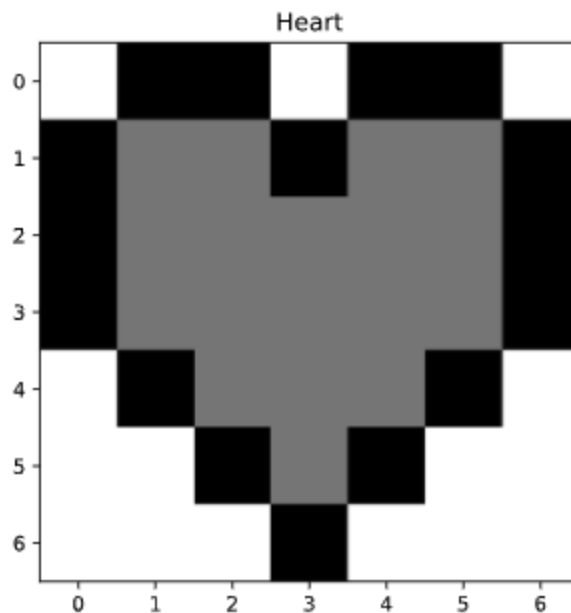
`image`: a NumPy array of RGB values

`name_identifier`: a string to title your photo (this will make it easy to keep track of what is being plotted throughout your project)

Use `show_image()` to plot `heart_img` with the title "Heart Image". If you get stuck, click the hint for help.

2.

You should now see the following plot in the web browser:



Can you figure out how the NumPy array `heart_img` creates that image you see? Take some time to think. Notice that `heart_img` is a 7x7 matrix and that the heart image itself is 7x7 pixels. Using `show_image()`, we mapped each value in `heart_img` to a 7x7 square matrix with grayscale colors that range from 0 (black) to 255 (white).

With this in mind, what are the `heart_img` values of the following sections of the heart image we see?

(6, 6) – bottom right

(3, 3) – center

(1, 3) – right below top center

Click the answer to see the answers!

Image Transformations

3.

Now that we understand how a NumPy array maps to an image let's do some fun image transformations!

Since our image is a matrix, we can do transformations based on linear algebra.

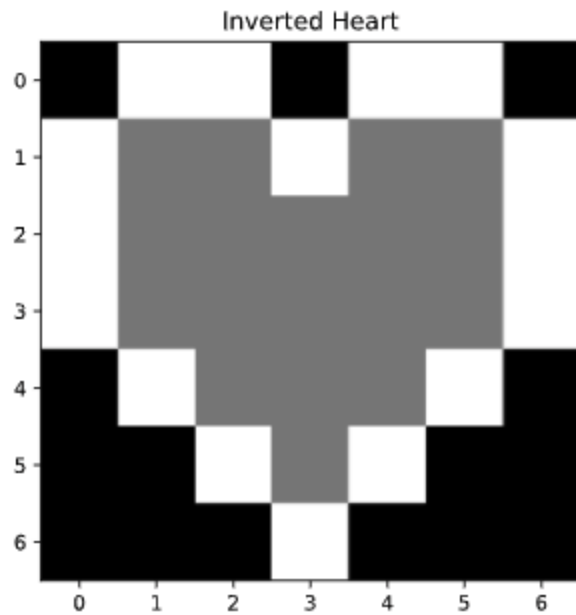
First, let's invert the colors of our heart image.

Create a new variable called `inverted_heart_img` that subtracts each value in `heart_img` from 255.

Show `inverted_heart_img` with the plot title "Inverted Heart Image."

4.

You should now see the following plot in the web browser:



Notice that the white pixels are now black, the black pixels are now white, while the gray pixels are the same since $255 - (255/2) = 255/2$

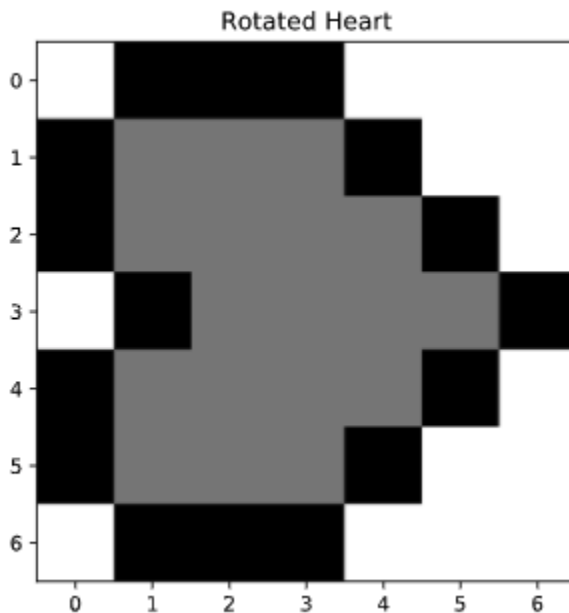
Let's do another image transformation. This time let's plot a rotation of `heart_img`.

Create a new variable called `rotated_heart_img` that swaps the rows and columns of `heart_img`.

Show `rotated_heart_img` with the plot title "Rotated Heart Image."

5.

You should now see the following plot in the web browser:



The rows of `heart_img` have now become the columns of `rotated_heart_img` as we plotted the transpose of `heart_img`.

Let's dive into some more complicated operations next!

More Advanced Operations

6.

Below the comment `# Random Image`, we have the following variable:

```
random_img = np.random.randint(0,255, (7,7))
```

Copy to Clipboard

Plot that image using `show_image()` with the title "Random Image."

7.

We want to solve for the matrix that creates `heart_img` given the following equation:

`random_img · x = heart_img`

$$\text{random_img} \cdot x = \text{heart_img}$$

Create a variable called `x` that is the solution to this equation. If you are not sure how to do this, check the hint for some guidance.

Plot `x` using `show_image()` with the title "x."

8.

Create a variable called `solved_heart_img` that is equal to the product of `random_img` and `x`.

Plot `solved_heart_img` with the plot title "Solved Heart Image."

What image do you see in the web browser?

Extra

9.

Congrats! You have finished your exploration of image transformations with NumPy and linear algebra. If you want to continue the fun, here are some more things you can try:

- Create a new shape with a NumPy array.

- Transform your image with a permutation matrix.

- Change the color scheme from grayscale. Check the `cmap` parameter of the `imshow()` function.

- Create a complicated image with more pixels and/or a shape that is not a square matrix.

These are just some of the things you can do. Happy coding!

Inference Specialist

Algerian Forest Fires

In this project, you'll examine the `forests` dataset that includes variables on weather and fire risk for locations in Algerian forests from two different regions. By plotting and running multiple linear regressions, you'll explore the relationships among variables including:

- `temp` – maximum temperature in degrees Celsius
- `humid` – relative humidity as a percentage
- `region` – location in Bejaia in the northeast of Algeria or Sidi Bel-abbes in the northwest of Algeria
- `fire` – whether a fire occurred (`True`) or didn't (`False`)
- `FFMC` – Fine Fuel Moisture Code: measure of forest litter fuel moisture that incorporates temperature, humidity, wind, and rain
- `ISI` – Initial Spread Index: estimates spread potential of fire
- `BUI` – Buildup Index: estimates potential release of heat
- `FWI` – Fire Weather Index: measure of general fire intensity potential that incorporates ISI and BUI

The dataset is a subset of data that was downloaded from the [UCI Machine Learning Repository](#) and then cleaned for analysis.

A file called **solutions.py** has been loaded for you. While we encourage you to attempt this project without looking at the solutions, you can always check the solution code for help when you get stuck!

Finally, a quick note about plotting: in order to prevent plots from getting layered on top of each other, you'll need to show then clear each plot before creating the next one using the following code:

```
plt.show() # Show the plot
plt.clf()  # Clear the plot
```

Copy to Clipboard

Dataset Citation

Faroudja ABID et al., Predicting Forest Fire in Algeria using Data Mining Techniques: Case Study of the Decision Tree Algorithm, International Conference on Advanced Intelligent Systems for Sustainable Development (AI2SD 2019) , 08 - 11 July , 2019, Marrakech, Morocco.

Tasks

0/16 complete

[Mark the tasks as complete by checking them off](#)

Predicting Relative Humidity

1.

The dataset has been loaded for you as `forests`. Since we will be running several multiple linear regressions in this project, we want to check the multicollinearity assumption before we get started. Create a table of correlations for the quantitative variables in `forests` and call it `corr_grid`. Then plot the correlations with a heat map to look for potential collinear variables. Make sure to show, then clear the plot.

Are there any variables that should not go into a model together as predictors?

2.

Let's explore the relationship between relative humidity (`humid`) and maximum temperature (`temp`) by creating a scatter plot with `humid` on the y-axis and `temp` on the x-axis. Color the points by `region`. Make sure to show, then clear the plot.

Does the relationship between humidity and temperature appear different for locations in Bejaia compared to those in Sidi Bel-abbes?

3.

Fit a multiple linear regression model predicting humidity with temperature and region as predictors. Save the fitted model as `modelH` and print the coefficients. Does the coefficient on `temp` match the relationship you expected from the plot you made in the previous step?

4.

Using the model coefficients you printed in the last step, write out the full regression equation as a comment. Then write out the regression equation for locations in Bejaia and the regression equation for Sidi Bel-abbes as additional comments. How are these equations similar and how are they different?

5.

In a comment, write an interpretation for the coefficient on `temp` in the full regression equation. Then, for each region's regression equation, interpret the intercept in terms of the regression line and in the context of predicting humidity from temperature. Do the intercept interpretations make practical sense?

6.

Finally, let's put our model back into a visualization to see if it matches our expectations. Create the same scatter plot of humidity and temperature from step 2, but this time add the two regression lines you found for each region. Make sure to show, then clear the plot.

Do these lines seem to fit the data well?

Predicting FFMC: Interaction

7.

In this section, we will fit the first of two regression models predicting the Fine Fuel Moisture Code (`FFMC`), a measurement of fuel moisture that contributes to the assessment of fire risk. First, let's explore the relationship between FFMC and temperature and see if that relationship looks different for areas that ended up experiencing a fire compared to those that didn't. Create a scatter plot of FFMC on the y-axis and temperature on the x-axis with points colored by the binary variable for fire status. Make sure you show, then clear the plot.

Do the groups seem to have the same or different slopes?

8.

Since the groups look like they may have regression lines with different slopes, fit a model predicting FFMC with predictors `temp`, `fire`, and a term for their interaction. Save the results as `resultsF` and print the resulting model coefficients. What do we learn about the regression line created with these coefficients?

9.

As a comment, write the full regression equation from the coefficients you printed in the last step. Then, write an equation for each group of `fire`. How are the equations similar, and how are they different?

10.

Using the equations you created in the previous step, write as a comment an interpretation of the coefficient on `temp` for each group of `fire`. What does the difference in these coefficients say about the relationship between FFMC measure and temperature in areas where a fire didn't occur compared to where one did occur?

11.

Let's visualize our model results on the scatter plot from step 7 by adding lines to the plot for the regression equation for each group of `fire`. Make sure you show, then clear the plot.

Do these lines seem to fit the data better than two parallel lines (i.e., lines with the same slope)?

Predicting FFMC: Polynomial

12.

Now, let's try predicting FFMC from just relative humidity. First, let's get an idea of what the relationship looks like in a scatter plot of FFMC on the y-axis and relative humidity on the x-axis. Make sure to show, then clear the plot. Will a straight line produce the best fit here?

13.

Since we see a curve in the pattern of points on the plot, run a multiple regression model predicting FFMC with humid and humid raised to the second power (squared) as predictors. Save the results as resultsP and print the coefficients. Does it make sense to interpret the coefficient on humid as we would normally interpret coefficients on quantitative variables?

14.

In a comment, use the coefficients you found in the previous step to write the regression equation. Using the equation and the model coefficients you found in the last step, find the FFMC for the following relative humidity levels: 25%, 35%, 60%, and 70%. What do you notice about the difference in FFMC when increasing from 25% to 35% compared to increasing from 60% to 70%?

15.

Using the plot from step 12 and the sample predicted values from step 14, write as a comment an interpretation of the relationship between FFMC and relative humidity. What would a straight regression line have implied about their relationship?

16.

So far, we've used weather, location, and fire status variables to predict humidity and a measure of forest fuel moisture. Finally, let's do some exploring using other variables in the dataset. The FFMC is derived from data on humidity, temperature, wind, and rain. Run a multiple regression predicting FFMC from all four variables. How could you better examine these relationships and decide whether to use interaction terms?

The Fire Weather Index (FWI), which measures of general fire intensity potential, takes ISI and BUI as inputs (measures of fire spread potential and heat release potential, respectively). Try running a multiple regression model predicting FWI from the predictors ISI and BUI and printing the coefficients. Are the relationships what you thought they'd be?

Craigslist Analysis

In this project, you'll examine some data about housing rentals on Craigslist — an online classifieds site. The provided housing data contains the following variables:

```
price: monthly rental price in U.S.D.
type: type of housing (eg., 'apartment', 'house', 'condo', etc.)
sqfeet: housing area, in square feet
beds: number of beds
baths: number of baths
comes_furnished: 1 means yes, 0 means no
laundry_options: laundry availability with values 'laundry on site', 'no laundry on site', 'w/d
hookups', or 'w/d in unit'
parking_options: parking availability with values 'carport', 'detached garage', 'no parking', 'off-
street parking', 'street parking', or 'valet'
smoking_allowed: 1 means yes, 0 means no
cats_allowed: 1 means yes, 0 means no
dogs_allowed: 1 means yes, 0 means no
```

Note that a **solution.py** file is loaded for you in the workspace, which contains solution code for this project. We highly recommend that you complete the project on your own without checking the solution, but feel free to take a look if you get stuck or want to check your answers when you're done!

Tasks

0/14 complete

Mark the tasks as complete by checking them off

Fit Some Models

1.
Fit a model that predicts `price` using `type`, `sqfeet`, `beds`, and `baths` as predictors. Save the fitted model as `model1`.
2.
Fit a model that predicts `price` using `type`, `sqfeet`, `beds`, `baths`, `comes_furnished`, `laundry_options`, `parking_options`, and `smoking_allowed` as predictors. Save the fitted model as `model2`.
Note that `model1` and `model2` are *nested models* because `model2` contains all of the predictors in `model1`.
3.
Fit a model that predicts `price` using `type`, `sqfeet`, `beds`, `baths`, `comes_furnished`, `laundry_options`, `parking_options`, `smoking_allowed`, `cats_allowed`, and `dogs_allowed` as predictors. Save the fitted model as `model3`.
Note that `model3`, `model2`, and `model1` are *nested models* because `model2` contains all of the predictors in `model1` and `model3` contains all of the predictors in `model2`.

Find the Best Model to Fit the Data

4.
Print the R-squared for all three models. Approximately what proportion of variation in rental prices can be described using the largest predictor set (`model3`)?
5.
Print out the adjusted R-squared for all three models. Based on adjusted R-squared, which model fits the data best?
Note that the two extra predictors in `model3` (compared to `model2`) are related to pet policies (`cats_allowed` and `dogs_allowed`). Based on your answer to the above: holding all other predictors constant, is there a significant relationship between a housing option's pet policy and its price?
6.
Use the `anova_lm()` function from `statsmodels` (which has already been imported for you in `script.py`) to run an F-test comparing `model2` and `model3`, then print the results.
Using a significance threshold of 0.05, are the coefficients on `cats_allowed` and `dogs_allowed` significantly different from zero? In other words: holding all other predictors constant, is there a significant relationship between a housing option's pet policy and its price?
Does your answer based on the F-test match your answer based on adjusted R-squared? Note that these two criteria don't have to agree!

Find the Best Model for Out-of-Sample Prediction

7.
Print the log-likelihood for all three models. Which model has the largest log-likelihood? Does this make sense?
8.
Print the AIC for all three models. Based on AIC, which model fits the data best?
Would you choose the same model based on AIC as you would based on adjusted R-squared and the F-test?
9.
Print the BIC for all three models. Based on BIC, which model fits the data best?
Note that BIC tends to favor simpler models with fewer predictors. Would you choose the same model based on BIC as you would based on AIC, adjusted R-squared and the F-test?
10.
We've provided you with code in `script.py` to split the `housing` data into training and test sets. These are saved as `housing_train` and `housing_test`, respectively.
Re-fit `model2` and `model3` using the training dataset and re-save the fitted models as `model2_train` and `model3_train`.
- 11.

Calculate the fitted values for the test dataset based on `model2_train` and `model3_train`. Save them as `fitted_mod2` and `fitted_mod3`, respectively.

12.

Calculate and print the predicted root mean squared error (PRMSE) for models 2 and 3. Based on PRMSE, which model performs best with respect to out-of-sample prediction?

Bonus Explorations and Next Steps!

13.

In this project, we saw that `model2` and `model3` performed very similarly. `model3` edged out `model2` in most comparisons, but only by a small amount.

Note that the process of calculating PRMSE involves randomly splitting the data into training and test datasets. Depending on how we split the data, we'll calculate slightly different PRMSE values. If two models have very similar PRMSEs, then different models may "win" depending on how we split the data.

Toward the beginning of **script.py** we've set a random seed using `np.random.seed(1)` to control the way the data is split. Try changing the random seed to a different number besides `1`, then re-run the code. Does model 3 still have a smaller PRMSE?

Try a few more times with different numbers. Can you get a sense for whether model 3 wins out more often — or is it a toss-up?

14.

Now that you've explored three potential models for housing prices, see if you can improve upon these models using additional terms. Are there any interactions or polynomial terms that you think may improve the model?

Data Scientist: Inference Specialist

Impact of Cover Crops on Wheat Crop Yields

<https://www.codecademy.com/paths/data-science-inf/tracks/dsinf-causal-inference/modules/dsinf-matching-and-weighting-methods/projects/propensity-scores-project>

As the global population continues to increase, scientists and policymakers are looking for ways to improve our agricultural systems so that we can feed more people.

The data used in this project is adapted from the [2017 Census of Agriculture](#) performed by the U.S. Department of Agriculture (USDA). This survey is a complete count and characterization of U.S. farms and agricultural producers. The data include various information about agricultural land area and use, farming practices, and crop yields at the county level.

There are several farming strategies that promote the long-term health of agricultural land:

Agricultural land easements: contracts between the government and farmers where the government pays landowners to leave areas of the land untouched.

Conservation tillage: a method that helps protect the soil surface and reduce erosion from wind and water between crop harvest and planting.

Cover crops: plants used to slow erosion, return nutrients to the soil, and help smother weeds and control pests.

In this project, we'll use inverse probability of treatment weighting (IPTW) to determine whether the use of cover crops causes an increase in crop yields. We encourage you to try all the project tasks in the file **notebook.Rmd**, but we have also provided solutions for reference in the file **solution.Rmd**.

Tasks

0/13 complete

[Mark the tasks as complete by checking them off](#)

Data Exploration

1.

Let's start by loading the dataset. We've provided a file named **farms.csv** in the workspace. Load this file into a dataframe named `farm_df`.

2.

Take a look at the head of the dataframe `farm_df`. Make sure to click through the arrows in the header to see all available variables.

Outcome:

`total_yield`: represents the average total yield of wheat in bushels per acre.

Treatment:

`cover_10`: indicates whether at least 10% of farms in a county employ cover crops.

Predictors:

`region`: geographic region where the county is located.

`total_avg`: average total size of farms (thousands of acres).

`age_avg`: average age of the primary farm operator (years).

`experience_avg`: average number of years of farming experience of the primary farm operator (years).

`insurance_avg`: average area of land with crop insurance (thousands of acres).

`easement_p`: indicates the average percent of land under an easement in the county.

`conservation_till_avg`: average area of land that uses conservation tillage methods (acres).

`fertilizer_per_area`: average cost of fertilizers used per acre (hundreds of thousands of dollars).

Examine Initial Overlap and Balance

3.

First, let's assess overlap and balance visually for a couple of variables to compare counties with AT LEAST 10% of farms using cover crops and counties with LESS THAN 10% of farms using cover crops. Create a balance plot for the `age_avg` variable. Do the treatment and control distributions appear to be centered in the same location and have similar spreads?

4.

Now create a balance plot for the categorical variable `region`. Are the proportions of counties in the treatment versus control groups similar across the four regions?

5.

So far, it looks like we have quite a bit of imbalance between the treatment groups to deal with. Let's assess balance numerically to quantify this imbalance more precisely. Create a balance table to show standardized mean differences (SMD) and variance ratios for all the predictor variables according to the treatment group. Check whether the balance measurements fall outside of the guidelines of ± 0.1 for SMDs and between 0.5 and 2 for variance ratios.

Perform IPTW Procedure and Re-check Balance

6.

Now it's time to perform the inverse probability of treatment weighting (IPTW) procedure to see if we can minimize imbalance between the treatment groups before estimating the causal treatment effect. To get the most out of this procedure, we have to carefully think about the form of the propensity score model. In other words, we should think about what variables in the dataset are predictive of cover crop use. Let's start with a propensity score model with a limited set of variables:

`region`: The ability to grow cover crops depends in part on regional variation in climate.
`total_avg`: Cover crops might be more feasible to implement on farms that are smaller.
`insurance_avg`: Many farmers have crop insurance, but certain states offer subsidies to offset insurance costs if farmers decide to use cover crops.
`fertilizer_per_area`: Cover crops require the use of fertilizer. Higher fertilizer costs might prevent some farmers from using cover crops.

Perform an IPTW procedure using `region`, average farm size, average number of farms with crop insurance, and average cost of fertilizer per acre as predictors in the propensity score model. Specify in your code to calculate weights in order to estimate the average treatment effect (ATE). Save the output to a weighting object named `farm_iptw`.

7.

We need to perform a check of the IPTW procedure to see whether we have achieved a better balance between groups. Let's look at the SMD values visually to see whether they now fall within our ± 0.1 guidelines.

Create a Love plot showing the SMD between treatment groups for each variable in the propensity score model. Be sure to display the SMD for binary variables and the SMD threshold of ± 0.1 .

Refine Propensity Score Model

8.

The balance is pretty good, but let's see if we can improve it further by tweaking the propensity score model. Re-run the IPTW procedure but with a new propensity score model. This time, remove the `fertilizer_per_area` variable from the model. Then add to the model:

the average age of the primary producer
the average experience of the primary producer
the average percent of land under easement
the average number of farms using conservation tillage Save the new model as `farm_iptw2`.

9.

Let's find out whether our expanded propensity score model leads to a better balance in the weighted data.

Create a Love plot showing the SMD between treatment groups for each variable using the new propensity score model. Be sure to display the SMD for binary variables and the SMD threshold of ± 0.1 .

10.

The Love plot shows SMD values closer to zero than the previous model! We can also inspect the distribution of propensity scores between treatment groups to see how well the IPTW procedure worked. Generate a balance plot to display the distribution of propensity scores in each treatment group before AND after the weighting process. Do the distributions of the weighted propensity scores look closer to identical, and are they overlapping each other?

Estimate Causal Treatment Effect

11.

You're almost there — great job so far! The last step of the analysis is to fit the final outcome model to estimate the causal effect of cover crop usage on crop yields.

First, let's fit the outcome regression model with total crop yield as the outcome, cover crop usage as the treatment variable, and the other variables from the second propensity score model as the additional predictors. Remember to include the weights from the IPTW procedure in the regression model. Save this regression model to an object name `yield_mod`.

12.

We're not quite done yet! Remember that when we use IPTW to estimate the causal treatment effect, we need to use a robust standard error estimate.

Estimate the regression parameters for the weighted regression model using the `coefTest()` function. Incorporate a robust standard error estimator from the sandwich package.

13.

Take a look at the regression parameter for the treatment variable. How would you interpret this value? What do you think it says about the effect of cover crops on wheat yields? Write out your interpretation of the results.

Data Scientist: Inference Specialist

Effect of Emergency Weather Systems on Transit Times

<https://www.codecademy.com/paths/data-science-inf/tracks/dsinf-causal-inference/modules/dsinf-regression-discontinuity-design-and-instrumental-variables/projects/rdd-project>

As of the year 2018, 55% of the world's population lived in urban areas. By 2050, the United Nations expects this number to jump to 68%. Major metropolitan areas will have to adapt and become more efficient in order to accommodate billions of new residents in the coming decades. One infrastructure system in particular that will feel the burden of growing urban populations is transportation.

Imagine a large, growing city that is located in an area that gets heavy snowfall in the fall and winter seasons. City officials have partnered with a large tech company to collect anonymous navigation data about residents' transit times during peak rush hours.

Preliminary data seems to verify what city officials suspected: transit times are substantially longer in the winter months when the city gets a lot of snow. The city officials decide to institute a "snow emergency" system between the months of November and March intended to reduce average transit times during severe weather. A snow emergency will be issued whenever the city experiences at least 4 inches of snow in a 24-hour period. During snow emergencies, the city will restrict car traffic to only major roads and will waive fares for public transportation.

The leaders hope that these actions will incentivize people to use the city's efficient train system instead of commuting via car, which will reduce traffic congestion and transit times. After three winter seasons, the city decides to analyze the navigation data to see whether the snow emergency system was effective at reducing transit times.

In this project, you will use regression discontinuity design (RDD) to assess the system's effectiveness. We encourage you to try all tasks on your own in **notebook.Rmd**, but there are solutions available for reference in **solution.Rmd**.

Tasks

0/12 complete

Mark the tasks as complete by checking them off

Data Exploration

1.
First, let's import the dataset for this project. We've provided a file named **snow.csv** in the workspace. Load this file into a dataframe named `snow_df`.
2.
Now that the dataset is imported and ready to work with, let's take a look at the first few rows of the dataframe `snow_df`. This dataset contains several variables:

Forcing Variable

`snowfall`: The amount of snowfall on recorded date (inches)

Treatment Variable

`emergency`: Whether or not a snow emergency was issued on the recorded date ("Emergency" vs. "No Emergency")

Outcome Variable

`minutes`: Average commute time on recorded date (minutes)

Others

`date`: Recorded date

Assessing RDD Assumptions

3.

Use the `ggplot2` package to create a scatter plot of the data with the forcing variable on the x-axis and the outcome variable on the y-axis. Use colors and shapes to distinguish between dates with a snow emergency and dates without a snow emergency.

Save the base plot as `scatter_base` and view the plot.

4.

Let's add a little more detail to the plot. Add a vertical dashed line to `scatter_base` at the snowfall cutoff that determines emergency status. Save the updated plot to `scatter_cutpoint` and print it to the console to view the plot.

Does it look like this is a sharp RDD or a fuzzy RDD?

5.

To verify that there is actually a discontinuity in the outcome at the cutpoint, add linear best fit lines to `scatter_cutpoint`. Separate best fit lines should be plotted for dates with a snow emergency and dates without a snow emergency. Save this plot as `scatter_fit` and print it to view the plot.

Does there appear to be a discontinuity in the outcome variable around the cutpoint?

Calculating IK Bandwidth

6.

Calculate the optimal IK bandwidth for the data and save the result to `snow_ik_bw`. Print `snow_ik_bw` to the console. Is the bandwidth large or small relative to the scale of the forcing variable?

7.

Add solid vertical lines to `scatter_cutpoint` to plot the range of the optimal bandwidth stored in `snow_ik_bw`. Save the updated plot to `scatter_bw` and print it to the console to view the plot.

Does a lot of the data fall outside of the bandwidth? How do you think this could affect the treatment effect estimate?

Estimating Causal Treatment Effect

8.

Using the correct cutpoint and optimal bandwidth you just calculated, fit a local linear regression model using the `RDeestimate()` function. Save the results to `snow_rdd`.

9.

Print the results of the local linear regression results stored in `snow_rdd`. Take note of the estimate of the LATE across different bandwidths. Are the values similar?

Standard Errors and Conclusion

10.

Print the number of observations in each of the three bandwidths provided in `snow_rdd`. How does sample size change across the optimal bandwidth, half the bandwidth, and double the bandwidth?

11.

Print the standard errors of the LATE estimates at each bandwidth provided in `snow_rdd`. How did the standard errors change as the sample size increased?

12.

Great job! Using the optimal bandwidth of 2.63 inches, we calculated a local average treatment effect (LATE) of -11.04. This means that in this dataset, the use of the snow emergency system resulted in an average decrease in commute time of 11.04 minutes.

Given what we know about interpreting the LATE, think about where this causal effect is generalizable — do you think the bandwidth was wide enough to cover many different snowfall events?

Machine Learning/AI Engineer

School Catalogue

Let's put your knowledge of classes to the test by creating a digital school catalog for the New York City Department of Education. The Department of Education wants the catalog to hold quick reference material for each school in the city.

We need to create classes for primary and high schools. Because these classes share properties and methods, each will inherit from a parent `School` class. Our parent and three child classes have the following properties, getters, setters, and methods:

School

Properties: `name` (string), `level` (one of three strings: 'primary', 'middle', or 'high'), and `numberOfStudents` (integer)

Getters: all properties have getters

Setters: the `numberOfStudents` property has a setter

Methods: A `__repr__` method that displays information about the school.

Primary

Includes everything in the `School` class, plus one additional property

Properties: `pickupPolicy` (string, like "Pickup after 3pm")

Middle

Does not include any additional properties or methods

High

Includes everything in the `School` class, plus one additional property

Properties: `sportsTeams` (list of strings, like ['basketball', 'tennis'])

Tasks

0/16 complete

[Mark the tasks as complete by checking them off](#)

Create the School Class

1.

Let's start by making a parent class for `Primary`, `Middle`, and `High` classes.

Create an empty class named `School`.

Use the following syntax to create a new class:

```
class myNewClass:
```

Copy to Clipboard

2.

Inside the `School` class, create a constructor that accepts three parameters in addition to `self`. The names of these parameters should match the properties listed in the narrative above.

A constructor is created using `__init__(self, parameter1, parameter2, ...)`

3.

Inside the constructor, set the `School` properties equal to the values passed into the constructor. Remember to use the `self` keyword when setting these properties.

For example, we used `self.name = name` to create a property named `self.name` and set it equal to the value passed into the constructor.

4.

Create getters for the `name`, `level`, and `numberOfStudents` properties. Each getter should return the value saved to the property.

For example, here is our getter for the `name` property:

```
def getName(self):  
    return self.name
```

Copy to Clipboard

5.

Create a setter for `numberOfStudents`. This method should take one parameter (in addition to `self`) and set `self.numberOfStudents` equal to the value passed into the method.

We called our parameter `newNumberOfStudents` and set `self.numberOfStudents = newNumberOfStudents`.

6.

Create a `__repr__()` method so when a `School` is printed, it displays relevant information about the object. You can choose what you want displayed, but we printed:

A {level} school named {name} with {numberOfStudents} students

Recall that the `__repr__()` method should return the string that you want to be displayed when you print an object of the class.

7.

At this point, it's probably a good idea to test your code if you haven't already. Try creating a `School` object, use the getter and setter methods you created, and print that object

Our code looked like this:

```
a = School("Codecademy", "high", 100)  
print(a)  
print(a.getName())  
print(a.getLevel())  
a.setNumberOfStudents(200)  
print(a.getNumberOfStudents())
```

Copy to Clipboard

Create the `PrimarySchool` class

8.

Next, we'll build a `PrimarySchool` class that inherits from `School`. If you feel comfortable building the `PrimarySchool` class on your own, give it a shot. If not, use the steps below to help you along the way.

Whether you want to follow the steps or not, it's important to note, the `PrimarySchool` class only has one additional property, `pickupPolicy`.

Create an empty `PrimarySchool` class that inherits from `School`.

The following code creates a class named `A` that inherits from `B`:

```
class A(B):
```

Copy to Clipboard

9.

Inside the `PrimarySchool` class, create a constructor that accepts three arguments in addition to `self`. Think about which three properties we'll need arguments for.

We need arguments for `name`, `numberOfStudents`, and `pickupPolicy`. The `level` of a `PrimarySchool` will automatically be set to `primary`.

10.

Call `super().__init__()` on the first line of the `PrimarySchool`'s constructor. Pass it any arguments that the parent constructor uses.

Since this is the `PrimarySchool` class, pass `'primary'` as the argument for the `level` parameter in the parent constructor.

The arguments for the call to the superclass' constructor should look like `name, "primary", numberOfStudents`.

11.

You've used two of the arguments you've passed to the `PrimarySchool` constructor. Now use the third argument to set the value of a new property named `self.pickupPolicy`.

Because we named that argument `pickupPolicy`, we have `self.pickupPolicy = pickupPolicy`.

12.

Since our `PrimarySchool` class inherits `Schools`'s properties and getters, we only need to create one new getter in the `PrimarySchool` class. Add this getter to the `PrimarySchool` class. Each getter should return the value saved to that property.

We want to create a `getPickupPolicy()` method that returns `self.pickupPolicy`.

13.

Finally, we probably want the `__repr__()` method for a `PrimarySchool` to display information about the `pickupPolicy` variable. Override the `__repr__()` method so when a `PrimarySchool` is printed, information about the pickup policy is displayed in addition to the other information about the school.

You can call `super().__repr__()` to avoid repeating some code.

This is what our overridden `__repr__()` method looks like. Notice we call the parent class' `__repr__()` method so we don't repeat work.

```
def __repr__(self):
    parentRepr = super().__repr__()
    return parentRepr + "The pickup policy is {pickupPolicy}".format(pickupPolicy =
self.pickupPolicy)
```

Copy to Clipboard

14.

Create a `PrimarySchool` object and verify your new constructor, getter method, and `__repr__()` method work as expected.

Our test code looked like this:

```
b = PrimarySchool("Codecademy", 300, "Pickup Allowed")
print(b.getPickupPolicy())
print(b)
```

Copy to Clipboard

Create the HighSchool class

15.

In this task, you will create a `HighSchool` class that inherits from the `School` class. In addition to the properties, getters, and methods in `School`, the `HighSchool` includes the following:

Properties: `sportsTeams` (list of strings)

Getters: A getter for the `sportsTeams` array. The getter should return the list of the sports teams.

Override the `__repr__()` method to display information about `sportsTeams`.

Take a look at the Hint for step-by-step instructions.

Follow these steps to complete the task:

Create an empty `HighSchool` class that inherits from `School`.

Create a constructor that accepts arguments for `name`, `numberOfStudents`, and `sportsTeams`.

On the first line of the constructor, call `super().__init__()` and pass it `name`, `'high'`, and `numberOfStudents` as arguments.

On the next line, set the `sportsTeams` property using `self.sportsTeams = sportsTeams`

Under the constructor, create a getter for `sportsTeams`.

Override the `__repr__()` method to display information about `sportsTeams`.

16.

Once again, let's test our code to make sure it works correctly. Create a `High School` object and verify your new constructor, getter method, and `__repr__()` method work as expected.

Our test code looked like this:

```
c = HighSchool("Codecademy High", 500, ["Tennis", "Basketball"])
print(c.getSportsTeams())
print(c)
```


Machine Learning/AI Engineer

Create your own Higher Order Functions

To further your understanding of functional programming, you will create your own higher-order functions and then use them to process data from a CSV file.

The functions you will create are:

`count()`: will return the frequency of an element in an iterable

`average()`: will return the average of elements in an iterable of numbers

The `count()` function will be a special type of `filter()` function that returns the number of occurrences of an element instead of returning a Boolean value. The `count()` function will accept the following two parameters:

`predicate`: when evaluated to `True` will allow a counter to increment by one.

`itr`: the collection containing the element of interest.

The `average()` function will compute the arithmetic mean of a collection. You will implement this function recursively to adhere to functional programming principles. `average()` will only accept one parameter: `itr`, the collection to be averaged.

Before we begin, please keep in mind the following:

The `reduce()` function can accept a third parameter that serves as the initial value, thereby allowing an initial condition that is not of the same type as the collection `reduce()` is operating on. Syntax:

```
reduce(lambda x: x+ y, some_collection, initial_value)
```

Copy to Clipboard

You will use these functions to process sales data from a CSV file called **1kSalesRec.csv**.

Note that a **solution.py** file is also loaded for you in the workspace, which contains solution code for this project. We highly recommend that you complete the project on your own without checking the solution, but feel free to take a look if you get stuck or want to check your answers when you're done!

Let's begin!

Tasks

0/11 complete

[Mark the tasks as complete by checking them off](#)

The count() Function

1.

Let's start with creating the `count()` function.

Inside the `count()` function, write a `filter()` function that will "filter" `itr` based on `predicate`. Save this value to a variable called `count_filter`.

The `filter()` function should have the same `predicate` as `count()`.

2.

Inside of `count()`, use `reduce()` to process the iterator returned from `filter()` so the accumulator increments by one for each `True` evaluation from `filter()`.

Remember this definition of `reduce()`:

```
reduce(lambda x, y: x + y, some_collection, initial_value)
```

Copy to Clipboard

Save your `reduce()` function to a variable called `count_reduce`. Make sure to return `count_reduce` at the end of your function.

Click the hint or check **solution.py** if you feel stuck.

You want to set up your `reduce()` function in the following way:

Use a lambda function that increments the accumulator by one:

```
lambda x, y: x + 1
```

Copy to Clipboard

`x` will be the current accumulated value from previous evaluations, and `y` will be the current value given to the accumulator.

The `some_collection` in the `reduce()` function is `count_filter`.

The initial value is zero.

Therefore, you should have the following:

```
count_reduce = reduce(lambda x, y: x + 1, count_filter, 0)
```

Copy to Clipboard

The average() Function

3.

Let's create the `average()` function that will compute an average recursively. This is something you may not have seen before, so we will walk you through it.

The `average()` function will contain a helper function called `avg_helper()` that will be responsible for implementing a loop to compute an average.

Create a function called `avg_helper()`. That accepts the following three parameters:

`curr_count`: represents the current count of elements.

`itr`: represents the collection.

`curr_sum`: represents the current running total of elements in the collection.

Your function should look like this:

```
def avg_helper(curr_count, itr, curr_sum)
```

Copy to Clipboard

4.

We want `avg_helper()` to implement the following loop recursively:

```
curr_count = 0
curr_sum = 0
for i in iterable:
    curr_sum += i
    curr_count += 1
```

Copy to Clipboard

In our case, `curr_sum` will be provided from the function call. To obtain the next value in an iterable (if there is one), we have to use `next()`. If there are no more elements to obtain from the iterable, `next()` should return the string "null". The `next()` function is called like so:

```
next(iterable, default_value)
```

In your `avg_helper()` function, do the following:

Use `next()` to obtain the next value in the iterable. Set this value to a variable called `next_num`. Add this value to `curr_sum`.

Increment the counter, `curr_count`, by one.

In the next task, we will see how to handle "null" values.

You should have the following code so far:

```
next_num = next(itr, "null")
curr_sum += next_num
```

Copy to Clipboard

5.

A recursive function performs the next iteration in the loop by making a call to itself and terminates the loop when a base case is reached.

In our case, the loop should terminate when all the elements in the iterable are processed; i.e., `next()` returns "null".

When `next()` returns "null", `avg_helper()` should compute the average and return that value.

Implement this functionality in `avg_helper()`. To do this:

Increment `curr_count` by one on each iteration.

Return average if `next()` returns "null" (use an if statement)

`curr_sum` contains the total sum of elements.

`curr_count` is a counter which keeps track of how many elements have been found so far.

Therefore to calculate the average return `curr_sum/curr_count`.

Your if statement should be:

```
if next_num == "null":
    return curr_sum/curr_count
```

Copy to Clipboard

After your if statement, you should have two increments:

```
curr_count += 1
curr_sum += next_num
```

Copy to Clipboard

6.

If "null" is not returned, that means there are still more values to add up.

The next iteration of the loop is computed by making a call to `avg_helper()` with the new `curr_count` and `curr_sum` supplied.

Call `avg_helper()` in the proper location and return the value.

Add the following line of code to the end of your `avg_helper()` function:

```
return avg_helper(curr_count, itr, curr_sum)
```

Copy to Clipboard

7.

Typically, recursive functions follow this style:

```
def recursive_function():
    # Base Case
    # Computation
    # Recursive call
```

Copy to Clipboard

If your code is not written in this style, refactor your `avg_helper()` function to match it.

The base case is when `next()` returns "null". If this is true, return the computed average. If it is not, the function should perform the other computations and make the recursive call.

[Click here to see what your function should look like!](#)

```
def avg_helper(curr_count, itr, curr_sum):
    next_num = next(itr, "null")
    if next_num == "null":
        return curr_sum/curr_count
    curr_count += 1
    curr_sum += next_num
    return avg_helper(curr_count, itr, curr_sum)
```

Copy to Clipboard

8.

Inside `average()`, make the initial call to `avg_helper()` with the proper initial values for `curr_count` and `curr_sum`. Return the value found by `avg_helper()`.

Since no elements have been processed yet, the current total is zero, and the current count is also zero. Use this information to provide `avg_helper()` with the proper initial values.

[Click here to see what your function should look like!](#)

```
def average(itr):
    iterable = iter(itr)
    return avg_helper(0, iterable, 0)
```

Copy to Clipboard

Using `count()` and `average()`

9.

You will now use your `count()` and `average()` functions on data obtained from a CSV file called "1kSalesRec.csv". This file contains order data from a warehouse.

The fields in this CSV file are:

```
["Region", "Country", "Item Type", "Sales Channel", "Order Priority", "Order Date", "Order ID", "Ship Date", "Units Sold", "Unit Price", "Unit Cost", "Total Revenue", "Total Cost", "Total Profit"]
```

Copy to Clipboard

Under the section of code that opens a CSV file and creates a `reader` object, use your `count()` function to count the number of orders for the country "Belgium".

To do this:

Replace `count_belgiums = None` with your `count()` function.

Within your `count()` function call, use the following lambda function:

```
lambda x: x[1] == "Belgium"
```

Copy to Clipboard

Recall that a record is read in a list. In this case, Country is the second item in the list (index 1).

What should your second argument be?

The predicate in your `count()` function should check to see when a record is entered where the Country variable is "count_belgium".

Click here to see what your function call should look like!

```
count_belgium = count(lambda x: x[1] == "Belgium", reader)
```

Copy to Clipboard

10. Use your `average()` function to compute the average total profit (index 13) for Portugal. Replace `avg_portugal = None` with your function call.

To do this:

You first need to use `filter()` to extract all records with Country "Portugal"

Then you need to use `map()` to extract the Total Profit (index 13).

Be careful about which order you use `filter()` and `map()` in your `average()` functional call.

Note: the entries in the list that represents the records are all of type string. You must convert them to float by using `float()`.

Your final `average()` function call should look like the following:

```
avg_portugal = average(map(lambda x: float(x[13]), filter(lambda x: x[1] == "Portugal", reader)))
```

Copy to Clipboard

Congrats!

11. Fantastic job finishing this project! This was a challenging project where we introduced new material and pushed you to do some challenging analysis. We hope you enjoyed it. If you are looking to do more, you can:

Analyze different parts of **1kSalesRec.csv**

Create more higher order functions, such as `median()` or `standard_deviation()`

Come up with different ways to calculate the values from tasks nine and ten.

Happy coding!

```
import csv
from functools import reduce

def count(predicate, iterable):
    pass # Remove this keyword when you start working on this function.

def average(itr):
    # If itr is not iterable, this will generate an iterator.
    iterable = iter(itr)

with open('1kSalesRec.csv', newline = '') as csvfile:
    reader = csv.reader(csvfile, delimiter=',', quotechar='|')
    fields = next(reader)
    belgiums = None
    print(belgiums)
    csvfile.seek(0)
    avg_portugal = None
    print(avg_portugal)
```

ATM Logging

You are developing a mobile ATM application that will handle bank transactions such as deposits and withdrawals. With the original design of this application, you used print statements to print helpful information to the console, such as amounts entered for each transaction, transaction status, and date and timestamps to see when each transaction occurred.

The goal of this project is to modify the existing BankAccount methods to utilize the logging module instead of print statements. Using the logging module over print statements will help make the project code more readable and maintainable after these modifications are made.

A **solution.py** file is also loaded for you in the workspace, which contains solution code for this project. We highly recommend that you complete the project on your own without checking the solution, but feel free to take a look if you get stuck or want to check your answers when you're done!

Tasks

9/9 complete

Mark the tasks as complete by checking them off

Replacing Print Statements

1. Start off by looking over the starting code in **script.py**. To try it out yourself, type `python3 script.py` in the console and click Enter.
Note that if you make any edits to the code, you must click **Save** before running the code again.
2. Create a logger object that directs logged messages to the console. Do this above the `BankAccount` class.
You will also need to import two libraries.
Create a `getLogger` object and a `StreamHandler` object. Make sure to import `logging` and `sys`.
3. Set your log level to `INFO`.
You should use the `.setLevel()` method.
4. Determine which print statements should be replaced by logged messages. Depending on the content of the message, select the appropriate logging level to use for each statement.
Note that this will be the most important and time-consuming part of the project. If you feel stuck at all, click the hint below or look at **solution.py** for some inspiration.
Notice that a lot of the print statements have words like "info", "warning", and "error". This should provide some guidance for how you can use logs to replace many of the print statements.
For example, you can replace:

```
print("Warning! You entered a negative number to deposit.")
```


Copy to Clipboard
with:

```
logger.warning("You entered a negative number to deposit.")
```


if you created a logger object called `logger`.
5. Reduce repetitive code within the `Transaction Info` sections by including a date timestamp in each log message.
This can be accomplished via the `Formatter` class when formatting the logger object. You should:
Create a `Formatter` object.
Use `setFormatter()` to add your `Formatter` object to your `StreamHandler` object.

Executing Logged Messages

6. Add an additional handler to also direct the logged messages to a log file called **formatted.log**.
7. Execute the code and supply the following inputs:
Deposit: \$50
Withdraw: \$200
View the saved log file or the console and verify that an error was logged for insufficient funds for the withdrawal transaction.
8. Re-execute the code and supply the following inputs:

Deposit: \$50
Withdraw: \$10

View the saved log file to see the successful deposit and withdrawal transactions.

Extra

9. Congratulations! You have completed your Python journey with logging. Some more things you can try are:
 - Playing around with the logging level and format of your logs
 - Have unique logging levels and formatting between logs sent to the terminal and logs sent to **formatted.log**
 - Creating and reading to multiple log files
- Happy coding!

SOLUTIONS:

```
#### Solution Code

import logging
import random
import sys

logger = logging.getLogger(__name__)
stream_handler = logging.StreamHandler(sys.stdout)
file_handler = logging.FileHandler('formatted.log')

formatter = logging.Formatter('%(asctime)s - %(message)s')
stream_handler.setFormatter(formatter)
file_handler.setFormatter(formatter)

logger.addHandler(stream_handler)
logger.setLevel(logging.INFO)

class BankAccount:
    def __init__(self):
        self.balance=100
        print("Hello! Welcome to the ATM Depot!")

    def authenticate(self):
        while True:
            pin = int(input("\nEnter account pin: "))
            while pin != 1234:
                logger.error("Invalid pin.")
                pin = int(input("\nTry again: "))
            return None

    def deposit(self):
        try:
            amount=float(input("Enter amount to be deposited: "))
            if amount < 0:
                logger.warning("You entered a negative number to deposit.")
                self.balance += amount
                logger.info("Amount Deposited: {amount}".format(amount=amount))
                logger.info("Transaction Info:")
                logger.info("Status: Successful")
                logger.info("Transaction #{number}".format(number=random.randint(10000, 1000000)))
            except ValueError:
                logger.error("You entered a non-number value to deposit.")
                logger.info("\nTransaction Info:")
                logger.info("Status: Failed")
                logger.info("\nTransaction #{number}".format(number=random.randint(10000, 1000000)))

    def withdraw(self):
        try:
            amount = float(input("Enter amount to be withdrawn: "))
            if self.balance >= amount:
                self.balance -= amount
                logger.info("\nYou withdrew: {amount}".format(amount=amount))
                logger.info("\nTransaction Info:")
                logger.info("Status: Successful")
                logger.info("Transaction #{number}".format(number=random.randint(10000, 1000000)))
            else:
```

```
        logger.error("Insufficient balance to complete withdraw.")
        logger.info("\nTransaction Info:")
        logger.info("Status: Failed")
        logger.info("\nTransaction #{number}".format(number=random.randint(10000, 1000000)))
    except ValueError:
        logger.error("You entered a non-number value to deposit.")
        logger.info("\nTransaction Info:")
        logger.info("Status: Failed")
        logger.info("\nTransaction #{number}".format(number=random.randint(10000, 1000000)))

    def display(self):
        print("\nAvailable Balance =", self.balance)

acct = BankAccount()
acct.authenticate()
acct.deposit()
acct.withdraw()
acct.display()
```

Bicycle World

Welcome to Bicycle World, the premier text-based bicycle shop! This shop is only accessible to programmers like you, who are familiar with the command line.

In this project, you'll use the commands you learned to navigate and edit the filesystem of this special shop.

The starting filesystem is shown below. (Bicycle World recently changed owners, who named the main directory `bicycle-world-ii`.)

```
bicycle-world-ii
├── brands.txt
├── freight/
│   ├── messenger/
│   └── porteur/
├── mountain/
│   ├── downhill/
│   │   ├── heavyweight/
│   │   └── lightweight/
│   └── hardtail/
├── racing/
│   ├── road/
│   └── track/
```

Copy to Clipboard

If you get stuck during this project or would like to see an experienced developer work through it, click **"Get Unstuck"** to see a **project walkthrough video**.

Tasks

0/16 complete

[Mark the tasks as complete by checking them off](#)

1.

Print the working directory.

If you ever get lost, you can return to this directory using `cd`.

```
$ pwd
/home/ccuser/workspace/bicycle-world-ii
```

Copy to Clipboard

Return here by entering:

```
$ cd /home/ccuser/workspace/bicycle-world-ii
```

Copy to Clipboard

2.

List the files and directories in the working directory.

```
$ ls
brands.txt  freight  mountain  racing
```

Copy to Clipboard

3.

Change directories to the **freight/** directory.

```
$ cd freight
```

Copy to Clipboard

4.

List the files and directories in the working directory.

```
$ ls
messenger  porteur
```

Copy to Clipboard

5.

Change directories to the **porteur/** directory.

```
$ cd porteur
```

Copy to Clipboard

6.

Change directories up two levels to the **bicycle-world-ii/** directory.

List the files and directories in the **bicycle-world-ii/** directory.

```
$ cd ../../
```



```
$ ls
brands.txt  freight  mountain  racing
```

Copy to Clipboard

If you see an error like `No such file or directory`, you might be in the wrong directory! Use `cd ../../` from within **porteur/**.

7.

Change directories to the **mountain/downhill/** directory.

```
$ cd mountain/downhill
```

Copy to Clipboard

8.

Make a file called **dirt.txt**.

```
$ touch dirt.txt
```

Copy to Clipboard

9.

Make a file called **mud.txt**.

```
$ touch mud.txt
```

Copy to Clipboard

10.

List the files and directories in the **downhill/** directory.

```
$ ls
dirt.txt  heavyweight  lightweight  mud.txt
```

Copy to Clipboard

11.

Downhill biking is dangerous: In the **downhill/** directory, make a directory called **safety/**.

```
$ mkdir safety
```

Copy to Clipboard

12.

Change directories to the **bicycle-world-ii/** directory.

```
$ cd ../../
```

Copy to Clipboard

If you see an error like `No such file or directory`, you might be in the wrong directory! Use `cd ../../` from within **downhill/**.

13.

List the contents of the **bicycle-world-ii/** directory.

```
$ ls
brands.txt  freight  mountain  racing
```

Copy to Clipboard

14.

The shop is adding a new type of bike!

In **bicycle-world-ii/**, make a directory called **bmX/**.

```
$ mkdir bmX
```

Copy to Clipboard

15.

Without changing directories from **bicycle-world-ii/**, make a file in the **bmX/** directory called **tricks.txt**.

```
$ touch bmX/tricks.txt
```

Copy to Clipboard

16.

List all files and directories in the current directory.

```
$ ls
bmX  brands.txt  freight  mountain  racing
```

Copy to Clipboard

Artusi

Let's use commands we just learned to manipulate the files and directories of Artusi, an arts supply Store.

If you get stuck during this project or would like to see an experienced developer work through it, click ["Get Unstuck"](#) to see a **project walkthrough video**.

Tasks

1/26 complete

[Mark the tasks as complete by checking them off](#)

1.

Print the working directory.

To print out the current directory you are in, use

```
pwd
```

Copy to Clipboard

2.

List all contents in the current directory, including hidden files and directories.

To display a directory's contents, use `ls`. Since we want to also include hidden files, we can use the option `-a`.

```
ls -a
```

Copy to Clipboard

3.

List all contents, in long format.

To display a directory's contents, use `ls`. Since we want to list with long format, we can use the option `-l`.

```
ls -l
```

Copy to Clipboard

4.

List all contents, including hidden files and directories, in long format, ordered by the date and time they were last modified.

Since we want to list all directories, including hidden ones, in long format, and sorted by last time modified, we can combine the options `-a`, `-l`, and `-t`, making `-alt`.

```
ls -alt
```

Copy to Clipboard

5.

Change directories to the **pencils/** directory. The relative path of the pencils directory is **drawing/pencils/**.

The current directory is

```
/home/ccuser/workspace/artusi
```

Copy to Clipboard

and from our `ls` command, we know **drawing/** is a directory inside the current directory. Since we're told **pencils/** is inside **drawing/**, we simply need to change directory like so:

```
cd drawing/pencils
```

Copy to Clipboard

6.

List all contents in the current working directory, including hidden files and directories.

To display a directory's contents, use `ls`. Since we want to also include hidden files, we can use the option `-a`.

```
ls -a
```

Copy to Clipboard

7.

Copy the file **color.txt** to **graphite.txt**.

Use the `cp` command. We have a source file **color.txt** and a destination file **graphite.txt**, so we can use `cp` like so:

```
cp color.txt graphite.txt
```

Copy to Clipboard

You can use the following commands to confirm the result of the above `cp` statement:

```
cat color.txt
```

Copy to Clipboard

and

```
cat graphite.txt
```

Copy to Clipboard

(**cat** is a command that outputs the contents of a file)

8.

Change directories into the **charcoal/** directory. The relative path to the charcoal directory is **../charcoal/**.

The current directory is

```
/home/ccuser/workspace/artusi/drawing/pencils
```

Copy to Clipboard

charcoal/ is another directory inside the directory **drawing/**. That means we need to go up a directory level to **drawing/** using **..**, and from there go into **charcoal/**. So our **cd** statement would be:

```
cd ../charcoal
```

Copy to Clipboard

9.

Copy the file **compressed.txt** to **vine.txt**.

Use the **cp** command. We have a source file **compressed.txt** and a destination file **vine.txt**, so we can use **cp** like so:

```
cp compressed.txt vine.txt
```

Copy to Clipboard

You can use the following commands to confirm the result of the above **cp** statement:

```
cat compressed.txt
```

Copy to Clipboard

and

```
cat vine.txt
```

Copy to Clipboard

10.

Copy the file **vine.txt** to **pencils/color.txt**. The relative path of the latter is **../pencils/color.txt**.

Use the **cp** command. We have a source file **vine.txt** and a destination file **../pencils/color.txt**, so we can use **cp** like so:

```
cp vine.txt ../pencils/color.txt
```

Copy to Clipboard

You can use the following commands to confirm the result of the above **cp** statement:

```
cat vine.txt
```

Copy to Clipboard

and

```
cat ../pencils/color.txt
```

Copy to Clipboard

11.

From the **charcoal/** directory, change directories to the **painting/** directory. The relative path is **../../painting/**.

The current directory is

```
/home/ccuser/workspace/artusi/drawing/charcoal/
```

Copy to Clipboard

painting/ is another directory inside the directory **artusi/**. That means we need to go up two directory levels to **artusi/** using **../../**, and from there go into **painting/**. So our **cd** statement would be:

```
cd ../../painting
```

Copy to Clipboard

12.

Print the working directory.

To print out the current directory you are in, use

```
pwd
```

Copy to Clipboard

13.

List all contents, in long format, including the hidden files and directories, ordered by the date and time they were last modified.

Since we want to list all directories, including hidden ones, in long format, and sorted by last time modified, we can combine the options **-a**, **-l**, and **-t**, making **-alt**.

```
ls -alt
```

Copy to Clipboard

14.

Change directories to the **brushes/** directory.

The current directory is

```
/home/ccuser/workspace/artusi/painting
```

Copy to Clipboard

and from our `ls` command from the last step, we know **brushes/** is a directory inside the current directory. So our `cd` command would simply be this:

```
cd brushes
```

Copy to Clipboard

15.

List all contents, in long format, including the hidden files and directories, ordered by the date and time they were last modified.

Use `ls` and combine the options `-a`, `-l`, and `-t`, making `-alt`.

```
ls -alt
```

Copy to Clipboard

16.

Copy the files starting with the letter **f** from the **brushes/** directory to the **paint/** directory. The path to the **paint/** directory is `../paint/`. Without changing directories, list the files and directories of the **paint/** directory.

First, let's copy the files using `cp`.

The source file(s) are all `.txt` files starting with `f`, which we can shorthand as `f*.txt` (`*` is wildcard).

The destination is `../paint/`.

Putting it all together, the `cp` command would be:

```
cp f*.txt ../paint/
```

Copy to Clipboard

To look at the `../paint/` directory, use `ls`.

```
ls ../paint/
```

Copy to Clipboard

What you see should confirm that your `cp` command worked!

17.

Change directories to the **sculpting/** directory. The relative path is `../sculpting/`

The current directory is

```
/home/ccuser/workspace/artusi/painting/brushes/
```

Copy to Clipboard

sculpting/ is another directory inside the directory **artusi/**. That means we need to go up two directory levels to **artusi/** using `../..`, and from there go into **sculpting/**. So our `cd` statement would be:

```
cd ../../sculpting.
```

Copy to Clipboard

18.

List all contents, in long format, including the hidden files and directories, ordered by the date and time they were last modified.

Use `ls` and combine the options `-a`, `-l`, and `-t`, making `-alt`.

```
ls -alt
```

Copy to Clipboard

19.

Change directories into the **clay/polymer/** directory, and list all contents of the directory.

The current directory is

```
/home/ccuser/workspace/artusi/sculpting
```

Copy to Clipboard

clay/ is a directory inside the current one so we can change directories like so:

```
cd clay/polymer
```

Copy to Clipboard

Now view the contents in the current directory:

```
ls
```

Copy to Clipboard

20.

Move **airdry.txt** into the **ceramic/** directory. The relative path to this directory is `../ceramic/`.

Use the `mv` command to move a file without making a copy. We have a file to move called **airdry.txt** and a destination `../ceramic/`, so we can put together a command like so:

```
mv airdry.txt ../ceramic/
```

Copy to Clipboard

You can use the following commands to confirm the result of the above move:

```
ls
```

```
Copy to Clipboard
to make sure airdry.txt is no longer in the current directory and
ls ../ceramic/
```

```
Copy to Clipboard
to see airdry.txt in the destination directory.
```

21.
Change directories into the **ceramic/** directory.

As the step before mentioned, the relative path to **ceramic/** is **../ceramic/**. so we can change directories like so:

```
cd ../ceramic
```

```
Copy to Clipboard
You can confirm you changed into the correct directory using
pwd
```

```
Copy to Clipboard
```

22.
List all contents, including hidden files and directories.

To display a directory's contents, use **ls**. Since we want to also include hidden files, we can use the option **-a**.

```
ls -a
```

```
Copy to Clipboard
```

23.
Remove **earthenware.txt** from the current directory.

To remove a single file, we can use the **rm** command. This current directory contains **earthenware.txt**, so we don't need to specify a path.

```
rm earthenware.txt
```

```
Copy to Clipboard
You can confirm the removal of the file using
ls
```

```
Copy to Clipboard
to see the updated contents of the directory.
```

24.
Change directories one level up back to the **clay/** directory.

To change directories one level up, we can use **cd** with **..**

```
cd ..
```

```
Copy to Clipboard
You can confirm you changed into the correct directory using
pwd
```

```
Copy to Clipboard
```

25.
Take a look at the contents of the current directory **clay/**. Delete the **dough/** directory.

To remove a directory, we should use the **rm** command with the option **-r**, which deletes the directory and all its contents (even directories nested inside). We could combine the option **-r** with the option **-f**, which will delete forcibly without individually confirming each piece of content inside the directory.

Since **dough/** is a directory, our command would look like this:

```
rm -rf dough
```

```
Copy to Clipboard
```

26.
Change directories two levels up back to the **artusi/** directory. Print the working directory.

To change directories two levels up, we can use **cd** with two **../**s separated by a **/**:

```
cd ../../
```

```
Copy to Clipboard
Let's confirm the current directory using
pwd
```

```
Copy to Clipboard
```

Machine Learning/AI Engineer

Athletica

In this project, you'll use the commands you just learned to redirect files in Athletica, a sporting events directory.

If you get stuck during this project or would like to see an experienced developer work through it, click "**Get Unstuck**" to see a **project walkthrough video**.

Tasks

0/15 complete

Mark the tasks as complete by checking them off

1.

Print the working directory.

```
pwd
```

Copy to Clipboard

2.

List all files and directories in long format, including hidden files.

```
ls -la
```

Copy to Clipboard

3.

Use `cat` to view the contents of **basketball.txt**.

```
cat basketball.txt
```

Copy to Clipboard

4.

Use `cat` to view the contents of **hockey.txt**.

```
cat hockey.txt
```

Copy to Clipboard

5.

Redirect the standard output of **basketball.txt** into **hockey.txt**. Then view the contents of **hockey.txt**.

```
cat basketball.txt > hockey.txt
```

```
cat hockey.txt
```

Copy to Clipboard

6.

View the contents of **lacrosse.txt**.

```
cat lacrosse.txt
```

Copy to Clipboard

7.

Append the contents of **lacrosse.txt** to the contents of **tennis.txt**. Then, view the contents of **tennis.txt**.

```
cat lacrosse.txt >> tennis.txt
```

```
cat tennis.txt
```

Copy to Clipboard

8.

Redirect the contents of **gymnastics.txt** as standard input for the `cat` command.

```
cat < gymnastics.txt
```

Copy to Clipboard

9.

Pipe the standard output of `cat lacrosse.txt` to the wordcount command.

```
cat lacrosse.txt | wc
```

Copy to Clipboard

10.

Use `cat` to view the contents of **equipment.txt**.

```
cat equipment.txt
```

Copy to Clipboard

11.

Sort the contents of **equipment.txt** in alphabetical order.

```
sort equipment.txt
```

Copy to Clipboard

12.

Use the `uniq` command to filter out adjacent, duplicate lines in **equipment.txt**.

```
sort equipment.txt | uniq
```

Copy to Clipboard

13.

Use the `grep` command to search **roster.txt** for players from Japan.

```
grep Japan roster.txt
```

Copy to Clipboard

14.

Use the `grep` command to search for the string 'player' in the current directory (`.`), and output filenames and lines for matched results.

```
grep -R player .
```

Copy to Clipboard

15.

Use `cat` to view the contents of **games.txt**. Then, use `sed` to replace all instances of the word 'loss' with 'win' in **games.txt**.

```
sed 's/loss/win/g' games.txt
```


Attica

Attica, a clothing store, has asked you to help them configure their environment.

If you get stuck during this project or would like to see an experienced developer work through it, click **"Get Unstuck"** to see a **project walkthrough video**.

Tasks

0/18 complete

[Mark the tasks as complete by checking them off](#)

1.

Print the working directory.

To display the current working directory, use:

```
pwd
```

Copy to Clipboard

2.

Create and open a file in nano called **hello.txt**.

Create and open a file with nano like so:

```
nano filename.txt
```

Copy to Clipboard

3.

Save the string, "Hello, I am nano" in **hello.txt**.

Save the file, exit nano, and clear the terminal.

Once nano is open, type in the string. To save and exit, type these keys in the following order:

```
Ctrl + O  
Enter  
Ctrl + X
```

Copy to Clipboard

To clear the terminal, use the `clear` command.

4.

Use nano to open the bash profile.

Open a file with nano like so:

```
nano filename.txt
```

Copy to Clipboard

The path to the bash profile is `~/.bash_profile`

5.

In the bash profile, add a greeting with the word "Hello" using the `echo` command. Save the file, exit nano, and clear the terminal.

Once the bash profile is open, type in `echo` and the string "Hello" on the same line.

To save and exit, type these keys in the following order:

```
Ctrl + O  
Enter  
Ctrl + X
```

Copy to Clipboard

To clear the terminal, use the `clear` command.

6.

Use the `source` command to make the greeting available in the current session.

You should see the greeting you created.

To execute the content that we've just updated the bash profile with, use:

```
source ~/.bash_profile
```

Copy to Clipboard

7.

Open the bash profile, and create two aliases. The first alias should be `p` for the `pwd` command and `ll` for the `ls -la` command.

Save the file, exit nano, and clear the terminal.

To make an alias in bash profile, use the following format on a new line:

```
alias aliascommand="originalcommand"
```

Copy to Clipboard

8.

Use the `source` command to make the aliases available in the current session.

To execute the content that we've just updated the bash profile with, use:

```
source ~/.bash_profile
```

Copy to Clipboard

9.

Test out the aliases you created for the `pwd` and `ls -la` command.

The alias we created for `pwd` is:

```
p
```

Copy to Clipboard

The alias we created for `ls -la` is:

```
ll
```

Copy to Clipboard

Type each of them after the command prompt.

10.

Open the bash profile and create and export the `USER` environment variable, setting it equal to your name.

Save the file, exit nano, and clear the terminal.

To export an environment variable, type the following format in the bash profile:

```
export VARIABLE="Value"
```

Copy to Clipboard

In this case, the environment variable is `USER` and the value is your name.

11.

Use the `source` command to make the aliases available in the current session.

To execute the content that we've just updated the bash profile with, use:

```
source ~/.bash_profile
```

Copy to Clipboard

12.

Echo the `USER` variable.

To echo an environment variable, use `echo` and the variable name (with a `$` in front of it).

```
echo $VARIABLE
```

Copy to Clipboard

13.

Open the bash profile and create and export the `PS1` environment variable, setting it equal to `">> "`.

Save the file, exit nano, and clear the terminal.

To export an environment variable, type the following format in the bash profile:

```
export VARIABLE="Value"
```

Copy to Clipboard

In this case, the environment variable is `PS1` and the value should be `">> "`.

14.

Use the `source` command to make the prompt available in the current session.

To execute the content that we've just updated the bash profile with, use:

```
source ~/.bash_profile
```

Copy to Clipboard

15.

Test out the prompt by typing the two aliases you created earlier.

The alias we created for `pwd` is:

```
p
```

Copy to Clipboard

The alias we created for `ls -la` is:

```
ll
```

Copy to Clipboard

Type each of them after the new command prompt.

16.

Echo the `HOME` variable.

To echo an environment variable, use `echo` and the variable name (with a `$` in front of it).

```
echo $VARIABLE
```

Copy to Clipboard

17.

Echo the `PATH` variable.

To echo an environment variable, use `echo` and the variable name (with a `$` in front of it).

```
echo $VARIABLE
```

Copy to Clipboard

18.

Return a list of environment variables.

```
env
```

Copy to Clipboard

returns and displays the list of environment variables.

Lingua Franca

Lingua Franca, a translation service, has asked you to help them configure their environment.

If you get stuck during this project or would like to see an experienced developer work through it, click **"Get Unstuck"** to see a **project walkthrough video**.

Tasks

16/16 complete

[Mark the tasks as complete by checking them off](#)

1.

Print the working directory.

```
pwd
```

Copy to Clipboard

2.

Create and open the bash profile.

```
nano ~/.bash_profile
```

Copy to Clipboard

3.

In the bash profile, add a greeting of your choice.

Save the bash profile, exit nano, and clear the terminal window.

```
echo "Hello, Hola, Ni Hao, Bonjour, and Kon'nichiwa! Welcome to our translation services"
```

Copy to Clipboard

```
Ctrl + O
```

```
Enter
```

```
Ctrl + X
```

Copy to Clipboard

4.

Source the bash profile to make the greeting available in the current session. You should see the greeting you creating in the above step.

```
source ~/.bash_profile
```

Copy to Clipboard

5.

Open the bash profile, and create three aliases, one for the `mkdir` command, one for `date`, and one for `history`.

Save the bash profile, exit nano, and clear the terminal window.

```
alias md="mkdir"
```

```
alias d="date"
```

```
alias hy="history"
```

Copy to Clipboard

6.

Source the bash profile to make the aliases available in the current session.

```
source ~/.bash_profile
```

Copy to Clipboard

7.

Test out the aliases you created for the `mkdir`, `date`, and `history` commands. (Recall that when you test out the alias for `mkdir` you will need to pass in a directory name as an argument such as `translations`.)

```
md translations
```

```
d
```

```
hy
```

Copy to Clipboard

8.

Type the `less` command, with the filename `japanese.txt`. After you do this, type `q` to quit.

The `less` command is similar to `cat`, but is better adapted to handling larger files. It displays files in the terminal one page at a time.

```
less javanese.txt
```

Copy to Clipboard

```
q
```

Copy to Clipboard

9.

The `less` command is a special command that we can configure in the bash profile through the `LESS` environment variable. Open the bash profile, and create and export a new environment variable called `LESS`, setting it equal to the option `"-N"`. The `-N` option adds line numbers to the file.

Save the bash profile, exit nano, and clear the terminal window.

```
export LESS="-N"
```

Copy to Clipboard

10.

Source the bash profile to make the `LESS` line number option available for the current session.

```
source ~/.bash_profile
```

Copy to Clipboard

11.

Type the `less` command again now, with the filename `javanese.txt`. The file should appear, along with line numbers. Press `q` to quit.

```
less javanese.txt
```

Copy to Clipboard

```
q
```

Copy to Clipboard

12.

Open the bash profile. Create and export the `PS1` environment variable, setting it equal to a prompt of your choice. Be sure to leave a space before the close of the quotations.

Save the bash profile, exit nano, and clear the terminal window.

```
export PS1="c[0_T•]D "
```

Copy to Clipboard

13.

Source the bash profile to make the new prompt available in the current session.

```
source ~/.bash_profile
```

Copy to Clipboard

14.

Test out the prompt by typing the names of the aliases you created.

```
md script
d
hy
```

Copy to Clipboard

15.

Return a list of environment variables.

```
env
```

Copy to Clipboard

16.

Last but not least, return the value for the environment variable for `LESS`.

```
env | grep LESS
```

Manhattan Zoo

Ready to try out some of your new Git knowledge?

In this project, you'll use Git to keep track of meal guidelines for animals at the Manhattan Zoo.

If you get stuck during this project or would like to see an experienced developer work through it, click **"Get Unstuck"** to see a **project walkthrough video**.

Tasks

0/10 complete

[Mark the tasks as complete by checking them off](#)

Manhattan Zoo

1.

Initialize a new Git repository.

All git commands begin with `git`, followed by the specific instruction. `init` initializes a new repository.

2.

Check the status of the repository.

`status` is the git command to check the repository status.

3.

Add `meal-regimens.txt` to the staging area.

4.

Make a commit.

`commit` is used to make git commits. the `-m` flag followed by a message in double-quotes ("`<message>`") attaches a message to the commit.

5.

Include this new info in `meal-regimens.txt`.

```
3. Long-Tailed Chinchillas
```

```
Meal: 1 bag animal pellets, 1 bag dried fruit, 1/2 bag cashews, 5 carrots, 3 stalks kale
```

```
Times: 8:00 am
```

```
Directions: disperse contents throughout Chinchilla habitat
```

Copy to Clipboard

Click Save.

6.

Add `meal-regimens.txt` to the staging area.

7.

Check the status of the Git project.

You should see `meal-regimens.txt` listed as "modified".

8.

Make a commit.

9.

View your Git commit history.

If your cursor is stuck in Git log mode, press "q" on your keyboard to escape.

The git command `log` can be used to view git history.

10.

Here's two more animal reports. Include each in `meal-regimens.txt`, making a new commit for each animal added.

```
4. Poison Dart Frogs
```

```
Meal: 1 bag small crickets
```

```
Times: 6:00 am
```

```
Directions: empty bag in frog habitat once daily. Do not touch frogs! Extremely poisonous.
```

```
5. Western Lowland Gorilla
```

```
Meal: (Morning) 20 lbs. kale, 10 lbs. celery, 10 lbs. green beans, 5 lbs. carrots, 1 bag sweet potatoes. (Evening) 10 Bananas, 10 apples, 5 oranges, 5 mango, 20 lbs. grapes, 10
```

```
lbs. turnips, 5 lbs. white potatoes
```

Times: 6:30 am, 12:00 pm, 7:00 pm

Directions: feed Gorillas in the morning as group, spread forage items during noon meal, and divide quantities for individual feeding in evening

Copy to Clipboard

SnapFit Robots, Inc.

Now that you've had more practice with the Git workflow, let's solidify your new skills even more.

In this project, you will be working on assembly instructions for Snap-Fit Robots Inc., a build-it-yourself robot company.

If you get stuck during this project or would like to see an experienced developer work through it, click **"Get Unstuck"** to see a **project walkthrough video**.

Tasks

0/11 complete

Mark the tasks as complete by checking them off

SnapFit Robots Inc.

1.
Initialize a new Git project.
2.
Check the status of the Git project.
You will see multiple files listed in the output as "Untracked".
3.
Add each file to the Git staging area.
4.
Check the status of the Git project again.
5.
Make a commit.
6.
View your Git commit log.
If your cursor is stuck in Git log mode, press "q" on your keyboard to escape.
7.
Include this line in **disclaimer.txt**:

```
Warning: For best battery life, do not leave robot battery  
charging overnight.
```

Copy to Clipboard

Click Save.

8.
Add the file to the staging area.
9.
Now make a commit.
10.
View your Git commit log again to identify your commit.
11.
Revise each file in whatever ways you'd like. Then add your changes to the staging area and make another commit.

Poem Fiasco

Let's get some practice with Git backtracking.

In this project, changes have been made to a series of poems and you want to change them back.

If you get stuck during this project or would like to see an experienced developer work through it, click **"Get Unstuck"** to see a **project walkthrough video**.

Tasks

0/13 complete

[Mark the tasks as complete by checking them off](#)

Poems Fiasco

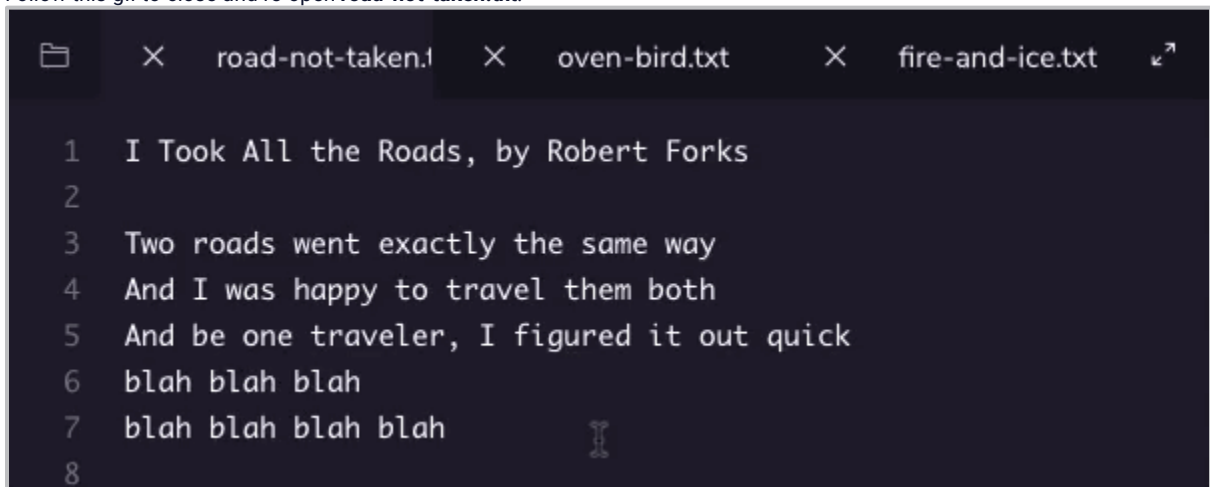
1.

The file **road-not-taken.txt** doesn't look right at all! Perhaps a rival poet snuck in and changed it while you were getting coffee. Good thing you've been committing often.

Checkout the `HEAD` version of **road-not-taken.txt** to discard changes made to the working directory.

Close and re-open the file to see the result.

Follow this gif to close and re-open **road-not-taken.txt**:



2.

Now, finish the poem by adding a line. Here's a suggestion:

And that has made all the difference.

Copy to Clipboard

Then click Save.

3.

Take a look at **oven-bird.txt** to see if it has also been tampered with.

Indeed it has! We'll want to discard changes in the working directory again.

There's a commonly used shortcut for this command:

```
git checkout -- filename
```

Copy to Clipboard

It does the same exact thing that `git checkout HEAD filename` does. Try it with **oven-bird.txt**.

4.

Now, finish "Oven Bird" by adding a line. Here's a suggestion:

Is what to make of a diminished thing.

Copy to Clipboard

Then click Save.

5.

Click on **fire-and-ice.txt**.

This file has *not* been altered, but just to be sure, check the diff for this file.

Then, add these last two lines to the poem:

```
Is also okay  
And would suffice.
```

Copy to Clipboard

6.

Now that you've restored and completed **road-not-taken.txt** and **oven-bird.txt** and added a line to **fire-and-ice.txt**, add all three of the files to the staging area with a single command.

7.

fire-and-ice.txt could be better. Before you commit, remove this file from the staging area.

8.

Now that you've removed **fire-and-ice.txt**, make a commit.

9.

You get the crazy idea to change your poems in a big way.

Make some drastic changes to each of the three poems. Remember to click Save after each file change.

10.

Now add all three files to the staging area.

11.

Make a commit.

12.

A little later you take a look at the current state of your poems and regret your last commit.

Reset your Git project to the commit before you made those drastic changes.

13.

There's a problem: you reset `HEAD` to a previous commit, but the changes you want to get rid of are still in the working directory.

What Git backtracking command that you already know can discard changes to the working directory, restoring the files to the way they look in the `HEAD` commit?

Stuck? Here's a [hint](#).

ASCII Portfolio

ASCII art is art made from only the letters, numbers and symbols you can type on your keyboard.

In this project, you'll use Git backtracking commands to undo mistakes made to an ASCII art portfolio!

If you get stuck during this project or would like to see an experienced developer work through it, click **"Get Unstuck"** to see a **project walkthrough video**.

Tasks

0/11 complete

[Mark the tasks as complete by checking them off](#)

ASCII Portfolio

1.

The ASCII face in **portrait.txt** had a goatee in the last commit, but it was deleted in the working directory. Taking a second look, you want the goatee back.

Discard changes in the working directory for **portrait.txt**.

Close and reopen **portrait.txt** to see the result.

2.

That's much better! Now, give your portrait some eyebrows. It's up to you how to do it. One way is to use a few `=` symbols on the line above the eyes, like this:

```
====      ===  
O          O
```

[Copy to Clipboard](#)

Click Save.

3.

Add the file to the staging area.

4.

Make a commit.

5.

It looks like every file has its date written incorrectly.

Change the completion dates on every file, clicking Save after each change.

6.

Next, add all your changes to the staging area with a single command.

7.

Make a commit.

8.

You forgot to add your "artist name" to each file. Under the date, include the name:

```
L. Da Vinci
```

[Copy to Clipboard](#)

Or whatever name you'd like. Click Save after each file change.

9.

If you know you want to add every file in the working directory to the staging area, instead of adding each file individually, you can use a shortcut:

```
git add .
```

[Copy to Clipboard](#)

The `.` means "all files". Adding files to the staging area with `.` is faster than specifying each file individually, but it's easy to accidentally add files you don't want there. Make sure you always know what you're adding.

10.

It turns out that **house.txt** is an experiment and doesn't belong in the commit you're staging. Reset the staging area to remove **house.txt**.

11.

Now make a commit.

Sam's Surf Shop

Welcome to Sam's Surf Shop! This project will exercise your knowledge of errors and unit testing practices in Python. It will also give you a small taste of testing a full application.

You've been hired to create a handful of tests for the shopping cart software at the surf shop. Once that is done, you'll implement some improvements for these tests using more advanced unit testing features (skipping, parameterization, and expected failures). Finally, you'll have the opportunity to fix bugs that were exposed by your tests.

The shopping cart software for Sam's Surf Shop lives inside of the file called `surfshop.py`. Look over the files and familiarize yourself with their contents. Most of our work will take place in `tests.py`.

If you get stuck, you can look at a final version of the project by looking at `solution_surf.py` and `solution_test.py` - but try it out on your own first.

Let's get started!

Tasks

2/13 complete

[Mark the tasks as complete by checking them off](#)

Create your tests

1.

Let's get some basic setup out of the way.

First, import both the `surfshop` and `unittest` modules in `tests.py`.

Modules are imported by using the `import` keyword followed by the module name.

2.

Next, create a class which will contain all of your tests. The class can be named whatever you'd like, but it should inherit from `unittest.TestCase`.

Remember that classes can inherit from other classes by including a set of parentheses with a parent class following the class name, like this:

```
class MyClass(ParentClass):
```

Copy to Clipboard

3.

The features you need to test have been implemented in the `surfshop.ShoppingCart` class. In order to test the inner workings of a class, you will need to create a new instance of the shopping cart. Don't worry - you will handle that in the next tasks! For now, it's important that every test has a new `ShoppingCart` object to work with so that way the test always starts on a clean slate.

In your class, create a setup fixture that runs before every test. It should instantiate a new `ShoppingCart` object and assign it to an instance variable called `self.cart`. Your tests can then use `self.cart` to reference your instance of the `ShoppingCart` class.

Inside of a `TestCase` class, you can define a method called `setUp()`. This method runs automatically before each test.

Remember that new objects are instantiated by calling the class name followed by parentheses, as follows:

```
surfshop.ShoppingCart
```

Copy to Clipboard

4.

It's time to create your first test! Let's test the `add_surfboards()` method of the cart.

The `ShoppingCart.add_surfboards()` method takes an integer as its only argument and updates the number of surfboards in the cart. Define a test method that calls this function with an argument of `1` and checks that `'Successfully added 1 surfboard to cart!'` is returned.

Remember that:

test methods must start with the word `test`

the `self.assertEqual()` method can be used to verify that two strings are the same

5.

Let's test another input for the `.add_surfboards` method. Create another test method which calls `ShoppingCart.add_surfboards()`, but this time, passes an argument of `2`. It should test that the return value is `'Successfully added 2 surfboards to cart!'`

6.

The shopping cart has a limit of 4 surfboards per customer. Create a test to check that a `surfshop.TooManyBoardsError` (a custom exception) is raised when `ShoppingCart.add_surfboards()` is called with an argument of 5.

The `self.assertRaises()` method takes an exception as its first argument and a function as its second. Any additional arguments get passed into the function. The test fails if the exception is not raised. Alternatively, you can pass the arguments directly into the function.

7.

The shopping cart has a feature that applies rental discounts for locals called `apply_locals_discount()`. When this function is called, it sets the `self.locals_discount` property to `True`.

Create a test that calls `ShoppingCart.apply_locals_discount()` and then checks that `ShoppingCart.locals_discount` is `True`.

The `self.assertTrue()` method checks that the passed argument evaluates to `True`.

Run and maintain your tests

8.

It's time to start running your tests! At the bottom of `tests.py`, call `unittest.main()`.

If you've implemented your tests correctly, they should all pass - except for one! It seems that the `ShoppingCart.apply_locals_discount()` function is not working as expected. While you wait for the development team to fix this bug, you don't want it to cause our tests to fail. Mark this test as an expected failure.

The `@unittest.expectedFailure` decorator can mark tests as expected to fail.

9.

Sam, the owner of Sam's Surf Shop, has just informed us that the shop is heading into the off season and business has slowed down. The store's shopping cart no longer needs to enforce the 4 surfboards per customer rule - at least until business picks up again.

Go back and modify the test you wrote in task 5 which checks for a `surfshop.TooManySurfboardsError` so that it is skipped.

You can use the `@unittest.skip` decorator to skip a test.

10.

Parameterize the test you wrote in task 4 so that it runs 3 times, passing 2, 3, and 4 as the arguments to `surfshop.add_surfboards()`. This allows us to easily test a single function with a variety of inputs. Remember to modify the expected return value with the correct number of surfboards.

You can use the `unittest.subTest()` decorator inside of a for loop to parameterize a test.

Improve the software

11.

Sam has noticed all of your hard work and the fact that your tests found a bug. You can now start working on the actual shopping cart software!

Take a look in `surfshop.py`. Recall that the `ShoppingCart.apply_locals_discount` is not setting the `ShoppingCart.locals_discount` attribute to `True`, as it should be. Can you fix it?

When you do, comment out the expected failure decorator and see if all the tests pass.

12.

Next, make an improvement to the exception that gets thrown when too many surfboards are added to the cart. Modify `TooManySurfboardsError` so that when raised, it has the message `'Cart cannot have more than 4 surfboards in it!'`.

The `__str__` method of an exception class determines the exception message.

13.

Congratulations on your first successful software testing project! In this project, you successfully:

- Implemented a suite of unit tests for existing software.

- Ran and modified the tests according to results.

- Improved the existing software.

If you want to challenge yourself even further, take a look at the `ShoppingCart.set_checkout_date()` function. This function takes a `datetime.datetime` object as an argument and raises a `surfshop.CheckoutDateError` if the date is not in the future. Can you write a test that validates this behavior?

Machine Learning/AI Engineer

Sam's Surf Shop

Welcome to Sam's Surf Shop! This project will exercise your knowledge of errors and unit testing practices in Python. It will also give you a small taste of testing a full application.

You've been hired to create a handful of tests for the shopping cart software at the surf shop. Once that is done, you'll implement some improvements for these tests using more advanced unit testing features (skipping, parameterization, and expected failures). Finally, you'll have the opportunity to fix bugs that were exposed by your tests.

The shopping cart software for Sam's Surf Shop lives inside of the file called `surfshop.py`. Look over the files and familiarize yourself with their contents. Most of our work will take place in `tests.py`.

If you get stuck, you can look at a final version of the project by looking at `solution_surf.py` and `solution_test.py` - but try it out on your own first.

Let's get started!

Tasks

1/13 complete

[Mark the tasks as complete by checking them off](#)

Create your tests

1.

Let's get some basic setup out of the way.

First, import both the `surfshop` and `unittest` modules in `tests.py`.

Modules are imported by using the `import` keyword followed by the module name.

2.

Next, create a class which will contain all of your tests. The class can be named whatever you'd like, but it should inherit from `unittest.TestCase`.

Remember that classes can inherit from other classes by including a set of parentheses with a parent class following the class name, like this:

```
class MyClass(ParentClass):
```

Copy to Clipboard

3.

The features you need to test have been implemented in the `surfshop.ShoppingCart` class. In order to test the inner workings of a class, you will need to create a new instance of the shopping cart. Don't worry - you will handle that in the next tasks! For now, it's important that every test has a new `ShoppingCart` object to work with so that way the test always starts on a clean slate.

In your class, create a setup fixture that runs before every test. It should instantiate a new `ShoppingCart` object and assign it to an instance variable called `self.cart`. Your tests can then use `self.cart` to reference your instance of the `ShoppingCart` class.

Inside of a `TestCase` class, you can define a method called `setUp()`. This method runs automatically before each test.

Remember that new objects are instantiated by calling the class name followed by parentheses, as follows:

```
surfshop.ShoppingCart
```

Copy to Clipboard

4.

It's time to create your first test! Let's test the `add_surfboards()` method of the cart.

The `ShoppingCart.add_surfboards()` method takes an integer as its only argument and updates the number of surfboards in the cart. Define a test method that calls this function with an argument of `1` and checks that `'Successfully added 1 surfboard to cart!'` is returned.

Remember that:

test methods must start with the word `test`

the `self.assertEqual()` method can be used to verify that two strings are the same

5.

Let's test another input for the `.add_surfboards` method. Create another test method which calls

`ShoppingCart.add_surfboards()`, but this time, passes an argument of `2`. It should test that the return value is `'Successfully added 2 surfboards to cart!'`

6.

The shopping cart has a limit of 4 surfboards per customer. Create a test to check that a `surfshop.TooManyBoardsError` (a custom exception) is raised when `ShoppingCart.add_surfboards()` is called with an argument of 5.

The `self.assertRaises()` method takes an exception as its first argument and a function as its second. Any additional arguments get passed into the function. The test fails if the exception is not raised. Alternatively, you can pass the arguments directly into the function.

7.

The shopping cart has a feature that applies rental discounts for locals called `apply_locals_discount()`. When this function is called, it sets the `self.locals_discount` property to `True`.

Create a test that calls `ShoppingCart.apply_locals_discount()` and then checks that `ShoppingCart.locals_discount` is `True`.

The `self.assertTrue()` method checks that the passed argument evaluates to `True`.

Run and maintain your tests

8.

It's time to start running your tests! At the bottom of `tests.py`, call `unittest.main()`.

If you've implemented your tests correctly, they should all pass - except for one! It seems that the `ShoppingCart.apply_locals_discount()` function is not working as expected. While you wait for the development team to fix this bug, you don't want it to cause our tests to fail. Mark this test as an expected failure.

The `@unittest.expectedFailure` decorator can mark tests as expected to fail.

9.

Sam, the owner of Sam's Surf Shop, has just informed us that the shop is heading into the off season and business has slowed down. The store's shopping cart no longer needs to enforce the 4 surfboards per customer rule - at least until business picks up again.

Go back and modify the test you wrote in task 5 which checks for a `surfshop.TooManySurfboardsError` so that it is skipped.

You can use the `@unittest.skip` decorator to skip a test.

10.

Parameterize the test you wrote in task 4 so that it runs 3 times, passing 2, 3, and 4 as the arguments to `surfshop.add_surfboards()`. This allows us to easily test a single function with a variety of inputs. Remember to modify the expected return value with the correct number of surfboards.

You can use the `unittest.subTest()` decorator inside of a for loop to parameterize a test.

Improve the software

11.

Sam has noticed all of your hard work and the fact that your tests found a bug. You can now start working on the actual shopping cart software!

Take a look in `surfshop.py`. Recall that the `ShoppingCart.apply_locals_discount` is not setting the `ShoppingCart.locals_discount` attribute to `True`, as it should be. Can you fix it?

When you do, comment out the expected failure decorator and see if all the tests pass.

12.

Next, make an improvement to the exception that gets thrown when too many surfboards are added to the cart. Modify `TooManySurfboardsError` so that when raised, it has the message `'Cart cannot have more than 4 surfboards in it!'`.

The `__str__` method of an exception class determines the exception message.

13.

Congratulations on your first successful software testing project! In this project, you successfully:

- Implemented a suite of unit tests for existing software.

- Ran and modified the tests according to results.

- Improved the existing software.

If you want to challenge yourself even further, take a look at the `ShoppingCart.set_checkout_date()` function. This function takes a `datetime.datetime` object as an argument and raises a `surfshop.CheckoutDateError` if the date is not in the future. Can you write a test that validates this behavior?

Concurrent Programming

In this project, you are going to be calculating the average of three lists using four different approaches:

- sequential approach
- async approach
- threading approach
- multiprocessing approach

Your goal will be to make this calculation as efficient as possible and see which approaches work well and which ones struggle.

Note that a **solution.py** file is also loaded for you in the workspace, which contains solution code for this project. We highly recommend that you complete the project on your own without checking the solution, but feel free to take a look if you get stuck or want to check your answers when you're done!

In this project, we are using a terminal. To run your code, click **Save** and then type the command `python3 script.py` into the terminal and press **Enter** on your keyboard.

Tasks

0/14 complete

[Mark the tasks as complete by checking them off](#)

Starter Code

1.

Before diving in, look at the starter code in **script.py**. There are six functions defined:

```
cal_average()
```

This helper function takes in a list of numbers, `num`, and calculates the average of the numbers within the list. Notice that we have added a `time.sleep(1)` line before returning the average. This extra second is an added barrier to test the efficiency of each approach, as you will use this helper function in your sequential, threading, and multiprocessing programs.

```
main_sequential()
```

This is where you will write your sequential program. The lines `s = time.perf_counter()` and `elapsed = time.perf_counter() - s` are there to time your program (they are in the rest of the functions as well.).

```
cal_average_async()
```

This helper function is the same as `calc_average()` except it is an asynchronous function you will use for your async program.

```
main_async()
```

This is where you will write your async program.

```
main_threading()
```

This is where you will write your threading program.

```
main_multiprocessing()
```

This is where you will write your multiprocessing program.

Finally, the code after `main_multiprocessing()` tests your sequential, async, threading, and multiprocessing functions for you each time you run **script.py**. There is no need to touch it during the project.

Sequential Approach

2.

In `main_sequential()`, write a program that calculates the average of three lists sequentially using `cal_average()` as a helper function.

Your code should go in between `s = time.perf_counter()` and `elapsed = time.perf_counter() - s`.

Before running **script.py**, estimate how long these calculations should take when using `main_sequential()`.

In the sequential program, you should call `cal_average()` three times on each list, like so:

```
cal_average(list1)
cal_average(list2)
cal_average(list3)
```

[Copy to Clipboard](#)

3.

Execute **script.py**.

How long did it take for the calculation to complete? Does it line up with your estimate?

It should take about three seconds for the `main_sequential()` function to calculate the average of all three lists. This makes sense since a sequential program will wait one second between each average calculation using `cal_average()`.

This is inefficient. Imagine if we had thousands or millions of calculations to perform! Even if each calculation was not very computationally expensive, the run time could increase exponentially.

Try out the other methods to see if any better handle the `time.sleep(1)` inconvenience.

Async Approach

4.

Now it's time to try out an async approach.

In the `main_async()` function, create a variable called `tasks` and assign it to a list that contains three function calls on `list1`, `list2`, and `list3` using `cal_average_async()`.

Your `tasks` variable should look like the following:

```
tasks = [cal_average_async(list1), cal_average_async(list2),
cal_average_async(list3)]
```

Copy to Clipboard

5.

Run each task concurrently. To do this, you will need to:

- Use the `await` keyword

- Group your tasks together

Click the hint if you get stuck.

Before running `script.py`, estimate how long these calculations should take when using `main_async()`.

To run the asynchronous tasks concurrently, use `asyncio.gather()` and pass in `tasks` as an unpacked list like the following:

```
await asyncio.gather(*tasks)
```

Copy to Clipboard

* unpacks `tasks` so that it can be used as an argument in `.gather()`.

6.

Run `script.py`.

How long did it take for the calculation to complete? Does it line up with your estimate?

It should take about one second for the `main_async()` function to calculate the average of all three lists.

When each task reaches `await asyncio.sleep(1)`, the function tells the event loop nothing is happening and pushes for the next task to occur. This allows all three calculations to run in about one second rather than three seconds.

Threading Approach

7.

Now it's time to try out the threading approach. Do the following in the `main_threading()` function:

- Create a variable called `lists` containing three elements: `list1`, `list2`, and `list3`.

- Create an empty list called `threads`.

8.

Still within `main_threading()`, create a for loop that iterates through `range(len(lists))`. At each iteration:

- Create a thread called `x`. Your thread should run `cal_average()` and pass in one of the lists as its argument.

- Add each thread to `threads`.

- Start the created thread.

Next, create another for loop. This time loop through each thread in `threads`. In this for loop, you should join each thread using the `.join()` method.

Before running `script.py`, estimate how long these calculations should take when using `main_threading()`.

Your thread should look like the following:

```
x = threading.Thread(target=cal_average, args=(lists[index],))
```

Copy to Clipboard

Note that `index` is what we used as the "dummy" variable in the for loop.

To use the `.join()` method, you should have something similar to the following code:

```
for t in threads:
    t.join()
```

Copy to Clipboard

9.

Run `script.py`.

How long did it take for the calculation to complete? Does it line up with your estimate?

`main_threading()` also happens in about a second, similar to the async approach.

Multiprocessing Approach

10.

Now it's time to try out the multiprocessing approach. Within `main_multiprocessing()`, create a variable called `lists` containing three elements: `list1`, `list2`, and `list3`.

11.

Still within `main_multiprocessing()`, create a process object for each list within `lists`. You can either use a for loop or list comprehension.

If you use a list comprehension, you should have code that looks like the following:

```
processes = [Process(target=cal_average, args=(lists[x],)) for x in range(len(lists))]
```

Copy to Clipboard

12.

Still within `main_multiprocessing()`, create two separate iterations:

One iteration should go through each process within your processes list and spawn each one using the `.start()` method.

The second iteration should also go through each process within your processes list and join each one using the `.join()` method.

Note that you should use two separate iterations to start the processes before you join them together.

Before running **script.py**, estimate how long these calculations should take when using `main_multiprocessing()`.

Your for loop to start the processes should look like the following:

```
for p in processes:
    p.start()
```

Copy to Clipboard

13.

Run **script.py**.

How long did it take for the calculation to complete? Does it line up with your estimate?

`main_multiprocessing()` takes about one second to run to completion.

Extra

14.

Fantastic job finishing this project! This was a challenging project where we pushed you to do some challenging analysis.

We hope you enjoyed it. If you are looking to do more, you can:

Create a new helper function to test the different approaches against.

Add print statements in your helper function. Do they print out in the same order for each approach?

Use a larger dataset instead of a small list, such as data from an Excel worksheet. (You would need to do this on your own computer locally.)

Create a new helper function to test the different approaches against various types of problems, not just finding the average of a list

Happy coding!

```
import time
import threading
import asyncio
from multiprocessing import Process

def cal_average(num): # Average function used for sequential programming, threading, and multiprocessing
    sum_num = 0
    for t in num:
        sum_num = sum_num + t
    avg = sum_num / len(num)
    time.sleep(1)
    return avg

def main_sequential(list1, list2, list3): # Main wrapper for sequential example
    s = time.perf_counter()
    # your code goes here

    elapsed = time.perf_counter() - s
    print("Sequential Programming Elapsed Time: " + str(elapsed) + " seconds")
```

```

async def cal_average_async(num): # Average function used for asynchronous programming only ( needs await
    asyncio.sleep() )
    sum_num = 0
    for t in num:
        sum_num = sum_num + t
    avg = sum_num / len(num)
    await asyncio.sleep(1)
    return avg

async def main_async(list1, list2, list3): # Main wrapper for asynchronous example
    s = time.perf_counter()
    # your code goes here

    elapsed = time.perf_counter() - s
    print("Asynchronous Programming Elapsed Time: " + str(elapsed) + " seconds")

def main_threading(list1, list2, list3): # Main wrapper for threading example
    s = time.perf_counter()
    # your code goes here

    elapsed = time.perf_counter() - s
    print("Threading Elapsed Time: " + str(elapsed) + " seconds")

def main_multiprocessing(list1, list2, list3): # Main wrapper for multiprocessing example
    s = time.perf_counter()
    # your code goes here

    elapsed = time.perf_counter() - s
    print("Multiprocessing Elapsed Time: " + str(elapsed) + " seconds")

if __name__ == '__main__': # Need to use this if-statement so multiprocessing doesn't cause an infinite loop
    l1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] # Three lists we are trying to calculate average on
    l2 = [2, 4, 6, 8, 10]
    l3 = [1, 3, 5, 7, 9, 11]
    main_sequential(l1, l2, l3)
    loop = asyncio.get_event_loop()
    loop.run_until_complete(main_async(l1, l2, l3))
    main_threading(l1, l2, l3)
    main_multiprocessing(l1, l2, l3)

```

SOLUTIONS:

```

import time
import threading
import asyncio
from multiprocessing import Process

def cal_average(num): # Average function used for sequential programming, threading, and multiprocessing
    sum_num = 0
    for t in num:
        sum_num = sum_num + t
    avg = sum_num / len(num)
    time.sleep(1)
    return avg

def main_sequential(list1, list2, list3): # Main wrapper for sequential example
    s = time.perf_counter()
    cal_average(list1)
    cal_average(list2)
    cal_average(list3)
    elapsed = time.perf_counter() - s
    print("Sequential Programming Elapsed Time: " + str(elapsed) + " seconds")

```

```

async def cal_average_async(num): # Average function used for asynchronous programming only ( needs await
    asyncio.sleep() )
    sum_num = 0
    for t in num:
        sum_num = sum_num + t
    avg = sum_num / len(num)
    await asyncio.sleep(1)
    return avg

async def main_async(list1, list2, list3): # Main wrapper for asynchronous example
    s = time.perf_counter()
    tasks = [cal_average_async(list1), cal_average_async(list2), cal_average_async(list3)]
    await asyncio.gather(*tasks) # Starts all asynchronous tasks and waits for them to complete
    elapsed = time.perf_counter() - s
    print("Asynchronous Programming Elapsed Time: " + str(elapsed) + " seconds")

def main_threading(list1, list2, list3): # Main wrapper for threading example
    s = time.perf_counter()
    lists = [list1, list2, list3]
    threads = list()
    for index in range(len(lists)):
        x = threading.Thread(target=cal_average, args=(lists[index],)) # Create 1 thread per list
        threads.append(x) # Add each thread to the list of 'threads'
        x.start() # Start all threads
    for thread in threads:
        thread.join()
    elapsed = time.perf_counter() - s
    print("Threading Elapsed Time: " + str(elapsed) + " seconds")

def main_multiprocessing(list1, list2, list3): # Main wrapper for multiprocessing example
    s = time.perf_counter()
    lists = [list1, list2, list3]
    processes = [Process(target=cal_average, args=(lists[x],)) for x in range(len(lists))] # This creates a
process per list
    for p in processes:
        p.start() # Starts each of the processes
    for p in processes:
        p.join()
    elapsed = time.perf_counter() - s
    print("Multiprocessing Elapsed Time: " + str(elapsed) + " seconds")

if __name__ == '__main__': # Need to use this if-statement so multiprocessing doesn't cause an infinite loop
    l1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] # Three lists we are trying to calculate average on
    l2 = [2, 4, 6, 8, 10]
    l3 = [1, 3, 5, 7, 9, 11]
    main_sequential(l1, l2, l3)
    loop = asyncio.get_event_loop()
    loop.run_until_complete(main_async(l1, l2, l3))
    main_threading(l1, l2, l3)
    main_multiprocessing(l1, l2, l3)

```

Build a Build Script

One common use of bash scripts is for releasing a “build” of your source code. Sometimes your private source code may contain developer resources or private information that you don’t want to release in the published version.

In this project, you’ll create a release script to copy certain files from a **source** directory into a **build** directory.

Tasks

0/13 complete

[Mark the tasks as complete by checking them off](#)

Start the script

1.

Take a look at the **build** and **source** folders. The objective of our script is to copy files from **source** to **build**, with a couple of exceptions and modifications.

Get started on the script by adding a header to **script.sh**, identifying the type of script.

Start the script with:

```
#!/bin/bash
```

Copy to Clipboard

2.

Let’s welcome the user to the script. Feel free to use your own style here.

Make sure to save your script. Test your script in the terminal using `./script.sh`.

For example:

```
echo "███Beginning build!! ███"
```

Copy to Clipboard

3.

Since we are creating a new build, let’s verify with the user that they have updated **changelog.md** with the current release version.

The first line of the file contains a version number with markdown formatting like so:

```
## 1.1.1
```

Copy to Clipboard

Read the first line of this file into a variable `firstline`. You can use the linux command [head](#) for this purpose.

By default, `head` returns 10 lines. But in this example, we want the first line. We can use the `-n` argument to specify the number of lines:

```
firstline=$(head -n 1 source/changelog.md)
```

Copy to Clipboard

4.

We want just the version number without the markdown formatting. The command [read](#) can be used to split a string into an array using the `-a` argument.

Split the string `firstline` into the array `splitfirstline`.

The syntax for splitting a string `foo` into an array `bar` is:

```
read -a bar <<< $foo
```

Copy to Clipboard

To split `firstline` into an array and assign it to `splitfirstline`:

```
read -a splitfirstline <<< $firstline
```

Copy to Clipboard

5.

Now we are ready to set the value of the version of the script. It is located in index `1` of the array `splitfirstline`.

The syntax for accessing the value at `index` of an array `foo` is:

```
${foo[index]}
```

Copy to Clipboard

Save the version to a variable, `version`.

Print a statement to the terminal notifying the user of the version they are building.

The version is at index 1, so the following code will get the version number and print it to the terminal:

```
version=${splitfirstline[1]}  
echo "You are building version" $version
```

Copy to Clipboard

User Input

6.

Let's give the user a chance to exit the script if they need to make a change to the version.

Ask the user to enter "1" (for yes) to continue and "0" (for no) to exit.

Assign their response to the variable `versioncontinue`.

The following will get user input and assign it to `versioncontinue`:

```
echo 'Do you want to continue? (enter "1" for yes, "0" for no) '  
read versioncontinue
```

Copy to Clipboard

7.

Add a conditional. If the user said "1" to the continue question, we will execute the rest of our script. For now, respond "OK".

If the user did not, tell them "Please come back when you are ready".

Here we use `-eq` because we are checking for equality between two numbers:

```
if [ $versioncontinue -eq 1 ]  
then  
    echo "OK"  
else  
    echo "Please come back when you are ready"  
fi
```

Copy to Clipboard

Copying files

8.

Now we want to copy every file from **source** to **build** except for **secretinfo.md**.

Within the positive conditional (where we told the user "OK"), start by iterating over all the files in the **source** directory and printing their names to the terminal.

Use the wildcard `*` to access files within the source directory:

```
for filename in source/*  
do  
    echo $filename  
done
```

Copy to Clipboard

9.

Check if the `filename` is "source/secretinfo.md". If it is, inform the user that it is not being copied.

Otherwise, inform the user that it is being copied.

Make sure to use spaces in your string conditional.

Here we are comparing two strings, so we use `==` to check equality:

```
if [ "$filename" == "source/secretinfo.md" ]  
then  
    echo "Not copying" $filename  
else  
    echo "Copying" $filename  
fi
```

Copy to Clipboard

10.

Now we can actually copy the files. After informing the user the file is being copied, copy the file into the `build` directory.

You can use the terminal to make sure the right files have been copied:

```
ls build/
```

Copy to Clipboard

We are copying the file using the `cp` command:

```
if [ "$filename" == "source/secretinfo.md" ]
```

```
then
  echo "Not copying" $filename
else
  echo "Copying" $filename
  cp $filename build/.
fi
```

Copy to Clipboard

Listing files

11.

The final thing we want to do is list the files in the **build** directory for the user.

Outside of the loop over the filenames in the directory, use the script to change the directory to the build directory. So that we don't forget, also add the command to change back to the directory with the script.

Use `cd` to change directories:

```
cd build/
# We'll add more code here later
cd ..
```

Copy to Clipboard

12.

Add code to notify the user what files are currently in the build directory.

Be sure to reference the version in your message.

Use `ls` to list the contents of a directory:

```
echo "Build version $version contains:"
ls
```

Copy to Clipboard

13.

You now have a build script for this repository. Feel free to play around with making it more robust. Some ideas:

- Copy **secretinfo.md** but replace "42" with "XX".

- Give the script more character with emojis.

Birthday Party

Let's practice some Git branching.

In this project, you'll be using Git to make a 1-page website for your friend Kay's birthday party.

index.html is written in HTML. Don't be afraid! You won't be asked to write any HTML from scratch. If you're interested in learning HTML, check out Codecademy's [HTML course](#).

If you get stuck during this project or would like to see an experienced developer work through it, click **"Get Unstuck"** to see a **project walkthrough video**.

Tasks

0/12 complete

[Mark the tasks as complete by checking them off](#)

Birthday Party

1.

From the terminal, list the Git branches.

2.

Kay wasn't sure where she wanted to host the party, so you made several Git branches to explore different locations. Some of the branches are no longer needed. Delete the following branches:

```
moma
whitney
```

Copy to Clipboard

You'll need the `-D` option, because these feature branches were never merged into `master`:

```
git branch -D branchname
```

Copy to Clipboard

3.

Kay wants to see a version of the webpage that includes an unordered list with bullet points instead of a paragraph to show information about the party.

Create a new branch called `unordered-list` and switch over to it.

4.

In **index.html**, replace:

```
<p>Description: Join Kay in celebrating their 29th birthday with free food and
beverages, karaoke and a special appearance by Willy Nelson. Also, feel free to
explore the Met museum before or after you stop by! Presents for Kay
optional.</p>
```

Copy to Clipboard

with this unordered list:

```
<ul>
  <li>Join Kay in celebrating their 29th birthday with free food and
beverages</li>
  <li>karaoke and a special appearance by Willy Nelson</li>
  <li>explore the Met museum before or after you stop by!</li>
  <li>Birthday presents optional</li>
</ul>
```

Copy to Clipboard

Click Save.

5.

Add **index.html** to the staging area.

6.

Now make a commit.

7.

Kay approves the changes you made in the `unordered-list` branch.

Switch over to `master`. Then, merge `unordered-list` into `master`. This will be a *fast forward* merge.

8.

Kay wants the heading to be way bigger. Create a new branch called `big-heading`.

9.

Now, switch over the `big-heading` branch.

To make the heading bigger, replace the line below:

```
<h1>Kay's Birthday Party</h1>
```

Copy to Clipboard

With this line:

```
<h1 style="font-size: 72px">Kay's Birthday Party</h1>
```

Copy to Clipboard

Then click Save.

10.

Add `index.html` to the staging area.

11.

Make a commit.

12.

Kay approves of the giant heading!

Switch back over to the `master` branch. Then, merge `big-heading` into `master`.

Ruby Time Calculator

Merge conflicts are challenging even for expert Git users, so it's good to get as much practice as possible with them.

In this project, you'll have to resolve merge conflicts in two markdown files. [Markdown](#) is a file format that converts easily into HTML. You won't have to write any markdown, just identify the differences between lines.

Take a deep breath. You can do this!

If you get stuck during this project or would like to see an experienced developer work through it, click **"Get Unstuck"** to see a **project walkthrough video**.

Tasks

7/7 complete

[Mark the tasks as complete by checking them off](#)

Ruby Time Calculator

1.

You are currently on `master`.

Merge the `edits` branch into the `master` branch.

This will create two merge conflicts: **README.md** and **examples.md**.

Run this command in the terminal:

```
git merge edits
```

Copy to Clipboard

2.

Using the file navigator, open **README.md** and **examples.md**. Identify the merge conflicts.

If you open the files in the code editor, you should now see merge conflicts. For example, here's the content of **README.md**:

```
## Famous speeches by technology advocates and contributors

<<<<<<< HEAD
This repository is a compilation of some of my favorite speeches and quotes
from renowned technology advocates and contributors.

They are contained in the examples.md file.
=====
This repository is a compilation of speeches and quotes from renowned
technology advocates and contributors including [Grace
Hopper] (https://en.wikipedia.org/wiki/Grace_Hopper), [Edsger
Dijkstra] (https://en.wikipedia.org/wiki/Edsger_W._Dijkstra), and [Alan
Turing] (https://en.wikipedia.org/wiki/Alan_Turing).
>>>>>>> edits
```

Copy to Clipboard

3.

In **README.md**, keep the file changes from the `edits` branch. Delete the file changes from `HEAD`.

Don't forget to delete *all* of Git's special markings that indicate a merge conflict.

Click Save.

README.md should now look like this:

```
## Famous speeches by technology advocates and contributors

This repository is a compilation of speeches and quotes from renowned
technology advocates and contributors including [Grace
Hopper] (https://en.wikipedia.org/wiki/Grace_Hopper), [Edsger
Dijkstra] (https://en.wikipedia.org/wiki/Edsger_W._Dijkstra), and [Alan
Turing] (https://en.wikipedia.org/wiki/Alan_Turing).
```

Copy to Clipboard

4.

Add **README.md** to the staging area.

Run this command in the terminal:

```
git add README.md
```

Copy to Clipboard

5.

Follow the same order of actions as above for **examples.md**: keep the `edits` branch file changes and delete the `HEAD` changes.

Click Save.

The **examples.md** file should now look like this:

```
### Grace Hopper - Nanoseconds
```

```
> They started talking about circuits that acted in nanoseconds, billionths of a second. ... We need to define the problem instead of the procedures.
```

```
### Edsger Dijkstra - The Humble Programmer
```

```
> > As a result of a long sequence of coincidences I entered the programming profession ... careful when we give advice to younger people; sometimes they follow it!
```

```
### Alan Turing - Intelligent Machinery (1948)
```

```
> A man provided with paper, pencil, and rubber, and subject ... The new form of the problem can be described in terms of a game which we call the 'imitation game'.
```

Copy to Clipboard

Note: The above is abridged the `...`s represent additional text we omitted in the above.

6.

Add **examples.md** to the staging area and make a commit. Your commit message could be "Resolve merge conflict".

Run this command in the terminal to add **examples.md** to the staging area:

```
git add examples.md
```

Copy to Clipboard

Then run this command in the terminal to make your commit:

```
git commit -m "Resolve merge conflict"
```

Copy to Clipboard

7.

Delete the `edits` branch.

Run this command in the terminal to delete the `edits` branch since we no longer need it:

```
git branch -d edits
```

Copy to Clipboard

Code Review

Get targeted feedback about your code and how well it meets the objective of the project. You can request 1 AI-powered review per project when you complete all task

JavaScript Homework

Let's keep applying the Git collaboration skills we're learning.

In this project, we'll be using Git to write comments on your student's JavaScript homework. Don't worry! You don't need to know JavaScript to do the project.

If the code below piques your interest, check out Codecademy's [JavaScript](#) course here.

If you get stuck during this project or would like to see an experienced developer work through it, click "**Get Unstuck**" to see a **project walkthrough video**.

Tasks

0/9 complete

[Mark the tasks as complete by checking them off](#)

JavaScript Homework

1.

There is a remote called **maryrose-hw** which contains your student's JavaScript homework. Clone the remote, giving it a name of your choice.

2.

Use the terminal command `cd` to go into your cloned repository.

```
cd my_clone
```

Copy to Clipboard

Don't forget to replace `my_clone` with the name you gave your clone in the previous step.

3.

Use the file navigator to open **homework.js** in the code editor.

Be sure to open **homework.js** in your clone, *not* in the remote.

4.

From the terminal, create a new branch with Git. Your branch will be commenting on Mary Rose's JavaScript homework, so name the branch appropriately.

Next, switch over to your new branch.

5.

Time to start commenting on Mary Rose's JavaScript homework.

Comments are used to leave notes for other programmers or yourself. In JavaScript, you can make a single-line comment by typing `//` at the beginning of any line. Then type your comment. For example:

```
// I'm a comment!
```

Copy to Clipboard

There are a number of comments already in **homework.js** to guide you.

Make whatever comments you'd like to Mary Rose's JavaScript homework. Then click Save.

6.

Add your changes to the Git staging area.

7.

Now, make a commit.

8.

Push your branch up to the remote.

9.

Use:

```
cd ../maryrose-hw
```

Copy to Clipboard

To change directories back to the remote. Use your Git knowledge to see the branch you just pushed.



Recipe Book

Let's continue practicing our Git collaboration skills.

In this project, you'll be playing the role of two collaborators working on a recipe book.

If you get stuck during this project or would like to see an experienced developer work through it, click "**Get Unstuck**" to see a **project walkthrough video**.

Tasks

0/10 complete

[Mark the tasks as complete by checking them off](#)

Recipe Book

1.

Go into the **veggie-favorites** remote by using the `cd` command:

```
cd veggie-favorites
```

[Copy to Clipboard](#)

2.

Once inside **veggie-favorites** make a change to **chili.txt** and/or **squash-lasagna.txt** in the code editor.

Click Save, add your changes to the Git staging area, and then make a commit.

3.

Go into your clone with:

```
cd ../veggie-clone
```

[Copy to Clipboard](#)

Check out Codecademy's Command Line course [here](#) to learn more about `cd ..`

4.

Fetch all new work from the remote.

5.

Merge `origin/master` into your local `master` branch .

6.

Create a new branch, then switch over to it to work on **new-recipe.txt**. The recipe can be whatever dish you'd like.

Click Save.

7.

Add your file changes to the staging area and make a commit.

8.

Fetch one more time just for good measure (there won't be a change).

9.

Push your branch up to the remote.

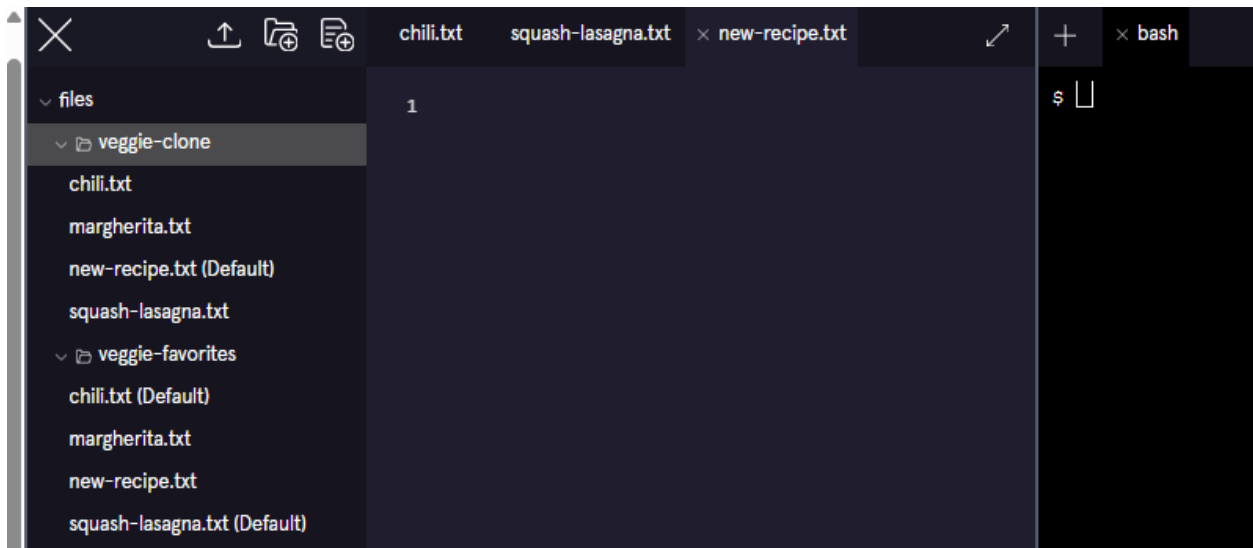
10.

Navigate back to the remote with:

```
cd ../veggie-favorites
```

[Copy to Clipboard](#)

Confirm the presence of your new Git branch there.



Off-Platform Project: Excursion

<https://www.codecademy.com/paths/machine-learning-engineer/tracks/mle-software-engineering-for-machine-learning-ai-engineers/modules/mle-learn-git-ii-git-for-deployment/projects/f1-excursion>

In this project, you'll create a web page which advertises a product called "Excursion." You'll use your growing toolset including HTML, CSS, Command Line Interface, Git, and GitHub. You'll be proud of yourself when it all comes together!

The web page we'll build advertises a mobile app which helps users record and share their experiences, so we'll use video and landscape imagery to set the scene. A landing page is a vital tool in marketing a product these days, and the goal will be to entice potential customers into using the product.

We'll work with Git and GitHub on our local machines, so if you haven't yet, refer to the articles on Command Line Interface Setup and Git Setup.

A preview of the page is available [here](#)!

If you get stuck during this project or would like to see an experienced developer work through it, click "**Get Unstuck**" to see a **project walkthrough video**.

Tasks

0/13 complete

[Mark the tasks as complete by checking them off](#)

Tasks

1.
Download the [design mock](#) and [redline mock](#). Review these, and think about how you might implement the web page with HTML and CSS.
2.
Open your command line interface. Make sure your current working directory matches your expectations. Make a new directory called **excursion**.
3.
Make a new file named **index.html**. Make a new directory named **resources**. Inside **resources**, make another directory called **css**. Inside **css**, create a single file for your page's custom CSS, **style.css**.
4.
Make sure your current working directory is **excursion** and then initialize a Git repository.
5.
Add the HTML and CSS files to Git's staging area. Make your first commit with a simple message.
6.
Let's set up a remote GitHub repository. Go to GitHub's homepage to make a new repository. Call it **excursion**. Follow the instructions to add an existing Git repository, since you have already initialized yours on your own computer.
After you pushed to your remote repository, refresh your repository page on GitHub. You should see your new files!
7.
Open **index.html** and add the necessary boilerplate code, including the link to your CSS file. Then make the top-most header and link as seen on the design mock.
8.
Add the updated **index.html** file to Git's staging area and commit again. An appropriate message might be "Add boilerplate HTML and begin page content".
9.
Continue replicating the mock by adding the video component, supporting text, and images. Here are the resources you'll need:
[MP4 stock video](#)

[Landscape image](#)

[Cellphone icon](#)

We're leaving you to complete this web page on your own, we trust you can do it. Here are some notes to help make it the best you can:

The white "Cellphone icon" image linked above will appear blank against the white background. We promise it's there!

Make new directories inside the **resources** directory called **images** and **videos** to store these items.

Since your images, videos, and stylesheets are all in the **resources** directory, you'll need to include that in the path when you link to them. For example, the path to your stylesheet will be `resources/css/style.css`.

By default, images will appear on the same line as the element before them. In the mock, all our images are centered on their own line. To effect this, make a new paragraph element and put the image element inside of it.

10.

When you have added a feature, or made any other concise change to the HTML or CSS, commit again. Add a commit message which describes the changes you've made. Continue to commit at opportune moments.

11.

Push your local changes to your repository on GitHub. Reload your GitHub repository page in your browser to see that your new commits saved to the cloud. If you completed all of the GitHub Pages setup steps, this project should be automatically deployed. Navigate to `http://<username>.github.io/<repository-name>` to view the deployed version of your project.

12.

To complete the page, add the footer and any remaining CSS. Review the redline mock and confirm your progress.

13.

When you are satisfied with your work, make sure to commit your changes once again. Your commit message should reflect that you feel this is complete! Finally, push your changes to GitHub.

Now take a step back, reflect, and be proud; you've accomplished a lot!



Machine Learning/AI Engineer

Random Forests Project

In this project, we will be using a dataset containing census information from UCI's Machine Learning Repository. By using this census data with a random forest, we will try to predict whether or not a person makes more than \$50,000. Let's get started!

Datasets

The original data set is available at the UCI Machine Learning Repository:

<https://archive.ics.uci.edu/ml/datasets/census+income>

The dataset has been loaded for you in `script.py` and saved as a dataframe named `df`. Some of the input and output features of interest are:

age: continuous
workclass: Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, Local-gov, State-gov, Without-pay, Never-worked.
education: Bachelors, Some-college, 11th, HS-grad, Prof-school, Assoc-acdm, Assoc-voc, 9th, 7th-8th, 12th, Masters, 1st-4th, 10th, Doctorate, 5th-6th, Preschool
race: White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black
sex: Female, Male
capital-gain: continuous
capital-loss: continuous
hours-per-week: continuous
native country: discrete
income: discrete, >50K, <=50K

Tasks

17/17 complete

[Mark the tasks as complete by checking them off](#)

Investigate the data

1. We will build a random forest classifier to predict the income category. First, take a look at the distribution of income values – what percentage of samples have incomes less than 50k and greater than 50k?
2. There's a small problem with our data that is a little hard to catch – every string has an extra space at the start. For example, the first row's native-country is " United-States", but we want it to be "United-States". One way to fix this is to select all columns of type `object` and use the string method `.str.strip()`.
3. Create a features dataframe `x`. This should include only features in the list `feature_cols` and convert categorical features to dummy variables using `pd.get_dummies()`. Include the paramter `drop_first=True` to eliminate redundant features.
4. Create the output variable `y`, which is binary. It should be 0 when income is less than 50k and 1 when it is greater than 50k.
5. Split the data into a train and test set with a test size of 20%.

Build and Tune Random Forest Classifiers by Depth

6. Instantiate an instance of a `RandomForestClassifier()` (with default parameters). Fit the model on the train data and print the score (accuracy) on the test data. This will act as a baseline to compare other model performances.
7. We will explore tuning the random forest classifier model by testing the performance over a range of `max_depth` values. Fit a random forest classifier for `max_depth` values from 1-25. Save the accuracy score for the train and test sets in the lists `accuracy_train`, `accuracy_test`.
8. Find the largest accuracy and the depth this occurs on the test data.
9. Plot the training and test accuracy of the models versus the `max_depth`.
10. Refit the random forest model using the `max_depth` from above; save the feature importances in a dataframe. Sort the results and print the top five features.

Create Additional Features and Re-Tune

11.

Looking at the education feature, there are 16 unique values – from preschool to professional school. Rather than adding dummy variables for each value, it makes sense to bin some of these values together. While there are many ways to do this, we will take the approach of combining the values into 3 groups: `High school and less`, `College to Bachelors` and `Masters and more`. Create a new column in `df` for this new features called `education_bin`.

12.

Like we did previously, we will now add this new feature into our feature list and recreate `X`.

13.

As we did before, we will tune the random forest classifier model by testing the performance over a range of `max_depth` values. Fit a random forest classifier for `max_depth` values from 1-25. Save the accuracy score for the train and test sets in the lists `accuracy_train`, `accuracy_test`.

14.

Find the largest accuracy and the depth this occurs on the test data. Compare the results from the previous model tuned.

15.

Plot the training and test accuracy of the models versus the `max_depth`. Compare the results from the previous model tuned.

16.

Refit the random forest model using the `max_depth` from above; save the feature importances in a dataframe. Sort the results and print the top five features. Compare the results from the previous model tuned.

17.

Nice work! Note that the accuracy of our final model increased and one of our added features is now in the top 5 based on importance!

There are a few different ways to extend this project:

- Are there other features that may lead to an even better performance? Consider creating new ones or adding additional features not part of the original feature list.

- Consider tuning hyperparameters based on a different evaluation metric – our classes are fairly imbalanced, AUC or F1 may lead to a different result

- Tune more parameters of the model. You can find a description of all the parameters you can tune in the [Random Forest Classifier documentation](#). For example, see what happens if you tune `max_features` or `n_estimators`.

SOLUTIONS:

```
import pandas as pd
import numpy as np
import codecademylib3
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, BaggingClassifier, RandomForestRegressor
from sklearn import tree
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

col_names = ['age', 'workclass', 'fnlwgt', 'education', 'education-num',
             'marital-status', 'occupation', 'relationship', 'race', 'sex',
             'capital-gain', 'capital-loss', 'hours-per-week', 'native-country', 'income']
df = pd.read_csv('adult.data', header=None, names = col_names)

#Distribution of income
print(df.income.value_counts(normalize=True))

#Clean columns by stripping extra whitespace for columns of type "object"
for c in df.select_dtypes(include=['object']).columns:
    df[c] = df[c].str.strip()

feature_cols = ['age',
                'capital-gain', 'capital-loss', 'hours-per-week', 'sex', 'race']
#Create feature dataframe X with feature columns and dummy variables for categorical features
X = pd.get_dummies(df[feature_cols], drop_first=True)
#Create output variable y which is binary, 0 when income is less than 50k, 1 when it is greater than 50k
```

```

y = np.where(df.income=='<=50K', 0, 1)

#Split data into a train and test set
x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=1, test_size=.2)

#Instantiate random forest classifier, fit and score with default parameters
rf = RandomForestClassifier()
rf.fit(x_train, y_train)
rf.score(x_test, y_test)
print(f'Accuracy score for default random forest: {round(rf.score(x_test, y_test)*100,3)}%')

#Tune the hyperparameter max_depth over a range from 1-25, save scores for test and train set
np.random.seed(0)
accuracy_train=[]
accuracy_test = []
depths = range(1,26)
for i in depths:
    rf = RandomForestClassifier(max_depth=i)
    rf.fit(x_train, y_train)
    y_pred = rf.predict(x_test)
    accuracy_test.append(accuracy_score(y_test, rf.predict(x_test)))
    accuracy_train.append(accuracy_score(y_train, rf.predict(x_train)))

#Find the best accuracy and at what depth that occurs
best_acc= np.max(accuracy_test)
best_depth = depths[np.argmax(accuracy_test)]
print(f'The highest accuracy on the test is achieved when depth: {best_depth}')
print(f'The highest accuracy on the test set is: {round(best_acc*100,3)}%')

#Plot the accuracy scores for the test and train set over the range of depth values
plt.plot(depths, accuracy_test,'bo--',depths, accuracy_train,'r*:')
plt.legend(['test accuracy', 'train accuracy'])
plt.xlabel('max depth')
plt.ylabel('accuracy')
plt.show()

#Save the best random forest model and save the feature importances in a dataframe
best_rf = RandomForestClassifier(max_depth=best_depth)
best_rf.fit(x_train, y_train)
feature_imp_df = pd.DataFrame(zip(x_train.columns, best_rf.feature_importances_), columns=['feature',
'importance'])
print('Top 5 random forest features:')
print(feature_imp_df.sort_values('importance', ascending=False).iloc[0:5])

#Create two new features, based on education and native country
df['education_bin'] = pd.cut(df['education-num'], [0,9,13,16], labels=['HS or less', 'College to
Bachelors', 'Masters or more'])

feature_cols = ['age',
                'capital-gain', 'capital-loss', 'hours-per-week', 'sex', 'race','education_bin']
#Use these two new additional features and recreate X and test/train split
X = pd.get_dummies(df[feature_cols], drop_first=True)

x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=1, test_size=.2)

#Find the best max depth now with the additional two features
np.random.seed(0)
accuracy_train=[]
accuracy_test = []
depths = range(1,10)
for i in depths:
    rf = RandomForestClassifier(max_depth=i)
    rf.fit(x_train, y_train)
    y_pred = rf.predict(x_test)
    accuracy_test.append(accuracy_score(y_test, rf.predict(x_test)))
    accuracy_train.append(accuracy_score(y_train, rf.predict(x_train)))

```

```

best_acc= np.max(accuracy_test)
best_depth = depths[np.argmax(accuracy_test)]
print(f'The highest accuracy on the test is achieved when depth: {best_depth}')
print(f'The highest accuracy on the test set is: {round(best_acc*100,3)}%')

plt.figure(2)
plt.plot(depths, accuracy_test, 'bo--', depths, accuracy_train, 'r*:')
plt.legend(['test accuracy', 'train accuracy'])
plt.xlabel('max depth')
plt.ylabel('accuracy')
plt.show()

#Save the best model and print the two features with the new feature set
best_rf = RandomForestClassifier(max_depth=best_depth)
best_rf.fit(x_train, y_train)
feature_imp_df = pd.DataFrame(zip(x_train.columns, best_rf.feature_importances_), columns=['feature',
'importance'])
print('Top 5 random forest features:')
print(feature_imp_df.sort_values('importance', ascending=False).iloc[0:5])

```

Boosting

In this project, we will be using a dataset containing census information from UCI's Machine Learning Repository. By using this census data with boosting algorithms, we will try to predict whether or not a person makes more than \$50,000.

Let's get started!

Datasets

The original data set is available at the UCI Machine Learning Repository:

<https://archive.ics.uci.edu/ml/datasets/census+income>

Tasks

0/10 complete

[Mark the tasks as complete by checking them off](#)

Explore and prepare the data

1. Take a look at the distribution of the target column, `income`. What percentage of samples have incomes greater than 50k and less than or equal to 50k?

Use `df[target_column].value_counts(normalize=True)` or `df['income'].value_counts(normalize=True)`.

2. We have identified a set of features to explore. The features are stored in a variable `raw_feature_cols`. Take a look at the datatypes of these columns. Are they what you expected based on the data dictionary provided in the description?

Use `df[raw_feature_cols].dtypes`.

Preparing the features

3. Create a features dataframe `x` with the features listed in the stored variable `raw_feature_cols`. Since the columns `workclass`, `sex`, and `race` are all low cardinality categorical variables, we will convert them to dummy variables using `pd.get_dummies()`. Set the parameter `drop_first = True` in `pd.get_dummies()`. This drops the first categorical instance in each of the categorical variable columns because it is redundant. Make sure you understand why we do not have to worry about dropping the redundant variable before moving on.

Note: `pd.get_dummies()` is clever enough that it will only create dummy variables for the categorical columns. It will not create dummy variables for the `int64` columns.

Take a look at the first 5 rows of the features dataframe by using the `.head(n=5)` method.

Use `x = pd.get_dummies(df[raw_feature_cols], drop_first=True)` to prepare the features and `x.head(n=5)` to take a look at the first 5 rows.

4. Convert the target variable to a binary value and store it in a variable `y`. Set it to 0 when `income <= 50K` and 1 when `income > 50K`.

Use `train_test_split` from `sklearn` with `test_size = 0.3`.

Build and Train the AdaBoost and Gradient Boosted Trees Classifiers

5. Perform a train-test split. Create the base estimator for the AdaBoost classifier in the form a decision stump using `DecisionTreeClassifier()` and store it in a variable named `decision_stump`. The decision stump should be an instance of `DecisionTreeClassifier()` with parameter `max_depth=1`.
- 6.

Create an instance of `AdaBoostClassifier()` and store it in a variable `ada_classifier`. Keep most of the parameters set as their default value, except the `base_estimator` parameter which should be set to `decision_stump`.

Use `AdaBoostClassifier(base_estimator=decision_stump)` to instantiate the estimator.

7.

Create an instance of `GradientBoostingClassifier()` and store it in a variable `grad_classifier`. Keep all the parameters set as their default value.

Use `GradientBoostingClassifier` to instantiate the estimator.

8.

Fit each of the instantiated models on the training data. Calculate and store the predictions on the test data in separate variables `y_pred_ada` and `y_pred_grad`. Print the `accuracy` and `f1` score for the predictions from each model.

To train each of the classifiers use the method `.fit(X_train, y_train)`. Use `GradientBoostingClassifier` to instantiate the estimator.

To get the predictions for the AdaBoost model use `y_pred_ada = ada_classifier.predict(X_test)`.

To get the accuracy score for the AdaBoost model use `accuracy_score(y_test, y_pred_ada)`

Explore Hyperparameters

9.

For AdaBoost the default `n_estimators` is 50 and for Gradient Boosting it is 100. We've created a new list `n_estimators_list = [10, 30, 50, 70, 90]` to search from and determine the value of this parameter that gives us the most performant model. Use `GridSearchCV` with AdaBoost to fit the data and search this parameter space.

Remember that `GridSearchCV` takes in the argument, `param_grid`, a dictionary of parameters to search through. Something like `estimator_parameters = {'n_estimators': n_estimators_list}` would work here!

10.

Calculate the `mean_test_score` for each of these fits and store it as a list, `ada_scores_list`. Plot it against `n_estimators_list` to pick the best value for `n_estimators`.

The attribute `.cv_results_` contains the results of the cross validation.

SOLUTIONS:

```
import pandas as pd
import numpy as np
import codecademylib3

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.ensemble import GradientBoostingClassifier

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

path_to_data = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data"

col_names = [
    'age', 'workclass', 'fnlwgt', 'education', 'education-num', 'marital-status',
    'occupation', 'relationship', 'race', 'sex', 'capital-gain', 'capital-loss',
    'hours-per-week', 'native-country', 'income'
]

df = pd.read_csv(path_to_data, header=None, names = col_names)
print(df.head())

#Clean columns by stripping extra whitespace for columns of type "object"
for c in df.select_dtypes(include=['object']).columns:
    df[c] = df[c].str.strip()
```

```

target_column = "income"
raw_feature_cols = [
    'age',
    'education-num',
    'workclass',
    'hours-per-week',
    'sex',
    'race'
]

##1. Percentage of samples with income < and > 50k
print(df[target_column].value_counts(normalize=True))

##2. Data types of features
print(df[raw_feature_cols].dtypes)

##3. Preparing the features

X = pd.get_dummies(df[raw_feature_cols], drop_first=True)
X.head(n=5)

##4. Convert target variable to binary
y = np.where(df[target_column] == '<=50K', 0, 1)

##5a. Create train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=123)
##5b. Create base estimator and store it as decision_stump
decision_stump = DecisionTreeClassifier(max_depth=1)

##6. Create AdaBoost Classifier
ada_classifier = AdaBoostClassifier(base_estimator=decision_stump)

##7. Create GradientBoost Classifier
grad_classifier = GradientBoostingClassifier()

##8a. Fit models and get predictions
ada_classifier.fit(X_train, y_train)
y_pred_ada = ada_classifier.predict(X_test)

grad_classifier.fit(X_train, y_train)
y_pred_grad = grad_classifier.predict(X_test)

##8b. Print accuracy and F1
print(f"AdaBoost accuracy: {accuracy_score(y_test, y_pred_ada)}")
print(f"AdaBoost f1-score: {f1_score(y_test, y_pred_ada)}")

print(f"Gradient Boost accuracy: {accuracy_score(y_test, y_pred_grad)}")
print(f"Gradient Boost f1-score: {f1_score(y_test, y_pred_grad)}")

##9. Hyperparameter Tuning
n_estimators_list = [10, 30, 50, 70, 90]

from sklearn.model_selection import GridSearchCV
estimator_parameters = {'n_estimators': n_estimators_list}
ada_gridsearch = GridSearchCV(ada_classifier, estimator_parameters, cv=5, scoring='accuracy',
verbose=True)
ada_gridsearch.fit(X_train, y_train)

##10. Plot mean test scores
ada_scores_list = ada_gridsearch.cv_results_['mean_test_score']
plt.scatter(n_estimators_list, ada_scores_list)
plt.show()
plt.clf()

```


Building ML Pipelines

In this project, we will be using a dataset containing bone marrow transplantation characteristics for pediatric patients from UCI's Machine Learning Repository.

We will use this dataset to build a pipeline, containing all preprocessing and data cleaning steps, and then selecting the best classifier to predict patient survival.

Let's get started!

Tasks

0/16 complete

[Mark the tasks as complete by checking them off](#)

Investigate the data

1.

The dataset has been loaded for you in `script.py` and saved as a dataframe named `df`. Some of the input and output features of interest are:

- `donor_age` - Age of the donor at the time of hematopoietic stem cells apheresis
- `donor_age_below_35` - Is donor age less than 35 (yes, no)
- `donor_ABO` - ABO blood group of the donor of hematopoietic stem cells (O, A, B, AB)
- `donor_CMV` - Presence of cytomegalovirus infection in the donor of hematopoietic stem cells prior to transplantation (present, absent)
- `recipient_age` - Age of the recipient of hematopoietic stem cells at the time of transplantation
- `recipient_age_below_10` - Is recipient age below 10 (yes, no)
- `recipient_age_int` - Age of the recipient discretized to intervals (0,5], (5, 10], (10, 20]
- `recipient_gender` - Gender of the recipient (female, male)
- `recipient_body_mass` - Body mass of the recipient of hematopoietic stem cells at the time of the transplantation
- ...
- `survival_status` - Survival status (0 - alive, 1 - dead)

We will build a classification pipeline to predict the survival status. As part of the pipeline, we will process the numerical and categorical columns separately. First, we will define lists for each of these columns. However, one thing to note that the data imported is ALL numeric – binary columns, like `donor_age_below_35`, are encoded as numerical values (0 and 1). Similarly, columns like `donor_ABO`, with four categories, are encoded as -1,0,1, and 2. So we cannot just look for columns of numeric types or object types to define these lists.

First, calculate the number of unique values for each column – how might we use this to determine categorical variables?

Use `df.nunique()` to count the number of unique values in each column.

2.

Save the target, `survival_status`, as `y` and the remaining columns (minus `survival_time`) as `X`.

Use `X = df.drop(columns=['survival_time', 'survival_status'])` and `y=df.survival_status`.

3.

After exploring the unique value counts in each column, we will use 7 as a threshold – for columns with more than 7 unique values, we will consider this a numeric variable. For columns with 7 or less unique values, we will consider this a categorical variable. Define `num_cols` as a list of columns in `X` with more than 7 unique values and `cat_cols` as a list of columns in `X` with 7 or fewer unique values.

Subset the columns of `X` with `X.columns[X.nunique()>7]` and define each list accordingly.

4.

Check to see what, if any, columns in `X` have missing values and print them.

Use `X.columns[X.isnull().sum()>0]`.

5.

Split the data into a train and test set with a test size of 20%.

Use `train_test_split(X,y, random_state=1, test_size=.2)`.

Create Preprocessing Pipeline

6.

Create a categorical preprocessing pipeline called `cat_vals` that contains two steps – the first will fill in missing values using the mode and the second will one-hot-encode the variables. Make sure to use parameters `sparse=False`, `drop='first'`, `handle_unknown = 'ignore'` for the `OneHotEncoder`.

Define `cat_vals = Pipeline([("imputer", SimpleImputer(strategy='most_frequent')), ("one", OneHotEncoder(sparse=False, drop='first', handle_unknown = 'ignore'))])`.

7.

Create a numerical preprocessing pipeline called `num_vals` that contains two steps – the first will fill in missing values using the mean and the second will scale features using `StandardScaler()`.

Define `num_vals = Pipeline([("imputer", SimpleImputer(strategy='mean')), ("scale", StandardScaler())])`.

8.

Create a column transformer named `preprocess` that will preprocess the numerical and categorical features separately. This will contain the previous two pipelines, `cat_vals` and `num_vals`, as transformers on their respective columns, `cat_cols` and `num_cols`.

Define `preprocess = ColumnTransformer( transformers=[ ("cat_process", cat_vals, cat_cols), ("num_process", num_vals, num_cols) ] )`.

Create Classification Pipeline and Tune

9.

Now that we have finished all the preprocessing, let's create a classification pipeline called `pipeline` – it will contain three steps. The first is the `preprocess` created above. The second is `PCA()` – this will select a number of principal components from the processed data features. The third and last step will be the classifier, a logistic regression classifier.

Define `pipeline as Pipeline([("preprocess", preprocess), ("pca", PCA()), ("clf", LogisticRegression())])`.

10.

Fit `pipeline` on the training data and predict the accuracy score on the test data.

Use `pipeline.fit(x_train, y_train)` and `pipeline.score(x_test, y_test)`.

11.

Update the search space of hyperparameters (in the dictionary `search_space`) to contain the number of PCA dimensions for each classifier. Use `np.linspace(30, 37, 3).astype(int)` for the values of the dimensions.

As the PCA step is named `pca`, update the parameters using `pca__n_components': np.linspace(5, 35, 7).astype(int)`.

12.

Search over the hyperparameters in `search_space` for the pipeline using `GridSearchCV`. Fit on the training set.

Define the grid search with `GridSearchCV(pipeline, search_space)` and fit using `.fit(x_train, y_train)`.

13.

Save the best estimator from the grid search as `best_model`.

Access the best estimator from the grid search with `.best_estimator_`.

14.

Print attributes of `best_model` – the type of classifier it is, the hyperparameters of the classifier, and the number of components selected from PCA.

To access named steps of a pipeline, use `best_model.named_steps['clf']` and `best_model.named_steps['pca']`.

15.

Evaluate the best estimator on the test set and print the final accuracy score. How does this compare to the initial model?

Evaluate `best_model.score(x_test, y_test)`.

16.

Nice work! Note that the accuracy of our final model increased from 79% to 89% – and we used pipelines to write all the preprocessing, feature selection, and modeling step in our code!

There are a few different ways to extend this project:

- Are there other classification models that may lead to an even better performance? Consider adding additional types to the grid search and their respective hyperparameters.

- Tune more parameters of the model. You can find a description of all the parameters you can tune in the scikit-learn documentation – for example [Random Forest Classifier documentation](#). See what happens if you tune `max_features` or `n_estimators`.

- Consider other types of feature selection/creation steps other than PCA – substitute these in place of PCA in the final pipeline and see how this impacts the model performance.

- And many others – pipeline help you tune parameters across all steps in your model and change steps easily – options are unlimited!

SOLUTIONS:

```
import numpy as np
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
from sklearn.preprocessing import LabelEncoder, StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
```

```

from sklearn.decomposition import PCA
from sklearn.metrics import confusion_matrix
from scipy.io import arff

data = arff.loadarff('bone-marrow.arff')
df = pd.DataFrame(data[0])
df.drop(columns=['Disease'], inplace=True)

#Convert all columns to numeric, coerce errors to null values
for c in df.columns:
    df[c] = pd.to_numeric(df[c], errors='coerce')

#Make sure binary columns are encoded as 0 and 1
for c in df.columns[df.nunique()==2]:
    df[c] = (df[c]==1)*1.0

# 1. Calculate the number of unique values for each column
print('Count of unique values in each column:')
print(df.nunique())

# 2. Set target, survival_status, as y; features (dropping survival status and time) as X
y = df.survival_status
X = df.drop(columns=['survival_time', 'survival_status'])

# 3. Define lists of numeric and categorical columns based on number of unique values
num_cols = X.columns[X.nunique()>7]
cat_cols = X.columns[X.nunique()<=7]

# 4. Print columns with missing values
print('Columns with missing values:')
print(X.columns[X.isnull().sum()>0])

# 5. Split data into train/test split
x_train, x_test, y_train, y_test = train_test_split(X, y, random_state=0, test_size=0.2)

# 6. Create categorical preprocessing pipeline
# Using mode to fill in missing values and OHE
cat_vals = Pipeline([("imputer", SimpleImputer(strategy='most frequent')),
("ohe", OneHotEncoder(sparse=False, drop='first', handle_unknown = 'ignore'))])

# 7. Create numerical preprocessing pipeline
# Using mean to fill in missing values and standard scaling of features
num_vals = Pipeline([("imputer", SimpleImputer(strategy='mean')), ("scale", StandardScaler())])

# 8. Create column transformer that will preprocess the numerical and categorical features separately
preprocess = ColumnTransformer(
    transformers=[
        ("cat_process", cat_vals, cat_cols),
        ("num_process", num_vals, num_cols)
    ]
)

# 9. Create a pipeline with preprocess, PCA, and a logistic regression model
pipeline = Pipeline([("preprocess", preprocess),
("pca", PCA()),
("clf", LogisticRegression())])

# 10. Fit the pipeline on the training data
pipeline.fit(x_train, y_train)
#Predict the pipeline on the test data
print('Pipeline Accuracy Test Set:')
print(pipeline.score(x_test, y_test))

# 11. Define search space of hyperparameters
search_space = [{'clf': [LogisticRegression()],
                    'clf__C': np.logspace(-4, 2, 10),
                    'pca__n_components': np.linspace(30, 37, 3).astype(int)},
                ]

```

```
#12. Search over hyperparameters abolve to optimize pipeline and fit
gs = GridSearchCV(pipeline, search_space, cv=5)
gs.fit(x_train, y_train)

# 13. Save the best estimator from the gridsearch and print attributes and final accuracy on test set
best_model = gs.best_estimator_

# 14. Print attributes of best_model
print('The best classification model is:')
print(best_model.named_steps['clf'])
print('The hyperparameters of the best classification model are:')
print(best_model.named_steps['clf'].get_params())
print('The number of components selected in the PCA step are:')
print(best_model.named_steps['pca'].n_components)

# 15. Print final accuracy score
print('Best Model Accuracy Test Set:')
print(best_model.score(x_test, y_test))
```

NLP

Data Scientist: Natural Language Processing Specialist

Discover Insights into Classic Texts

Novels and text contain insights into ideologies and places that are often originally unknown to the reader. By reading a written piece, you uncover the opinions of the author on their chosen topic and come to understand both the topic and how the author thinks.

In this project you will perform a natural language parsing analysis to gain deeper insight into one of two famous and often discussed novels in the public domain: [Oscar Wilde's *The Picture of Dorian Gray*](#) or [Homer's *The Iliad*](#)! Fear not if you haven't heard or read the novels, one of the beauties of natural language parsing with regular expressions is the ability to gain insight into lengthy pieces of text without a formal read!

By the end of this project, you will find out the main topics of discussion in the novel of your choosing and can begin to discern some of the author's thoughts and beliefs!

Setup Instructions

You have two options of completing this assignment. Either here, within Codecademy's output terminal, or on your own, in case you're more comfortable using a Jupyter notebook. If you choose to do this project on your computer instead of Codecademy, you can download what you'll need by clicking the "Download" button below. If you need help setting up your computer, be sure to check out our setup guides:

[Command Line Interface Setup](#)
[Introducing Jupyter Notebook](#)
[Setting up Jupyter Notebook](#)
[Getting Started with Jupyter](#)
[Getting More out of Jupyter Notebook](#)

Open **Discover Insights into Classic Texts.ipynb** and follow the steps in the Jupyter Notebook. If you get stuck, you can look at **Discover Insights into Classic Texts_Solution.ipynb** for the answer.

Download

Tasks

0/17 complete

Mark the tasks as complete by checking them off

Import and Preprocess Text Data

1.

Given to you in the code editor are text files for the *The Picture of Dorian Gray*, named `dorian_gray.txt`, and *The Iliad*, named `the_iliad.txt`, sourced from [Project Gutenberg](#). Import the text of your choosing, convert it to lowercase, and name it `text` using the following line of code.

```
text = open("_____.txt", encoding='utf-8').read().lower()
```

Copy to Clipboard

Replace the blank with the name of the text file for the novel you choose to analyze!

You can import the text of your choosing using Python's `open()` function and lowercase the text with `.lower()`.

If you choose to analyze *The Picture of Dorian Gray*:

```
text = open("dorian_gray.txt", encoding='utf-8').read().lower()
```

Copy to Clipboard

If you choose to analyze *The Iliad*:

```
text = open("the_iliad.txt", encoding='utf-8').read().lower()
```

Copy to Clipboard

2.

With the text imported, now you need to split the text into individual sentences and then individual words. This allows you to perform a sentence-by-sentence parsing analysis!

Provided to you in the code editor is a customized function `word_sentence_tokenize()` that will sentence tokenize a text and then word tokenize each sentence, returning a list of word tokenized sentences. Call the function with `text` as an argument and save the result to a variable named `word_tokenized_text`.

Call `word_sentence_tokenize()` as follows:

```
word_tokenized_text = word_sentence_tokenize(_____)
```

Copy to Clipboard

Replace the blank with the text you are splitting into sentences and words.

3.

Save any word tokenized sentence in `word_tokenized_text` to a variable named `single_word_tokenized_sentence`. Print `single_word_tokenized_sentence` as a check to visualize what you have done so far!

You can save and print the word tokenized sentence at index 100 in `word_tokenized_text` as follows:

```
single_word_tokenized_sentence = word_tokenized_text[100]
print(single_word_tokenized_sentence)
```

Copy to Clipboard

Part-of-speech Tag Text

4.

Next you can part-of-speech tag each sentence to allow for syntax parsing! Begin by creating a list named `pos_tagged_text` that will hold each part-of-speech tagged sentence from the novel.

You can create an empty list in Python with either of the below syntax:

```
new_list = list()
```

Copy to Clipboard

OR

```
new_list = []
```

Copy to Clipboard

5.

Loop through each word tokenized sentence in `word_tokenized_text` and part-of-speech tag each sentence using `nltk's pos_tag()` function. Append the result to `pos_tagged_text`.

You can loop through each word tokenized sentence as shown below:

```
for word_tokenized_sentence in word_tokenized_text:
```

Copy to Clipboard

Then part-of-speech tag each word tokenized text with `nltk's pos_tag()` function:

```
pos_tagged_text.append(pos_tag(word_tokenized_sentence))
```

Copy to Clipboard

6.

Save any part-of-speech tagged sentence in `pos_tagged_text` to a variable named `single_pos_sentence`. Print `single_pos_sentence` as a check to visualize what you have done so far!

You can save and print the part-of-speech tagged sentence at index 100 in `pos_tagged_text` as follows:

```
single_pos_sentence = pos_tagged_text[100]
print(single_pos_sentence)
```

Copy to Clipboard

Chunk Sentences

7.

Now that you have part-of-speech tagged your text, you can move on to syntax parsing!

Begin by defining a piece of chunk grammar `np_chunk_grammar` that will chunk a noun phrase. Remember, a noun phrase consists of an optional determiner `DT`, followed by any number of adjectives `JJ`, followed by a noun `NN`.

A noun phrase can be expressed in chunk grammar as shown below:

```
np_chunk_grammar = "NP: {<DT>?<JJ>*<NN>}"
```

Copy to Clipboard

8.

Create a `nltk.RegexpParser` object named `np_chunk_parser` using the noun phrase chunk grammar you defined as an argument.

You can create a `RegexpParser` object with the following syntax:

```
np_chunk_parser = RegexpParser(_____)
```

Copy to Clipboard

Fill the blank with the noun phrase chunk grammar you defined earlier!

9.

Define a piece of chunk grammar named `vp_chunk_grammar` that will chunk a verb phrase of the following form: noun phrase, followed by a verb `VB`, followed by an optional adverb `RB`.

A verb phrase can be expressed in chunk grammar as shown below:

```
vp_chunk_grammar = "VP: {<DT>?<JJ>*<NN><VB.*><RB.??>}"
```

Copy to Clipboard

10.

Create a `nltk.RegexpParser` object named `vp_chunk_parser` using the verb phrase chunk grammar you defined as an argument.

You can create a `RegexpParser` object with the following syntax:

```
chunk_parser = RegexpParser(_____)
```

Copy to Clipboard

Fill the blank with the verb phrase chunk grammar you defined earlier!

11.

Create two empty lists `np_chunked_text` and `vp_chunked_text` that will hold the chunked sentences from your text.

You can create an empty list in Python with either of the below syntax:

```
new_list = list()
```

Copy to Clipboard

OR

```
new_list = []
```

Copy to Clipboard

12.

Loop through each part-of-speech tagged sentence in `pos_tagged_text` and noun phrase chunk each sentence using your `RegexpParser`'s `.parse()` method. Append the result to `np_chunked_text`.

You can loop through each part-of-speech tagged sentence as shown below:

```
for pos_tagged_sentence in pos_tagged_text:
```

Copy to Clipboard

Then within the for-loop, noun phrase chunk each sentence with `np_chunk_parser`'s `.parse()` method and append to `np_chunked_text`:

```
np_chunked_text.append(np_chunk_parser.parse(pos_tagged_sentence))
```

Copy to Clipboard

13.

Within the same loop you defined in the previous task, verb phrase chunk each part-of-speech tagged sentence using your `RegexpParser`'s `.parse()` method. Append the result to `vp_chunked_text`.

Verb phrase chunk each part-of-speech tagged sentence with `vp_chunk_parser`'s `.parse()` method and append to `vp_chunked_text`:

```
vp_chunked_text.append(vp_chunk_parser.parse(pos_tagged_sentence))
```

Copy to Clipboard

Make sure this line of code is within the same loop you defined in the last task!

Analyze Chunks

14.

Now that you have chunked your novel, you can analyze the chunk frequencies to gain insights!

A function `np_chunk_counter()` that returns the 30 most common NP-chunks from a list of chunked sentences has been imported to the workspace for you. Call `np_chunk_counter()` with `np_chunked_text` as an argument and save the result to a variable named `most_common_np_chunks`. Print `most_common_np_chunks`. What sticks out to you about the most common noun phrase chunks? Are you surprised by anything? Open the hint to see our analysis.

Want to see how `np_chunk_counter()` works? Use the file navigator to open `chunk_counters.py` and inspect `np_chunk_counter()`.

You can call `np_chunk_counter()` to count the most common noun phrase chunks as follows:

```
most_common_np_chunks = np_chunk_counter(np_chunked_text)
```

Copy to Clipboard

Print `most_common_np_chunks` to view the 30 most common chunks in descending order, given in the form `(NP-Chunk, Count)`.

Analysis for *The Picture of Dorian Gray*

Looking at `most_common_np_chunks`, you can identify characters of importance in the text such as `henry`, `harry`, `dorian gray`, and `basil`, based on their frequency. Additionally another noun phrase `the picture` appears to be very relevant.

Analysis for *The Iliad*

Looking at `most_common_np_chunks`, you can identify characters of importance in the text such as `hector` and `jove` based on their frequency. Additionally a location of importance, `troy`, is mentioned often. A theme of `war` can also be implied by its high frequency count.

15.

A function `vp_chunk_counter()` that returns the 30 most common VP-chunks from a list of chunked sentences has been imported to the workspace for you. Call `vp_chunk_counter()` with `vp_chunked_text` as an argument and save the result to a variable named `most_common_vp_chunks`. Print `most_common_vp_chunks`. What sticks out to you about the most common verb phrase chunks? Are you surprised by anything? Open the hint to see our analysis.

Want to see how `vp_chunk_counter()` works? Use the file navigator to open `chunk_counters.py` and inspect `vp_chunk_counter()`.

You can call `vp_chunk_counter()` to count the most common verb phrase chunks as follows:

```
most_common_vp_chunks = vp_chunk_counter(vp_chunked_text)
```

Copy to Clipboard

Print `most_common_vp_chunks` to view the 30 most common chunks in descending order, given in the form `(VP-Chunk, Count)`.

Analysis for *The Picture of Dorian Gray*

Looking at `most_common_vp_chunks`, some interesting findings appear. The verb phrases `i want`, `i know` and `i have` occur frequently, indicating a theme of desire and need.

Analysis for *The Iliad*

Looking at `most_common_vp_chunks`, you can see that verb phrases of the form you defined in your chunk grammar do not appear as often in *The Iliad* as noun phrases. This can indicate a different style of writing taken by the author that does not follow traditional grammatical style (i.e. poetry). Even when chunks are not found, their absence can give you insight!

Go Further On Your Own!

16.

Amazing! You have performed a syntax parsing analysis on a novel and gained insight into both the meaning of the text and how the author thinks, without reading a page!

Now's your chance to get creative. Is there a different pattern of parts-of-speech you want to identify and count in the novel you selected? Add a new piece of chunk grammar and repeat the process of chunking. What do you find?

17.

Not the biggest fan of *The Picture of Dorian Gray* or *The Iliad*? No worries! Included in the file navigator is a blank text file named `my_text.txt`. Open the file and copy any text of your choice (novel, script, article, etc.) into the file. Save the file and then return to `script.py`. Update the opened text file to `my_text.txt` and rerun `script.py` to perform a syntax parsing analysis on your text! What insights or deeper meanings did you discover?

Data Scientist: Inference Specialist

Enhanced Recovery After Surgery

<https://www.codecademy.com/paths/data-science-inf/tracks/dsinf-causal-inference/modules/dsinf-difference-in-differences/projects/difference-in-differences-project>

Enhanced Recovery After Surgery (ERAS) is a set of protocols used in the management of surgical patients before, during, and after an operation. While the specific components of ERAS implemented depend on the type of surgery, there are a few general principles:

Before surgery: pre-surgery counseling, meetings with the surgical team, use of epidural for pain management.

During surgery: shorter incisions, laparoscopic approaches when possible, conservative opioid pain management.

After surgery: encouraging movement soon after surgery, quick transition to oral pain medications, resumption of normal food/fluid intake.

Prior studies have demonstrated that the use of ERAS programs is associated with increased patient satisfaction, improved use of hospital resources, decreased complications, and decreased morbidity and mortality.

Imagine that a hospital system is interested in making the ERAS protocol standard practice for all surgeries at all of its hospitals. One of the hospitals in the system, Hospital B, has been using ERAS since the beginning of 2015. Another hospital, Hospital A, does not use it.

The hospital's administrators commission a study to determine whether the use of ERAS at Hospital B led to a decrease in length of stay (LOS). To assess the effectiveness of ERAS, the hospital administrators decide to compare Hospital B to Hospital A. The average length of stay at Hospital B and Hospital A was calculated for each of the years between 2010 and 2019.

In this project, we'll use the *Difference in Differences* (DID) method to evaluate whether implementation of the ERAS protocol led to a reduction in the average hospital length of stay (LOS).

We encourage you to try all tasks on your own in **notebook.Rmd**. However, there are solutions provided in **solution.Rmd** for reference.

Tasks

0/11 complete

[Mark the tasks as complete by checking them off](#)

Data Exploration

1.
Let's start by loading the dataset. We've provided a file named **eras_df.csv** in the workspace.
Load this file into a dataframe named `eras`.
2.
Take a look at the head of the dataframe `eras`. Familiarize yourself with the following variables in the dataset:
 - `year`: indicates the year of the surgery.
 - `hospital`: indicates where each surgery was performed (Hospital A, Hospital B).
 - `los`: hospital length of stay (LOS) in days.
3.
The next step is to plot the data. Filter to just Hospital B and make a line graph to display the change in average hospital LOS over time for Hospital B. Add a vertical dashed line to indicate the point in time when Hospital B implemented the ERAS protocol for surgeries.

Checking for Parallel Trends

4.
Great job so far! We can definitely see a change from 2014 to 2015, but we can't tell how much is due to using the ERAS system. Let's plot the data again with Hospital A as a potential control group.
Generate another line graph but this time plot data for both Hospital A and Hospital B with a vertical dashed line at the year 2014.
Do the trends in LOS at each hospital appear similar up to the year 2014? Do you think that the assumption of *parallel trends* is met?

Calculating the ATT with Means

5.
Now that we have verified that the parallel trends assumption has been met up to the year 2015, we can continue with calculating the ATT. First, let's create a new dataset called `eras2` that uses just the observations from 2014 and 2015. Print `eras2` to see the means before and after treatment for both hospitals.
6.
Now take the difference in average LOS from 2014 to 2015 for each hospital. Call this difference `Adiff` for Hospital A and `Bdiff` for Hospital B.
- 7.

Now let's calculate the difference in differences using `Adiff` and `Bdiff` from the previous task.

Calculating the ATT with Regression

8.

We've successfully calculated the ATT manually, but let's see if we can accomplish the same task with regression. Before we begin modeling, create two indicator variables:

`treat`: a treatment group indicator that is 1 for the treated group and 0 for the control group.

`time`: a time indicator variable that is 1 for the year 2015 and 0 for 2014.

9.

Fit a linear regression model predicting LOS from treatment group, time, and an interaction term between treatment group and time. Save the model object as `hospital_model`. Print the results of the model.

10.

So how should we interpret the results of the regression model we just fit?

`(Intercept)`: the average LOS for a patient at Hospital A (control group) in 2014 was about 4.46 days.

`treat`: Prior to implementation of ERAS at Hospital B, the average LOS for patients at Hospital B was about 0.44 days shorter than for patients at Hospital A.

`time`: The LOS for patients at Hospital A was on average 0.76 days shorter in 2015 than in 2014.

`treat:time`: The change in LOS for patients at Hospital B was an additional 0.80 days fewer than the change in LOS of patients at Hospital A between 2014 and 2015.

But which regression coefficient represents the ATT (the causal estimand)? Print only the coefficient that corresponds with the ATT. Does this align with the ATT you calculated manually?

Conclusion

11.

Great job! Using both manual methods and regression, you found that the average treatment effect among the treated (ATT) for the ERAS protocol is -0.75. This means that use of the ERAS protocol led to a 0.75 day (or 18 hour) reduction in average hospital length of stay compared to the expected LOS had Hospital B NOT used ERAS. Not too shabby!

Mystery Friend

You've received an anonymous postcard from a friend who you haven't seen in years. Your friend did not leave a name, but the card is definitely addressed to you. So far, you've narrowed your search down to three friends, based on handwriting:

Emma Goldman
Matthew Henson
TingFang Wu

But which one sent you the card?

Just like you can classify a message as spam or not spam with a spam filter, you can classify writing as related to one friend or another by building a kind of friend writing classifier. You have past writing from all three friends stored up in the variable `friends_docs`, which means you can use scikit-learn's bag-of-words and Naive Bayes classifier to determine who the mystery friend is!

Ready?

Setup Instructions

You have two options of completing this assignment. Either here, within Codecademy's output terminal, or on your own, in case you're more comfortable using a Jupyter notebook. If you choose to do this project on your computer instead of Codecademy, you can download what you'll need by clicking the "Download" button below. If you need help setting up your computer, be sure to check out our setup guides:

[Command Line Interface Setup](#)
[Introducing Jupyter Notebook](#)
[Setting up Jupyter Notebook](#)
[Getting Started with Jupyter](#)
[Getting More out of Jupyter Notebook](#)

Open **Mystery Friend.ipynb** and follow the steps in the Jupyter Notebook. If you get stuck, you can look at **Mystery Friend Solution.ipynb** for the answer.

Download

Tasks

10/10 complete

[Mark the tasks as complete by checking them off](#)

Feature vectors are in the bag with scikit-learn

1.
Near the top of `script.py`, import `CountVectorizer` from `sklearn.feature_extraction.text`. Below it, import `MultinomialNB` from `sklearn.naive_bayes`.
Import statements in Python look like:

```
from some_library module import some_function_or_variable
```

Copy to Clipboard

2.
Define `bow_vectorizer` as an implementation of `CountVectorizer`.
You can create an implementation of `CountVectorizer` like this:

```
some_vectorizer = CountVectorizer()
```

Copy to Clipboard

3.
Use your newly minted `bow_vectorizer` to both `fit` (train) and `transform` (vectorize) all your friends' writing (stored in the variable `friends_docs`). Save the resulting vector object as `friends_vectors`.
Your syntax should look like this:

```
some_vector = some_vectorizer.fit_transform(some_documents)
```

Copy to Clipboard

4.
Create a new variable `mystery_vector`. Assign to it the vectorized form of `[mystery_postcard]` using the vectorizer's `.transform()` method.
(`mystery_postcard` is a string, while the vectorizer expects a list as an argument.)
Your syntax should look like this:

```
some_vector = some_vectorizer.transform(some_new_documents)
```

Copy to Clipboard

This mystery friend gets classified

5.
You've vectorized and prepared all the documents. Let's take a look at your friends' writing samples to get a sense of how they write.
Print out one document of each friend's writing — try any one between 0 and 140. (Your friend's documents are stored in `goldman_docs`, `henson_docs`, and `wu_docs`.)
You could access the 30th document like:

```
some_friend_docs[29]
```

Copy to Clipboard

6. Have an inkling about which friend wrote the mystery card? We can use a classifier to confirm those suspicions...

Implement a Naive Bayes classifier using `MultinomialNB`. Save the result to `friends_classifier`.

You can create an implementation of `MultinomialNB` like this:

```
some_naive_bayes_classifier = MultinomialNB()
```

Copy to Clipboard

7.

Train `friends_classifier` on `friends_vectors` and `friends_labels` using the classifier's `.fit()` method.

8.

Change `predictions` value from `["None Yet"]` to the classifier's prediction about which friend wrote the postcard. You can do this by calling the classifier's `.predict()` method on the `mystery_vector`.

```
some_prediction = some_classifier.predict(some_new_vector)
```

Copy to Clipboard

Mystery revealed!

9.

Uncomment the final print statement and save your code to see who your mystery friend was all along!

10.

But does it really work? Find some lines by Emma Goldman, Matthew Henson, and TingFang Wu on gutenberg.org and save them to `mystery_postcard` to see how the classifier holds up!

Try using the `.predict_proba()` method instead of `.predict()` and print out `predictions` to see the estimated probabilities that the `mystery_postcard` was written by each person.

What happens when you add in a recent email or text instead?

```
goldman_raw = """
The history of human growth and development is at the same time the
history of the terrible struggle of every new idea heralding the
approach of a brighter dawn. In its tenacious hold on tradition, the
old has never hesitated to make use of the foulest and cruellest means
to stay the advent of the New, in whatever form or period the latter
may have asserted itself. Nor need we retrace our steps into the
distant past to realize the enormity of opposition, difficulties, and
hardship placed in the path of every progressive and dangerous idea.
The thumbscrew, and the knout are still with us; so are the convict's
garb and the social wrath, all conspiring against the spirit that is
serenely marching on.

Anarchism could not hope to escape the fate of all other ideas of
innovation. Indeed, as the most revolutionary and uncompromising
innovator, Anarchism must needs meet with the combined ignorance and
venom of the world it aims to reconstruct.

To deal even remotely with all that is being said and done against
Anarchism would necessitate writing a whole volume. I shall
therefore meet only two of the principal objections. In so doing, I
shall attempt to elucidate what Anarchism really stands for.

The strange phenomenon of the opposition to Anarchism is that it
brings to light the relation between so-called intelligence and
ignorance. And yet this is not so very strange when we consider the
relativity of all things. The ignorant mass has in its favor that it
makes no pretense of knowledge or tolerance. Acting as it always
does, by mere impulse, its reasons are like those of a child.
"Why?" "Because." Yet the opposition of the uneducated to Anarchism
deserves the same consideration as that of the intelligent man.

What, then, are the objections? First, Anarchism is impractical,
though a beautiful ideal. Second, Anarchism stands for violence and
destruction. I must be writing as well as thinking.

Both the intelligent man and the ignorant mass judge not from a
thorough knowledge of the subject, but either from hearsay or false
interpretation.

A practical scheme, says Oscar Wilde, is either one already in
existence, or a scheme that could be carried out under the existing
conditions; but it is exactly the existing conditions that one
objects to, and any scheme that could accept these conditions is
wrong and foolish. The true criterion of the practical, therefore,
is not whether the latter can keep intact the wrong or foolish;
rather is it whether the scheme has vitality enough to leave the
stagnant waters of the old, and build, as well as sustain, new life.
In the light of this conception, Anarchism is indeed practical.
More than any other idea, it is helping to do away with the wrong and
foolish; more than any other idea, it is building and sustaining new
life.

The emotions of the ignorant man are continuously kept at a pitch by
the most blood-curdling stories about Anarchism. Not a thing too
outrageous to be employed against this philosophy and its exponents.
Therefore Anarchism represents to the unthinking what the proverbial
bad man does to the child,--a black monster bent on swallowing
everything; in short, destruction and violence.

Destruction and violence! How is the ordinary man to know that the
most violent element in society is ignorance; that its power of
destruction is the very thing Anarchism is combating? Nor is he
aware that Anarchism, whose roots, as it were, are part of nature's
forces, destroys, not healthful tissue, but parasitic growths that
feed on the life's essence of society. It is merely clearing the
soil from weeds and sagebrush, that it may eventually bear healthy
fruit.

Someone has said that it requires less mental effort to condemn than
to think. The widespread mental indolence, so prevalent in society,
proves this to be only too true. Rather than to go to the bottom of
any given idea, to examine into its origin and meaning, most people
will either condemn it altogether, or rely on some superficial or
prejudicial definition of non-essentials.

Anarchism urges man to think, to investigate, to analyze every
proposition; but that the great capacity of the average reader be not
taxed too much, I also shall begin with a definition, and then
elaborate on the latter.
```

ANARCHISM:--The philosophy of a new social order based on liberty, unrestricted by man-made law; the theory that all forms of government rest on violence, and are therefore wrong and harmful, as well as unnecessary.

The new social order rests, of course, on the materialistic basis of life; but while all Anarchists agree that the main evil today is an economic one, they maintain that the solution of that evil can be brought about only through the consideration of EVERY PHASE of life--individual, as well as the collective; the internal, as well as the external phases.

A thorough perusal of the history of human development will disclose two elements in bitter conflict with each other; elements that are only now beginning to be understood, not as foreign to each other, but as close, real and truly harmonious, if only placed in proper environment: the individual and social instincts. The individual and society have waged a relentless and bloody battle for ages, each striving for supremacy, because each was blind to the value and importance of the other. The individual and social instincts,--the one a most potent factor for individual endeavor for growth,--the aspiration, self-realization; the other an equally potent factor for mutual helpfulness and social well-being.

The explanation of the storm raging within the individual, and between him and his surroundings, is not far to seek. The primitive man, unable to understand his being, much less the unity of all life, felt himself absolutely dependent on blind, hidden forces, ever ready to mock and taunt him. Out of that attitude grew the religious concepts of man as a mere speck of dust dependent on superior powers on high, who can only be appeased by complete surrender. All the early sagas rest on that idea, which continues to be the LIT-MOTIF of the Biblical tales dealing with the relation of man to God, to the State, to society. Again and again the same motif, MAN IS NOTHING, THE GODS ARE EVERYTHING, would come. Only Jesus would make the condition of complete surrender. Man can have all the glories of the earth, but he must not become conscious of himself. The State, society, and moral laws all sing the same refrain: Man can have all the glories of the earth, but he must not become conscious of himself.

Anarchism is the only philosophy which brings to man the consciousness of himself, and maintains that God, the State, and society are non-existent; that their promises are null and void, since they can be fulfilled only through man's subordination. Anarchism is therefore the teacher of the unity of life; not merely in nature, but in man. There is no conflict between the individual and the social instincts, any more than there is between the heart and the lungs; the one the receptacle of a precious life essence, the other the pump that keeps the essence pure and strong. The individual is the heart of society, conserving the essence of social life; society is the lungs which are distributing the element to keep the life essence--that is, the individual--pure and strong.

"The one thing of value in the world," says Emerson, "is the active soul; this every man contains within him. The soul active sees absolute truth and creates it." In other words, the individual instinct is the thing of value in the world. It is the true soul that sees and creates the truth alive, out of which is to come a still greater truth, the re-born social soul.

Anarchism is the great liberator of man from the phantoms that have held him captive; it is the arbiter and pacifier of the two forces for individual and social harmony. To accomplish that unity, Anarchism has declared war on the pernicious influences which have so far prevented the harmonious blending of individual and social instincts, the individual and society.

Religion, the dominion of the human mind; Property, the dominion of human needs; and Government, the dominion of human conduct, represent the stronghold of man's enslavement and all the horrors it entails. Religion! How it dominates man's mind, how it humiliates and degrades his soul! God is everything, man is nothing, says religion. But out of that nothing God has created a kingdom so despotic, so tyrannical, so cruel, so terribly exacting that naught but gloom and tears and blood mark the way to man. Anarchism rouses man to rebellion against this black monster. Break your mental fetters, says Anarchism to man, for not until you think and judge for yourself will you get rid of the dominion of darkness, the greatest obstacle to all progress.

Property, the dominion of man's needs, the denial of the right to satisfy his needs. Time was when property claimed a divine right, when it made man a slave, even as religion. What a sacrifice! Abnegate! Submit! The spirit of Anarchism has lifted man from his prostrate position. He now stands erect, with his face toward the light. He has learned to see the insatiable, devouring, devastating nature of property, and he is preparing to strike the monster dead.

"Property is robbery," said the great French Anarchist, Proudhon. Yes, but without risk and danger to the robber. Monopolizing the accumulated efforts of man, property has robbed him of his birthright, and has turned him loose a pauper and an outcast. Property has even the time-worn excuse that man does not create enough to satisfy all needs. The A B C student of economics knows that the productivity of labor within the last few decades far exceeds normal demand a hundredfold. But what are normal demands to an abnormal civilization? The demand for property recognizes as its own gluttonous appetite for greater wealth, because wealth means power; the power to subdue, to crush, to exploit, the power to enslave, to outrage, to degrade. America is particularly boastful of her great power, her enormous national wealth. Poor America, of what avail is all her wealth, if the individuals comprising the nation are wretchedly poor? If they live in squalor, in filth, in crime, with hope and joy gone, a homeless, soilless army of human prey.

It is generally conceded that unless the returns of any business venture exceed the cost, bankruptcy is inevitable. But those engaged in the business of producing wealth have not yet learned even this simple lesson. Every year the cost of production in human life is growing larger (50,000 killed, 100,000 wounded in America last year); the returns to the masses, who help to create wealth, are ever getting smaller. Yet America continues to be blind to the inevitable bankruptcy of our business of production. Nor is this the only crime of the latter. Still more fatal is the crime of turning the producer into a mere particle of a machine, with less will and decision than his master of steel and iron. Man is being robbed not merely of the products of his labor, but of the power of free initiative, of originality, and the interest in, or desire for, the things he is making.

Real wealth consists in things of utility and beauty, in things that help to create strong, beautiful bodies and surroundings inspiring to live in. But if man is doomed to wind cotton around a spool, or dig coals or bury roses for thirty years of his life, there can be no talk of wealth. What he gives to the world is only gray and hideous things, reflecting a dull and hideous existence,--too weak to live, too cowardly to save. Strange to say, there are people who extol this deadening method of centralized production as the proudest achievement of our age. They fail utterly to realize that if we are to continue in machine subservience, our slavery is more complete than was ever that of the King. They do not want liberty, but also of health and beauty, of art and science, all these being impossible in a clock-like, mechanical atmosphere.

Anarchism cannot but repudiate such a method of production: its goal is the freest possible expression of all the latent powers of the individual. Oscar Wilde defines a perfect personality as, "one who develops under perfect conditions, who is not wounded, chained, or in danger." A perfect personality, then, is only possible in a state of society where man is free to choose the mode of work, the conditions of work, and the freedom to work. One to whom the making of a table,

the building of a house, or the tilling of the soil, is what the painter, the artist, or the discoverer of a scientific truth, the result of inspiration, of intense longing, and deep interest, in work as a creative force. That being the ideal of Anarchism, its economic arrangements must consist of voluntary productive and distributive associations, in which all live communally, as the best means of producing with the least waste of human energy. Anarchism, however, also recognizes the right of the individual, or numbers of individuals, to arrange at all times for other forms of work, in harmony with their tastes and desires.

Such free display of human energy being possible only under complete individual and social freedom, Anarchism directs its forces against the third and greatest foe of all social equality; namely, the State, organized authority, or statutory law,--the dominion of human conduct.

Just as religion has fettered the human mind, and as property, or the monopoly of things, has subdued and stifled man's needs, so has the State enslaved his spirit, dictating every phase of conduct. All government in essence," says Emerson, "is tyranny. It matters not whether it is government by divine right or majority rule, in every instance its aim is the absolute subordination of the individual.

Referring to the American government, the greatest American Anarchist, David Thoreau, said: "Government, what is it but a tradition, though a recent one, endeavoring to transmit itself unimpaired to posterity, but each instance losing its integrity; it has not the vitality and force of a single living man, law never made man a whit more just; and by means of their respect for it, even the well disposed are daily made agents of injustice."

Indeed, the keynote of government is injustice. With the arrogance and self-sufficiency of the King who could do no wrong, governments ordain, judge, condemn, and punish the most insignificant offenses, while maintaining themselves as the greatest of all offenses, the annihilation of individual liberty. Thus Ouida is right when she maintains that "the State only aims at instilling those qualities in its public by which its demands are obeyed, and its exchequer is filled. Its highest attainment is the reduction of mankind to clockwork. In its atmosphere all those finer and more delicate liberties, which require treatment and spacious expansion, inevitably dry up and perish. The State requires a taxpaying machine--in which there is no life, no exchequer, which there is never a desire, and a public, monotonous, obedient, colorless, spiritless, moving, humbly like a flock of sheep along a straight high road between two walls."

Yet even a flock of sheep would resist the chicanery of the State, if it were not for the corruptive, tyrannical, and oppressive methods it employs to serve its purposes. Therefore Bakunin repudiates the State as synonymous with the surer of the liberty of every individual or small minorities,--the destruction of social relationship, the curtailment, or complete denial even, of life itself, of its own gratification. The State is the ally of all political freedom and strikes the religious altar, it is maintained for the purpose of human sacrifice.

The average man with his self-sufficiency, his ridiculously superior airs of patronage towards the female sex, is an impossibility for woman as depicted in the CHARACTER STUDY by Laura Marholm. Equally impossible for her is the man who can see in her nothing more than her mentality and her genius, and who fails to awaken her woman nature.

A rich intellect and a fine soul are usually considered necessary attributes of a deep and beautiful personality. In the case of the modern woman, these attributes serve as a hindrance to the complete assertion of her being. For over a hundred years the old form of marriage, based on the Biblical doctrine of procreation, has been denounced as an institution that stands for the sovereignty of the man over the woman, of her complete submission to his whims and commands, and absolute dependence on his name and support. Time and again it has been conclusively proved that the old matrimonial relation restricted woman to the function of a man's servant and the bearer of his children. And yet we find many emancipated women who prefer marriage, with all its deficiencies, to the narrowness of an unmarried life, and are glad to be released from the chains of moral and social prejudice that cramp and bind her nature.

The explanation of such inconsistency on the part of many advanced women as to why they never fully understand the meaning of emancipation. They thought that all that was needed was independence from external tyrannies; the internal tyrants, far more harmful to life and growth--ethical and social conventions--were left to take care of themselves, and they have taken care of themselves. They seem to get along as beautifully in the heads and hearts of the most active exponents of woman's emancipation, as in the heads and hearts of our grandmothers.

These internal tyrants, whether they be in the form of public opinion or what will mother say, or brother, father, aunt, or relative of any sort; what will Comstock, Mr. Comstock, the employer, the Board of Education say? All these pusbodies, moral detectives, jailers of the human spirit, what will they say? Until woman has learned to defy them all, to stand firmly on her own ground and to insist upon her own way, to listen to the voice of her own nature, whether it call for life's greatest treasure, love for a man, or her most glorious privilege, the right to give birth to a child, she cannot call herself emancipated. How many emancipated women are brave enough to acknowledge that the voice of love is a thing, wildly beating against their breasts, demanding to be heard, to be satisfied.

The French writer, Jean Reibrach, in one of his novels, NEW BEAUTY, attempts to picture the ideal, beautiful, emancipated woman. This ideal is embodied in a young girl, a physician. She talks very cleverly and wisely of how to feed infants; she is kind, and administers medicines free to poor mothers. She converses with a young man of her acquaintance about the sanitary conditions of the future, and how various bacilli and germs shall be exterminated by the use of stone walls and floors, and by the doing away with rugs and hangings. She is, of course, very plainly and practically dressed, mostly in black. The young man, who, at their first meeting, was overawed by the wisdom of his emancipated friend, gradually learns to understand her, and recognized one fine day that he loves her, they are young, and she is kind and beautiful, and though always in rigid attire, her appearance is softened by a sportlessly clean white collar and cuffs. One would expect that he would treat her of his love, but he is not one to commit romantic absurdities. Poetry and the enthusiasm of love cover their blushing faces before the pure beauty of the lady. He silences the voice of his nature, and remains correct. She too is always exact, always rational, always well behaved. I fear if they had formed a union, the young man would have risked freezing to death. I must confess that I can see nothing beautiful in this new beauty, who is as cold as the stone walls and floors she dreams of. Rather would I have the love songs of romantic poets, rather Don Juan and Madame Venus, rather an elopement by ladder and rope on a moonlight night, followed by the father's curse, mother's moans, and the moral comments of neighbors, than coldly measured privacy secured by varnish and sky. If one does not know how to give and take without restrictions, it is not love, but a transaction that never fails to lay stress on a plus and a minus.

The greatest shortcoming of the emancipation of the present day lies in its artificial stiffness and its narrow respectabilities, which produce an emptiness in woman's soul that will not let her drink from the fountain of life. I once remarked that there seemed to be a deeper relationship between the old-fashioned mother and hostess, ever on the alert for the happiness of her little ones and the comfort of those she loved, and the truly new woman, than between the latter and her average emancipated sister. The spectacles of emancipation pure and simple declared me a heathen, fit only for the stake. Their blind zeal did not let them see that my comparison between the old and the new was merely to prove that a goodly number

of our grandmother had more blood in their veins, far more humor and wit, and certainly a greater amount of naturalness, kind-heartedness, and simplicity than the majority of our emancipated professional women who fill the colleges, halls of learning, and various offices. This does not mean a wish to return to the past, nor does it condemn woman to her old sphere, the kitchen and the nursery.

Salvation lies in an energetic march onward towards a brighter and clearer future. We are in need of unhampered growth out of old traditions and habits. The movement for woman's emancipation has so far made but a feeble step in this direction. It is to be noted that the great gathering trenched to make another. The right to vote and equal civil rights, may be good demands, but true emancipation begins neither at the polls nor in courts. It begins in woman's soul. Education, the great power, must be given to the oppressed class, and education from its masters through its own efforts. It is necessary that woman learn that lesson, that she realize that her freedom will reach as far as her power to achieve her freedom reaches. It is, therefore, the duty of woman to turn to her higher regeneration, to cut loose from the weight of prejudices, traditions, and customs. The demand for equal rights in every vocation of life is just and fair; but, after all, the most vital right is the right to love and to be loved. When woman's emancipation is complete, when the love and true emancipation of woman, it will have to do away with the ridiculous notion that to be loved, to be sweetheart and mother, is synonymous with being slave or subordinate. It will have to do away with the distinction between the sexes, or that man and woman represent two antagonistic worlds.

Pettiness separates; breadth unites. Let us be broad and big. Let us overlook all things because of the bulk of our fires. A true conception of the relation of the sexes will not admit of conqueror and conquered; it knows of but one great thing: to give of one's self boundlessly, in order to find one's self richer, deeper, better. That alone can fill the emptiness, and transform the tragedy of woman's emancipation into joy, limitless joy.

The popular notion about marriage and love is that they are synonymous, that they spring from the same motives, and cover the same human needs. Like most popular notions this also rests not on actual facts, but on superstition.

Marriage and love have nothing in common; they are as far apart as the poles; are, in fact, antagonistic to each other. No doubt some marriages have been the result of love. Not, however, because love is the cause of marriage, but because marriage is the result of love. People can completely outgrow a convention. There are today large numbers of men and women to whom marriage is naught but a farce, but who are nevertheless married. It is not because they are married, while it is true that some marriages are based on love, and while it is equally true that in some cases love continues in married life, I maintain that it does so regardless of marriage and not because of it.

On the other hand, it is utterly false that love results from marriage. On rare occasions one does hear of a miraculous case of a married couple falling in love after marriage, but on these occasions one will find that the marriage was a complete failure. The inevitable - certainly the growing - used to each other is far away from the spontaneity, the intensity, and beauty of love, without which the intimacy of marriage must prove degrading to both the woman and the man.

Marriage is primarily an economic arrangement, an insurance pact. It differs from the ordinary life insurance agreement only in that it is more binding, more exacting. Its returns are insignificantly small compared with the investments. In taking out an insurance policy one pays for it in dollars and cents, always at liberty to discontinue the policy at any time. In taking out a marriage policy one pays for it with her name, her privacy, her self-respect, her very life, "until death doth part." Moreover, the marriage insurance condemns her to life-long dependency, to parasitism, to complete uselessness, to a life as a parasite on the life of another person. The economic sphere is wider, marriage does not limit him as much as woman. He feels his chains more in an economic sense.

Thus Dante's motto over Inferno applies with equal force to marriage.
"Ye who enter here leave all hope behind."

That marriage is a failure none but the very stupid will deny. One has but to glance over the statistics of divorce to realize how bitter a failure marriage really is. Nor will the stereotyped Phillistine argument that the laxity of divorce laws and the growing looseness of woman account for the fact that, first, every twelfth married couple divorce, second, that in 1911, 151,000 divorces increased from 28 to 100 for every hundred thousand population; third, that adultery, since 1867, as ground for divorce, has increased 270.8 per cent.; fourth, that desertion increased 369.8 per cent.

Added to these startling figures is a vast amount of material, dramatic and literary, further elucidating this subject. Robert Herrick, in TOGETHER; Pintero, in MID-CHANNEL; Eugene Walter, in PAID IN FULL; and scores of other writers are discussing the barrenness, the monotony, the sordidness, the inadequacy of marriage as a factor for harmony and understanding.

The thoughtful social student will not content himself with the popular superficial excuse for this phenomenon. He will have to dig deeper into the very life of the sexes to know why marriage proves so disastrous.

Edward Carpenter says that behind every marriage stands the life-long environment of the two sexes; an environment so differentiated from each other that men and women are strangers separated by an insurmountable wall of superstition, custom, and habit, marriage has not the potentiality of developing knowledge of, and respect for, each other, without which every union is doomed to failure.

Henrik Ibsen, the greater of all social shams, was probably the first to realize this great truth. Nora leaves her husband, not--as the stupid critic would have it--because she is tired of her responsibilities or feels the need of woman's rights, but because she has found another person who can understand her as a real stranger, and borne him children. Can there be anything more humiliating, more degrading than a life-long proximity between two strangers? No need to ask. The answer is obvious. The knowledge of the man, the knowledge of the woman--what is there, to know except that she has a pleasing appearance? We have not yet outgrown the theologic myth that woman has no soul, that she is a mere appendix to man, made out of clay for the sake of the man, for the pleasure of the gentleman who was so strong that he was afraid of his own shadow.

Perchance the pogg quality of the material whence woman comes is responsible for her inferiority. At any rate, woman has no soul, what is there to know about her? Besides, the less soul a woman has, the more she is a wife, for, after all, she absorbs herself in her husband. It is this slavish acquiescence to man's superiority that has kept the marriage institution seemingly so strong and lasting. Now, however, when woman is coming into her own, now that she is actually growing aware of herself as being outside of the master's drudge, the sacred institution of marriage is being undermined, and no amount of sentimental lamentation can stay it.

From infancy, almost, the average girl is told that marriage is her ultimate goal; therefore her training and education must be directed towards that end. Like the mute beast fattened for slaughter, she is brought up to be a wife and mother, and to know how to perform her less about her function as wife and mother than the ordinary artisan of his trade. It is indecent and filthy for a respectable girl to be seen in the company of men, or to be associated with anything that is of respectability, that needs the marriage vow to turn something which is filthy into the purest and most sacred arrangement that none dare tamper with. It is one of the most sacred and most sacred of all the average upholder of marriage that the prospective wife and mother is

kept in complete ignorance of her only asset in the competitive field--sex. Thus she enters into life-long relations with a man only to find herself shocked, repelled, outraged beyond measure by the most natural and healthy instinct, sex. It is safe to say that a large percentage of the unhappiness, misery, distress, and physical suffering of matrimony is due to the criminal ignorance in sex matters that is being extolled as a great virtue. Nor is it at all an exaggeration when I say that more than one home has been broken up because of this deplorable fact.

If, however, woman is free and big enough to learn the mystery of sex without the sanction of State or Church, she will stand condemned as utterly unfit to become the wife of a "good" man, his goodness consisting of an empty brain and plenty of money. Can there be anything more outrageous than the idea that a healthy, grown woman, full of life and passion, must deny nature's demand, must subdue her most intense craving, undermine her health and break her spirit, must stunt her vision, abstain from the depth and glory of sex experience until a "good" man comes along to take her unto himself as a wife? That is precisely what marriage means. How can such an arrangement end except in failure? This is one, though not the least important, factor of marriage, which differentiates it from love.

Ours is a practical age. The time when Romeo and Juliet risked the wrath of their fathers for love, when Gretchen exposed herself to the gossip of her neighbors for love, is no more. If, on rare occasions, young people allow themselves the luxury of romance, they are taken in care by the elders, drilled and pounded until they become "sensible."

The moral lesson instilled in the girl is not whether the man has aroused her love, but rather is it "How much?" The important and only God of practical American life: Can the man make a living? can he support a wife? That is the only thing that justifies marriage. Gradually this saturates every thought of the girl; her dreams are not of moonlight and kisses, of laughter and tears; she dreams of shopping tours and bargain counters. This soul poverty and sordidness are the elements inherent in the marriage institution. The State and Church approve of no other ideal, simply because it is the one that necessitates the State and Church control of men and women.

Doubtless there are people who continue to consider love above dollars and cents. Particularly this is true of that class whom economic necessity has forced to become self-supporting. The tremendous change in woman's position, wrought by that mighty factor, is indeed phenomenal when we reflect that it is but a short time since she has entered the industrial arena. Six million women wage workers; six million women, who have equal right with men to be exploited, to be robbed, to go on strike; aye, to starve even. Anything more, my lord? Yes, six million wage workers in every walk of life, from the highest brain work to the mines and railroad tracks; yes, even detectives and policemen. Surely the emancipation is complete.

Yet with all that, but a very small number of the vast army of women wage workers look upon work as a permanent issue, in the same light as does man. No matter how depreciable the task, they have been taught to be independent, self-supporting. Oh, I know that no one is really independent in our economic treadmill; still, the poorest specimen of a man hates to be a parasite; to be known as such, at any rate.

The woman considers her position as worker transitory, to be thrown aside for the first bidder. That is why it is infinitely harder to organize women than men. "Why should I join a union? I am going to get married, to have a home. Has she not been taught from infancy to look upon that as her ultimate calling? She learns soon enough that the home, though not so large a prison as the factory, has more solid doors and bars and has a keeper so faithful that night can escape him. The most tragic part, however, is that the home no longer frees her from wage slavery; it only increases her task."

```
goldman docs = goldman raw.split(". ")
```

```
henson raw = ""
```

When the news of the discovery of the North Pole, by Commander Peary, was first sent to the world, a distinguished citizen of New York City, well versed in the affairs of the Peary Arctic Club, made the statement, that he was sure that Matt Henson had been with Commander Peary on the day of the discovery.

There were not many people who knew who Henson was, or the reason why the gentleman had made the remark, and, when asked why he was so certain, he explained that, for the best part of the twenty years of Commander Peary's Arctic work, his faithful and often only companion was Matthew Alexander Henson.

To-day there is a more general knowledge of Commander Peary, his work and his success, and a vague understanding of the fact that Commander Peary's sole companion from the realm of civilization, when he stood at the North Pole, was Matthew A. Henson, a Colored Man.

To satisfy the demand of perfectly natural curiosity, I have undertaken to write a brief autobiography, giving particularly an account of my Arctic work.

I was born in Charles County, Maryland, August 8, 1866. The place of my birth was on the Potomac River, about forty-four miles below Washington, D. C. Slavery days were over forever when I was born. Besides, my parents were both free born before me, and in my mother's veins ran some white blood. At an early age, my parents were induced to leave the country and remove to Washington, D. C. My mother died when I was seven years old. I was taken in charge by my uncle, who sent me to school, the "N Street School" in Washington, D. C., which I attended for over six years. After leaving school I went to Baltimore, Md., where I shipped as cabin-boy, on board a vessel bound for China. After my first voyage I became an able-bodied seaman, and for four years followed the sea in that capacity, sailing to China, Japan, Manilla, North Africa, Spain, France, and through the Black Sea to Southern Russia.

It was while I was in Washington, D. C., in 1888, that I first attracted the attention of Commander Peary, who at that time was a civil engineer in the United States Navy, with the rank of lieutenant, and it was with the instinct of my race that I recognized in him the qualities that made me willing to engage myself in his service. I accompanied him as his body-servant to Nicaragua. I was his messenger at the League Island Navy Yard, and from the beginning of his second expedition to the Arctic regions, in 1891, I have been a member of every expedition of his, in the capacity of assistant: a term that covers a multitude of duties, abilities, and responsibilities.

The narrative that follows is a record of the last and successful expedition of the Peary Arctic Club, which had as its attainment the discovery of the North Pole, and is compiled from notes made by me at different times during the course of the expedition. I did endeavor to keep a diary or journal of daily events during my last trip, and did not find it difficult aboard the ship while sailing north, or when in winter-quarters at Cape Sheridan, but I found it impossible to make daily entries while in the field, on account of the constant necessity of concentrating my attention on the real business of the expedition. Entries were made daily of the records of temperature and the estimates of distance traveled; and when solar observations were made the results were always carefully noted. There were opportunities to complete the brief entries on several occasions while out on the ice, notably the six days' enforced delay at the "Big Lead," 84 north, the twelve hours preceding the return of Captain Bartlett at 87 47' north, and the thirty-three hours at North Pole, while Commander Peary was determining to a certainty his position. During the return from the Pole to Cape Columbia, we were so urged by the knowledge of the supreme necessity of speed that the thought of recording the events of that part of the journey did not occur to me so forcibly as to compel me to pay heed to it, and that story was written aboard the ship while waiting for favorable conditions to sail toward home lands.

* * * * *

It was in June, 1891, that I started on my first trip to the Arctic regions, as a member of what was known as the "North Greenland Expedition." Mrs. Peary accompanied her husband, and among the members of the expedition were Dr. Frederick A. Cook, of Brooklyn, N. Y., Mr. Langdon Gibson, of Flushing, N. Y., and Mr. Eivind Astrp, of Christiania, Norway, who had the honor of being the companion of Commander Peary in the first crossing of North Greenland--and of having an Esquimo at Cape York become so fond of him that he named his son for him! It was on this voyage north that Peary's leg was broken.

Mr. John M. Verhoeff, a stalwart young Kentuckian, was also an enthusiastic member of the party. When the expedition was ready to sail home the following summer, he lost his life by falling in a crevasse in a glacier. His body was never recovered. On the first and the last of Peary's expeditions, success was marred by tragedy. On the last expedition, Professor Ross G. Marvin, of Cornell University, lost his life by being drowned in the Arctic Ocean, on his return from his farthest north, a farther north than had ever been made by any other explorers except the members of the last expedition. Both Verhoeff and Marvin were good friends of mine, and I respect and venerate their memories.

Naturally the impressions formed on my first visit to the Land of Ice and Snow were the most lasting, but in the coming years I was to learn more and more that such a life was no picnic, and to realize what primitive life meant. I was to live with a people who, the scientists stated, represented the earliest form of human life, living in what is known as the Stone Age, and I was to revert to that stage of life by leaps and bounds, and to emerge from it by the same sudden means. Many and many a time, for periods covering more than twelve months, I have been to all intents an Esquimo, with Esquimos for companions, speaking their language, dressing in the same kind of clothes, living in the same kind of dens, eating the same food, enjoying their pleasures, and frequently sharing their griefs. I have come to love these people. I know every man, woman, and child in their tribe. They are my friends and they regard me as theirs.

After the first return to civilization, I was to come back to the savage, ice- and rock-bound country seven times more. It was in June, 1893, that I again sailed north with Commander Peary and his party on board the Falcon, a larger ship than the Kite, the one we sailed north in on the previous expedition, and with a much larger equipment, including several burros from Colorado, which were intended for ice-cap work, but which did not make good, making better dog-food instead. Indeed the dogs made life a burden for the poor brutes from the very start. Mrs. Peary was again a member of the expedition, as well as another woman, Mrs. Cross, who acted as Mrs. Peary's maid and nurse. It was on this trip that I adopted the orphan Esquimo boy, Kudlooktoo, his mother having died just previous to our arrival at the Red Cliffs. After this boy was washed and scrubbed by me, his long hair cut short, and his greasy, dirty clothes of skins and furs burned, a new suit made of odds and ends collected from different wardrobes on the ship made him a presentable Young American. I was proud of him, and he of me. He learned to speak English and slept underneath my bunk.

This expedition was larger in numbers than the previous one, but the results, owing to the impossible weather conditions, were by no means successful, and the following season all of the expedition returned to the United States except Commander Peary, Hugh J. Lee, and myself. When the expedition returned, there were two who went back who had not come north with us. Miss Marie Ahnighito Peary, aged about ten months, who first saw the light of day at Anniversary Lodge on the 12th of the previous September, was taken by her mother to her kinfolks in the South. Mrs. Peary also took a young Esquimo girl, well known among us as "Miss Bill," along with her, and kept her for nearly a year, when she gladly permitted her to return to Greenland and her own people. Miss Bill is now grown up, and has been married three times and widowed, not by death but by desertion. She is known as a "Holy Terror." I do not know the reason why, but I have my suspicions.

The memory of the winter of 1894 and 1895 and the summer following will never leave me. The events of the journey to 87° 6' in 1906 and the discovery of the North Pole in 1909 are indelibly impressed on my mind, but the recollections of the long race with death across the 450 miles of the ice-cap of North Greenland in 1895, with Commander Peary and Hugh Lee, are still the most vivid.

For weeks and weeks, across the seemingly never-ending wastes of the ice-cap of North Greenland, I marched with Peary and Lee from Independence Bay and the land beyond back to Anniversary Lodge. We started on April 1, 1895, with three sledges and thirty-seven dogs, with the object of determining to a certainty the northeastern terminus of Greenland. We reached the northern land beyond the ice-cap, but the condition of the country did not allow much exploration, and after killing a few musk-oxen we started on June 1 to make our return. We had one sledge and nine dogs.

We reached Anniversary Lodge on June 25, with one dog.

The Grim Destroyer had been our constant companion, and it was months before I fully recovered from the effects of that struggle. When I left for home and God's Country the following September, on board the good old Kite, it was with the strongest resolution to never again! no more! forever! leave my happy home in warmer lands. Nevertheless, the following summer I was again "Northward Bound," with Commander Peary, to help him secure, and bring to New York, the three big meteorites that he and Lee had discovered during the winter of 1894-1895.

The meteorites known as "The Woman" and "The Dog" were secured with comparative ease, and the work of getting the large seventy-ton meteor, known as "The Tent," into such a position as to insure our securing it the following summer, was done, so it was not strange that the following summer I was again in Greenland, but the meteorite was not brought away that season.

It is well known that the chief characteristic of Commander Peary is persistency which, coupled with fortitude, is the secret of his success. The next summer, 1897, he was again at the island after his prize, and he got it this time and brought it safely to New York, where it now reposes in the "American Museum of Natural History." As usual I was a member of the party, and my back still aches when I think of the hard work I did to help load that monster aboard the _Hope_.

It was during this voyage that Commander Peary announced his determination to discover the North Pole, and the following years (from 1898 to 1902) were spent in the Arctic.

In 1900, the American record of Farthest North, held by Lockwood and Brainard, was equaled and exceeded; their cairn visited and their records removed. On April 21, 1902, a new American record of 84° 17' was made by Commander Peary, further progress north being frustrated by a lack of provisions and by a lane of open water, more than a mile wide. This lead or lane of open water I have since become more familiarly acquainted with. We have called it many names, but it is popularly known as the "Big Lead." Going north, meeting it can be depended upon. It is situated just a few miles north of the 84th parallel, and is believed to mark the continental shelf of the land masses in the Northern Hemisphere.

During the four years from 1898 to 1902, which were continuously spent in the regions about North Greenland, we had every experience, except death, that had ever fallen to the lot of the explorers who had preceded us, and more than once we looked death squarely in the face. Besides, we had many experiences that earlier explorers did not meet. In January, 1899, Commander Peary froze his feet so badly that all but one of his toes fell off.

After the return home, in 1902, it was three years before Commander Peary made another attack on the Pole, but during those years he was not resting.

He was preparing to launch his final and "sincerely to be hoped" successful expedition, and in July, 1905, in the newly built ship, _Roosevelt_, we were again "Poleward-bound." The following September, the _Roosevelt_ reached Cape Sheridan, latitude 82° 27' north, under her own steam, a record unequalled by any other vessel, sail or steam.

Early the next year, the negotiation of the Arctic Ocean was commenced, not as oceans usually are negotiated, but as this ocean must be, by men, sledges, and dogs. The field party consisted of twenty-six men, twenty sledges, and one hundred and thirty dogs.

That was an open winter and an early spring, very desirable conditions in some parts of the world, but very undesirable to us on the northern coast of Greenland. The ice-pack began disintegrating much too early that year to suit, but we pushed on, and had it not been for furious storms enforcing delays and losses of many precious days, the Pole would have been reached. As it was, Commander Peary and his party got to 87° 6' north, thereby breaking all records, and in spite of incredible hardships, hunger and cold, returned safely with all of the expedition, and on Christmas Eve the _Roosevelt_, after a most trying voyage, entered New York harbor, somewhat battered but still seaworthy.

Despite the fact that it was to be his last attempt, Commander Peary no sooner reached home than he announced his intention to return, this time to be the last, and this time to win.

However, a year intervened, and it was not until July 6, 1908, with the God-Speed and good wishes of President Roosevelt, that the good ship named in his honor set sail again. The narrative of that voyage, and the story of the discovery of the North Pole, follow.

The ages of the wild, misgiving mystery of the North Pole are over,

to-day, and forever it stands under the folds of Old Glory.

July 6, 1908: We're off! For a year and a half I have waited for this order, and now we have cast off. The shouting and the tumult ceases, the din of whistles, bells, and throats dies out, and once again the long, slow surge of the ocean hits the good ship that we have embarked in. It was at one-thirty P. M. to-day that I saw the last hawse-line cast adrift, and felt the throb of the engines of our own ship. Chief Wardwell is on the job, and from now on it is due north.

Oyster Bay, Long Island Sound: We are expecting President Roosevelt. The ship has been named in his honor and has already made one voyage towards the North Pole, farther north than any ship has ever made.

July 7: At anchor, the soft wooded hills of Long Island give me a curious impression. I am waiting for the command to attack the savage ice- and rock-bound fortress of the North, and here instead we are at anchor in the neighborhood of sheep grazing in green fields.

Sydney, N. S., July 17, 1908: All of the expedition are aboard and those going home have gone. Mrs. Peary and the children, Mr. Borup's father, and Mr. Harry Whitney, and some other guests were the last to leave the _Roosevelt_, and have given us a last good-by from the tug, which came alongside to take them off.

Good-by all. Every one is sending back a word to some one he has left behind, but I have said my good-bys a long time ago, and as I waved my hand in parting salutation to the little group on the deck of the tug, my thoughts were with my wife, and I hoped when she next heard of me it would be with feelings of joy and happiness, and that she would be glad she had permitted me to leave her for an absence that might never end.

The tenderfeet, as the Commander calls them, are the Doctor, Professor MacMillan, and young Mr. Borup. The Doctor is a fine-looking, big fellow, John W. Goodsell, and has a swarthy complexion and straight hair; on meeting me he told me that he was well acquainted with me by reputation, and hoped to know me more intimately.

Professor Donald B. MacMillan is a professor in a college in Massachusetts, near Worcester, and I am going to cultivate his acquaintance.

Mr. George Borup is the kid, only twenty-one years old but well set up for his age, always ready to laugh, and has thick, curly hair. I understand he is a record-breaker in athletics. He will need his athletic ability on this trip. I am making no judgments or comments on these fellows now. Wait; I have seen too many enthusiastic starters, and I am sorry to say some of them did not finish well.

All of the rest of the members of the expedition are the same as were on the first trip of the _Roosevelt_ :--Commander Peary, Captain Bartlett, Professor Marvin, Chief Engineer Wardwell, Charley Percy the steward, and myself. The crew has been selected by Captain Bartlett, and are mostly strangers to me.

Commander Peary is too well known for me to describe him at length; thick reddish hair turning gray; heavy, bushy eyebrows shading his "sharpshooter's eyes" of steel gray, and long mustache. His hair grows rapidly and, when on the march, a thick heavy beard quickly appears. He is six feet tall, very graceful, and well built, especially about the chest and shoulders; long arms, and legs slightly bowed. Since losing his toes, he walks with a peculiar slide-like stride. He has a voice clear and loud, and words never fail him.

Captain Bartlett is about my height and weight. He has short, curly, light-brown hair and red cheeks; is slightly round-shouldered, due to the large shoulder-muscles caused by pulling the oars, and is as quick in his actions as a cat. His manner and conduct indicate that he has always been the leader of his crowd from boyhood up, and there is no man on this ship that he would be afraid to tackle. He is a young man (thirty-three years old) for a ship captain, but he knows his job.

Professor Marvin is a quiet, earnest person, and has had plenty of practical experience besides his splendid education. He is rapidly growing bald; his face is rather thin, and his neck is long. He has taken great interest in me and, being a teacher, has tried to teach me. Although I hope to perfect myself in navigation, my knowledge so far consists only of knot and splice seamanship, and I need to master the mathematical end.

The Chief Engineer, Mr. Wardwell, is a fine-looking, ruddy-complexioned giant, with the most honest eyes I have ever looked into. His hair is thinning and is almost pure white, and I should judge him to be about forty-five years old. He has the greatest patience, and I have never seen him lose his temper or get rattled.

Charley Percy is Commander Peary's oldest hand, next to me. He is our steward, and sees to it that we are properly fed while aboard ship, and he certainly does see to it with credit to himself.

From Sydney to Hawks Harbor, where we met the _Erik_, has been uneventful except for the odor of the _Erik_, which is loaded with whale-meat and can be smelled for miles. We passed St. Paul's Island and Cape St. George early in the day and through the Straits of Belle Isle to Hawks Harbor, where there is a whale-factory. From here we leave for Turnavik.

We have been racing with the _Erik_ all day, and have beaten her to this place. Captain Bartlett's father owns it, and we loaded a lot of boots and skins, which the Captain's father had ready for us. From here we sail to the Esquimo country of North Greenland, without a stop if possible, as the Commander has no intention of visiting any of the Danish settlements in South Greenland.

Cape York is our next point, and the ship is sailing free. Aside from

the excitement of the start, and the honor of receiving the personal visit of the President, and his words of encouragement and cheer, the trip so far has been uneventful; and I have busied myself in putting my cabin in order, and making myself useful in overhauling and stowing provisions in the afterhold.

July 24: Still northward-bound, with the sea rolling and washing over the ship; and the Erik in the distance seems to be getting her share of the wash. She is loaded heavily with fresh whale-meat, and is purposely keeping in leeward of us to spare us the discomfort of the odor.

July 25 and 26: Busy with my carpenter's kit in the Commander's cabin and elsewhere. There has been heavy rain and seas, and we have dropped the Erik completely. The Roosevelt is going fine. We can see the Greenland coast plainly and to-day, the 29th, we raised and passed Disco Island. Icebergs on all sides. The light at midnight is almost as bright as early evening twilight in New York on the Fourth of July and the ice-blink of the interior ice-cap is quite plain. We have gone through Baffin's Bay with a rush and raised Duck Island about ten A. M. and passed and dropped it by two P. M.

I was ashore on Duck Island in 1891, on my first voyage north, and I remember distinctly the cairn the party built and the money they deposited in it. I wonder if it is still there? There is little use for money up here, and the place is seldom visited except by men from the whalers, when their ships are locked in by ice.

From here it is two hundred miles due north to Cape York.

August 1: Arrived at Cape York Bay and went ashore with the party to communicate with the Esquimos of whom there were three families. They remembered us and were dancing up and down the shore, and waving to us in welcome, and as soon as the bow of the boat had grazed the little beach, willing hands helped to run her up on shore. These people are hospitable and helpful, and always willing, sometimes too willing. As an example, I will tell how, at a settlement farther north, we were going ashore in one of the whale-boats. Captain Bartlett was forward, astraddle of the bow with the boat-hook in his hands to fend off the blocks of ice, and knew perfectly well where he wanted to land, but the group of excited Esquimos were in his way and though he ordered them back, they continued running about and getting in his way. In a very short while the Captain lost patience and commenced to talk loudly and with excitement; immediately Sipsoo took up his language and parrot-like started to repeat the Captain's exact words: "Get back there, get back--how in ---- do you expect me to make a landing?" And thus does the innocent lamb of the North acquire a civilized tongue.

It is amusing to hear Kudlooktoo in the most charming manner give Charley a cussing that from any one else would cause Charley to break his head open.

For the last week I have been busy, with "Matt! The Commander wants you," "Matt do this," and "Matt do that," and with going ashore and trading for skins, dogs, lines, and other things; and also walrus-hunting. I have been up to my neck in work, and have had small opportunity to keep my diary up to date. We have all put on heavy clothing; not the regular fur clothes for the winter, but our thickest civilized clothing, that we would wear in midwinter in the States. In the middle of the day, if the sun shines, the heat is felt; but if foggy or cloudy, the heavy clothing is comfortable.

All of the Esquimos want to come aboard and stay aboard. Some we want and will take along, but there are others we will not have or take along on a bet, and the pleasant duty of telling them so and putting them ashore falls to me. It is not a pleasant job to disappoint these people, but they would be a burden to us and in our way. Besides, we have left them a plentiful supply of needfuls, and our trading with them has been fair and generous.

The "Crow's-Nest" has been rigged upon the mainmast, and this morning, after breakfast, Mr. Whitney, three Esquimos, and myself started in Mr. Whitney's motor-boat to hunt walrus. The motor gave out very shortly after the start, and the oars had to be used. We were fortunate in getting two walrus, which I shot, and then we returned to the ship for the whale-boat. We left the ship with three more Esquimos in the whale-boat, and got four more walrus.

Sunday, at Kangerdlooksoah; the land of the reindeer, and the one pleasant appearing spot on this coast. Mr. Whitney and his six Esquimo guides have gone hunting for deer, and I have been ashore to trade for dogs and furs, and have gotten twenty-seven dogs, sealskin-lines for lashings, a big bearskin, and some foxskins. I try to get furskins from animals that were killed when in full fur and before they have started to shed, but some of the skins I have traded in are raw, and will have to be dried.

I have had the disagreeable job of putting the undesirable ashore, and it was like handling a lot of sulky school children.

Seegloo, the dog-owner, is invited to bring his pack aboard and is easily persuaded. He will get a Springfield rifle and loading-outfit and also a Winchester, if he will sell, and he is more than willing.

And this is the story of day after day from Cape York to Etah Harbor, which we reached on August 12.

""

```
henson docs = henson raw.encode('ascii', 'ignore').decode("utf-8").split(" ")
```

```
wu raw = ""
The Importance of Names
```

"What's in a name? That which we call a rose
By any other name would smell as sweet."

Notwithstanding these lines, I maintain that the selection of names is important. They should always be carefully chosen. They are apt to influence friendships or to excite prejudices according to their significance. We Chinese are very particular in this matter. When a son is born the father or the grandfather chooses a name for the infant boy which, according to his horoscope, is likely to insure him success, or a name is selected which indicates the wish of the family for the new-born child. Hence such names as "happiness", "prosperity", "longevity", "success", and others, with like propitious import, are common in China. With regard to girls their names are generally selected from flowers, fruits, or trees. Particular care is taken not to use a name which has a bad meaning. In Washington I once met a man in an elevator whose name was "Coffin". Was I to be blamed for wondering if the elevator would be my coffin? On another occasion I met a man whose name was "Death", and as soon as I heard his name I felt inclined to run away, for I did not wish to die. I am not superstitious. I have frequently taken dinner with thirteen persons at the table, and I do not hesitate to start on a journey on a Friday. I often do things which would not be done by superstitious persons in China. But to meet a man calling himself "Coffin" or "Death" was too much for me, and with all my disbelief in superstition I could not help showing some repugnance to those who bore such names.

Equally important, if not more so, is the selection of a name for a state or a nation. When the several states of America became independent they called themselves the "United States of America"--a very happy idea. The Union was originally composed of thirteen states, covering about 300,000 square miles; it is now composed of forty-eight states and three territories, which in area amount to 3,571,492 square miles, practically as large in extent as China, the oldest nation in the world. It should be noted that the name is most comprehensive: it might comprise the entire continent of North and South America. It is safe to say that the founders of the nation did not choose such a name without consideration, and doubtless the designation "United States of America" conceals a deep motive. I once asked a gentleman who said he was an American whether he had come from South or North America, or whether he was a Mexican, a Peruvian or a native of any of the countries in Central America? He replied with emphasis that he was an American citizen of the United States. I said it might be the United States of Mexico, or Argentina, or other United States, but he answered that when he called himself a citizen it could not mean any other than that of the United States of America. I have asked many other Americans similar questions and they all have given me replies in the same way. We Chinese call our nation "The Middle Kingdom"; it was supposed to be in the center of the earth. I give credit to the founders of the United States for a better knowledge of geography than that possessed by my countrymen of ancient times and do not assume that the newly formed nation was supposed to comprise the whole continent of North and South America, yet the name chosen is so comprehensive as to lead one naturally to suspect that it was intended to include the entire continent. However, from my observation of their national conduct, I believe their purpose was just and humane; it was to set a noble example to the sister nations in the Western Hemisphere, and to knit more closely all the nations on that continent through the bonds of mutual justice, goodwill and friendship. The American nation is, indeed, itself a pleasing and unique example of the principle of democracy. Its government is ideal, with a liberal constitution, which in effect declares that all men are created equal, and that the government is "of the people, for the people, and by the people." Anyone with ordinary intelligence and with open eyes, who should visit any city, town or village in America, could not but be impressed with the orderly and unostentatious way in which it is governed by the local authorities, or help being struck by the plain and democratic character of the people. Even in the elementary schools, democracy is taught and practised. I remember visiting a public school for children in Philadelphia, which I shall never forget. There were about three or four hundred children, boys and girls, between seven and fourteen years of age. They elected one of their students as mayor, another as judge, another as police commissioner, and in fact they elected for the control of their school community almost all the officials who usually govern a city. There were a few Chinese children among the students, and one of them was pointed out to me as the police superintendent. This not only eloquently spoke of his popularity, but showed goodwill and harmony among the several hundred children, and the entire absence of race feeling. The principals and teachers told me that they had no difficulty whatever with the students. If one of them did anything wrong, which was not often, he would be taken by the student policeman before the judge, who would try the case, and decide it on its merits, and punish or discharge his fellow student as justice demanded. I was assured by the school authorities that this system of self-government worked admirably; it not only relieved the teachers of the burden of constantly looking after the several hundred pupils, but each of them felt a moral responsibility to behave well, for the sake of preserving the peace and good name of the school. Thus early imbued with the idea of self-government, and entrusted with the responsibilities of its administration, these children when grown up, take a deep interest in federal and municipal affairs, and, when elected for office, invariably perform their duties efficiently and with credit to themselves.

It cannot be disputed that the United States with its democratic system of government has exercised a great influence over the states and nations in Central and South America. The following data showing the different nations of America, with the dates at which they turned their respective governments from Monarchies into Republics, all subsequent to the independence of the United States, are very significant.

Mexico became a Republic in 1823, Honduras in 1839, Salvador in 1839, Nicaragua in 1821, Costa Rica in 1821, Panama in 1903, Colombia in 1819, Venezuela in 1830, Ecuador in 1810, Brazil in 1889, Peru in 1821, Bolivia in 1825, Paraguay in 1811, Chile in 1810, Argentina in 1824, and Uruguay in 1828.

These Republics have been closely modelled upon the republican form of government of the United States; thus, nearly all the nations or states on the continent of America have become Republics. Canada still belongs to Great Britain. The fair and generous policy pursued by the Imperial Government of Great Britain accounts for the Canadians' satisfaction with their political position, and for the fact that they do not wish a change. It must be noted, however, that a section of the American people would like to see Canada incorporated with the United States. I remember that at a public meeting held in Washington, at which Sir Wilfrid Laurier, then Premier of Canada, was present, an eminent judge of the Federal Supreme Court jocularly expressed a wish that Canada should be annexed to the United States. Later, Mr. Champ Clark, a leader of the Democratic party in the House of Representatives, addressed the House urging the annexation of Canada. Even if these statements are not taken seriously they at least show the feelings of some people, and he would be a bold man who would prophesy the political status of Canada in the future. There is, however, no present indication of any change being desired by the Canadians, and it may be safely presumed that the existing conditions will continue for

many years to come. This is not to be wondered at, for Canada though nominally a British colony practically enjoys almost all the privileges of an independent state. She possesses a constitution similar to that of the United Kingdom, with a parliament of two houses, called the "Senate", and the "House of Commons". The Sovereign of Great Britain appoints only the Governor General who acts in his name, but the Dominion is governed by a responsible Ministry, and all domestic affairs are managed by local officials, without interference from the Home Government. Canadians enjoy as many rights as the inhabitants of England, with the additional advantage that they do not have to bear the burden of maintaining an army and navy. Some years ago, if I remember rightly, in consequence of some agitation or discussion for independence, the late Lord Derby, then Secretary of State for the Colonies, stated that if the Canadians really wished for independence, the Home Government would not oppose, but that they should consider if they would gain anything by the change, seeing that they already had self-government, enjoyed all the benefits of a free people, and that the only right the Home Government reserved was the appointment of the Governor-General, although it assumed the responsibility of protecting every inch of their territory from encroachment. Since this sensible advice from the Colonial Secretary, I have heard nothing more of the agitation for independence.

From a commercial point of view, and for the welfare of the people, there is not much to choose to-day between a Limited Monarchy and a Republic. Let us, for instance, compare England with the United States. The people of England are as free and independent as the people of the United States, and though subjects, they enjoy as much freedom as Americans. There are, however, some advantages in favor of a Republic. Americans until recently paid their President a salary of only \$50,000 a year; it is now \$75,000 with an additional allowance of \$25,000 for travelling expenses. This is small indeed compared with the Civil List of the King or Emperor of any great nation. There are more chances in a Republic for ambitious men to distinguish themselves; for instance, a citizen can become a president, and practically assume the functions of a king or an emperor. In fact the President of the United States appoints his own cabinet officials, ambassadors, ministers, etc. It is generally stated that every new president has the privilege of making more than ten thousand appointments. With regard to the administration and executive functions he has in practice more power than is usually exercised by a king or an emperor of a Constitutional Monarchy. On the other hand, in some matters, the executive of a Republic cannot do what a king or an emperor can do; for example, a president cannot declare war against a foreign nation without first obtaining the consent of Congress. In a monarchical government the king or the cabinet officials assume enormous responsibilities. Lord Beaconsfield (then Mr. D'Israeli), while he was Prime Minister of England, purchased in 1875 from the Khedive of Egypt 176,602 Suez Canal shares for the sum of 3,976,582 Pounds on his own responsibility, and without consulting the Imperial Parliament. When Parliament or Congress has to be consulted about everything, great national opportunities to do some profitable business must undoubtedly be sometimes lost. No such bold national investment as that made by Lord Beaconsfield could have been undertaken by any American president on his own responsibility. Mr. Cleveland, when president of the United States, said that "the public affairs of the United States are transacted in a glass house."

Washington, in his farewell address, advised his compatriots that on account of the detached and distant situation of their country they should, in extending their commercial relations with foreign nations, have as little political connection with them as possible; and he asked this pertinent and pregnant question, "Why, by interweaving our destiny with that of any part of Europe, entangle our peace and prosperity in the toils of European ambition, rivalry, interest, humors, and caprice?" In 1823, twenty-seven years after Washington's celebrated address, President Monroe in his annual message to Congress warned the European Powers not to plant any new colonies on any portion of the American hemisphere, as any attempt on their part to extend their system in that part of the world would be considered as dangerous to the peace and safety of the United States. This "Monroe Doctrine", as it has since been called, practically protects every state and country on the American continent from attack or interference by any foreign power, and it cannot be denied that it has been and is now the chief factor in preserving the integrity of all the countries on that continent. Thus the United States is assuming the role of guardian over the other American nations. In the city of Washington there is an International Bureau of the American Republics, in which all the Republics of Central and South America are represented. It is housed in a magnificent palace made possible by the beneficence of Mr. Andrew Carnegie, the American multi-millionaire and philanthropist, and the contributions of the different governments. It cost 750,000 gold dollars, and Mr. John Barrett, the capable and popular director of the Bureau, has well called it "a temple of friendship and commerce and a meeting place for the American Republics." The Bureau is supported by the joint contributions of the twenty-one American Republics and its affairs are controlled by a governing board composed of their diplomatic representatives in Washington, with the American Secretary of State as chairman ex officio. This institution no doubt strengthens the position of the United States and is calculated to draw the American Republics into closer friendship.

Chapter 2. American Prosperity

One of the main causes of the prosperity of the great American Republic is its natural resources. It possesses coal, oil, silver, gold, copper, and all the other mineral ores. Nature seems, indeed, to have provided almost everything that man needs. The soil is rich; wheat and every kind of fruit can be grown; but favorable as are these native conditions they could not be turned to any great advantage without the skill and industry of enterprising men. Many countries in Africa and Asia possess equal advantages, but they are not equally prosperous. This leads me to the consideration of another reason for America's growth. The men who have migrated to the United States have not been rich people. They went there to make a living. They were prepared to work, their purpose was to improve their condition, and they were willing to undertake any manual or mental labor to accomplish their object. They were hardy and strong and could bear a heavy strain. Their children inherited their good qualities, and so an American is generally more hard working and enterprising than most of the people in Europe and elsewhere.

Another reason for America's success is the great freedom which each citizen enjoys. Every man considers himself the equal of every other, and a young man who is ambitious will not rest until he reaches the top of his profession or trade. Thousands of Americans who were once very poor, have become millionaires or multi-millionaires. Many of them had no college education; they taught themselves, and some of them have become both literary and scholarly. A college or university education does not necessarily make a man learned; it only gives him the opportunity to learn. It is said that some college men have proven themselves to be quite ignorant, or rather that they do not know so much as those who have been self-taught. I do not in any way wish to disparage a college education; no doubt men who have been trained in a

university start in life with better prospects and with a greater chance of success, but those men who have not had such advantages have doubtless done much to make their country great and prosperous, and they ought to be recognized as great men.

The general desire of the American people to travel abroad is one of their good traits. People who never leave their homes cannot know much. A person may become well-informed by reading, but his practical knowledge cannot be compared with that of a person who has travelled. We Chinese are great sinners in this regard. A Chinese maxim says, "It is dangerous to ride on horseback or to go on a voyage". Hence until very recently we had a horror of going abroad. A person who remains all his life in his own town is generally narrow-minded, self-opinioned, and selfish. The American people are free from these faults. It is not only the rich and the well-to-do who visit foreign countries, but tradesmen and workmen when they have saved a little money also often cross the Atlantic. Some years ago a Senator in Washington told me that he crossed the Atlantic Ocean every summer and spent several months in Europe, and that the next trip would be his twenty-eighth voyage. I found, however, that he had never gone beyond Europe. I ventured to suggest that he should extend his next annual journey a little farther and visit Japan, China, and other places in the Far East which I felt sure he would find both interesting and instructive. I have travelled through many countries in Europe and South America, and wherever I have gone and at whatever hotel I have put up, I have always found some Americans, and on many occasions I have met friends and acquaintances whom I had known in Washington or New York. But it is not only the men who go abroad; in many cases ladies also travel by themselves. On several occasions lady friends from Washington, Philadelphia, and New York have visited me in Peking. This is one of 'the Americans' strong points, is it not wiser and much more useful to disburse a few hundred dollars or so in travelling and gaining knowledge, coming in contact with other peoples and enlarging the mind, than to spend large sums of money in gaudy dresses, precious stones, trinkets, and other luxuries?

In a large country like America where a considerable portion of the land still remains practically uncultivated or undeveloped, hardy, industrious, and patient workmen are a necessity. But the almost unchecked influx of immigrants who are not desirable citizens cannot but harm the country. In these days of international trade it is right that ingress and egress from one country to another should be unhampered, but persons who have committed crimes at home, or who are ignorant and illiterate, cannot become desirable citizens anywhere. They should be barred out of the United States of America. It is well known that foreigners take part in the municipal and federal affairs of the country as soon as they become citizens. Now if such persons really worked for the good of their adopted country, there could be no objection to this, but it is no secret that many have no such motives. That being so, it is a question whether steps should not be taken to limit their freedom. On the other hand, as many farms suffer from lack of workmen, people from whatever country who are industrious, patient, and persevering ought to be admitted as laborers. They would be a great boon to the nation. The fear of competition by cheap labor is causeless; regulations might be drawn up for the control of these foreign laborers, and on their arrival they could be drafted to those places where their services might be most urgently needed. So long as honest and steady workmen are excluded for no reason other than that they are Asiatics, while white men are indiscriminately admitted, I fear that the prosperity of the country cannot be considered permanent, for agriculture is the backbone of stable wealth. Yet at present it is the country's wealth which is one of the important factors of America's greatness. In the United States there are thousands of individuals whose fortunes are counted by seven or eight figures in gold dollars. And much of this money has been used to build railways or to develop manufactures and other useful industries. The country has grown great through useful work, and not on account of the army and navy. In 1881 America's army numbered only 26,622 men, and her navy consisted of only 24 iron-clads, 2 torpedo-boats, and 25 tugs, but in 1910 the peace strength of her army was 96,628 and the navy boasted 33 battleships and 120 armored cruisers of different sizes.

Within the last few years it has been the policy of many nations to increase the army and to build as many Dreadnaughts and super-dreadnaughts as possible. Many statesmen have been infected by this Dreadnaught fever. Their policy seems to be based on the idea that the safety of a nation depends on the number of its battleships. Even peaceful and moderate men are carried away by this hobby, and support it. It is forgotten that great changes have taken place during the last twenty or thirty years; that a nation can now be attacked by means quite beyond the reach of Dreadnaughts. The enormous sums spent on these frightful monsters, if applied to more worthy objects, would have a greater effect in preserving the nations' heritages than anything these monstrosities can do.

The nation which has a large army and a strong navy may be called powerful, but it cannot be considered great without other good requisites. I consider a nation as great when she is peacefully, justly, and humanely governed, and when she possesses a large number of benevolent and good men who have a voice in the administration. The greater the number of good men that a nation possesses the greater she becomes. America is known to have a large number of such men and women, men and women who devote their time and money to preaching peace among the nations. Mr. Andrew Carnegie is worth a hundred Dreadnaughts. He and others like him are the chief factors in safeguarding the interests and welfare of America. The territory of the United States is separated from Europe and other countries by vast oceans; so that it would be difficult, if not impossible, for a foe to successfully attack any portion of that country. But who wishes to attack her? She has scarcely an enemy. No country is invaded by another without cause, and as the United States is in friendly relations with all the Powers, there is no reason to fear foreign invasion. Even should a foreign power successfully attack her and usurp a portion of her territories, a supposition which is most improbable, would the enemy be able to hold what he seized? History shows that no conquered country has ever been successfully and permanently kept without the people's consent, and there is not the least chance that the Americans will ever consent to the rule of a foreign government.

It is to be hoped that the United States will not follow the example of other nations and unduly increase her armaments, but that she will take the lead in the universal peace movement and show the world that a great power can exist and maintain her position without force of arms. I am aware that general disarmament is not popular among statesmen, that it has been denounced by an eminent authority as a "will-o'-the-wisp", that arbitration has been styled a "Jack-o'-lantern", but this is not the first time a good and workable scheme has been branded with opprobrious names. The abolition of slavery was at one time considered to be an insane man's dream, now all people believe in it. Will the twentieth century witness the collapse of our present civilization?

Why are the world's armaments constantly increasing? To my mind it is due to two causes, one of which is mistrust. One nation begins to build Dreadnaughts, another does the same through fear and mistrust. The second cause is that it is the fashion of some nations to follow

the example of others that they may preserve their position as great naval powers. But it is unnecessary for the United States to show such mistrust, or to follow such fashion. The United States, as becomes a great and powerful nation, take an independent course of her own. If she sets the example other nations in due time will follow her. The peace of the world will be more surely guarded, and America will win the approbation, the respect, and the gratitude of all peace-loving people.

It would seem reasonable to expect that in yielding so fully to the wishes of the United States in this second negotiation the Chinese Government would not be called upon to make any further concessions in the interests or at the demand of the labor unions on the Pacific coast, but in this China was disappointed. Within a period of less than ten years an urgent application was made by the American Secretary of State for a new treaty amended so as to enable the Congress of the United States to still further restrict the privileges of Chinese laborers who had come to the United States. And when the Chinese Government hesitated to consent to the withdrawal of rights which the United States granted to the subjects of other Governments, Congress passed the Scott Act of 1888 prohibiting any Chinese person from entering the United States except Chinese officials, teachers, students, merchants or travellers for pleasure or curiosity and forbidding also Chinese laborers in the United States, after having left, from returning thereto. This, in the words of Hon. J. W. Foster, ex-Secretary of State and a distinguished international lawyer, "was a deliberate violation of the Treaty of 1880 and was so declared by the Supreme Court of the United States." In order to save the Executive of the United States from embarrassment, the Chinese Government, contrary to its own sense of justice, and of international comity, for a third time yielded to the wishes of the United States, and concluded the amended treaty of 1894 which gave Congress additional power of legislation respecting Chinese laborers. By Article I of this treaty it was agreed that for a term of ten years the coming of Chinese laborers to the United States should be absolutely prohibited. Article III distinctly provided that "the provisions of this convention shall not affect the right at present enjoyed of Chinese subjects, being officials, teachers, students, merchants, or travellers for curiosity or pleasure, of coming to the United States and residing therein." Thus it is clear that the prohibition affects only laborers, and not the other classes of Chinese. For a few years after the signing of this convention this was the view adopted and acted upon by the immigration officials, but afterward they changed their attitude, and the foregoing Article has since been interpreted to mean that only the above-mentioned five classes can be admitted to the United States, and that all the other classes of Chinese, however respectable and honorable, must be refused admission. Will my readers believe that a Chinese banker, physician, lawyer, broker, commercial agent, scholar or professor could all be barred out of the United States of America under the provisions of this convention? In the face of the plain language of the text it seems too absurd and unreasonable to be contemplated, and yet it is a fact.

This convention was proclaimed in December, 1894. According to its provisions, it was to remain in force only for a period of ten years, but that if six months before the end of that period neither Power should give notice of denunciation it should be extended for a similar period. Such notice was, however, given by China to the United States and accordingly the convention expired in December, 1904, and is now no longer in force. No serious attempt has since been made by the United States Government to negotiate a new treaty regarding Chinese laborers, so the customs and immigration officials continue to prohibit Chinese laborers from coming to America by virtue of the law passed by Congress. It will be seen that by the treaty of 1868, known as the "Burlingame Treaty", the United States Government formally agreed that Chinese subjects residing in the United States, should enjoy the same privileges and immunities as were enjoyed by the citizens or subjects of the most favored nation; that being so, and as the convention of 1894 has expired, according to the legal opinion of Mr. John W. Foster, and other eminent lawyers, the continuation of the exclusion of Chinese laborers and the restrictions placed upon Chinese merchants and others seeking admission to the United States are not only without international authority but in violation of treaty stipulations.

The enforcement of the exclusion laws against Chinese in the Hawaiian and Philippine Islands is still more inexcusable. The complaint in America against the immigration of Chinese laborers was that such immigration was detrimental to white labor, but in those Islands there has been no such complaint; on the contrary the enforcement of the law against the Chinese in Hawaii has been, and is, contrary to the unanimous wish of the local Government and the people. Free intercourse and immigration between those Islands and China have been maintained for centuries. What is most objectionable and unfair is that the Chinese should be singled out for discrimination, while all other Asiatics such as Japanese, Siamese, and Malays are allowed to enter America and her colonies without restraint. It is my belief that the gross injustice that has been inflicted upon the Chinese people by the harsh working of the exclusion law is not known to the large majority of the American people, for I am sure they would not allow the continuation of such hardships to be suffered by those who are their sincere friends. China does not wish special treatment, she only asks that her people shall be treated in the same way as the citizens or subjects of other countries. Will the great American nation still refuse to consent to this?

To solve the problem of immigration in a manner that would be satisfactory to all parties is not an easy task, as so many conflicting interests are involved. But it is not impossible. If persons interested in this question be really desirous of seeing it settled and are willing to listen to reasonable proposals, I believe that a way may be found for its solution. There is good reason for my optimistic opinion. Even the Labor Unions, unless I am mistaken, would welcome an amicable settlement of this complicated question. In 1902, while at Washington, I was agreeably surprised to receive a deputation of the leaders of the Central Labor Union of Binghamton, New York, inviting me to pay a visit to them and to deliver an address. As I did not wish to disappoint them I accepted their invitation. During my short stay there, I was very cordially and warmly received, and most kindly treated not only by the local authorities and inhabitants, but by the members of the Labor Union and the working men also. I found that the Union leaders and the working men were most reasonable, their platform being, as far as I could learn, to have no cheap labor competition but not necessarily discrimination against any race. If the United States Government would appoint a commission composed of members representing the Labor Unions, manufacturers and merchants, to treat with a similar commission nominated by the Chinese Government, the whole question in all its bearings could be discussed, and I feel certain that after free and candid exchange of views, the joint Commissioners would be able to arrive at a scheme which would put at rest once for all the conflicting claims, and settle the matter satisfactorily to both China and the United States.

wu docs = wu raw.split(". ")

Read the News Analysis

Newspapers and their online formats supply the public with the information we need to understand the events occurring in the world around us. From politics to sports, the news keeps us informed, in the loop, and ready to make decisions about how to act in a rapidly changing world.

Given the vast amount of news articles in circulation, identifying and organizing articles by topic is a useful activity. This can help you sift through the enormous amount of information out there so you can find the news relevant to your interests, or even allow you to build a news recommendation engine!

[The News International](#) is the largest English language newspaper in Pakistan, covering local and international news across a variety of sectors. A selection of articles from a [Kaggle Dataset of The News International articles](#) is provided in the workspace. In this project you will use term frequency-inverse document frequency (tf-idf) to analyze each article's content and uncover the terms that best describe each article, providing quick insight into each article's topic.

Let's get started!

Tasks

0/12 complete

[Mark the tasks as complete by checking them off](#)

Imports and Data Preparation

1.
In order to calculate tf-idf scores for the articles in the news dataset, you will need some help from scikit-learn. Begin by importing `CountVectorizer`, `TfidfTransformer`, and `TfidfVectorizer` from `sklearn.feature_extraction.text`.
To import `CountVectorizer`, `TfidfTransformer`, and `TfidfVectorizer` from `sklearn.feature_extraction.text`:

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfTransformer, TfidfVectorizer
```

Copy to Clipboard

2.
Provided in `articles.py` is a selection of 10 articles from The News International. Each article, stored as a string, is given as a corpus in the list `articles`.
In `script.py`, print one of the articles and read its contents.
To print one of the articles stored in `articles`, where `i` is an index from 0 to 9:

```
print(articles[i])
```

Copy to Clipboard

3.
Before proceeding, let's preprocess each article by performing tokenization and lemmatization.
Provided in `preprocessing.py` is a function `preprocess_text()` that accepts a string as input and returns a preprocessed string.
Preprocess each article in `articles` and store the processed articles in a list called `processed_articles`.
Print out one of the preprocessed articles.

You can use a list comprehension to preprocess each document in a corpus as follows:

```
processed_docs = [preprocess_text(document) for document in corpus]
```

Copy to Clipboard

You can also use a `for` loop to preprocess the documents!

```
processed_docs = list()
for document in corpus:
    processed_docs.append(preprocess_text(document))
```

Copy to Clipboard

Replace `corpus` with `articles` to preprocess each article in `articles`.

Calculate Tf-idf Scores

4.
You want to begin your analysis by starting off with simple word counts for each article. Initialize a `CountVectorizer` object assigned to a variable named `vectorizer`.
You can initialize a `CountVectorizer` object as follows:

```
my_vectorizer = CountVectorizer()
```

Copy to Clipboard

5.
Fit and transform your `vectorizer` on `processed_articles` to get the word counts for each article. Save the resulting counts to a variable named `counts`.
After you save the word counts to `counts`, you will see a DataFrame appear in the browser component. View the DataFrame to see the word counts for each article.

You can fit and transform a vectorizer object on a corpus as shown below:

```
word_counts = my_vectorizer.fit_transform(corpus)
```

Copy to Clipboard

Replace `corpus` with `processed_articles`.

6.
Now that you have the word counts for each article, let's convert them into tf-idf scores.
Initialize a `TfidfTransformer` object with keyword argument `norm=None` saved to a variable `transformer`.
You can initialize a `TfidfTransformer` object as follows:

```
my_transformer = TfidfTransformer(norm=None)
```

Copy to Clipboard

7.

Fit and transform your `transformer` on `counts` to convert the word counts into tf-idf scores for each article. Save the resulting tf-idf scores to a variable named `tfidf_scores_transformed`. After you save the tf-idf scores to `tfidf_scores_transformed`, you will see another DataFrame appear in the browser component. View the DataFrame to see the tf-idf scores for each article.

You can fit and transform a `TfidfTransformer` object on a set of word counts as shown below:

```
tfidf_scores = my_transformer.fit_transform(word_counts)
```

Copy to Clipboard

8.

Amazing! Now you have your tf-idf scores for each article. You want to confirm, however, that the `TfidfTransformer` gives the same results as directly using the `TfidfVectorizer`. Initialize a `TfidfVectorizer` object with keyword argument `norm=None` saved to a variable `vectorizer`.

You can initialize a `TfidfVectorizer` object as follows:

```
my_vectorizer = TfidfVectorizer(norm=None)
```

Copy to Clipboard

9.

Fit and transform your `vectorizer` on `processed_articles` to calculate the tf-idf scores for each article in one step. Save the resulting tf-idf scores to a variable named `tfidf_scores`. After you save the tf-idf scores to `tfidf_scores`, you will see another DataFrame appear in the browser component. View the DataFrame to see the tf-idf scores for each article.

Do the tf-idf scores given by `TfidfVectorizer` look the same as those given by `TfidfTransformer`?

You can fit and transform a `TfidfVectorizer` object on a corpus:

```
tfidf_scores = my_vectorizer.fit_transform(corpus)
```

Copy to Clipboard

10.

Let's confirm that the tf-idf scores given by `TfidfTransformer` and `TfidfVectorizer` are the same. Paste the following `if` statement into `script.py` under the comment "check if tf-idf scores are equal":

```
if np.allclose(tfidf_scores_transformed.todense(), tfidf_scores.todense()):
    print(pd.DataFrame({'Are the tf-idf scores the same?':['YES']}))
else:
    print(pd.DataFrame({'Are the tf-idf scores the same?':['No, something is wrong :(']}))
```

Copy to Clipboard

You should see that the tf-idf scores are, in fact, the same!

Paste the below code into `script.py` under the comment "check if tf-idf scores are equal":

```
if np.allclose(tfidf_scores_transformed.todense(), tfidf_scores.todense()):
    print(pd.DataFrame({'Are the tf-idf scores the same?':['YES']}))
else:
    print(pd.DataFrame({'Are the tf-idf scores the same?':['No, something is wrong :(']}))
```

Copy to Clipboard

Analyze the Results

11.

A simple way of identifying the "topic" of a document is to label the document with its highest-scoring tf-idf term. While this is a more naive approach than others, it is a quick and easy way of getting insight into the topic of a document. Scroll down to the bottom of `script.py`, and begin by writing a `for` loop that iterates a variable `i` through the values 1 to 10.

You can create a `for` loop using the `range()` function:

```
for i in range(start, stop)
```

Copy to Clipboard

`start` is the first value taken by `i` and `stop - 1` is the last value taken by `i`.

12.

The [Pandas Series method .idxmax\(\)](#) is a helpful tool for returning the index of the highest value in a DataFrame column. We will use this method to find the highest scoring tf-idf term for each article.

Within the `for` loop, paste the following code:

```
print(df_tf_idf[[f'Article {i}']].idxmax())
```

Copy to Clipboard

On each pass through the `for` loop, this code will print the index of the term with the highest tf-idf score for that article (from Article 1 to Article 10).

Compare the actual text of the articles to the selected term. Do printed terms give you any insight into the topic of the respective articles?

Paste the following code within the `for` loop you just created:

```
print(df_tf_idf[[f'Article {i}']].idxmax())
```

Copy to Clipboard

After pasting in the code and running `script.py`, you should get the following terms as having the highest tf-idf score for each article. While this is a crude approach to topic definition, it is helpful in gaining some quick insight into an articles meaning!

Article

Highest Scoring Term

Article 1	fare
Article 2	hong
Article 3	sugar
Article 4	petrol
Article 5	engine
Article 6	australia
Article 7	car
Article 8	railway
Article 9	cabinet
Article 10	china

```
import codecademylib3 seaborn
import pandas as pd
import numpy as np
from articles import articles
from preprocessing import preprocess_text

# import CountVectorizer, TfidfTransformer, TfidfVectorizer

# view article

# preprocess articles

# initialize and fit CountVectorizer
```

```

# convert counts to tf-idf

# initialize and fit TfidfVectorizer

# check if tf-idf scores are equal

# get vocabulary of terms
try:
    feature_names = vectorizer.get_feature_names()
except:
    pass

# get article index
try:
    article_index = [f"Article {i+1}" for i in range(len(articles))]
except:
    pass

# create pandas DataFrame with word counts
try:
    df_word_counts = pd.DataFrame(counts.T.todense(), index=feature_names,
    columns=article_index)
    print(df_word_counts)
except:
    pass

# create pandas DataFrame(s) with tf-idf scores
try:
    df_tf_idf = pd.DataFrame(tfidf_scores_transformed.T.todense(),
    index=feature_names, columns=article_index)
    print(df_tf_idf)
except:
    pass

try:
    df_tf_idf = pd.DataFrame(tfidf_scores.T.todense(), index=feature_names,
    columns=article_index)
    print(df_tf_idf)
except:
    pass

# get highest scoring tf-idf term for each article

```

```

article 1 = '''KARACHI: The Sindh government has decided to bring down public transport
fares by 7 per cent due to massive reduction in petroleum product prices by the federal
government, Geo News reported.Sources said reduction in fares will be applicable on public
transport, rickshaw, taxi and other means of traveling. Meanwhile, Karachi Transport
Ittehad (KTI) has refused to abide by the government decision.KTI President Irshad Bukhari
said the commuters are charged the lowest fares in Karachi as compare to other parts of the
country, adding that 80pc vehicles run on Compressed Natural Gas (CNG). Bukhari said
Karachi transporters will cut fares when decrease in CNG prices will be made.'''

```

```

article 2 = '''HONG KONG: Hong Kong shares opened 0.66 percent lower Monday following a
tepid lead from Wall Street, as the first full week of the new year kicked off. The
benchmark Hang Seng Index dipped 158.63 points to 23,699.19.'''

```

```

article 3 = '''KARACHI: Wholesale market rates for sugar dropped to less than Rs 50 per kg
following the resumption of sugar cane crushing by sugar mills in Sindh. Within two days,

```

the rate dropped by Rs 1.70 to Rs 49.80 per kg in Karachi Whole Sale Market. According to dealers, the resumption of sugar cane crushing by the mills stabilised the supply to the market with an immediate effect on price as well. Industry experts said that the quality of sugar cane is excellent in Sindh and approximately 100 kg of sugar cane can produce 11 kg of sugar.'''

article 4 = '''ISLAMABAD: Long queues of vehicles on fuel stations were visible in different parts of the country as the petrol became rare commodity on Thursday. Federal Minister for Petroleum Shahid Khaqan Abbasi says "it may take up to ten days to bring the situation to normality". He claimed that northern areas of Pakistan had been facing the petrol shortage. The minister cited the recent decline in petroleum prices and delay in a shipment as reasons for the shortage. He said situation would improve as soon as shipment reached Pakistan. Sources told Geo News that due to financial restraints the Pakistan State Oil has been unable to import petrol.'''

article 5 = '''KARACHI: The final shipment of Chinese manufactured Rail Engines arrived in Pakistan on Friday. Federal Railways Minister, Khawaja Saad Rafique says, the inclusion of the new engines will help ease the shortfall faced by Pakistan Railways. The shipment includes 2000 and 3000-horse-power engines which will be used to pull freight bogeys. Rafique told journalists, the inclusion of 15 new engines has brought Pakistan Railways total strength to 268 engines however more engines are still required.'''

article 6 = '''SYDNEY: Cricket fever has gripped Australia with the World Cup just days away. Fans from around the world have thronged to the country and hotels are capitalising. Prices of rooms have almost doubled to 300 dollars and hotels are experiencing full bookings. Experts estimate that during the mega event Australia will generate 1.5 million US dollars just from hotel bookings. If the cost of internal air travel, taxis and tickets is taken into consideration, Australia stands to generate two million US dollars during the World Cup.'''

article 7 = '''SAN FRANCISCO: Apple Inc aims to begin producing electric vehicles as early as 2020, Bloomberg reported. The report cited people with knowledge of the matter as saying, a seemingly aggressive target for a mobile devices maker with little experience in car manufacture. The iPhone maker is pushing its "car team" of about 200 people to meet that goal. But Apple may decide to scrap its car-making effort, or delay it, if executives grew unhappy with its progress, the news agency said.'''

article 8 = '''LAHORE: Federal Minister for Railways, Khawaja Saad Rafique Tuesday announced good news of pay-raise for the employees of Pakistan Railways. In a media statement, the Minister disclosed that a summary for increase in salaries for the employees of Pakistan Railways has been forwarded to the Prime Minister. He also said that the government had also chalked out a plan to build houses for the Railways workers. Khawaja Saad Rafique said it was expected that the salaries of Railway Police may witness a jump of 20 percent. He also announced the government's plan to launch a new train service between Karachi and Islamabad.'''

article 9 = '''ISLAMABAD: The Federal Cabinet on Tuesday approved the budget strategy paper, sources revealed to Geo News. During the cabinet meeting, Prime Minister Nawaz Sharif said tax rate had to be reduced to increase revenue. He added that people would happily pay taxes if the rate was reduced. The prime minister directed the cabinet to provide maximum relief to people in the budget, emphasising that the economic impact should reach people.'''

article 10 = '''BEIJING: China will keep the yuan basically stable against a basket of currencies and there is no basis for continued yuan depreciation, central bank vice governor Yi Gang said on Sunday. China also will keep foreign exchange reserves at appropriate levels, Yi said.'''

articles = [article_1, article_2, article_3, article_4, article_5, article_6, article_7, article_8, article_9, article_10]

Data Scientist: Natural Language Processing Specialist

U.S.A. Presidential Vocabulary

Whenever a United States of America president is elected or re-elected, an [inauguration ceremony](#) takes place to mark the beginning of the president's term. During the ceremony, the president gives an inaugural address to the nation, dictating the tone and focus of the next four years of leadership.

In this project you will have the chance to analyze the inaugural addresses of the presidents of the United States of America, as collected by [the Natural Language Toolkit](#), using word embeddings.

By training sets of word embeddings on subsets of inaugural address versus the collection of presidents as a whole, we can learn about the different ways in which the presidents use language to convey their agenda.

Let's get started!

Tasks

0/19 complete

[Mark the tasks as complete by checking them off](#)

Preprocessing the Data

1.

Provided in the workspace are **.txt** files for each of the 58 U.S.A. presidential inaugural addresses from 1789 to 2017. The files are named with the following convention:

YEAR-PRESIDENT.txt

where **YEAR** is the year the inaugural address was given and **PRESIDENT** is the name of the president who gave the address. If there is only one president with a given last name, only the last name is used. For presidents with the same last name, first name and middle initial(s), separated by a **-**, are included.

Open the file navigator and click on a few of the files to view them.

2.

Navigate back to **script.py**. At the top of the file we have imported some libraries that will be helpful for your analysis, as well as some helper functions from the file **president_helper.py**.

In order to create word embeddings on the corpus of all the presidents' speeches, we need to read the text data from each file, separate the files into sentences on a word by word basis, and then merge all the sentences across the speeches into one big list of lists.

Let's start by finding all the file names for the **.txt** files we will be analyzing. Paste the following code into **script.py** to store a list of all **.txt** files in a variable **files**.

```
files = sorted([file for file in os.listdir() if file[-4:] == '.txt'])
```

Copy to Clipboard

Print **files** to view the names of each inauguration speech file.

The **os** package function **listdir()** returns a list of all files in a given directory (folder). In the provided code, we filter out all files that do not end with **.txt** and sort the file names using the **sorted()** function.

Since there are 58 inaugural addresses, the length of **files** should be 58.

3.

Imported from **president_helper.py** is a function called **read_file()** which takes a file name as an argument and returns the opened file as one big string.

Call **read_file()** on each file in **files**, and save all the resulting string versions to a list named **speeches**.

You can call some function `my_func()` on each item in a list `my_list` using a list comprehension as follows:

```
result = [my_func(item) for item in my_list]
```

Copy to Clipboard

Similarly, you can also use a `for` loop:

```
result = list()
for item in my_list:
    result.append(my_func(item))
```

Copy to Clipboard

Replace `my_func()` with `read_file()` and `my_list` with `files` in order to read each file into a string and store all the resulting speeches in a list.

4.

Now that we have a list of speeches as strings, we need to breakdown the speeches into words on a sentence by sentence basis.

For example, if we have the following sentences:

```
"The cat in the hat is sad. He didn't have a mouse."
```

Copy to Clipboard

The processed data would look as follows:

```
[["the", "cat", "in", "the", "hat", "is", "sad"], ["he", "didn't", "have", "a", "mouse"]]
```

Copy to Clipboard

We've imported the function `process_speeches()` from `president_helper.py` in order to perform this preprocessing for you. The function takes a list of strings as an argument and returns a list of lists. Each inner list represents one inaugural address and is a list of lists as well. Each inner list of the inaugural address list represents a sentence of that address, and each item in the sentence list is a word token in that sentence (see the hint for further explanation of this structure).

Call `process_speeches()` with `speeches` as an argument and save the resulting list to `processed_speeches`.

You can call some function `my_func()` with an argument `my_arg` as follows:

```
result = my_func(my_arg)
```

Copy to Clipboard

Replace `my_func()` with `process_speeches()` and `my_arg` with `speeches` in order to process each speech in `speeches`.

After calling `process_speeches()` with `speeches` as an argument, the data will be formatted such that:

`processed_speeches[0]` represents the first inaugural address in `processed_speeches`.

`processed_speeches[0][0]` represents the first sentence in the first inaugural address in `processed_speeches`.

`processed_speeches[0][0][0]` represents the first word in the first sentence in the first inaugural address in `processed_speeches`.

5.

In order to build a custom set of word embeddings using `gensim`, we need to convert our data into a list of lists, where each inner list is a sentence and each item in the inner list is a word token.

Currently, our speech data is in the following form:

```
speeches = [[["monkeys", "eat", "bananas"], ["they", "hang", "from", "trees"], ...],
             [["penguins", "are", "cute"], ["they", "cannot", "fly"], ...], ...]
```

Copy to Clipboard

Where we have two speeches, one about monkeys and another about penguins. We want to remove the outer layer of list so our data is as so:

```
speeches = [["monkeys", "eat", "bananas"], ["they", "hang", "from", "trees"], ...,
             ["penguins", "are", "cute"], ["they", "cannot", "fly"], ...]
```

Copy to Clipboard

We've imported a function `merge_speeches()` from **president_helper.py** that takes a list of all our processed speeches and returns a list of lists where each inner list is a sentence and each item in the inner list is a word token.

Call `merge_speeches()` on `processed_speeches` and save the result to `all_sentences`.

Our data is now processed and ready for some analysis!

You can call some function `my_func()` with an argument `my_arg` as follows:

```
result = my_func(my_arg)
```

Copy to Clipboard

Replace `my_func()` with `merge_speeches()` and `my_arg` with `processed_speeches` in order to merge all the speeches into one big list of sentences.

All Presidents

6.

To get a better understanding of the data, let's take a look at the most frequently used words across all the inaugural addresses.

Imported from **president_helper.py** is a function `most_frequent_words()`, which uses the `Counter()` function from Python's `collections` module to find the most common words in a list of lists, where each item in the inner list is a word token.

Call `most_frequent_words()` with `all_sentences` as an argument and save the result to `most_freq_words`. Print `most_freq_words` to the terminal.

Once you have taken a look at the list of the commonly used words, you can comment out your print statement so your output terminal doesn't display the words each time you run your code.

You can call some function `my_func()` with an argument `my_arg` as follows:

```
result = my_func(my_arg)
```

Copy to Clipboard

Replace `my_func()` with `most_frequent_words()` and `my_arg` with `all_sentences` to find the most commonly used words by presidents in their inaugural addresses.

7.

Finally, it's word embedding time! Create a word embedding model with `gensim` using the following function and keyword arguments:

```
gensim.models.Word2Vec(_____, size=96, window=5, min_count=1, workers=2, sg=1)
```

Copy to Clipboard

Save the model to a variable named `all_prez_embeddings`.

Replace the _____ with your variable containing all the inaugural address sentences.

You can build a `gensim` word embedding model on some `corpus` with the `gensim.models.Word2Vec()` function as follows:

```
my_embedding_model = gensim.models.Word2Vec(corpus, size=96, window=5, min_count=1, workers=2, sg=1)
```

Copy to Clipboard

Make sure to update the value for `corpus` to your variable containing all the inaugural address sentences, `all_sentences`.

8.

Now that we have our word embeddings, let's have some fun exploring them! The concept of "freedom" is prevalent in the speeches made by the presidents. Find the top 20 words that are used in similar contexts to "freedom", and save the results to a variable named `similar_to_freedom`.

Print `similar_to_freedom` to the terminal.

To find the words with "similar" word vectors to a given word, you can use the `gensim` word embedding model method `.most_similar()`, as follows:

```
similar_to_my_word = my_word_embedding_model.most_similar("my_word", topn=20)
```

Copy to Clipboard

Replace `"my_word"` with the word for which you want to find similar word embeddings.

9.

What other words in the corpus of inaugural addresses do you want to analyze? Pick a word from `most_freq_words` and find other words that are used similarly. Are you surprised by the words that are used in the same contexts?

One President

10.

A fun aspect of word embeddings is to see how different corpora result in different word embeddings, alluding to differences in how words are used between writers/authors/speakers.

Let's train a word embedding model on a single president and see how their word embeddings differ from the collection of all presidents.

Provided in **script.py** is a function `get_president_sentences()` that takes a president's name as a string argument and returns a list of processed sentences from every inaugural address given by the president.

Call `get_president_sentences()` with `"franklin-d-roosevelt"` as an argument and save the result to `roosevelt_sentences`.

The function `get_president_sentences()` accomplishes most of the work we performed using multiple functions in previous tasks.

11.

To get a better understanding of President Franklin D Roosevelt's speeches, let's take a look at the most frequently used words across his inaugural addresses.

Call `most_frequent_words()` with `roosevelt_sentences` as an argument and save the result to `roosevelt_most_freq_words`. Print `roosevelt_most_freq_words` to the terminal.

Once you have taken a look at the list of the commonly used words, you can comment out your print statement so your output terminal doesn't display the words each time you run your code.

You can call some function `my_func()` with an argument `my_arg` as follows:

```
result = my_func(my_arg)
```

Copy to Clipboard

Replace `my_func()` with `most_frequent_words()` and `my_arg` with `roosevelt_sentences` to find the most commonly used words by President Roosevelt in his inaugural addresses.

12.

Now that we have the sentences from President Roosevelt's speeches, create another word embedding model with `gensim`, using the same keyword arguments as in the earlier word embedding model you created, trained on just the inaugural address sentences for President Roosevelt. Save the embeddings to a variable named `roosevelt_embeddings`.

You can build a `gensim` word embedding model trained on some `corpus` with the `gensim.models.Word2Vec()` function as follows:

```
my_embedding_model = gensim.models.Word2Vec(corpus, size=96, window=5,
min_count=1, workers=2, sg=1)
```

Copy to Clipboard

Make sure to update the value for `corpus` to your variable containing all the inaugural address sentences for President Roosevelt, `roosevelt_sentences`.

13.

Like with our previous word embedding model, let's explore `roosevelt_embeddings`! Find the top 20 words that are used in similar contexts to "freedom", and save the results to a variable named `roosevelt_similar_to_freedom`.

Print `roosevelt_similar_to_freedom` to the terminal.

How do the words similar to "freedom" in President Roosevelt's embeddings compare to the words similar to "freedom" in the embeddings of all the presidents' speeches?

Are there any surprises?

To find the words with "similar" word vectors to a given word, you can use the `gensim` word embedding model method `.most_similar()`, as follows:

```
similar_to_my_word = my_word_embedding_model.most_similar("my_word", topn=20)
```

Copy to Clipboard

Replace `"my_word"` with the word for which you want to find similar word embeddings.

The results you see from calling `roosevelt_embeddings.most_similar("freedom", topn=20)` are less than satisfying (many of the words with similar embeddings are stop words, or commonly used words that give little insight into topic or context). This is because even for President Roosevelt, who gave the largest number of inaugural addresses, there is not enough data to produce robust word embeddings.

In the next section, we will explore looking at the speeches of multiple presidents to increase our corpus size and produce better word embeddings.

Selection of Presidents

14.

You may have noticed that the results from `roosevelt_embeddings.most_similar("freedom", topn=20)` are less than satisfying. This is because we were working with a limited dataset, producing less robust and generalizable word embeddings.

Let's increase our corpus size and find more defined word embeddings by training a word embedding model on the inaugural addresses of a collection of multiple presidents.

Provided in **script.py** is a function `get_presidents_sentences()` that takes a list of multiple presidents' names as an argument and returns a list of processed sentences from every inaugural address given by the group of presidents.

Call `get_presidents_sentences()` with `["washington", "jefferson", "lincoln", "theodore-roosevelt"]` as an argument and save the result to `rushmore_prez_sentences`.

The function `get_presidents_sentences()` accomplishes most of the preprocessing work we performed using multiple functions in previous tasks.

The four presidents featured on [Mount Rushmore in Keystone, South Dakota](#) are George Washington, Thomas Jefferson, Theodore Roosevelt, and Abraham Lincoln.

15.

To get a better understanding of President Washington, Jefferson, Lincoln, and Theodore Roosevelt's speeches, let's take a look at the most frequently used words across their inaugural addresses.

Call `most_frequent_words()` with `rushmore_prez_sentences` as an argument and save the result to `rushmore_most_freq_words`. Print `rushmore_most_freq_words` to the terminal.

Once you have taken a look at the list of the commonly used words, you can comment out your print statement so your output terminal doesn't display the words each time you run your code.

You can call some function `my_func()` with an argument `my_arg` as follows:

```
result = my_func(my_arg)
```

Copy to Clipboard

Replace `my_func()` with `most_frequent_words()` and `my_arg` with `rushmore_prez_sentences` to find the most commonly used words across President Washington, Jefferson, Lincoln and Roosevelt's inaugural addresses.

16.

Now that we have the sentences from the presidents featured on Mount Rushmore, create another word embedding model with `gensim`, using the same keyword arguments as in the earlier word embedding model you created, trained on just the inaugural address sentences for these presidents. Save the embeddings to a variable named `rushmore_embeddings`.

You can build a `gensim` word embedding model trained on some `corpus` with the `gensim.models.Word2Vec()` function as follows:

```
my_embedding_model = gensim.models.Word2Vec(corpus, size=96, window=5, min_count=1, workers=2, sg=1)
```

Copy to Clipboard

Make sure to update the value for `corpus` to your variable containing all the inaugural address sentences for the presidents featured on Mount Rushmore, `rushmore_prez_sentences`.

17.

Like with our previous word embedding models, let's explore `rushmore_embeddings`! Find words that are used in similar contexts to "freedom", and save the results to a variable named `rushmore_similar_to_freedom`.

How do the words similar to “freedom” in these presidents’ embeddings compare to the words similar to “freedom” in the embeddings of all the presidents’ speeches? And to just Franklin D Roosevelt’s?

Are there any surprises?

To find the words with “similar” word vectors to a given word, you can use the `gensim` word embedding model method `.most_similar()`, as follows:

```
similar_to_my_word = my_word_embedding_model.most_similar("my_word", topn=20)
```

Copy to Clipboard

Replace `"my_word"` with the word for which you want to find similar word embeddings.

18.

What other words in the corpus of Mount Rushmore presidents’ inaugural addresses do you want to analyze? Pick a word from `rushmore_most_freq_words` and find other words that are used similarly. Are you surprised by the words that are used in the same contexts?

19.

Who are your favorite presidents? Do you want to see how their choice of words compares to presidents you view less favorably? Or controversial presidents?

Perhaps you want to analyze presidents by political party affiliation?

Create a new word embedding model trained on a corpus of sentences from the speeches of a selection of presidents that you decide. Find the words used similarly to “freedom”, and explore the embeddings of other words in the corpus.

How do the embeddings from your model compare to the other models you have built? What surprises do you find?

```
import os
import gensim
import spacy
from president_helper import read_file, process_speeches, merge_speeches,
get_president_sentences, get_presidents_sentences, most_frequent_words

# get list of all speech files

# read each speech file

# preprocess each speech

# merge speeches

# view most frequently used words

# create gensim model of all speeches

# view words similar to freedom
```

```

# get President Roosevelt sentences

# view most frequently used words of Roosevelt

# create gensim model for Roosevelt

# view words similar to freedom for Roosevelt

# get sentences of multiple presidents

# view most frequently used words of presidents

# create gensim model for the presidents

# view words similar to freedom for presidents

```

PRESIDENT_HELPER.py

```

import os
from nltk.tokenize import PunktSentenceTokenizer
from collections import Counter

def read_file(file_name):
    with open(file_name, 'r+', encoding='utf-8') as file:
        file_text = file.read()
    return file_text

def process_speeches(speeches):
    word_tokenized_speeches = list()
    for speech in speeches:
        sentence_tokenizer = PunktSentenceTokenizer()
        sentence_tokenized_speech = sentence_tokenizer.tokenize(speech)
        word_tokenized_sentences = list()
        for sentence in sentence_tokenized_speech:
            word_tokenized_sentence = [word.lower().strip('.').strip('?').strip('!') for word in
            sentence.replace(",","").replace("-"," ").replace(":","").split()]
            word_tokenized_sentences.append(word_tokenized_sentence)
        word_tokenized_speeches.append(word_tokenized_sentences)
    return word_tokenized_speeches

def merge_speeches(speeches):
    all_sentences = list()
    for speech in speeches:
        for sentence in speech:
            all_sentences.append(sentence)
    return all_sentences

def get_president_sentences(president):
    files = sorted([file for file in os.listdir() if president.lower() in file.lower()])
    speeches = [read_file(file) for file in files]
    processed_speeches = process_speeches(speeches)
    all_sentences = merge_speeches(processed_speeches)
    return all_sentences

def get_presidents_sentences(presidents):

```

```
all_sentences = list()
for president in presidents:
    files = sorted([file for file in os.listdir() if president.lower() in file.lower()])
    speeches = [read_file(file) for file in files]
    processed_speeches = process_speeches(speeches)
    all_prez_sentences = merge_speeches(processed_speeches)
    all_sentences.extend(all_prez_sentences)
return all_sentences

def most_frequent_words(list_of_sentences):
    all_words = [word for sentence in list_of_sentences for word in sentence]
    return Counter(all_words).most_common()
```

Files = <https://www.codecademy.com/paths/data-science-nlp/tracks/nlp-language-quantification/modules/nlp-word-embeddings/projects/word-embeddings-presidential>

Source / download = https://avalon.law.yale.edu/subject_menus/inaug.asp

Data Scientist: Natural Language Processing Specialist

Echo Chatbot

Even with just a little bit of Python knowledge, you can build a small chatbot! In this project, you'll apply your knowledge of:

- variables
- string concatenation
- output
- user input

to create "Echo Chatbot," a chatbot that echoes whatever a user inputs.

Tasks

0/8 complete

[Mark the tasks as complete by checking them off](#)

Introductions are in order...

1.

First, print out a greeting message for the user, welcoming them to your first chatbot.

For example:

```
print("Welcome to Koma's first chatbot!")
```

Copy to Clipboard

2.

Now that you've welcomed the user, explain what they can expect to happen: the chatbot will repeat what they say.

For example:

```
print("I will repeat after you.")
```

Copy to Clipboard

3.

After saving your code, test your program so far to make sure everything is working as it should.

To do this, enter the following in the bash terminal:

```
python3 script.py
```

Copy to Clipboard

This tells the terminal to run a Python 3 program in the file **script.py**.

Did both of your messages print out?

If you used the example statements from the hints, you will see:

```
Welcome to Koma's first chatbot!  
I will repeat after you.
```

Copy to Clipboard

Asking for input

4.

Prompt your user to enter something for the program to repeat.

Assign the user's response to a new variable called `stuff_to_echo`.

For example:

```
stuff_to_echo = input("Enter something for me to repeat: ")
```

Copy to Clipboard

5.

After saving your code, test out your program:

```
python3 script.py
```

Copy to Clipboard

Make sure:

Everything is still working (there are no errors thrown).

The program prompts you for input as you expect it to.

Echo time!

6.

Print out `"You said: "` followed by the user input you collected.

Use string concatenation to combine `"You said: "` with your variable.

7.

Test everything out again!

```
python3 script.py
```

Copy to Clipboard

Does your Echo Chatbot work as you expect it to?

Wrapping up

8.

Congratulations on building your very first chatbot!

In creating Echo Chatbot, you're already implementing some patterns that you'll use as you develop more complex chatbot programs:

Greeting a user.

Informing the user that they are interacting with a bot (and not a human).

Instructing the user on how to interact with the bot.

Accepting user input.

Setting up output that is dependent on user input.

Want to extend this project further? Try adding more lines of input and output to craft a funny sort of conversation. You can also ask for the user's name and then use it in subsequent outputs.

Data Scientist: Natural Language Processing Specialist

Coffee Chatbot

But first, coffee.

Whether it's to get us ready to jump-start our day or to get us through a late-night cram session, many of us need our regular caffeine fix! Ordering coffee is just one example of a process that can be automated with the help of a chatbot.

You're given the task of creating a Python chatbot that can help cut the wait time of a normal coffee run by taking customers' orders in advance. Write your code in the file called **script.py** and run it by entering `python3 script.py` in the terminal.

Let's get started!

Tasks

0/14 complete

[Mark the tasks as complete by checking them off](#)

1.

Define a function called `coffee_bot()` that prints out the statement:

```
Welcome to the cafe!
```

Copy to Clipboard

At the bottom of the script, call your function. Run the script to see the greeting!

As you progress through the project, feel free to run the script at any point to see your changes. Don't forget to click the **Save** button first.

Define `coffee_bot()`:

```
def coffee_bot():  
    print('Welcome to the cafe!')
```

Copy to Clipboard

Call `coffee_bot()`:


```
coffee_bot()
```

Copy to Clipboard

2.

Next, define a function called `get_size()` that will ask the user for the size of their drink. Use the `input()` function to get the user input and store the response in a variable called `res`:

```
res = input('What size drink can I get for you? \n[a] Small \n[b] Medium \n[c] Large \n> ')
```

Copy to Clipboard

Return `res` from the function. It should contain the letter corresponding to the selection.

Inside your `coffee_bot()` definition, call the `get_size()` function and store the returned value in a variable called `size`. Print out `size` to see the value!

Define `get_size()`:

```
def get_size():  
    res = input('What size drink can I get for you? \n[a] Small \n[b] Medium \n[c] Large \n> ')  
  
    return res
```

Copy to Clipboard

Call `get_size()` inside `coffee_bot()`:

```
def coffee_bot():  
    print('Welcome to the cafe!')  
  
    size = get_size()  
  
    print(size)
```

Copy to Clipboard

3.

Now, update the `get_size()` function so that it returns the selected size instead of the letter corresponding to the user's selection.

Specifically, delete your previous return statement and add conditional statements such that:

If `res` is equal to `'a'`, return the string `'small'`
Else if `res` is equal to `'b'`, return the string `'medium'`
Else if `res` is equal to `'c'`, return the string `'large'`

Try running your script now and see what gets printed out.

Add the conditional statements to `get_size()`:

```
def get_size():  
  
    res = input('What size drink can I get for you? \n[a] Small \n[b]  
Medium \n[c] Large \n> ')  
  
    if res == 'a':  
        return 'small'  
  
    elif res == 'b':  
        return 'medium'  
  
    elif res == 'c':  
        return 'large'
```

Copy to Clipboard

4.

You may have noticed that if the user entered an input other than `'a'`, `'b'`, or `'c'`, `None` gets printed out. This is because we have not accounted for these kinds of responses in the conditional statements, and by default, a function will return `None`.

Let's fix this! Add an `else` block to your conditional statements. Inside this block, call the `get_size()` function and return its returned value. This will continue prompting the user for a size selection until the user's response matches one of our expected inputs.

Notice that the `get_size()` function is now calling itself, making it a *recursive function*. See [this exercise on recursion](#) to learn more about recursive functions.

Save your code and run the script by entering `python3 script.py` in the terminal to see it in action!

Add the `else` block to `get_size()` that will make it a *recursive function*:

```
def get_size():  
  
    res = input('What size drink can I get for you? \n[a] Small \n[b] Medium \n[c] Large \n> ')  
  
    if res == 'a':  
        return 'small'  
    elif res == 'b':  
        return 'medium'  
    elif res == 'c':  
        return 'large'  
    else:  
        return get_size()
```

Copy to Clipboard

5.

The `get_size()` function is looking great! Lastly, just to make it slightly more user friendly, let's have it print out a message in the `else` block before it prompts the user for an input again.

First, define a separate function called `print_message()` that prints the message:

```
I'm sorry, I did not understand your selection. Please enter the  
corresponding letter for your response.
```

Copy to Clipboard

Then, inside the `else` block of your `get_size()` function, call the `print_message()` function right before the return statement.

Define `print_message()`:

```
def print_message():  
    print('I\'m sorry, I did not understand your selection. Please enter  
the corresponding letter for your response.')
```

Copy to Clipboard

Call `print_message()` inside the *else* block of `get_size()`:

```
def get_size():  
    res = input('What size drink can I get for you? \n[a] Small \n[b]  
Medium \n[c] Large \n> ')  
  
    if res == 'a':  
        return 'small'  
    elif res == 'b':  
        return 'medium'  
    elif res == 'c':  
        return 'large'  
    else:  
        print_message()  
        return get_size()
```

Copy to Clipboard

6.

Next up, define a function called `get_drink_type()` that will ask the user for their drink order:

```
What type of drink would you like?
```

```
[a] Brewed Coffee
```

```
[b] Mocha
```

```
[c] Latte
```

Copy to Clipboard

Use the `input()` function to get the user input and store the response in a variable called `res`. Return `res` for now.

Inside your `coffee_bot()` definition, under your call to `get_size()`, call the `get_drink_type()` function and store the returned value in a variable called `drink_type`. Print out `drink_type` to see the value!

Define `get_drink_type()`:

```
def get_drink_type():  
  
    res = input('What type of drink would you like? \n[a] Brewed Coffee  
\n[b] Mocha \n[c] Latte \n> ')  
  
    return res
```

Copy to Clipboard

Call `get_drink_type()` inside `coffee_bot()`:

```
def coffee_bot():  
  
    print('Welcome to the cafe!')  
  
  
    size = get_size()  
    print(size)  
  
  
    drink_type = get_drink_type()  
    print(drink_type)
```

Copy to Clipboard

7.

Similar to `get_size()`, delete your previous return statement in `get_drink_type()` and add conditional statements such that:

If `res` is equal to `'a'`, return the string `'brewed coffee'`

Else if `res` is equal to `'b'`, return the string `'mocha'`

Else if `res` is equal to `'c'`, return the string `'latte'`

Otherwise, call `print_message()` and return the value that is returned from calling

`get_drink_type()`

Add the conditional statements to `get_drink_type()`:

```
def get_drink_type():  
    res = input('What type of drink would you like? \n[a] Brewed Coffee  
\n[b] Mocha \n[c] Latte \n> ')  
  
    if res == 'a':  
        return 'brewed coffee'  
    elif res == 'b':  
        return 'mocha'  
    elif res == 'c':  
        return 'latte'  
    else:  
        print_message()  
        return get_drink_type()
```

Copy to Clipboard

8.

Now, let's take it a step further! If the user orders a latte, we'll ask them what kind of milk they would like.

Define a function called `order_latte()` that prompts the user:

And what kind of milk for your latte?

[a] 2% milk

[b] Non-fat milk

[c] Soy milk

Copy to Clipboard

Use the `input()` function to get the user input and store the response in a variable called `res`.

Define `order_latte()`:

```
def order_latte():  
  
    res = input('And what kind of milk for your latte? \n[a] 2% milk  
\n[b] Non-fat milk \n[c] Soy milk \n> ')
```

Copy to Clipboard

9.

Add conditional statements to your `order_latte()` function such that:

If `res` is equal to `'a'`, return the string `'latte'`

Else if `res` is equal to `'b'`, return the string `'non-fat latte'`

Else if `res` is equal to `'c'`, return the string `'soy latte'`

Otherwise, call `print_message()` and return the value that is returned from calling `order_latte()`

Add the conditional statements to `order_latte()`:

```
def order_latte():  
  
    res = input('And what kind of milk for your latte? \n[a] 2% milk  
\n[b] Non-fat milk \n[c] Soy milk \n> ')  
  
    if res == 'a':  
        return 'latte'  
  
    elif res == 'b':  
        return 'non-fat latte'  
  
    elif res == 'c':  
        return 'soy latte'  
  
    else:
```

```
print_message()

return order_latte()
```

Copy to Clipboard

10.

In `get_drink_type()`, instead of returning the string `'latte'` if the user selects latte as their drink of choice, return the value that is returned from calling `order_latte()`.

Try running your script and see what gets printed out when you select latte as your drink!

Update `get_drink_type()` to call `order_latte()` if a latte is ordered:

```
def get_drink_type():

    res = input('What type of drink would you like? \n[a] Brewed Coffee\n[b] Mocha \n[c] Latte \n> ')

    if res == 'a':

        return 'brewed coffee'

    elif res == 'b':

        return 'mocha'

    elif res == 'c':

        return order_latte()

    else:

        print_message()

        return get_drink_type()
```

Copy to Clipboard

11.

Now, let's put it all together! In `coffee_bot()`, remove any print statements you currently have besides the welcome greeting.

Then, add a print statement at the very end that outputs:


```
Alright, that's a {size} {drink_type}!
```

Copy to Clipboard

Update `coffee_bot()` to print the order:

```
def coffee_bot():  
    print('Welcome to the cafe!')  
  
    size = get_size()  
    drink_type = get_drink_type()  
    print('Alright, that\'s a {} {}'.format(size, drink_type))
```

Copy to Clipboard

12.

Next, prompt the user for their name:

```
Can I get your name please?
```

Copy to Clipboard

Use the `input()` function to get the user input and store the response in a variable called `name`.

Prompt the user for their name:

```
def coffee_bot():  
    print('Welcome to the cafe!')  
  
    size = get_size()  
    drink_type = get_drink_type()  
    print('Alright, that\'s a {} {}'.format(size, drink_type))  
  
    name = input('Can I get your name please? \n> ')
```

Copy to Clipboard

13.

Finally, print out a statement that says:

```
Thanks, {name}! Your drink will be ready shortly.
```

Copy to Clipboard

Order complete!

Print final statement confirming the order:

```
def coffee_bot():  
    print('Welcome to the cafe!')  
  
    size = get_size()  
    drink_type = get_drink_type()  
    print('Alright, that\'s a {} {}'.format(size, drink_type))  
  
    name = input('Can I get your name please? \n> ')  
    print('Thanks, {}! Your drink will be ready shortly.'.format(name))
```

Copy to Clipboard

Bonus Task

14.

Try adding some additional functionality to the chatbot. For example, how can we ask the user if they would like a plastic cup or to use their own reusable cup? How about if they'd like their drink hot or iced? Or maybe if they'd want to order an additional drink?

Feel free to get creative!

To ask the user if they'd like a plastic cup or if they brought a reusable one, we could define a new function:

```
def extra_options():
```

```
res = input('Would you like a plastic cup or did you bring your  
own reusable cup? \n[a] I\'ll need a cup. \n[b] Brought my own!  
\n> ')
```

```
if res == 'a':  
    print('Okay, no problem! We\'ll get you a plastic cup.')  
elif res == 'b':  
    print('Great! We\'ll fill up your reusable cup.')  
else:  
    print_message()  
    return extra_options()
```

Copy to Clipboard

Then, we can call this function in `coffee_bot()`:

```
def coffee_bot():  
    print('Welcome to the cafe!')  
  
    size = get_size()  
    drink_type = get_drink_type()  
    print('Alright, that\'s a {} {}!'.format(size, drink_type))  
  
    extra_options()  
  
    name = input('Can I get your name please? \n> ')  
    print('Thanks, {}! Your drink will be ready  
shortly.'.format(name))
```

Data Scientist: Natural Language Processing Specialist

Alienbot

The 1950s ushered in a slew of UFO sightings and excitement about outer space. Incidentally, the decade also brought about a new idea of artificial intelligence. The computer scientist Alan Turing proposed that if a computer was able to fool a human into believing it was human, the computer would be considered “intelligent.”

While your bot may not pass a true Turing test, perhaps it could pass an “E.T.uring” test if you can successfully fool a human into believing that your chatbot is, in fact, an alien.

We’ve added a class called `AlienBot` to get you started; you’ll be digging into the main chatbot functionality. Take a look at what you have in **script.py**:

- the `regex` and `random` libraries are imported
- a tuple of negative responses (you can add more!)
- a tuple of commands for a user to exit the program
- a tuple of random starter questions an alien might ask (feel free to add more)
- a tuple of dictionaries called `alienbabble` with the chatbot’s main intent-utterance matching pairs — you’ll add to this later!

several methods for core chatbot functionality:

- `.greet()` will greet the user
- `.make_exit()` will check if a user has used an exit command
- `.chat()` will run the program allowing users to chat with the alien you created
- `.match_reply()` will match the response pairs in `alienbabble` for you

several methods associated with user intents

- `.describe_planet_intent()` will include responses about the alien’s planet
- `.answer_why_intent()` will include responses for why the alien is visiting
- `.cubed_intent()` is a method that returns the cube of a number, because these aliens are very good at math
- `.no_match_intent()` will respond with something general when none of the other intents are matched

As you add to the chatbot, you can check the functionality by:

Pressing the **Save** button

Running the following from the terminal, with:

```
python3 script.py
```

Copy to Clipboard

Tasks

31/31 complete

Mark the tasks as complete by checking them off

Greeting the user

1.

The first step in any chatbot program is to greet the user. Over the next few steps, you will call the `.greet()` method and program the chatbot to welcome a user.

Inside `.greet()`, ask the user for their name with an input method and assign it to a `self.name` variable.

Stuck on how to assign `name` using `input()`? Try this:

```
self.name = input("Hello there, what's your name? ")
```

Copy to Clipboard

2.

Below the previous `input`, create a `will_help` variable and assign it the user's response to the following string:

```
"Hi NAME, I'm Etcetera. I'm not from this planet. Will you help me learn about your planet? "
```

Copy to Clipboard

In this example, NAME is the value saved to `self.name`.

You can use an f-string, like the one below, to include the user's name in the statement:

```
my_variable = "print"
print(f"This is how you interpolate a variable's value with an f-string: {my_variable}.")
```

Copy to Clipboard

3.

Next, we want to see if the user even wants to chat. Create a conditional that checks if the

`will_help` response is in `self.negative_responses`. If it is, then we need to:

Print a goodbye statement to the terminal (we suggest "Ok, have a nice Earth day!")

Return from the function

For the condition, you can check if `will_help` is in `self.negative_responses` with the following syntax:

```
if will_help in self.negative_responses:  
    # do stuff here
```

Copy to Clipboard

Inside of the `if` block and after printing the goodbye statement, you need to add a `return`.

4.

If the user didn't respond in the negative, we know the user is interested in chatting. Outside the `if` statement, call `self.chat()`. The `.chat()` method will be responsible for handling the conversation.

5.

Let's test `.greet()` out!

At the bottom of `script.py`, outside of the `AlienBot` class, create an instance of the `AlienBot` class. Then call the `.greet()` method.

In the terminal enter `python3 script.py`. See if you can have the following two conversations:

Yes, you do want to have a conversation:

```
Hello there, what's your name? Codecademy  
Hi Codecademy, I'm Etcetera. I'm not from thisplanet. Will you  
help me learn about your planet? yes
```

Copy to Clipboard

No, you do not want to have a conversation:

```
Hello there, what's your name? Codecademy  
Hi Codecademy, I'm Etcetera. I'm not from this planet. Will you  
help me learn about your planet? no  
Ok, have a nice Earth day!
```

Copy to Clipboard

You can create an instance of `AlienBot` with the following:

```
my_bot = AlienBot()
```

Copy to Clipboard

Then, you can call the `.greet()` method:

```
my_bot.greet()
```

Copy to Clipboard

Remember, if everything is set up correctly, when you say you won't help, you should get a goodbye statement.

Handle the conversation

6.

In the next few tasks, you will create a loop that continues the conversation until the user replies with an exit command.

Let's start by asking the user a random question in the `.chat()` method. You can do this with the following line of code:

```
reply = input(random.choice(self.random_questions)).lower()
```

Copy to Clipboard

This code will randomly select a question from `random_questions`, ask it to the user, then save the user's response.

7.

Before setting up the loop in `.chat()`, let's set up a stop condition for the loop.

Inside of the `.make_exit()` method, you will check if the user wants to stop talking to the chatbot.

First, use a `for` loop to iterate over the words in `self.exit_commands`.

The following is the syntax for iterating over a tuple:

```
my_tuple = ("String 1", "String 2", "String 3")

for my_string in my_tuple:
    ...
```

Copy to Clipboard

8.

Inside the `for` loop, create an `if` statement to check if the current exit command is in `reply`.

Inside the `if` statement, print "Ok, have a nice Earth day!" and then return `True`.

The following loop will check if `string_to_check` is in any of the strings from `my_tuple`.

```
my_tuple = ("String 1", "String 2", "String 3")
string_to_check = "This will match String 1"

for my_string in my_tuple:
    if my_string in string_to_check:
        ...
```

Copy to Clipboard

9.

Jump back inside the `.chat()` method.

Below where you set `reply`, create a `while` loop that continues for as long as the `.make_exit()` method returns `False`. Include the following features in this loop:

The `.make_exit()` method should accept the user's response, saved to the `reply` variable.

Inside the loop, create an `input()` function that asks the user “How are you?” and saves the response to `reply` (we’ll change this later).

You can create a while loop with the following syntax:

```
while CONDITION:  
    ...
```

Copy to Clipboard

Use the following condition to continue looping as long as the `.make_exit()` method returns false:

```
not self.make_exit(reply)
```

Copy to Clipboard

Inside the `while` loop, use the following to save the user’s response to “How are you?”:

```
reply = input("How are you? ")
```

Copy to Clipboard

10.

Now, let’s try testing your code. Press the Save button, then type `python3 script.py`.

At this point, you should be able to recreate the following conversation:

```
Hello there, what's your name? Codecademy  
Hi Codecademy, I'm Etcetera. I'm not from this planet. Will you  
help me learn about your planet? yes  
Why are you here? To talk with you  
How are you? I'm okay  
How are you? I'm okay  
How are you? exit  
Ok, have a nice Earth day!
```

Copy to Clipboard

Remember, you can always exit the conversation using one of the exit commands.

Matching the user's response

11.

Now, we need to setup the functionality to match a user’s response to an intent. Over the next few steps, you will:

- Call the `self.match_reply()` method inside of `.chat()`.

- Loop over the intent-regular expression pairs in the `alienbabble` dictionary.

- Check if the user’s response matches a regular expression in the `alienbabble` dictionary.

Let’s get started. Inside `.chat()`, replace “How are you?” with a call to `self.match_reply()`.

(This is still inside `input()`.)

Pass the `reply` variable into the method.

Inside the `while` loop:

```
while not self.make_exit(reply):  
    reply = input("How are you? ")
```

Copy to Clipboard

replace `input("How are you? ")` with the following:

```
input(self.match_reply(reply))
```

Copy to Clipboard

12.

Inside of `.match_reply()`, loop over all the items in `self.alienbabble`. For each item, save the key to a variable called `intent` and the value to a variable called `regex_pattern`.

You can use the following code to loop over each key-value pair in a dictionary:

```
my_dict = {'key 1': 'value 1', 'key 2': 'value 2'}  
  
for key, value in my_dict.items():  
    ...
```

Copy to Clipboard

13.

Inside of the `for` loop, use the regular expression `.match()` method to check the user's reply for the `regex_pattern`. Save the result to a variable called `found_match`.

At this point, there are no regular expressions in the `alienbabble` variable. You will add them in a few steps.

Use the `re.match()` method as follows to match a regular expression to the user's response:

```
regex = r'.*text.*'  
reply = r'This is text'  
found_match = re.match(regex, reply)
```

Copy to Clipboard

The describe_planet_intent

14.

Over the next several steps, you are going to create a long `if-elif-else` conditional block to select the corresponding intent when a match is found.

After the line with `re.match()`, write a conditional statement that checks if a match was found and if the current `intent` is `'describe_planet_intent'`.

Inside the `if` block, return `self.describe_planet_intent()`.

The condition for this `if` statement should be:

```
found_match and intent == 'describe_planet_intent'
```

Copy to Clipboard

15.

Next, locate the `AlienBot` constructor method and add a regular expression to

`self.alienbabble` that matches the `'describe_planet_intent'` key.

Write a regular expression that matches the following two statements:

"Can you tell me about your planet?"

"I am interested in your planet."

The regular expression should not match:

"Why are you here?"

"I think I am the only human who has met you."

"Can you cube the number 3?"

"Why are you asking me so many questions?"

Scroll to the `self.alienbabble` dictionary:

```
self.alienbabble = {'describe_planet_intent': r'',
                    'answer_why_intent': r'',
                    'cubed_intent': r'',
                    'unsure_response_intent': r''
                    }
```

Copy to Clipboard

For the value that matches `'describe_planet_intent'`, add a regular expression like:

```
r'.*\s*your planet.*'
```

Copy to Clipboard

16.

At this point, you can check if your regular expression worked. Run `python3 script.py` and see if you can get the chatbot to respond with, `"Inside .describe_planet_intent()"`.

One example may look like:

```
Hello there, what's your name? Codecademy
Hi Codecademy, I'm Etcetera. I'm not from this planet. Will you
help me learn about your planet? yes
What do you consume for sustenance? tell me about your planet
Inside describe_planet_intent
```

Copy to Clipboard

Scroll to the `self.alienbabble` dictionary:

```
self.alienbabble = {'describe_planet_intent': r'',
                    'answer_why_intent': r'',
                    'cubed_intent': r'',
                    'unsure_response_intent': r''
                    }
```

Copy to Clipboard

For the value that matches 'describe_planet_intent', add the regular expression:

```
r'.*\s*your planet.*'
```

Copy to Clipboard

17.

Inside `.describe_planet_intent()`, we want to answer with a response about the alien's planet. Delete the current return statement.

In place, create a tuple called `responses` with the following two strings:

```
"My planet is a utopia of diverse organisms and species. "  
"I am from Opidipus, the capital of the Wayward Galaxies. "
```

You can create a tuple of strings using the following syntax:

```
some_tuple = ("oh hi there", "i love cheese")
```

Copy to Clipboard

18.

Next, return one of the strings in the `responses` tuple. You can randomly select a response using the following syntax:

```
random.choice(responses)
```

Copy to Clipboard

The answer_why_intent

19.

Under the `if` block, add an `elif` that checks if a match was found and if the current `intent` is 'answer_why_intent'.

Inside the `elif` block return `self.answer_why_intent()`.

The condition for this `elif` statement should be:

```
found_match and intent == 'answer_why_intent'
```

Copy to Clipboard

20.

Next, you need to add a regular expression to `self.alienbabble` that matches the 'answer_why_intent' key.

Write a regular expression that matches the following two statements:

```
"Why are you here?"  
"Why are you asking me so many questions?"
```

The regular expression should not match:

```
"Can you cube the number 3?"  
"Can you tell me about your planet?"  
"I think I am the only human who has met you."
```

"I am interested in your planet."

Scroll to the `self.alienbabble` dictionary. For the value that matches `'answer_why_intent'`, add a regular expression like:

```
r'why\sare.*'
```

Copy to Clipboard

21.

At this point, you can check if your regular expression worked. Run `python3 script.py` and see if you can get the chatbot to respond with, "Inside `.answer_why_intent()`".

One example may look like:

```
Hello there, what's your name? Codecademy
Hi Codecademy, I'm Etcetera. I'm not from this planet. Will you
help me learn about your planet? yes
What do you consume for sustenance? why are you here?
Inside answer_why_intent
```

Copy to Clipboard

Remember, you can exit the conversation at any time using an exit command.

22.

Inside `.answer_why_intent()`, we want to answer with a response about why the alien is visiting. Create a tuple called `responses` with the following strings:

"I come in peace. "

"I am here to collect data on your planet and its inhabitants. "

"I heard the coffee is good. "

You can make a tuple with the following syntax:

```
my_tuple = ("String 1", "String 2")
```

Copy to Clipboard

23.

Next, return one of the strings in the `responses` tuple. You can randomly select a response using the following syntax:

```
random.choice(responses)
```

Copy to Clipboard

The cubed_intent

24.

This alien chatbot is also capable of doing some math. Over the next few steps, you will create an intent that takes a user's input, with a number, and cubes it.

Under the last `elif` block, add an `elif` that checks if a match was found and if the current `intent` is `'cubed_intent'`.

Time for some entity recognition.

Inside the `elif` block return the output from `self.cubed_intent()`, and pass the first group from `found_match` as an argument. (This group should have the number the user wants cubed – our entity!)

The condition for this `elif` statement should be:

```
found_match and intent == 'cubed_intent'
```

Copy to Clipboard

You can access the first group from the match (the number entity) using the following syntax:

```
found_match.groups()[0]
```

Copy to Clipboard

25.

Next, you need to add a regular expression to `self.alienbabble` that matches the `'cubed_intent'` key.

Write a regular expression that matches the following two statement:

“Can you cube the number 3?”

The regular expression should not match:

“Why are you here?”

“Why are you asking me so many questions?”

“Can you tell me about your planet?”

“I think I am the only human who has met you.”

“I am interested in your planet.”

Scroll to the `self.alienbabble` dictionary. For the value that matches `'cubed_intent'`, add a regular expression like:

```
r'.*cube.*(\d+)'
```

Copy to Clipboard

This regular expression will match any statement that includes the word “cube” and then is followed by a number, which we can capture using regex.

26.

Let’s check if your regular expression worked. Run `python3 script.py` and see if you can get the chatbot to respond with, `Inside .answer_why_intent()`.

One example may look like:

```
Hello there, what's your name? Codecademy
Hi Codecademy, I'm Etcetera. I'm not from this planet. Will you
help me learn about your planet? yes
What do you consume for sustenance? can you cube the number 5?
Inside cubed_intent
```

Copy to Clipboard

Remember, you can exit the conversation at any time using an exit command.

27.

Inside `.cubed_intent()`, we need to return the following:

```
The cube of NUMBER is CUBED_NUMBER. Isn't that cool? "
```

Copy to Clipboard

Follow these steps to set up the above string:

On the first line of the method, use the `int` function to change the value saved to `number` to an integer (right now it's a string).

On the second line, multiply `number` by itself three times, then save the response to `cubed_number`.

On the third line return an f-string to respond to the user with the string above.

Use the following code to change `number` to an integer.

```
number = int(number)
```

Copy to Clipboard

Use the following to calculate the cube of `number`:

```
cubed_number = number * number * number
```

Copy to Clipboard

Interpolate `number` and `cubed_number` into the string:

```
some_number = 28
f"You can add {some_number} into an f-string like this."
```

Copy to Clipboard

Handling responses that don't match an intent

28.

Finally, we need to handle the situation when none of the intents are matched. Under the last

`elif` statement in `.match_reply()`, add an `else`.

Inside the `else` block, return `self.no_match_intent()`.

The `else` statement should look something like:

```
...
elif (condition):
    ...
else:
    # Call self.no_match_intent()
```

Copy to Clipboard

29.

Inside the method, we want to answer with a response that is applicable to many statements. In this way, we simulate that the chatbot understands what the user is saying despite it not matching any intent.

Create a tuple with the following strings and save it to `responses`:

```
"Please tell me more. "  
"Tell me more! "  
"Why do you say that? "  
"I see. Can you elaborate? "  
"Interesting. Can you tell me more? "  
"I see. How do you think? "  
"Why? "  
"How do you think I feel when you say that? "
```

You can make a tuple with the following syntax:

```
my_tuple = ("String 1", "String 2")
```

Copy to Clipboard

30.

Next, return the response to one of the strings in the `responses` tuple. You can randomly select a response using the following syntax:

```
random.choice(responses)
```

Copy to Clipboard

Now try out your `AlienBot` and see how it does!

Adding to this chatbot

31.

Great work!

You built a chatbot that simulates a real conversation. It's unlikely that a human would be fooled by the chatbot. However, it's worth mentioning that it can be easily extended by adding conditions and intents to the `if-elif-else` block.

At this point, you've proven that you understand the basic architecture and features of a rule-based chatbot. You can build upon these skills with more sophisticated regular matching and response generation. Good luck as you continue developing these skills to create increasingly human-like bots!

Some ideas to expand the project:

- Add some more responses to your current intents.

- Try adding more intents.

Try to capture other types of entities and use them to broaden the conversation.

Add in [reflections](#).

Code Review

Get targeted feedback about your code and how well it meets the objective of the project. You can request 1 AI-powered review per project when you complete all tasks.

```
# importing regex and random libraries

import re

import random


class AlienBot:

    # potential negative responses

    negative_responses = ("no", "nope", "nah", "naw", "not a chance",
                          "sorry")

    # keywords for exiting the conversation

    exit_commands = ("quit", "pause", "exit", "goodbye", "bye", "later")

    # random starter questions

    random_questions = (

        "Why are you here? ",

        "Are there many humans like you? ",

        "What do you consume for sustenance? ",

        "Is there intelligent life on this planet? ",

        "Does Earth have a leader? ",
```



```
        "What planets have you visited? ",

        "What technology do you have on this planet? "

    )

def __init__(self):

    self.alienbabble = {'describe_planet_intent': r'',

                        'answer_why_intent': r'',

                        'cubed_intent': r''

                        }

# Define .greet() below:

def greet(self):

    pass

# Define .make_exit() here:

def make_exit(self, reply):

    pass

# Define .chat() next:

def chat(self):
```

```
pass

# Define .match_reply() below:

def match_reply(self, reply):

    pass


# Define .describe_planet_intent():

def describe_planet_intent(self):

    return "Inside .describe_planet_intent()"


# Define .answer_why_intent():

def answer_why_intent(self):

    return "Inside .answer_why_intent()"


# Define .cubed_intent():

def cubed_intent(self, number):

    return "Inside .cubed_intent()"


# Define .no_match_intent():

def no_match_intent(self):
```

```
return "Inside .no_match_intent()"
```

```
# Create an instance of AlienBot below:
```

Cyborg Cantina

Retrieval-based chatbots are commonly used in customer service environments. User questions are constrained to the specific service being provided, which approximates a closed-domain conversation. However, people are known to be especially picky with their food. Can you create a chatbot system that effectively answers all of a diner's questions about the food on a restaurant menu?

In this project you will build a retrieval-based chatbot system for a restaurant serving Mexican cuisine. You'll use a combination of tf-idf scoring, word embedding models, and a set of use-defined functions in order to answer any number of questions from a restaurant diner.

By the end of this project, you'll have constructed a full retrieval-based chatbot system, conversed with your own chatbot creation, and improved the system by adding additional candidate responses!

Tasks

11/12 complete

[Mark the tasks as complete by checking them off](#)

Chatbot Input/Output

1.

Let's begin at the end! We must provide a way for our user to end a conversation once they have had all their questions answered. Note that `exit_commands`, a list of strings commonly used as exit commands from a chatbot system, is already defined in the workspace.

Define a `.make_exit()` method with `self` and `user_message` as parameters.

Within `.make_exit()`, write a `for` loop over each object in `exit_commands`.

For each object in `exit_commands`, check if the object also occurs in `user_message`.

If the object does occur in `user_message`, print a goodbye message to the console, and return `True`.

If the object does **not** occur in `user_message`, do nothing.

Your `.make_exit()` method should look something like this:

```
def make_exit(self, user_message):
    for exit_command in exit_commands:
        if exit_command in user_message:
            print("Ok, bye!")
            return True
```

[Copy to Clipboard](#)

2.

Of course, we should provide a method that allows our chatbot to chat:

Define a `.chat()` method with `self` as a parameter.

Within the `.chat()` method, write a welcoming prompt for a user question, using the `input()` function. Set the result equal to `user_message`.

Create a `while not` loop that checks whether `.make_exit(user_message)` is `True` at each iteration.

Within the while loop, call `.respond()` on `user_message` and assign the result to `user_message`.

Your `.chat()` method should look something like this:

```
def chat(self):
    user_message = input("Hey! What would you like to know about our menu today? ")
    while not self.make_exit(user_message):
        user_message = self.respond(user_message)
```

[Copy to Clipboard](#)

3.

Let's test out the `.make_exit()` functionality of the bot:

Outside of the `ChatBot` class, initialize a `ChatBot` instance.

Call the `.chat()` method on the `ChatBot` instance.

Run `python3 script.py` in the console to start your chatbot!

Ask a question that includes one of the terms in `exit_commands`.

Initialize your `ChatBot` and call the `.chat()` method like this:

```
cantina_bot = ChatBot()
cantina_bot.chat()
```

Copy to Clipboard

Type `python3 script.py` in the console to run the program. Then try asking, "where is the exit?"

Intent Classification

4.

We've already imported a collection of functions created throughout the [Retrieval-based Chatbots](#) lesson into the workspace, as well as a set of pre-defined responses for our retrieval-based bot.

Check out the `user_functions.py` file to refresh your understanding of the `preprocess()`, `compare_overlap()`, `extract_nouns()`, and `compute_similarity()` functions.

Look over the `responses.py` file to see the collection of responses already written for our bot.

5.

Let's build a set of BoW models from our data:

Define a `.find_intent_match()` method with `self`, `responses`, and `user_message` as parameters.

In the body of `.find_intent_match()`, call `preprocess()` on `user_message`, then call `Counter()` on the result to create a bag-of-words (BoW) model.

Call `preprocess()` on each item in `responses`, then call `Counter()` on the result.

Your `.find_intent_match()` method at this point should look something like this:

```
def find_intent_match(self, responses, user_message):
    bow_user_message = Counter(preprocess(user_message))
    processed_responses = [Counter(preprocess(response)) for response in
                           responses]
```

Copy to Clipboard

6.

Now we can select the response that best matches the intent of the user message:

Still in the body of `.find_intent_match()`, apply `compare_overlap()` on each `response` in `processed_responses`. Save the resulting list item to `similarity_list`.

Use Python's `.index()` method and `max()` function to select the index of the highest similarity score in `similarity_list`. Save the result to `response_index`.

Use list indexing to return the element at index `response_index` in `responses`.

Your `.find_intent_match()` method should now look something like this:

```
def find_intent_match(self, responses, user_message):
    bow_user_message = Counter(preprocess(user_message))
    processed_responses = [Counter(preprocess(response)) for response in
                           responses]
    similarity_list = [compare_overlap(doc, bow_user_message) for doc in
                      processed_responses]
    response_index = similarity_list.index(max(similarity_list))
    return responses[response_index]
```

Copy to Clipboard

7.

Let's test our `.find_intent_match()` method:

Define a method called `.respond()`, with `self` and `user_message` as parameters.

Assign the result of calling `.find_intent_match(responses, user_message)` to a variable called `best_response`.

Within `.respond()`, print `best_response` to the console.

To allow for multiple questions, use the `input()` function to prompt the user for another question.

Assign the result to `input_message`.

Return `input_message`.

Run your script in the terminal to check if a response is returned!

Your `.respond()` method should look something like this:

```
def respond(self, user_message):
    best_response = self.find_intent_match(responses, user_message)
    print(best_response)
    input_message = input("Do you have any other questions? ")
    return input_message
```

Copy to Clipboard

Type `python3 script.py` in the console to run your program!

Entity Recognition

8.

Let's extract candidate entities from the user message:

Define a `.find_entities()` method with `self` and `user_message` as parameters.

In the body of `.find_entities()`, call `preprocess()` on `user_message`. Then call `pos_tag()` on the result.

Call `extract_nouns()` on `tagged_user_message`. Save the result to `message_nouns`.

Your `.find_entities()` method at this point should look something like this:

```
def find_entities(self, user_message):
    tagged_user_message = pos_tag(preprocess(user_message))
    message_nouns = extract_nouns(tagged_user_message)
```

Copy to Clipboard

9.

Now we can fit a word2vec model on our candidate entities:

Use `" ".join()` to create a concatenated string from `message_nouns`. Call `word2vec()` on this string. Save the result to a variable called `tokens`.

Call `word2vec()` on `blank_spot`. Save the result to a variable called `category`.

Call `compute_similarity()` on `tokens` and `category`. Save the result to `word2vec_result`.

Your `.find_entities()` method should now look something like this:

```
def find_entities(self, user_message):
    tagged_user_message = pos_tag(preprocess(user_message))
    message_nouns = extract_nouns(tagged_user_message)
    tokens = word2vec(" ".join(message_nouns))
    category = word2vec(blank_spot)
    word2vec_result = compute_similarity(tokens, category)
```

Copy to Clipboard

10.

Finally, let's select the entity with the highest similarity score:

Call `sort(key=lambda x: x[2])` on `word2vec_result`. This will sort the result list by ascending similarity score.

Write an `if` statement to check whether `word2vec_result` has at least one item. If `False`, return `blank_spot`. Otherwise, return the first element of the last list item in `word2vec_result`.

Your `.find_entities()` method should now look something like this:

```
def find_entities(self, user_message):
    # extract candidate entities
    tagged_user_message = pos_tag(preprocess(user_message))
    message_nouns = extract_nouns(tagged_user_message)
    tokens = word2vec(" ".join(message_nouns))
```

```

category = word2vec(blank_spot)
word2vec_result = compute_similarity(tokens, category)
word2vec_result.sort(key=lambda x: x[2])
if len(word2vec_result) < 1:
    return blank_spot
else:
    return word2vec_result[-1][0]

```

Copy to Clipboard

Response Selection

11.

Let's pull together the results from the Intent Classification and Entity Extraction tasks:

Within `.respond()`, delete the line which prints `best_response` to the console.

Directly after the call to `.find_intent_match`, assign the result of calling `.find_entities(user_message)` to `entity`.

Call Python's `.format()` method, with `entity` as an argument, on `best_response`. Print the result to the terminal.

Call your script from the terminal to test out your bot!

Your `.respond()` method should now look something like this:

```

def respond(self, user_message):
    best_response = self.find_intent_match(responses, user_message)
    entity = self.find_entities(user_message)
    print(best_response.format(entity))
    input_message = input("Do you have any other questions? ")
    return input_message

```

Copy to Clipboard

Type `python3 script.py` to test your program!

Improve your bot!

12.

While our the bot functions, you may find that after the first few user questions it's responses become a bit repetitive. One of the major limitations of retrieval-based chatbots is a reliance on a set of pre-defined responses. Try adding more responses to **responses.py** in order to extend the functionality of your bot!

Code Review

Get targeted feedback about your code and how well it meets the objective of the project.

```

from collections import Counter
from responses import responses, blank_spot
from user_functions import preprocess, compare_overlap, pos_tag, extract_nouns,
compute_similarity
import spacy
word2vec = spacy.load('en')

exit_commands = ("quit", "goodbye", "exit", "no")

class ChatBot:

    #define .make_exit() below:

    #define .chat() below:

    #define .find_intent_match() below:

    #define .find_entities() below:

    #define .respond() below:

```

```
#initialize ChatBot instance below:

#call .chat() method below:
```

```
user_functions.py

import re
from collections import Counter
import spacy
word2vec = spacy.load('en')
from nltk import pos_tag
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
stop_words = set(stopwords.words("english"))

def preprocess(input_sentence):
    input_sentence = input_sentence.lower()
    input_sentence = re.sub(r'^\w\s|', '', input_sentence)
    tokens = word_tokenize(input_sentence)
    input_sentence = [i for i in tokens if not i in stop_words]
    return(input_sentence)

def compare_overlap(user_message, possible_response):
    similar_words = 0
    for token in user_message:
        if token in possible_response:
            similar_words += 1
    return similar_words

def extract_nouns(tagged_message):
    message_nouns = list()
    for token in tagged_message:
        if token[1].startswith("N"):
            message_nouns.append(token[0])
    return message_nouns

def compute_similarity(tokens, category):
    output_list = list()
    for token in tokens:
        output_list.append([token.text, category.text, token.similarity(category)])
    return output_list
```

```
responses.py

response_a = "The {} has a gluten-free option, but it is not vegan"
response_b = "We have a selection of sides to go along with the {}, including mashed potatoes and steamed vegetables."
response_c = "{} includes habanero, so it is a bit spicy!"
blank_spot = "food"
```



```
responses = [response_a, response_b, response_c]
```

Build Deep Learning Models with TensorFlow

Implementing Neural Networks

The World Health Organization (WHO)'s Global Health Observatory (GHO) data repository tracks life expectancy for countries worldwide by following health status and many other related factors.

Although there have been a lot of studies undertaken in the past on factors affecting life expectancy considering demographic variables, income composition, and mortality rates, it was found that the effects of immunization and human development index were not taken into account.

This dataset covers a variety of indicators for all countries from 2000 to 2015 including:

- immunization factors
- mortality factors
- economic factors
- social factors
- other health-related factors

Ideally, this data will eventually inform countries concerning which factors to change in order to improve the life expectancy of their populations. If we can predict life expectancy well given all the factors, this is a good sign that there are some important patterns in the data. Life expectancy is expressed in years, and hence it is a number. This means that in order to build a predictive model one needs to use regression.

In this project, you will design, train, and evaluate a neural network model performing the task of regression to predict the life expectancy of countries using this dataset. Excited? Let's go!

Tasks

21/21 complete

[Mark the tasks as complete by checking them off](#)

Data loading and observing

1.

Load the **life_expectancy.csv** dataset into a pandas DataFrame by first importing pandas, and then using the `pandas.read_csv()` function to load the file and assign the resulting DataFrame to a variable called `dataset`.

The following code imports a module called `module` with a short name `md`:

```
import module as md
```

Copy to Clipboard

Replace `module` with `pandas` and `md` with `pd`.

The following code reads a csv file `my_csv.csv` into a DataFrame `df`:

```
df = pd.read_csv('my_csv.csv')
```

Copy to Clipboard

Replace `df` with `dataset` and `my_csv.csv` with the name of your csv file.

2.

Observe the data by printing the first five entries in the DataFrame `dataset` by using the `DataFrame.head()` function. Make sure to see what the columns are and what the data types are. Locate the feature we would like to predict: `life expectancy`. Use `DataFrame.describe()` to see the summary statistics of the data.

The following code prints the first five datapoints in a DataFrame `df`:

```
print(df.head())
```

Copy to Clipboard

Replace `df` with `dataset`.

The following code prints the summary statistics of a DataFrame `df`:

```
print(df.describe())
```

Copy to Clipboard

Replace `df` with `dataset`.

3.

Drop the Country column from the DataFrame using the `DataFrame` drop method. Why? To create a predictive model, knowing from which country data comes can be confusing and it is not a column we can generalize over. We want to learn a general pattern for all the countries, and not only those dependent on specific countries.

The following code drops a column called `MyColumn` from a DataFrame called `df`:

```
df = df.drop(['MyColumn'], axis = 1)
```

Copy to Clipboard

Replace `df` with `dataset` and `MyColumn` with `Country`.

4.

In the next two steps, you will split the data into labels and features. Labels are contained in the “Life expectancy” column. It’s the final column in the DataFrame. Create a new variable called `labels`. Use `iloc` indexing to assign the final column of `dataset` to it.

The following code assigns to a variable called `last_column` the last column of a DataFrame `df`:

```
last_column = df.iloc[:, -1] #select all the rows (:), and access the last column (-1)
```

Copy to Clipboard

Replace `df` with `dataset` and `last_column` with `labels`.

5.

Features span from the first column to the last column (not including it, since it’s a label column in our dataset). Create a new variable called `features`. Use `iloc` indexing to assign a subset of columns from first to last (not including the last column) to it.

The following code gets a subset of DataFrame’s `df` columns starting from the first one, and ending at the last one (not including it), and assigns it to `columns`:

```
columns = df.iloc[:, 0:-1] #select all the rows (:), and access columns from 0 to the last column
```

Copy to Clipboard

Replace `df` with `dataset` and `columns` with `features`.

Data Preprocessing

6.

When you observed your dataset you probably noticed that some columns are categorical. We learned in this lesson that categorical columns need to be converted into numerical columns using methods such as one-hot-encoding. Use `pandas.get_dummies(DataFrame)` to apply one-hot-encoding on all the categorical columns. Assign the result of the encoding back to the `features` variable.

The following code applies one-hot-encoding on a DataFrame `df` and assigns the result back to the original DataFrame:

```
import pandas as pd
df = pd.get_dummies(df)
```

Copy to Clipboard

Replace `df` with `features`.

7.

Split your data into training set and test sets using the `sklearn.model_selection.train_test_split()` function. Assign the training features to a variable called `features_train`
training labels to a variable called `labels_train`
test data to a variable called `features_test`
test labels to a variable called `labels_test`.

You can choose any percentage for `test_size` and any value for `random_state`.

The following code splits the data into training and testing data:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.20,
random_state=23)
```

Copy to Clipboard

Replace `X_train` with `features_train`, `X_test` with `features_test`, `y_train` with `labels_train`, and `y_test` with `labels_test`. You can choose any percentage for `test_size` and any value for `random_state`.

8.

The next step is to standardize/normalize your numerical features. You can pick whichever method you want. In this step, create a `sklearn.compose.ColumnTransformer` instance called `ct` to set up the normalization/standardization procedure. When initializing `ColumnTransformer` make sure to list all of the numerical features you have in your dataset. Or use `DataFrame.select_dtypes()` to select `float64` or `int64` feature types automatically.

The following code initializes a `ColumnTransformer` instance for applying a `StandardScaler` on a set of numeric features:

```
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import Normalizer
from sklearn.compose import ColumnTransformer
```

```
numerical_features = features.select_dtypes(include=['float64', 'int64'])
numerical_columns = numerical_features.columns

ct = ColumnTransformer([("only numeric", StandardScaler(), numerical_columns)],
remainder='passthrough')
```

Copy to Clipboard

9.

Fit your instance `ct` of `ColumnTransformer` to the training data and at the same time transform it by using the `ColumnTransformer.fit_transform()` method. Assign the result to a variable called `features_train_scaled`.

The following code fits and transforms a `ColumnTransformer` instance `ct`:

```
X_fitted = ct.fit_transform(X)
```

Copy to Clipboard

Replace `X_fitted` with `features_train_scaled` and `X` with `features_train`.

10.

Transform your test data instance `features_test` using the trained `ColumnTransformer` instance `ct`. Assign the result to a variable called `features_test_scaled`.

The following code transforms numerical data of a `DataFrame` `x` using a trained `ColumnTransformer` instance `ct`:

```
X_scaled = ct.transform(X)
```

Copy to Clipboard

Replace `X_scaled` with `features_test_scaled` and `X` with `features_test`.

Building the model

11.

Create an instance of the `tensorflow.keras.models.Sequential` model called `my_model`.

The following code creates an instance of the `Sequential` model:

```
from tensorflow.keras.models import Sequential
model = Sequential()
```

Copy to Clipboard

Replace `model` with `my_model`.

12.

Create the input layer to the network model using `tf.keras.layers.InputLayer` with the shape corresponding to the number of features in your dataset. Assign the resulting input layer to a variable called `input`.

The following code creates an input layer with the shape corresponding to the number of features in a `DataFrame` `df`:

```
from tensorflow.keras.layers import InputLayer
my_input = InputLayer(input_shape = (df.shape[1], ))
```

Copy to Clipboard

Replace `my_input` with `input`, and `df` with `features`.

13.

Add the input layer from the previous step to the model instance `my_model`.

The following code adds a layer `layer` to a model `model`:

```
model.add(layer)
```

Copy to Clipboard

Replace `layer` with `input` and `model` with `my_model`.

14.

Add one `keras.layers.Dense` hidden layer with any number of hidden units you wish. Use the `relu` activation function.

To add a `keras.layers.Dense` hidden layer to a model `model`:

```
from tensorflow.keras.layers import Dense
model.add(Dense(64, activation = "relu"))
```

Copy to Clipboard

Replace `model` with `my_model`.

15.

Add a `keras.layers.Dense` output layer with one neuron since you need a single output for a regression prediction.

The following code adds a `keras.layers.Dense` layer with one neuron to a model `model`:

```
from tensorflow.keras.layers import Dense
my_model.add(Dense(1))
```

Copy to Clipboard

16.

Print the summary of the model using the `Sequential.summary()` method.

The following code prints the summary of a model called `model`:

```
print(model.summary())
```

Copy to Clipboard

Replace `model` with `my_model`.

Initializing the optimizer and compiling the model

17.

Create an instance of the `Adam` optimizer with the learning rate equal to 0.01.

The following code initializes an `Adam` optimizer with the learning rate equal to 0.1:

```
from tensorflow.keras.optimizers import Adam
opt = Adam(learning_rate = 0.1)
```

Copy to Clipboard

Replace 0.1 with 0.01.

18.

Once your optimizer is initialized, compile the model using the `Sequential.compile()` method.

Assign the following parameters:

- For `loss` use the Mean Squared Error (`mse`)

- For `metrics` use the Mean Absolute Error (`mae`)

- For `opt` (the optimizer parameters) use the instance of the optimizer you created in the previous step.

The following code compiles a model `model`, with the Mean Squared Error loss, Mean Absolute Error learning metrics, and an optimizer:

```
from keras.optimizers import Adam
opt = Adam(learning_rate = 0.1)
model.compile(loss = 'mse', metrics = ['mae'], optimizer = opt)
```

Copy to Clipboard

Replace `model` with `my_model`.

Fit and evaluate the model

19.

Train your model with the `Sequential.fit()` method by passing it the following parameters:

- your preprocessed training data

- training labels

- number of epochs equal to 40

- batch size equal to 1

- verbose set to 1

The following code trains a `model` with the training data `x`, training labels `y`, number of epochs equal to 20, batch size equal to 5, and verbose set to 0:

```
model.fit(X, y, epochs = 20, batch_size = 5, verbose = 0)
```

Copy to Clipboard

Replace `x` by `features_train_scaled`, `y` with `labels_train`, `epochs = 20` to `epochs = 50`, `batch_size = 5` to `batch_size = 1`, and `verbose = 0` to `verbose = 1`

20.

Using the `Sequential.evaluate()` method, evaluate your trained model on the preprocessed test data set, and with the test labels. Set `verbose` to 0. Store the result of the evaluation in two variables: `res_mse` and `res_mae`.

The following code evaluates the `model` model on the preprocessed test data `x`, using test labels `y`, and returning the final value for loss and the metrics specified in the model compilation:

```
final_mse, final_mae = model.evaluate(X, y, verbose = 0)
```

Copy to Clipboard

Replace `final_mse` with `res_mse`, `final_mae` with `res_mae`, `model` with `my_model`, `x` with `features_test_scaled`, and `y` with `labels_test`.

21.

Print your final loss (RMSE) and final metric (MAE) to check the predictive performance on the test set.

The following code prints two variables in Python:

```
print(variable1, variable2)
```

Copy to Clipboard

Replace `variable1` with `res_mse`, and `variable2` with `res_mae`.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import Normalizer, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.metrics import mean_squared_error

# model
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import InputLayer
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

# READING THE DATA
df = pd.read_csv('life_expectancy.csv')

# Drop and encode
df = df.drop(['Country'], axis=1)
df = pd.get_dummies(df)

# Split features and labels
features = df.iloc[:, :-1]
labels = df.iloc[:, -1]

# Split
features_train, features_test, labels_train, labels_test = train_test_split(
    features, labels, test_size=0.2, random_state=23
)

# Scale numeric data
numerical_columns = features.select_dtypes(include=['float64', 'int64']).columns
ct = ColumnTransformer([('scaler', StandardScaler(), numerical_columns)],
    remainder='passthrough')

features_train_scaled = ct.fit_transform(features_train)
features_test_scaled = ct.transform(features_test)

# Clean data
features_train_scaled = np.nan_to_num(features_train_scaled)
labels_train = labels_train.fillna(labels_train.mean()).values.reshape(-1)

# Build model
my_model = Sequential()
my_model.add(InputLayer(input_shape=(features_train_scaled.shape[1],)))
my_model.add(Dense(64, activation='relu'))
my_model.add(Dense(1))
```

```
opt = Adam(learning_rate=0.01)
my_model.compile(loss='mse', metrics=['mae'], optimizer=opt)

my_model.summary()

# Fit
my_model.fit(features_train_scaled, labels_train, epochs=20, batch_size=5, verbose=0)

res_mse, res_mae = my_model.evaluate(features_train_scaled, labels_train, verbose = 0)

print(res_mse, res_mae)

predicted_labels = my_model.predict(features_test_scaled)

# calculate errors
errors = mean_squared_error(labels_test, predicted_labels)
# report error
print(errors)

# Root Mean Squared Error
RMSE = np.sqrt(errors)
print('Root Mean Squared Error: ', RMSE)
print('Mean of test labels: ', np.mean(labels_test))
```

Build Deep Learning Models with TensorFlow

Deep Learning Regression with Admissions Data

Overview

This project is slightly different than most on Codecademy. Instead of a step-by-step tutorial, this project contains a series of open-ended requirements that describe the project you'll be building. There are many possible ways to fulfill all of these requirements correctly, and you should expect to use the internet, Codecademy, and other resources when you encounter a problem that you cannot easily solve.

Project Goals

For this project, you will create a deep learning regression model that predicts the likelihood that a student applying to graduate school will be accepted based on various application factors (such as test scores).

By analyzing the parameters in this [graduate admissions dataset](#), you will use TensorFlow with Keras to create a regression model that can evaluate the chances of an applicant being admitted. You hope this will give you further insight into the graduate admissions world and improve your test prep strategy.

Setup Instructions

You have two options of completing this assignment. Either here, within Codecademy's output terminal, or on your own, in case you're more comfortable using a Jupyter notebook. If you choose to do this project on your computer instead of Codecademy, you can download what you'll need by clicking the "Download" button below. If you need help setting up your computer, be sure to check out our setup guides:

[Command Line Interface Setup](#)

[Introducing Jupyter Notebook](#)

[Setting up Jupyter Notebook](#)

[Getting Started with Jupyter](#)

[Getting More out of Jupyter Notebook](#)

Open **Deep Learning Regression with Admissions Data.ipynb** and follow the steps in the Jupyter Notebook. If you get stuck, you can look at **Deep Learning Regression with Admissions Data_Solution.ipynb** for the answer.

Download

Tasks

0/10 complete

[Mark the tasks as complete by checking them off](#)

Project Requirements

1.

If you take a look at **admissions_data.csv**, you'll see parameters that admissions officers commonly use to evaluate university applicants. This data is from [Kaggle](#) and provides information about 500 applications for various universities and what their chance of admittance is.

This is a regression problem because the probability of being admitted is a continuous label between 0 and 1.

Load the **csv** file into a DataFrame and investigate the rows and columns to get familiarity with the dataset.

To get more information about each parameter in **admissions_data.csv** click the hint below.

This dataset has nine different columns:

```
Serial No.: Index of each row (1-500)
GRE Score: GRE test score (out of 340)
TOEFL Score: TOEFL test score (out of 120)
University Rating: Evaluated university rating (out of 5)
SOP: Statement of Purpose Strength (out of 5)
LOR: Letter of Recommendation Strength (out of 5)
CGPA: Undergraduate GPA (out of 10)
Research: Has research experience (either 0 or 1)
Chance of Admit: Applicant's chance of being admitted (ranging from 0 to 1)
```

2.

Split it up the data into **feature** parameters and the **labels**.

You are creating a model that predicts an applicant's likelihood of being admitted to a master's program, so take some time to look at the features of your model and which column you are trying to predict. Also consider if there are any dataset features that should not be included as a predictor.

Make sure all of your variables are numerical.

If there are any categorical variables, be sure to map them to numerical values, using techniques such as one-hot-encoding, so they can be used in a regression analysis.

Our feature parameters should be:

```
GRE Score: GRE test score (out of 340)
TOEFL Score: TOEFL test score (out of 120)
```


University Rating: Evaluated university rating (out of 5)

SOP: Statement of Purpose Strength (out of 5)

LOR: Letter of Recommendation Strength (out of 5)

CGPA: Undergraduate GPA (out of 10)

Research: Has research experience (either 0 or 1) Do you notice any that are not included?

Your label parameter should be: `Chance of Admit`: Applicant's chance of being admitted (ranging from 0 to 1)

You can use the `iloc` function from the [pandas library](#) to split up the features and labels.

Luckily, there are no categorical variables in this dataset, so you do not have to perform *one-hot encoding*. You should always check this, however, before diving into analysis!

3.

Since you are creating a learning model, you must have a training set and a test set. Remember that this allows you to measure the effectiveness of your model.

You have created two DataFrames: one for `features` DataFrame and one for `labels`. Now, you must split each of these into a training set and a test set.

If you need a refresher on splitting a train and test set, use [scikit-learn's user guide](#) and any other online resources for help!

Click the hint below if you need any other guidance.

To split the data into a training set and a test, use the `.train_test_split()` function available in the `sklearn.model_selection` library. When using it, you should assign it to four different variables:

```
features_training_set
features_test_set
labels_training_set
labels_test_set
```

When splitting up the data into training and test sets, you can choose how much of your data is in the test set using the `test_size` attribute. Some common sizes for the test set range between 0.20 and 0.35.

4.

If you look through the `admissions_data.csv`, you may notice that there are many different scales being used. For example, the GRE Score is out of 340 while the University Rating is out of 5. Can you imagine why this might be a problem when using a regression learning model?

You should either scale or normalize your data so that all columns/features have equal weight in the learning model.

One possible solution would be to use the `StandardScaler()` function from the `sklearn.preprocessing` module. It then uses `.fit_transform()` on our `features_train_set` and `.transform()` on our `features_test_set` separately.

You can then reassign the `features_train_set` and the `features_test_set` to a DataFrame using the `.DataFrame()` pandas function.

Make sure to fit the scaler or the normalizer to the training data only. This way you avoid data leakage from the training set to the test set.

Read through the [sklearn.preprocessing documentation](#) for more information about scaling and transforming data as well as the [pandas guide](#) for help with DataFrames.

5.

Create a neural network model to perform a regression analysis on the admission data.

When designing your own neural network model, consider the following:

- The shape of your input
- Adding hidden layers as well as how many neurons they have
- Including activation functions
- The type of loss function and metrics you use
- The type of gradient descent optimizer you use
- Your learning rate

One possible model could include one hidden layer with 16 neurons and a ReLU function. Since this is a regression model, the loss function should be mean-squared error (MSE), with mean-average error (MAE) function as the metric.

When deciding what optimizer to use, you can consider:

- Stochastic Gradient Descent
- Adam

To keep your code organized, you should put your model into its own function, so that your code is more organized and readable.

If you feel stuck, you can use the [Keras Developer Guide](#) as a resource.

6.

It's time to test out the model you created!

Fit your model with your training set and test it out with your test set.

It's okay if it is not that accurate right now. You can play around with your model and tweak it to increase its accuracy.

To fit your model use the `.fit()` function from the `tensorflow.keras.Model` module. Then, to evaluate your model, use the `.evaluate()` function from the `tensorflow.keras.Sequential` module.

Each of these functions has many different required parameters (as well as optional ones). If you need a refresher on these, look at the [keras documentation](#).

7.

You have tested out your model. Now is the time to adjust your model's hyperparameters. You have a lot of choices to make. You can choose:

- the number of `epochs`
- the size of your `batch_size`
- to add more hidden layers
- your type of optimizer and/or activation functions.

Have fun in the hyperparameter playground. Test things out and see what works and what does not work. See what makes your model optimized between speed and accuracy. You have complete creative power!

Some suggestions:

- Use `EarlyStopping()` to make your model efficient.
- Play around with `batch_size` to optimize model efficiency with accuracy. If you're not sure where to start, we recommend you start with a small `batch_size` (maybe 2 or 4) and increase or play around with different sizes as you continue refining your model.
- Add a hidden layer and then adjust the number of neurons that are in the hidden layer.
- See what happens when you change the optimizer you use.

If you do need guidance along the way, the Hyperparameter Tuning Lesson can provide some good inspiration.

Solution

8.

Great work! Visit our [forums](#) to share your project with other learners. We recommend hosting your own solution on GitHub so you can share it more easily.

Your solution might look different from ours, and that's okay. There are multiple ways to solve these projects, and you'll learn more by seeing others' code.

Extensions

9.

Using the [Matplotlib Library](#), see if you can plot the model loss per epoch as well as the mean-average error per epoch for both training and validation data. This will give you an insight into how the model performs better over time and can also help you figure out better ways to tune your hyperparameters.

Because of the way Matplotlib plots are displayed in the learning environment, please use

`fig.savefig('static/images/my_plots.png')` at the end of your graphing code to render the plot in the browser. If you wish to display multiple plots, you can use `.subplot()` or `.add_subplot()` methods in the Matplotlib library to depict multiple plots in one figure.

Use the hint below if you have any struggles with displaying these graphs.

One way to create the plots is to use the following code:

```
fig = plt.figure()
ax1 = fig.add_subplot(2, 1, 1)
ax1.plot(history.history['mae'])
ax1.plot(history.history['val_mae'])
ax1.set_title('model mae')
ax1.set_ylabel('MAE')
ax1.set_xlabel('epoch')
ax1.legend(['train', 'validation'], loc='upper left')

# Plot loss and val_loss over each epoch
ax2 = fig.add_subplot(2, 1, 2)
ax2.plot(history.history['loss'])
ax2.plot(history.history['val_loss'])
ax2.set_title('model loss')
ax2.set_ylabel('loss')
ax2.set_xlabel('epoch')
ax2.legend(['train', 'validation'], loc='upper left')

# used to keep plots from overlapping each other
fig.tight_layout()
fig.savefig('static/images/my_plots.png')
```

Copy to Clipboard

10.

Let's say you wanted to evaluate how strongly the features in **admissions.csv** predict an applicant's admission into a graduate program. We can use something called an *R-squared* value. It is also known as the *coefficient of determination*; feel free to explore more about it [here](#).

Basically, we can use this calculation to see how well the features in our regression model make predictions. An R-squared value near close to 1 suggests a well-fit regression model, while a value closer to 0 suggests that the regression model does not fit the data well.

See if you can apply this to your model after it has been evaluated using a `.predict()` method on your `features_test_set` and the `r2_score()` function on your `labels_test_set`. Both of these functions are from the [scikit-learn library](#).

One possible way to implement an R-squared calculation using the following code:

```
# note these variable names may differ from yours
predicted_values = model.predict(features_test_set)
print(r2_score(labels_test_set, predicted_values))

import app
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras import layers

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import Normalizer
from sklearn.metrics import r2_score

# Do extensions code below
# if you decide to do the Matplotlib extension, you must save your plot in the directory by
# uncommenting the line of code below

# fig.savefig('static/images/my_plots.png')
```

<https://www.codecademy.com/paths/build-deep-learning-models-with-tensorflow/tracks/dlsp-getting-started-with-tensorflow/modules/dlsp-regression-challenge-project/projects/deep-learning-regression-with-admissions-data>

Classifying Galaxies

Classifying Galaxies Using Convolutional Neural Networks

<https://www.codecademy.com/paths/build-deep-learning-models-with-tensorflow/tracks/dlsp-classification-track/modules/dlsp-image-classification/projects/classifying-galaxies-deep-learning>

Around the clock, telescopes affixed to orbital satellites and ground-based observatories are taking millions of pictures of millions upon millions of celestial bodies. These data, of stars, planets and galaxies provide an invaluable resource to astronomers.

However, there is a bottleneck: until the data is annotated, it's incredibly difficult for scientists to put it to good use. Additionally, scientists are usually interested in subsets of the data, like galaxies with unique characteristics.

In this project, you will build a neural network to classify deep-space galaxies. You will be using image data curated by [Galaxy Zoo](#), a crowd-sourced project devoted to annotating galaxies in support of scientific discovery.

You will identify “odd” properties of galaxies. The data falls into four classes:

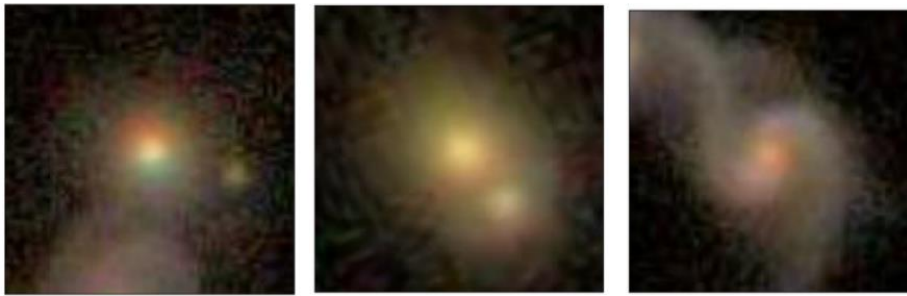
[1,0,0,0] - Galaxies with no identifying characteristics.



[0,1,0,0] - Galaxies with rings.



[0,0,1,0] - Galactic mergers.



[0,0,0,1] - "Other," Irregular celestial bodies.



Tasks

12/12 complete

Mark the tasks as complete by checking them off

1.

Because the dataset comprises over one thousand images, you'll use a custom function, `load_galaxy_data()` to load the compressed data files into the Codecademy learning environment as NumPy arrays. Take a look at the shape of the data.

Use `.shape` to print the dimensions of the `input_data` and `labels`.

What does the last dimension of the data indicate about the image data? What does the last dimension of the labels indicate about the labels?

Use `print(input_data.shape)` and `print(labels.shape)`.

Because the last dimension of the data is 3, you know that the image data is RGB/in color. Because the last dimension of the labels is 4, and there are four classes, you know that the labels are one-hot vectors. For example, `[1,0,0,0]` → Normal galaxy.

2.

Next, divide the data into training and validation data, using `sklearn's train_test_split()` function.

Set the `test_size` argument to be 0.20.

Shuffle the data.

Set the `random_state` to be 222.

Set `stratify=labels`. This ensures that ratios of galaxies in your testing data will be the same as in the original dataset.

Your code should look something like this:

```
x_train, x_valid, y_train, y_valid = train_test_split(input_data,
labels, test_size=0.20, stratify=labels, shuffle=True,
random_state=222)
```

Copy to Clipboard

3.

Now, it's time to preprocess the input.

Define an `ImageDataGenerator`, and configure it so that the object will normalize the pixels using the `rescale` parameter.

You can do this with the following line of code:

```
data_generator = ImageDataGenerator(rescale=1./255)
```

Copy to Clipboard

4.

Next, create two `NumpyArrayIterators` using the `.flow(x, y, batch_size=?)` method. We recommend using a batch size of 5. Significantly larger batch sizes may cause memory issues on the Codecademy platform.

Create a training data iterator by calling `.flow()` on your training data and labels.

Create a validation data iterator by calling `.flow()` on your training data and labels.

Your code should use the `.flow()` method like this:

```
training_iterator = data_generator.flow(x_train, y_train, batch_size=5)
validation_iterator = data_generator.flow(x_valid, y_valid,
batch_size=5)
```

Copy to Clipboard

5.

Next, build your model, starting with the input shape and output layer.

Create a `tf.keras.Sequential` model named `model`.

Add a `tf.keras.Input` layer. Refer back to the shape of the data. What should the input shape be?

Add a `tf.keras.layers.Dense` layer as your output layer. Make sure that it outputs 4 features, for the four classes ("Normal", "Ringed", "Merger", "Other").

Remember to use a `softmax` activation on this final layer.

Your input shape should be `(128, 128, 3)`. This is because the images are 128 pixels tall, 128 pixels wide, and have 3 channels: Red, Green, and Blue.

Your solution should add the layers to a `Sequential` model:

```
model = tf.keras.Sequential()
```

```
model.add(tf.keras.Input(shape=(128, 128, 3)))
model.add(tf.keras.layers.Dense(4, activation="softmax"))
```

Copy to Clipboard

6.

Before you finish designing your architecture, compile your model with an optimizer, loss, and metrics.

Use `model.compile(optimizer=?, loss=?, metrics=[?, ?])` to compile your model.

Use `tf.keras.optimizers.Adam` with a `learning_rate` of `0.001`.

Because the labels are one-hot categories, use `tf.keras.losses.CategoricalCrossentropy()` as your loss.

Set `[tf.keras.metrics.CategoricalAccuracy(), tf.keras.metrics.AUC()]` as your metrics.

Your code for compiling the model should look like this:

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss=tf.keras.losses.CategoricalCrossentropy(),
    metrics=[tf.keras.metrics.CategoricalAccuracy(), tf.keras.metrics.AUC()]
)
```

Copy to Clipboard

7.

Now, let's go back and finish fleshing out your architecture. An architecture that works well on this task is two convolutional layers, interspersed with max pooling layers, followed by two dense layers:

Conv2D: 8 filters, each 3x3 with strides of 2

MaxPooling2D: pool_size=(2, 2), strides=2

Conv2D: 8 filters, each 3x3 with strides of 2

MaxPooling2D: pool_size=(2, 2), strides=2

Flatten Layer

Hidden Dense Layer with 16 hidden units

Output Dense Layer

Try coding up this architecture yourself, using:

```
tf.keras.layers.Conv2D
tf.keras.layers.MaxPooling2D
tf.keras.layers.Flatten()
tf.keras.layers.Dense()
```

Don't forget to use "relu" activations for Dense and Conv2D hidden layers!

The full architecture of the model can be defined like this:

```

model = tf.keras.Sequential()
model.add(tf.keras.Input(shape=(128, 128, 3)))
model.add(tf.keras.layers.Conv2D(8, 3, strides=2, activation="relu"))
model.add(tf.keras.layers.MaxPooling2D(
    pool_size=(2, 2), strides=(2,2)))
model.add(tf.keras.layers.Conv2D(8, 3, strides=2, activation="relu"))
model.add(tf.keras.layers.MaxPooling2D(
    pool_size=(2,2), strides=(2,2)))
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(16, activation="relu"))
model.add(tf.keras.layers.Dense(4, activation="softmax"))

```

Copy to Clipboard

8.

At this point, your model should have 7,164 parameters. Use `model.summary()` to confirm this.

Add `model.summary()` after you have defined the model.

9.

Use `model.fit(...)` to train your model.

The first argument should be your training iterator.

Set `steps_per_epoch` to be the length of your training data, divided by your batch size.

Set `epochs` to be 8.

Set `validation_data` to be your validation iterator.

Set `validation_steps` to be the length of your validation data, divided by your batch size.

For example:

```

model.fit(
    training_iterator,
    steps_per_epoch=len(x_train)/5,
    epochs=8,
    validation_data=validation_iterator,
    validation_steps=len(x_valid)/5)

```

Copy to Clipboard

10.

Now you can run your code to train the model. Training may take a minute or two. After training for twelve epochs, your model's accuracy should be around 0.60-0.70, and your AUC should fall into the 0.80-0.90 range!

What do these results mean?

Your accuracy tells you that your model assigns the highest probability to the correct class more than 60% of the time. For a classification task with over four classes, this is no small feat: a random baseline model would achieve only ~25% accuracy on the dataset. Your AUC tells you that for a random galaxy, there is more than an 80% chance your model would assign a higher probability to a true class than to a false one.

11.

Congratulations on finishing the project! You have successfully trained a Convolutional Neural Network to classify galaxies.

Think you can do even better? If you would like, try tweaking your architecture. Can you find a better set of hyperparameters? Make sure to watch your parameter count: it's easy to accidentally create a model with more than tens of thousands of parameters, which could overfit to your relatively small dataset (or crash the Learning Environment).

Note that scores will fluctuate a bit, depending on how the weights are randomly initialized.

Good luck!

Here are a few parameters that you could consider tweaking:

- learning rate
- number of convolutional layers
- number of filters, strides, and padding type per layer
- stride and `pool_size` of max pooling layers
- size of hidden linear layers

12.

BONUS

Want to visualize how your convolutional neural network processes images?

Take a look at `visualize.py`. It contains a function, `visualize_activations()`.

This function loads in sample data, uses your model to make predictions, and then saves the feature maps from each convolutional layer. These feature maps showcase the activations of each filter as they are convolved across the input.

Try importing this function by adding the following to the end of `train.py`:

```
from visualize import visualize_activations
visualize_activations(model, YOUR_VALIDATION_ITERATOR)
```

Copy to Clipboard

And then replace `YOUR_VALIDATION_ITERATOR` with the `NumpyIterator` you defined in task four.

You are not required to understand the visualization code. However, the hint provides a summary.

`visualize_results` takes your Keras model and the validation iterator and does the following:

- It loads in a sample batch of data using your validation iterator.

- It uses `model.predict()` to generate predictions for the first sample images.

- Next, it compares those predictions with the true labels and prints the result.

It then saves the image and the feature maps for each convolutional layer using `matplotlib`.

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

from sklearn.model_selection import train_test_split
from utils import load_galaxy_data

import app

input_data, labels = load_galaxy_data()
```

Classify X-rays With Machine Learning

Overview

This project is slightly different than most on Codecademy. Instead of a step-by-step tutorial, this project contains a series of open-ended requirements that describe the project you'll be building. There are many possible ways to fulfill all of these requirements correctly, and you should expect to use the internet, Codecademy, and other resources when you encounter a problem that you cannot easily solve.

Project Goals

You are a researcher in a hospital lab and are given the task to develop a learning model that supports doctors with diagnosing illnesses that affect patients' lungs. At your disposal, you have a set [X-ray lung scans](#) with examples of patients who had either pneumonia, Covid-19, or no illness. Using the Keras module, you will create a classification model that outputs a diagnosis based on a patient's X-ray scan. You hope this model can help doctors with the challenge of deciphering X-ray scans and open a dialogue between your research team and the medical staff to create learning models that are as effective and interpretable as possible.

Setup Instructions

You have two options of completing this assignment. Either here, within Codecademy's output terminal, or on your own, in case you're more comfortable using a Jupyter notebook. If you choose to do this project on your computer instead of Codecademy, you can download what you'll need by clicking the "Download" button below. If you need help setting up your computer, be sure to check out our setup guides:

[Command Line Interface Setup](#)

[Introducing Jupyter Notebook](#)

[Setting up Jupyter Notebook](#)

[Getting Started with Jupyter](#)

[Getting More out of Jupyter Notebook](#)

Open **Covid 19 and Pneumonia Classification with Deep Learning.ipynb** and follow the steps in the Jupyter Notebook. If you get stuck, you can look at **Covid 19 and Pneumonia Classification with Deep Learning_Solution.ipynb** for the answer.

Download

Tasks

9/9 complete

[Mark the tasks as complete by checking them off](#)

Project Requirements

1.

If you take a look at [Covid19 dataset](#), you'll see that there are two different folders inside. Take some time to look around these directories and familiarize yourself with the images inside. As you peruse through them, think about the following:

What types of images are we working with?

How are they organized? How will this affect preprocessing?

Will this be binary classification or multi-class classification?

After you do this, you will be ready to get started on preprocessing! Click the hint below if you want to see some insights on the image data.

One of the first things you may notice is that the folders are split into **train** and **test** folders. This is immensely helpful for us during preprocessing as images have already been allocated and prepared for a learning model.

When you click in and view the images, you will notice that all the X-ray scans are in grayscale.

This is something to keep in mind when you dive into preprocessing the data.

Finally, when you click into either the **train** or **test** folder, you will see that there are three different folders within them:

Covid

Normal

Viral Pneumonia

These are each a different class that our learning model can output, indicating a multi-class classification problem rather than binary classification.

2.

Load in your image data and get it ready for the journey through a neural network. One possible way to do this is to use an `ImageGenerator` object; however, feel free to try other methods you may have experienced before this project.

When creating the object, remember that neural networks struggle with large integer values. Think about how you might want to get your image data ready for your neural network and get the best results.

If you decide to create an `ImageGenerator` object, check out TensorFlow's `preprocessing.image` [module](#).

You will also want to rescale your images with pixel normalization.

If you look at the documentation, `ImageDataGenerator()` contains numerous parameters, and you can play with them as you work on your neural network. If you flip or crop your images during data augmentation, note that you would have to create separate `ImageDataGenerator()` objects for both for your training and validation data to avoid hurting the performance on your test set.

3.

Now that you have set up your `ImageDataGenerator` object, it is time to actually load in your image data.

You will want to create two different iterable objects from this `ImageDataGenerator`: a train set and a test/validation set.

When you are creating these sets of images consider the following:

The directory the images come from

The types of images you are working with

The target size of the images

Click the hint below if you need any other guidance.

To split the data into a training set and a test/validation set, create two different iterable objects that can fit right into the network model: a `training_iterator` and a `validation_iterator`. Use the `flow_from_directory()` method in the `tensorflow.keras.preprocessing.image` module.

You can set your `class_mode` to `sparse` and use sparse categorical loss instead for this model as

well.

You can play with the `batch_size` parameter as you fine-tune your model.

4.

Now that your image data is loaded and ready for analysis, create a classification neural network model to perform on the medical data.

With image data, there are a ton of directions to go in. To get you grounded, we recommend you start by creating your input layer and output layer and compile it before diving into creating a more complex model.

When starting your neural network, consider the following:

- The shape of your input

- The shape of your output

- Using any activation functions for your output

- Your gradient descent optimizer

- Your learning rate

- Your loss functions and metrics

- Flattening the image data before the output layer

You should create a sequential model and be mindful of the shape of the image data that is being loaded into your neural network.

Next, you need to flatten your image data and create your output layer. In this case, you have three different classes. Because of this, you will need a dense layer with three different potential outputs.

Finally, when compiling your model, you have a lot of choices to make, including optimizer, learning rate, loss function, and metric evaluations.

5.

It's time to test out the model you created!

Fit your model with your training set and test it out with your validation/test set.

Since you have not added many layers yet or played around with hyperparameters, it may not be very accurate yet. Do not fret! Your next adventure will be to play with your model and mold it until you see more ideal results.

To fit your model use the `.fit()` function from the `tensorflow.keras.Model` [module](#). Be sure to fit it on your training data and test it with your validation data. You can also specify how many steps are taken during each epoch as well as how many epochs your model performs.

Each of these functions has many different required parameters (as well as optional ones). If you need a refresher on these, look at the [sklearn documentation](#).

6.

You have created a model and tested it out. Now it is time for the real fun! Start playing around with some hidden layers and hyperparameters.

When adding hidden layers, consider the type of hidden layers you add (remember this is image data).

As you add in layers, you should also adjust your model's hyperparameters. You have a lot of choices to make. You can choose:

- the number of `epochs`

The size of your `batch_size`

to add more hidden layers

your type of optimizer and/or activation functions

the size of your learning rate

Have fun in the hyperparameter playground. Test things out and see what works and what does not work. See what makes your model optimized between speed and accuracy. You have complete creative power!

Note: We suggest being thoughtful about the number of layers you add. Given the small size of the dataset, you should not train it on anything more than 10K parameters or roughly five layers. This will help you avoid overfitting the data as well as avoid crashing the learning environment. If you attempt to use `Dense` layers rather than `Conv2D` layers in the learning environment, your program will likely not successfully run to completion on the platform. Some suggestions:

Use convolutional layers as well as max pooling.

Use `EarlyStopping()` to make your model efficient.

Play around with `batch_size` to optimize model efficiency with accuracy.

Solution

7.

Great work! Visit our [forums](#) to share your project with other learners. We recommend hosting your own solution on GitHub so you can share it more easily.

Your solution might look different from ours, and that's okay. There are multiple ways to solve these projects, and you'll learn more by seeing others' code.

Extensions

8.

Plot the cross-entropy loss for both the train and validation data over each epoch using the [Matplotlib Library](#). You can also plot the AUC metric for both your train and validation data as well. This will give you an insight into how the model performs better over time and can also help you figure out better ways to tune your hyperparameters.

Because of the way Matplotlib plots are displayed in the learning environment, please use `fig.savefig('static/images/my_plots.png')` at the end of your graphing code to render the plot in the browser. If you wish to display multiple plots, you can use `.subplot()` or `.add_subplot()` methods in the Matplotlib library to depict multiple plots in one figure.

Use the hint below if you have any struggles with displaying these graphs.

One way to plot your metrics is with the following code:

```
# plotting categorical and validation accuracy over epochs
fig = plt.figure()
ax1 = fig.add_subplot(2, 1, 1)
ax1.plot(history.history['categorical_accuracy'])
ax1.plot(history.history['val_categorical_accuracy'])
ax1.set_title('model accuracy')
ax1.set_xlabel('epoch')
```

```

ax1.set_ylabel('accuracy')
ax1.legend(['train', 'validation'], loc='upper left')

# plotting auc and validation auc over epochs
ax2 = fig.add_subplot(2, 1, 2)
ax2.plot(history.history['auc'])
ax2.plot(history.history['val_auc'])
ax2.set_title('model auc')
ax2.set_xlabel('epoch')
ax2.set_ylabel('auc')
ax2.legend(['train', 'validation'], loc='upper left')

# used to keep plots from overlapping
fig.tight_layout()

fig.savefig('static/images/my_plots.png')

```

Copy to Clipboard

9.

Another potential extension to this project would be implementing a [classification report](#) and a [confusion matrix](#). These are not tools we have introduced you to; however, if you would like further resources to improve your neural network, we recommend looking into these concepts.

As a brief introduction, these concepts evaluate the nature of false positives and false negatives in your neural network taking steps beyond simple evaluation metrics like accuracy.

In the hint below, you will see a possible solution to calculate a `classification_report` and a `confusion_matrix`, but you will need to do some personal googling/exploring to acquaint yourself with these metrics and understand the outputs.

One way to calculate a `classification_report` and a `confusion_matrix` is the following:

```

test_steps_per_epoch =
numpy.math.ceil(validation_iterator.samples /
validation_iterator.batch_size)

predictions = model.predict(validation_iterator,
steps=test_steps_per_epoch)

test_steps_per_epoch =
numpy.math.ceil(validation_iterator.samples /
validation_iterator.batch_size)

predicted_classes = numpy.argmax(predictions, axis=1)

true_classes = validation_iterator.classes

class_labels = list(validation_iterator.class_indices.keys())

report = classification_report(true_classes, predicted_classes,
target_names=class_labels)

print(report)

```

```
cm=confusion_matrix(true_classes,predicted_classes)
print(cm)
```


PyTorch

Predicting Electric Vehicle Charging Loads

In this project, you will use PyTorch to train a neural network to predict residential electric vehicle charging loads using [real-world data](#) from apartment buildings in Norway.

In particular, you will use input features like

- plug-in duration
- whether the location is private/public
- the month
- the day of the week
- traffic density

to predict

- the actual charging load in kilowatt hours for a charging session

If it performs well, a model like this could be useful in predicting things like energy costs when developing EV charging infrastructure.

Note: we've included all of the data wrangling steps as tasks in this project, since it is important to practice these basic skills. Feel free to look at the solution if you don't remember some of the specific pandas syntax for these steps!

Tasks

0/19 complete

[Mark the tasks as complete by checking them off](#)

Task Group 1 - Load, Inspect, and Merge Datasets

1.

The file `'datasets/EV_charging_reports.csv'` contains electric vehicle (EV) charging data. These come from various residential apartment buildings in Norway. The data includes specific user and garage information, plug-in and plug-out times, charging loads, and the dates of the charging sessions.

Import this CSV file to a pandas DataFrame named `ev_charging_reports`.

Use the `.head()` method to preview the first five rows.

Use the pandas `pd.read_csv()` method to import the CSV file into a pandas DataFrame.

Use the `.head()` method to preview the first five rows of the pandas DataFrame.

2.

Import the file `'datasets/Local_traffic_distribution.csv'` to a pandas DataFrame named `traffic_reports`. This dataset contains the hourly local traffic density counts at 5 nearby traffic locations.

Preview the first five rows.

Use the pandas `pd.read_csv()` method to import the CSV file into a pandas DataFrame.

Use the `.head()` method to preview the first five rows of the pandas DataFrame.

3.

We'd like to use the traffic data together with charging data in our model. The same charging location may charge at different rates depending on the number of cars being charged, so this traffic data might help the model out.

Merge the `ev_charging_reports` and `traffic_reports` datasets together into a DataFrame named `ev_charging_traffic` using the columns:

```
Start_plugin_hour in ev_charging_reports
Date_from in traffic_reports
```

Use pandas `.merge()` function to combine two DataFrames using the `left_on` and `right_on` arguments to specify which columns to merge on.

4.

Use `.info()` to inspect the merged dataset. Specifically, pay attention to the data types and number of missing values in each column.

Use the following syntax to apply the `.info()` method: `df.info()`

Task Group 2 - Data Cleaning and Preparation

5.

Let's start by reducing the size of our dataset by dropping columns that won't be used for training. These include

- ID columns

- columns with lots of missing data

- non-numeric columns (for now, since we haven't yet covered using non-numeric data in neural networks)

Drop columns you don't want to use in training from `ev_charging_traffic`.

To match our solution, drop the columns

```
['session_ID', 'Garage_ID', 'User_ID', 'Shared_ID', 'Plugin_category',  
'Duration_category', 'Start_plugin', 'Start_plugin_hour',  
'End_plugout', 'End_plugout_hour', 'Date_from', 'Date_to']
```

Copy to Clipboard

Use the `.drop` pandas method with `axis=1` to drop a list of columns.

6.

Earlier we saw that the `El_kWh` and `Duration_hours` columns were object data types. Upon further inspection, we see that the reason is that the data is following European notation where commas `,` are used as decimals instead of periods.

Replace `,` with `.` in these three columns.

Use the `.str.replace()` pandas method on each of these columns.

7.

Next, convert the data types of all the columns of `ev_charging_traffic` to floats.

Use the `.astype()` pandas method.

Task Group 3 - Train Test Split

8.

Next, let's split the dataset into training and testing datasets.

The training data will be used to train the model and the testing data will be used to evaluate the model.

First, create two datasets from `ev_charging_traffic`:

- `x` contains only the input numerical features

- `y` contains only the target column `El_kWh`

The `y` variable should be a DataFrame containing only the target column `'El_kWh'`. The `x` DataFrame should contain the remaining columns.

To create `y`, use square brackets to index the DataFrame and select `El_kWh`.

To create `x`, drop `El_kWh` from the DataFrame.

9.

Use `sklearn` to split `x` and `y` into training and testing datasets. The training set should use 80% of the data. Set the `random_state` parameter to 2.

Import the `train_test_split` function from the `sklearn.model_selection` module to split `x` and `y` into training and testing data.

Use the `train_size` and `test_size` arguments to split the data into 80% training and 20% testing.

Set the random seed to 2 using the `random_state=2` argument.

Task Group 4 - Linear Regression Baseline

10.

This section is optional, but useful. The idea is to compare our neural network to a basic linear regression. After all, if a basic linear regression works just as well, there's no need for the neural network!

If you haven't done linear regression with scikit-learn before, feel free to use [our solution code](#) or to skip ahead.

Use Scikit-learn to train a Linear Regression model using the training data to predict EV charging loads.

The linear regression will be used as a baseline to compare against the neural network we will train later.

Import the `LinearRegression` class from the `sklearn.linear_model` module.

Instantiate the class to train a linear regression on the training data using the syntax:

```
from sklearn.linear_model import LinearRegression

linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
```

Copy to Clipboard

11.

Evaluate the linear regression baseline by calculating the MSE on the testing data. Use `mean_squared_error` from `sklearn.metrics`.

Save the testing MSE to the variable `test_mse` and print it out.

Generate test predictions on the testing data `x_test` using the `.predict()` method from the trained linear regression model.

Import the `mean_squared_error` function from the `sklearn.metrics` module.

Calculate the mean squared error of the testing data using `y_test` and the generated test predictions. Save the MSE to the variable `test_mse`.

Print out `test_mse`.

Task Group 5 - Train a Neural Network Using PyTorch

12.

First, we'll need to import the PyTorch library and modules.

Import the PyTorch library `torch`.

From `torch`, import `nn` to access built-in code for constructing networks and defining loss functions.

From `torch`, import `optim` to access built-in optimizer algorithms.

Use `import library` and `from library import module` syntax.

13.

Before training the neural network, we'll need to convert the training and testing sets into PyTorch tensors and specify `float` as the data type for the values.

Use the `torch.tensor()` function to convert the values of the training and testing data to PyTorch tensors.

Use the `dtype=torch.float` to convert the values to floats.

Use the `.view(-1, 1)` method to reshape the `y_train` and `y_test` tensors for training.

14.

Next, let's use `nn.Sequential` to create a neural network.

First, set a random seed using `torch.manual_seed(42)`.

Then, create a sequential neural network with the following architecture:

- input layer with number of nodes equal to the number of input features
- a first hidden layer with 56 nodes and a ReLU activation
- a second hidden layer with 26 nodes and a ReLU activation
- an output layer with 1 node

Save the network to the variable `model`.

Create a neural network using the `nn.Sequential` container.

Here is an example of a simple neural network with a single hidden layer and ReLU activation from the lesson:

```
model = nn.Sequential(  
    nn.Linear(2, 3),  
    nn.ReLU(),  
    nn.Linear(3, 1)  
)
```

Copy to Clipboard

15.

Next, let's define the loss function and optimizer used for training:

set the MSE loss function to the variable `loss`

set the Adam optimizer to the variable `optimizer` with a learning rate of 0.0007

From the `torch.nn` module use `nn.MSELoss()` and save it to variable `loss`.

Use PyTorch's implementation of the Adam optimizer from the `torch.optim` module: `optim.Adam()`. Be sure to specify the model parameters `model.parameters()` and the learning rate `lr` as arguments.

16.

Create a training loop to train our neural network for 3000 epochs.

Keep track of the training loss by printing out the MSE every 500 epochs.

The training loop should consist of the following steps:

- specify the number of epochs
- generate predictions using the forward pass
- calculate of the loss function
- compute the gradient of the loss function using the backward pass
- update the weights and biases
- reset the gradients at each training iteration

17.

Save the neural network in the `models` directory using the path `models/model.pth`.

Use the `torch.save()` function to save the neural network model inside the `models` directory.

18.

Evaluate the neural network on the testing set.

Save the testing data loss to the variable `test_loss` and use `.item()` to extract and print out the loss.

Set the PyTorch model to evaluation mode using the syntax `model.eval()`.

Disable gradient calculations using `with torch.no_grad()`.

Generate test predictions on the testing data tensor by passing it through the forward pass using `model(X_test_tensor)`. Save the test predictions to the variable `predictions`.

Calculate the testing MSE loss using the actual test values `y_test_tensor` and the test predictions `predictions`.

19.

We trained this same model for 4500 epochs locally. That model is saved as `models/model14500.pth`. Load this model using PyTorch and evaluate it. How well does the longer-trained model perform?

Use `torch.load()` and pass the filename as input.

Predicting Residential EV Charging Loads using Neural Networks

- [View Solution Notebook](#)
- [View Project Page](#)

```
# Setup - import basic data libraries
import numpy as np
import pandas as pd
```

Task Group 1 - Load, Inspect, and Merge Datasets

Task 1

The file `'datasets/EV_charging_reports.csv'` contains electric vehicle (EV) charging data. These come from various residential apartment buildings in Norway. The data includes specific user and garage information, plug-in and plug-out times, charging loads, and the dates of the charging sessions.

Import this CSV file to a pandas DataFrame named `ev_charging_reports`.

Use the `.head()` method to preview the first five rows.

What is the structure of the dataset?

- **session_ID** - the unique id for each EV charging session
- **Garage_ID** - the unique id for the garage of the apartment
- **User_ID** - the unique id for each user
- **User_private** - 1.0 indicates private charge point spaces and 0.0 indicates shared charge point spaces
- **Shared_ID** - the unique id if shared charge point spaces are used
- **Start_plugin** - the plug-in date and time in the format (day.month.year hour:minute)
- **Start_plugin_hour** - the plug-in date and time rounded to the start of the hour
- **End_plugout** - the plug-out date and time in the format (day.month.year hour:minute)
- **End_plugout_hour** - the start of the hour of the `End_plugout` hour
- **El_kWh** - the charged energy in kWh (charging loads)
- **Duration_hours** - the duration of the EV connection time per session
- **Plugin_category** - the plug-in time categorized by early/late night, morning, afternoon, and evening
- **Duration_category** - the plug-in duration categorized by 3 hour groups
- **month_plugin_{month}** - the month of the plug-in session
- **weekdays_plugin_{day}** - the day of the week of the plug-in session

Task 2

Import the file `'datasets/Local_traffic_distribution.csv'` to a pandas DataFrame named `traffic_reports`. This dataset contains the hourly local traffic density counts at 5 nearby traffic locations.

Preview the first five rows.

What is the structure of the dataset?

- **Date_from** - the starting time in the format (day.month.year hour:minute)
- **Date_to** - the ending time in the format (day.month.year hour:minute)
- **Location 1 to 5** - contains the number of vehicles each hour at a specified traffic location.

Task 3

We'd like to use the traffic data to help our model. The same charging location may charge at different rates depending on the number of cars being charged, so this traffic data might help the model out.

Merge the `ev_charging_reports` and `traffic_reports` datasets together into a Dataframe named `ev_charging_traffic` using the columns:

- `Start_plugin_hour` in `ev_charging_reports`
- `Date_from` in `traffic_reports`

Task 4

Use `.info()` to inspect the merged dataset. Specifically, pay attention to the data types and number of missing values in each column.

What do we notice about merged dataset under inspection?

We see that there are 39 columns and 6,833 rows in our merged dataset.

Some notable things we might have to address:

- We expected columns like `El_kWh` and `Duration_hours` to be floats but they are actually object data types.
- There are many identifying columns like `session_ID` and `User_ID` that might not be useful for training.

Task Group 2 - Data Cleaning and Preparation

Task 5

Let's start by reducing the size of our dataset by dropping columns that won't be used for training. These include

- ID columns
- columns with lots of missing data
- non-numeric columns (for now, since we haven't yet covered using non-numeric data in neural networks)

Drop columns you don't want to use in training from `ev_charging_traffic`.

To match our solution, drop the columns

```
['session_ID', 'Garage_ID', 'User_ID',  
 'Shared_ID',  
 'Plugin_category', 'Duration_category',  
 'Start_plugin', 'Start_plugin_hour', 'End_pluginout', 'End_pluginout_hour',  
 'Date_from', 'Date_to']
```

Task 6

Earlier we saw that the `El_kWh` and `Duration_hours` columns were object data types. Upon further inspection, we see that the reason is that the data is following European notation where commas `,` are used as decimals instead of periods.

Replace `,` with `.` in these three columns.

Task 7

Next, convert the data types of all the columns of `ev_charging_traffic` to floats.

Task Group 3 - Train Test Split

Next, let's split the dataset into training and testing datasets.

The training data will be used to train the model and the testing data will be used to evaluate the model.

Task 8

First, create two datasets from `ev_charging_traffic`:

- `X` contains only the input numerical features
- `y` contains only the target column `El_kWh`

Task 9

Use `sklearn` to split `X` and `y` into training and testing datasets. The training set should use 80% of the data. Set the `random_state` parameter to 2.

Task Group 4 - Linear Regression Baseline

This section is optional, but useful. The idea is to compare our neural network to a basic linear regression. After all, if a basic linear regression works just as well, there's no need for the neural network!

If you haven't done linear regression with scikit-learn before, feel free to use [our solution code](#) or to skip ahead.

Task 10

Use Scikit-learn to train a Linear Regression model using the training data to predict EV charging loads. The linear regression will be used as a baseline to compare against the neural network we will train later.

Task 11

Evaluate the linear regression baseline by calculating the MSE on the testing data. Use `mean_squared_error` from `sklearn.metrics`.

Save the testing MSE to the variable `test_mse` and print it out.

Looks like our mean squared error is around `131.4` (if you used different columns in your model than we did, you might have a different value). Remember, this is squared error. If we take the square root, we have about `11.5`. One way of interpreting this is to say that the linear regression, on average, is off by `11.5 kWh`.

Task Group 5 - Train a Neural Network Using PyTorch

Let's now create a neural network using PyTorch to predict EV charging loads.

Task 12

First, we'll need to import the PyTorch library and modules.

Import the PyTorch library `torch`.

From `torch`, import `nn` to access built-in code for constructing networks and defining loss functions.

From `torch`, import `optim` to access built-in optimizer algorithms.

Task 13

Before training the neural network, convert the training and testing sets into PyTorch tensors and specify `float` as the data type for the values.

Task 14

Next, let's use `nn.Sequential` to create a neural network.

First, set a random seed using `torch.manual_seed(42)`.

Then, create a sequential neural network with the following architecture:

- input layer with number of nodes equal to the number of training features
- a first hidden layer with `56` nodes and a ReLU activation
- a second hidden layer with `26` nodes and a ReLU activation
- an output layer with `1` node

Save the network to the variable `model`.

Task 15

Next, let's define the loss function and optimizer used for training:

- set the MSE loss function to the variable `loss`
- set the Adam optimizer to the variable `optimizer` with a learning rate of `0.0007`

Task 16

Create a training loop to train our neural network for 3000 epochs.

Keep track of the training loss by printing out the MSE every 500 epochs.

Task 17

Save the neural network in the `models` directory using the path `models/model.pth`.

Task 18

Evaluate the neural network on the testing set.

Save the testing data loss to the variable `test_loss` and use `.item()` to extract and print out the loss.

Task 19

We trained this same model for 4500 epochs locally. That model is saved as `models/model4500.pth`. Load this model using PyTorch and evaluate it. How well does the longer-trained model perform?

Pretty cool! The increased training improved our test loss to about `115.2`, a full `12%` improvement on our linear regression baseline. So the nonlinearity introduced by the neural network actually helped us out. That's the end of our project on predicting EV charging loads! Feel free to continue experimenting with this neural network model.

Some things you might want to investigate further include:

- explore different ways to clean and prepare the data
- we added traffic data, but there's no guarantee that more data converts to a better model. Test out different sets of input columns.
- test out different number of nodes in the hidden layers, activation functions, and learning rates
- train on a larger number of epochs

Predicting Hotel Booking Cancellations

<https://www.codecademy.com/courses/py-torch-for-classification/projects/predicting-hotel-booking-cancellations>

SOLUTIONS

<https://15b1e602141c4d6e8f056c2d40246083.cc-propeller.cloud/files/solutions.html>

In this project, you will use PyTorch and neural networks to predict hotel cancellations using [real-world hotel booking data](#) from a resort hotel.

The dataset contains many useful predictive input features such as

- the date of the booking
- the length of stay
- the number of adults, children, and babies
- the average daily rates

Your task is to build and train two neural networks:

- a binary network to predict if a customer will cancel their booking or not
- a multiclass network to predict if a customer will show up to their booking, cancel their booking ahead of time, or no-show

Training a well-performing model can provide immense value to a hotel company by helping to

- optimize revenue strategies
- better allocate resources such as staff and amenities
- assist in marketing strategies to target high-risk customers with ads and promotions

Tasks

0/26 complete

[Mark the tasks as complete by checking them off](#)

Import and Inspect

1.

Begin by importing the CSV file to a pandas DataFrame named `hotels`.

Preview the first five rows using the `.head()` method.

Use the pandas `pd.read_csv()` method to import the CSV file into a pandas DataFrame.

Use the `.head()` method to preview the first five rows of the pandas DataFrame.

2.

Let's explore the data types and whether any data is missing.

Use the `.info()` method on the `hotels` DataFrame to inspect the data.

Use the following syntax to apply the `.info()` method on the pandas DataFrame `df: df.info()`

3.

Let's now explore the cancellation column we want to predict.

Use the `.value_counts()` method on the `is_canceled` column to count the number **and** the percentage of overall cancellations.

Use the following syntax to apply the `.value_counts()` method on a column in the pandas DataFrame `df`:

```
df['column_name'].value_counts(0) to count the occurrences of each value
df['column_name'].value_counts(1) to compute the relative frequencies (in percentages)
of each value
```

4.

The `reservation_status` column tells us if the booking was canceled while also telling us if the customer was a no-show.

We need to be sure to exclude this column from the training set, otherwise this information will be *leaked* to our model resulting in inaccurate performance.

First, let's take a quick look at the values in this column.

Use the `.value_counts()` method on the `reservation_status` column to count the number **and** the percentage of overall cancellations.

Use the following syntax to apply the `.value_counts()` method on a column in the pandas DataFrame `df`:

```
df['column_name'].value_counts(0) to count the occurrences of each value
df['column_name'].value_counts(1) to compute the relative frequencies (in percentages)
of each value
```

5.

Before diving into building a model, let's continue to explore the dataset. It's important to understand how different columns interact with cancellations to guide our model structure!

Use the `.groupby()` method to group the data by the `arrival_date_month` column and apply the `.mean()` aggregation function on the `is_canceled` column. This will return the percent of reservations cancelled in each month.

Then, use the `.sort_values()` method to sort the percentages from lowest to highest.

Here is the syntax to use the `.groupby()` method to

group the data by a column

apply the `.mean()` aggregation function (when a column is Boolean, `mean` computes percentages)

sort the percentages from lowest to highest using the `.sort_values()` method:

```
grouped_pcts = df.groupby('grouped_column')['agg_column_name'].mean()
grouped_pcts.sort_values()
```

Copy to Clipboard

Data Cleaning and Preparation

6.

To get a sense of the categorical data in the dataset, let's start by previewing the first five rows of all columns with `object` datatype.

Create a list named `object_columns` containing only the names of the object columns (except for the reservation status columns). Select those columns from `hotels` and preview the first 5 rows.

We can select specific columns by their column name using `select_columns = ['column1', 'column2', 'column3']` and preview them using `df[select_columns].head()`.

7.

Typically, we don't want to use every column in training. For example, we may want to drop columns with many missing values or columns that are irrelevant to our prediction task.

Drop any columns you don't want to use to train a cancellation model (do not remove the target label column). Feel free to open our Hint to review the columns we chose to drop in our solution. Note that if you drop other columns, your results may not match ours. We'd recommend copying the list from the hint at first, so that our analytical notes in the notebook match your code.

Note: We don't want to drop the `reservation_status` column from the dataset quite yet because we'll be using this column to train our multiclass neural network.

Use the `.drop` pandas method with `axis=1` to drop a list of columns.

Here's a list of potential features to drop. Feel free to experiment on your own by dropping or keeping columns you might believe may contribute to training.

```
drop_columns = ['country', 'agent', 'company', 'reservation_status_date',
                'arrival_date_week_number', 'arrival_date_day_of_month',
                'arrival_date_year']

hotels = hotels.drop(labels=drop_columns, axis=1)
```

Copy to Clipboard

Here's why we chose these columns:

`country` - there are many countries that only appear a handful of times in the dataset which may make our model less generalizable and even discriminate against customers based on their country

`agent` - similar to `country`, there are many agents that only appear a handful of times which may make our model less generalizable (and there are many missing values!)

`company` - similar to `agent`, there are many companies that only appear a handful of times which may make our model less generalizable (and there are many missing values!)

`reservation_status_date` - tells us the date of the latest status change of the reservation which shouldn't be helpful and if anything may leak data

`arrival_date_week_number` - tells us the week of the year which may be too specific and prone to overfitting

`arrival_date_day_of_month` - tells us the day of the month which may be too specific and prone to overfitting

`arrival_date_year` - tells us the year of the booking which may not be helpful to predict future years

8.

Next, let's encode the `meal` column which tells us which type of meal(s) the customer booked:

`Undefined` and `SC` correspond to no meal packages

`BB` corresponds to breakfast only

`HB` (half board) corresponds to breakfast + lunch or dinner

`FB` (full board) corresponds to breakfast, lunch, and dinner.

Label encode the `meal` column with a meaningful order (# of meals booked) using the following scheme:

`Undefined` and `SC` to 0

`BB` to 1

`HB` to 2

`FB` to 3

We can label encode a categorical column using the `.replace()` method, passing in a dictionary that maps the category names to an integer value.

9.

Let's prepare the rest of the categorical columns using one-hot encoding.

Create a list named `one_hot_columns` containing the list of categorical column names (all the remaining categorical columns) to be one-hot encoded using the `pd.get_dummies()` method.

Preview the cleaned `hotels` DataFrame using the `.head()` method.

We can one-hot encode multiple categorical columns at once using the `pd.get_dummies()` method. Be sure to specify `dtype=int` to return integer values 1 for True and 0 for False.

```
one_hot_columns = ['column1', 'column2', 'column3']

df = pd.get_dummies(df, columns=one_hot_columns, dtype=int)
```

Copy to Clipboard

Create Training and Testing Sets

10.

Let's import the necessary PyTorch libraries and modules.

The necessary PyTorch libraries and modules include: `torch`, `torch.nn`, and `torch.optim`.

11.

We need to start by separating our training features from the target labels.

Create a list named `train_features` that contains all of the feature names (column names excluding the target variables `is_canceled` and `reservation_status`).

To select all of the columns in our data as features for training while excluding the target columns, we can use the syntax:

```
# Remove target columns
remove_cols = ['target_column1', 'target_column2']

# Select training features
train_features = [x for x in df.columns if x not in remove_cols]
```

Copy to Clipboard

12.

Using the list of training features in `train_features`, create `x` and `y` tensors:

`x` contains the data values from the `train_features` columns

`y` contains the binary labels in the `is_canceled` column in `hotels`

Both `x` and `y` should have the float datatype.

Be sure to set the correct view of `y` using `.view(-1,1)`.

When creating the tensors, be sure to extract the data values in the specified columns using `.values` as floats:

`x` should only include the data values contained in the `train_feature` columns

`y` should only include the binary label values in the target `is_canceled` column

13.

Let's now split our data contained in `x` and `y` into training and testing sets.

Import the `train_test_split` module from Scikit-learn's `sklearn.model_selection` library.

Split `X` and `y` using the following scheme:

- Use 80% of the data for the training set `X_train` and `y_train`

- Use 20% of the data for the testing set `X_test` and `y_test`

- Set the random state to 42 to match our solution

Print out the shape of `X_train` and `X_test` to see how many observations and columns are in the training and testing sets.

How many training features does our training set `X_train` have?

```
Import the train_test_split module from Scikit-learn's sklearn.model_selection library using
from sklearn.model_selection import train_test_split.
```

Split `X` and `y` into training and testing sets using 80% for training and 20% for testing.

Set the random seed to 42 using the `random_state=42` argument.

Train a Neural Network for Binary Classification

14.

Set a random seed to 42 using `torch.manual_seed(42)`.

Build the neural network architecture using `nn.Sequential` with the following:

- input layer with 65 nodes (equal to the number of training features)

- first hidden layer with 36 nodes and a ReLU activation

- second hidden layer with 18 nodes and a ReLU activation

- output layer with 1 node and a Sigmoid activation

Save the network to the variable `model`.

Create a neural network using the `nn.Sequential` container.

Here is an example of a simple neural network with the following architecture:

- input layer with 2 nodes

- a single hidden layer with 3 nodes and ReLU activation

- output layer with 1 node and Sigmoid activation

```
model = nn.Sequential(
    nn.Linear(2, 3),
    nn.ReLU(),
    nn.Linear(3, 1),
    nn.Sigmoid()
)
```

Copy to Clipboard

15.

Next, let's define the loss function and optimizer used for training:

- set the **binary cross-entropy** loss function to the variable `loss`

- set the **Adam** optimizer to the variable `optimizer` with a learning rate of 0.005

From the `torch.nn` module use `nn.BCELoss()` and save it to variable `loss`.

Use PyTorch's implementation of the Adam optimizer from the `torch.optim` module: `optim.Adam()`. Be sure to specify the model parameters `model.parameters()` and the learning rate `lr` as arguments.

16.

Let's build the training loop to train our neural network.

Train the neural network for `1000` epochs.

Keep track of the training performance by printing out the binary cross-entropy loss and accuracy score every `100` epochs.

Before calculating accuracy, convert the model's predicted probabilities to binary labels (as integers) using `0.5` as the threshold.

The training loop should consist of the following steps:

- specify the number of epochs
- generate predictions using the forward pass
- calculate the loss using the loss function
- compute the gradient of the loss function using the backward pass
- update the weights and biases
- reset (zero) the gradients at each training iteration

17.

Let's evaluate the trained neural network on the testing set:

Set the model to **evaluation mode**

Turn off gradient calculations

Generate predicted probabilities on `X_test`. Save the probabilities to the variable `test_predictions`.

Convert the predicted probabilities to binary labels using `0.5` as the threshold. Save the labels to the variable `test_predicted_labels`.

Set the PyTorch model to evaluation mode using the syntax `model.eval()`.

Disable gradient calculations using `with torch.no_grad()`.

Generate predicted probabilities on the testing data tensor by passing it through the forward pass using `model(X_test)`. Save the test predictions to the variable `test_predictions`.

Convert the predicted probabilities to binary labels with `0.5` as the threshold using the syntax:

`(test_predictions >= 0.5).int()`. Save the predicted labels to the variable `test_predicted_labels`.

18.

Recall that the number of cancellations is much lower than the number of non-cancellations (27.8% canceled vs 72.2% did not cancel).

To evaluate our neural network effectively, compute the accuracy, precision, recall, and F1 scores using the `sklearn.metrics` module:

use the `accuracy_score` function to compute the overall accuracy

use the `classification_report` function to compute the precision, recall, and F1 scores

Print out the accuracy and classification report.

Import the evaluation functions in Scikit-learn using the syntax: `from sklearn.metrics import accuracy_score, classification_report`.

Each evaluation function takes in the actual labels and predicted labels as input using the syntax:

`accuracy_score(y_test, test_predicted_labels)`.

Train a Neural Network for Multiclass Classification

19.

First, let's label encode the three categories in the `reservation_status` column:

Check-Out to 2

Canceled to 1

No-Show to 0

We can label encode a categorical column using the `.replace()` method, passing in a dictionary that maps the category names to an integer value.

20.

Using the same list of training features in `train_features`, create the `x` and `y` tensors where:

`x` contains the data values from the `train_features` columns

`y` contains the multiclass data values in the `reservation_status` column

Make sure that `y` uses the `long` datatype.

When creating the tensors, be sure to extract the data values in the specified columns using `.values` as floats:

`x` should only include the data values contained in the `train_feature` columns

`y` should only include the binary label values in the target `reservation_status` column

The `y` tensor should have the long datatype using the argument `dtype=torch.long`.

21.

Similar to before, split the `x` and `y` tensors into training and testing splits using the following scheme:

Use 80% of the data for the training set `x_train` and `y_train`

Use 20% of the data for the testing set `x_test` and `y_test`

Set the random state to 42

Import the `train_test_split` module from Scikit-learn's `sklearn.model_selection` library using `from sklearn.model_selection import train_test_split`.

Split `x` and `y` into training and testing sets using 80% for training and 20% for testing.

Set the random seed to 42 using the `random_state=42` argument.

22.

Set a random seed using `torch.manual_seed(42)`.

Next, let's construct the multiclass neural network with the following architecture:

input layer with 65 nodes (equal to the number of training features)

first hidden layer with 65 nodes and a ReLU activation

second hidden layer with 36 nodes and a ReLU activation

final output layer with 3 nodes corresponding to each of the categories in `reservation_status`

Save the network to the variable `multiclass_model`.

Create a neural network using the `nn.Sequential` container.

Here is an example of a simple neural network with the following architecture:

input layer with 12 nodes

single hidden layer with 6 nodes and ReLU activation

output layer with 4 nodes corresponding to a target column with 4 categories

```
multiclass_model = nn.Sequential(  
    nn.Linear(12, 6),  
    nn.ReLU(),  
    nn.Linear(6, 4)  
)
```


Copy to Clipboard

23.

Next, let's define the loss function and optimizer used for multiclass training:

set the **cross-entropy** loss function for multiclass to the variable `loss`

set the **Adam** optimizer to the variable `optimizer` with a learning rate of `0.01`

From the `torch.nn` module use `nn.CrossEntropyLoss()` and save it to variable `loss`.

Use PyTorch's implementation of the Adam optimizer from the `torch.optim` module: `optim.Adam()`. Be sure to specify the model parameters `model.parameters()` and the learning rate `lr` as arguments.

24.

Let's build the training loop to train our neural network.

Train the neural network for `500` epochs.

Keep track of the training performance by printing out the cross-entropy loss and accuracy score every `100` epochs.

Be sure to convert the output probabilities of the multiclass model to labels using the `torch.argmax()` function.

The training loop should consist of the following steps:

specify the number of epochs

generate predictions using the forward pass

calculate the loss using the loss function

compute the gradient of the loss function using the backward pass

update the weights and biases

reset (zero) the gradients at each training iteration

25.

Let's evaluate the trained neural network on the testing set:

Set the multiclass model to **evaluation mode**

Turn off gradient calculations

Generate predicted probabilities on `X_test`. Save the predicted probabilities to the variable `multiclass_predictions`.

Select the class with the largest predicted probability using the `torch.argmax()` function. Save the predicted classes to the variable `multiclass_predicted_labels`.

Set the PyTorch model to evaluation mode using the syntax `multiclass_model.eval()`.

Disable gradient calculations using `with torch.no_grad()`.

Generate predicted probabilities on the testing data tensor by passing it through the forward pass using `multiclass_model(X_test)`. Save the test predictions to the variable `multiclass_predictions`.

Select the class with the highest predicted probability using the `torch.argmax()` syntax: `torch.argmax(multiclass_predictions, dim=1)`. Save the predicted labels to the variable `multiclass_predicted_labels`.

26.

Lastly, let's evaluate the multiclass neural network by calculating the overall accuracy, precision, recall, and F1 scores.

Using the `sklearn.metrics` module:

use the `accuracy_score` function to compute and save the overall accuracy to the variable `multiclass_accuracy`

use the `classification_report` function to compute and save the classification metrics for each class to the variable `multiclass_report`

Print the overall accuracy and classification report for our multiclass model.

Import the evaluation functions in Scikit-learn using the syntax: `from sklearn.metrics import accuracy_score, classification_report`.

Each evaluation function takes in the actual labels and predicted labels as input using the syntax: `accuracy_score(y_test, multiclass_predicted_labels)`.

24 OpenAI API Coding in Python

OpenAI API Coding with Python

OpenAI Python API: Recipe Blog

As a new food blogger looking to create the next big culinary hub for your readers, get ready to transform your coding skills into a flavorful experience!

In this project, you'll build a recipe generator that provides tailored recipe recommendations by providing the model with context about the user's dietary preferences, ingredients on hand, and cuisine favorites. You'll practice prompt engineering to format the model output recipes in the best format for your blog.

Let's get cooking!

Tasks

0/14 complete

Mark the tasks as complete by checking them off

Setting Up the Environment

1.

Start the project by importing the `OpenAI` class from the `openai` module.

Use the following syntax:

```
from module_name import ClassName
```

Copy to Clipboard

2.

Create an instance of the `OpenAI` class and assign it to a variable named `client`.

Use the following syntax:

```
variable = ClassName()
```

Copy to Clipboard

Create a User Profile

3.

Create a dictionary named `user_profile` with no entries in it.

Use the following syntax:

```
dictionary_name = {}
```

Copy to Clipboard

4.

Update the previously defined dictionary `user_profile` by adding a new key-value pair. The key should be `"dietary_restrictions"` and the value should be a string listing the user's dietary restrictions.

The restrictions can be separated by commas or written in any other format that clearly specifies the various restrictions.

Use the following syntax:

```
dictionary_name['new_key'] = 'new_value'
```

Copy to Clipboard

5.

Add to the `user_profile` dictionary by inserting a new key-value pair. Assign the key `"cuisine_preferences"` and set its value to a string that indicates the user's favorite cuisines.

Format this list with commas or any other clear way to denote the different preferences.

Use the following syntax:

```
dictionary_name['new_key'] = 'new_value'
```

Copy to Clipboard

6.

Finish updating the `user_profile` dictionary by adding a key called `"ingredients_available"`.

The value for this key should be a string listing the ingredients the user has on hand.

The list can be separated by commas or written in a format that clearly outlines the various ingredients available to the user.

Use the following syntax:

```
dictionary_name['new_key'] = 'new_value'
```

Copy to Clipboard

Prepare the Prompts

7.

Create a dictionary called `system_prompt` which will define the instructions for the AI. This dictionary should instruct the AI to generate HTML code for a recipe blog that takes into account specific dietary restrictions, chosen cuisine type, and a provided list of ingredients.

Ensure that the dictionary includes a key named `"role"` with the value `"system"`, and another key named `"content"` where the value is the instruction string for the AI.

Example Prompt

Use the following syntax:

```
dictionary_name = {  
    'key1': 'value1',  
    'key2': 'value2'  
}
```

Copy to Clipboard

8.

Create a string variable named `user_content1` that begins the user prompt. This string should start with a sentence indicating your intention to create a recipe blog post. Then, proceed to include the relevant data from the `user_profile` dictionary, making sure to specify what each piece of data from `user_profile` represents.

For example, the string should clearly label the dietary restrictions, cuisine preferences, and available ingredients as provided in the `user_profile`.

Example Prompt

Use the following syntax:

```
string variable = f"PROMPT STRING WITH {variable}"
```

Copy to Clipboard

9.

Construct a string called `user_content2` to continue shaping the user prompt. This string should outline the structure of a blog post, including requests for specific elements such as the title, description, ingredients, and instructions.

Within this string, you have the option to include examples illustrating the preferred format for listing ingredients and instructions, guiding the AI on how to present these sections effectively.

Example Prompt

Use the following syntax:

```
string_variable = "PROMPT STRING"
```

Copy to Clipboard

10.

Construct a string with the name `user_content3` that establishes certain limitations for the recipe creation. This string should specify that:

- the recipes must be made using only the ingredients listed in the `user_profile`
- the AI's output should be limited to a single blog post
- the recipe instructions should not exceed six steps.

These constraints will guide the AI in generating content that adheres to the given parameters.

Example Prompt

Use the following syntax:

```
string_variable = "PROMPT STRING"
```

Copy to Clipboard

11.

Define a dictionary named `user_prompt` that will hold the instructions for the AI. This dictionary should include a key `"role"` with the value `"user"`.

Additionally, add another key `"content"` where the value is a concatenation of the strings `user_content1`, `user_content2`, and `user_content3`.

When combining these strings, ensure you use appropriate punctuation and spacing to distinguish each part clearly. You may use the newline character `"\n"` to insert a line break between each string, which will help maintain a readable and organized format for the prompt.

Use the following syntax:

```
dictionary_name = {  
    'key1': 'value1',  
    'key2': variable1 + "\n" + variable2 + "\n" + variable3  
}
```

Copy to Clipboard

Make the Chat Completion

12.

Utilize the `client` variable to initiate a chat completion. Pass the following keyword arguments to the function:

Assign the `model` keyword argument a value of either `"gpt-3.5-turbo"` or `"gpt-4-turbo-preview"`, depending on which model you want to use.

For the `messages` keyword argument, provide a list that contains the dictionary `system_prompt` as its first item, followed by the dictionary `user_prompt` as the second item.

Assign the return value of the chat completion to a variable called `response`.

Use the following syntax:

```
response = client.chat.completions.create(  
    model=MODEL_STRING,  
    messages=[prompt1, prompt2]  
)
```

Copy to Clipboard

13.

Output the chat completion reply content to the terminal.

Use the following code to output:

```
print(response.choices[0].message.content)
```

Copy to Clipboard

14.

The result of this process should be HTML code that is prepared for review and can then be published on a website.

Employing generative AI for automating such tasks significantly streamlines the content creation workflow, making it more efficient for producing various types of content.

